

Analysis of Key Economic Datasets

1. 10-Year Breakeven Inflation Rate (T10YIE)

Definition and Formula

The 10-year breakeven inflation rate is calculated as:

$$10Y_{\text{Breakeven}} = 10Y_{\text{Nominal Treasury Yield}} - 10Y_{\text{TIPS Yield}}$$

This represents the market's expectation of average inflation over the next 10 years.

Historical Trends and Implications

- Breakeven inflation rates tend to decline during economic crises (e.g., 2008, 2020) due to deflation fears.
- They increase during inflationary periods (e.g., 2021-2022) when investors anticipate higher inflation.
- It is influenced by central bank policies, investor risk appetite, and liquidity conditions.

Applications in Economic Analysis

- Used by policymakers to gauge long-term inflation expectations.
- Helps investors decide between nominal and inflation-protected bonds.
- A rising breakeven rate can signal growing concerns about inflation.

Relationship to Other Datasets

- Closely linked to Treasury yields and TIPS yields.
- Related to short-term inflation measures like CPI and PCE.

2. 5-Year, 5-Year Forward Inflation Expectation Rate (T5YIFR)

Definition and Derivation

This metric represents expected inflation for the five-year period beginning five years from today. It is calculated as:

$$5Y5Y = \left[\frac{(1 + \frac{10Y_{\text{nom}}}{100})^{10}}{(1 + \frac{5Y_{\text{TIPS}}}{100})^5} \right]^{\frac{1}{5}} - 1$$

Historical Trends and Monetary Policy Role

- Typically remains close to the Fed's 2% inflation target.
- Declined during deflationary periods (2009, 2015, early 2020) and rose during inflationary fears (2021-2022).
- Used by the Fed to monitor inflation expectations.

Applications in Predicting Inflation Expectations

- Helps forecast long-term inflation trends.
- Used in pricing long-dated securities and contracts.

Relationship with Breakevens and Yields

- Related to the 10-year breakeven inflation rate and nominal Treasury yields.
- A rising 5Y5Y can signal market concerns about unanchored inflation expectations.

3. Consumer Price Index for All Urban Consumers (CPIAUCNS)

CPI Calculation

$$CPI = \frac{\text{Cost of basket in current period}}{\text{Cost of basket in base period}} \times 100$$

Historical Inflation Trends

- CPI inflation was high in the 1970s (~14% in 1980), moderate in the 1990s (2-3%), and surged again in 2021-2022 (>7%).
- Affects real incomes and purchasing power.

Applications in Economic Analysis

- Used for cost-of-living adjustments in wages, pensions, and tax brackets.
- Fed watches CPI as an inflation measure, though it prefers PCE.

Comparison with PCE Price Index (PCEPI)

- CPI is a fixed-basket Laspeyres index, while PCE is a chain-weighted index.
 - CPI is typically higher than PCE due to different weighting and methodology.
-

4. Real Gross Domestic Product (GDPC1)

GDP Calculation

$$Real\ GDP = \frac{Nominal\ GDP}{GDP\ Deflator} \times 100$$

Trends in Real GDP

- Post-WWII average GDP growth was ~3% per year.
- Deep contractions occurred in 2008 (-4%) and 2020 (-9%).
- Long expansions (e.g., 1990s, 2010s) saw steady growth.

Use in Measuring Economic Performance

- Key measure of economic health.
- Central to fiscal and monetary policy decisions.

Relationship with Unemployment and Inflation

- Related to unemployment via **Okun's Law** (1% rise in unemployment ≈ 2% GDP loss).
 - Connected to inflation via the **Phillips Curve** (tight labor markets push up wages/prices).
-

5. Unemployment Rate (UNRATE)

Calculation Formula

$$Unemployment\ Rate = \frac{Unemployed}{Labor\ Force} \times 100$$

Historical Unemployment Trends

- Peaked at **14.7% in April 2020**, the highest since WWII.
- Long-term average ~5.8%, with low points near **3.5%** (1969, 2019, 2022).

Applications in Labor Market Analysis

- Indicates economic slack or tightness.
- Used in macro models and recession predictions.

Relationship with GDP and Inflation

- Linked to GDP via **Okun's Law**.
 - Related to inflation via the **Phillips Curve**.
-

6. Federal Funds Effective Rate (FEDFUNDS)

Determination and Mechanics

- Set by the **Federal Open Market Committee (FOMC)**.
- Influences short-term borrowing costs and liquidity conditions.

Historical Interest Rate Cycles

- Peaked at **20% in 1980** to combat inflation.
- Cut to **0% in 2008 and 2020** for economic stimulus.
- Raised to **5%+ in 2023** to control inflation.

Applications in Monetary Policy

- Primary tool for influencing economic growth and inflation.
- Affects mortgage rates, business investment, and consumer spending.

Influence on Inflation, GDP, and Treasury Yields

- Higher rates slow inflation but risk recession.
 - Directly impacts Treasury yields and credit markets.
-

7. Personal Consumption Expenditures: Chain-type Price Index (PCEPI)

PCE Price Index Calculation

- Chain-weighted index accounts for **consumer substitution**.
- Fed's preferred inflation measure.

Applications in Inflation Tracking

- Used for forecasting and economic modeling.
- Monitored for Fed's 2% inflation target.

Comparison with CPI

- PCE is generally lower than CPI due to different weights and methodology.
-

8. Nominal Broad U.S. Dollar Index (DTWEXBGS)

Calculation Methodology

- Trade-weighted index of USD against a basket of currencies.

Historical Dollar Trends

- Peaked in **1985, early 2000s, and 2022**.
- Strong dollar tends to increase trade deficits.

Use in Forex Analysis

- Indicates USD strength relative to major trading partners.
 - Impacts inflation via import price effects.
-

9. Crude Oil Prices: WTI (DCOILWTICO)

Oil Price Determinants

- Driven by **supply-demand dynamics**, OPEC policy, and geopolitical risks.

Historical Price Fluctuations

- Reached **\$147 in 2008**, **negative in April 2020**, **\$120 in 2022**.

Impact on Inflation and Economy

- High oil prices contribute to inflation.
 - Price swings impact energy markets and consumer spending.
-

10. Market Yield on U.S. Treasury Securities at 10-Year Constant Maturity (DGS10)

How 10-Year Yields Are Determined

- Interpolated yield from Treasury curve.

Economic Significance

- Benchmark for long-term borrowing rates.
- Inversions predict recessions.

Relationship with Inflation Expectations

- Higher expected inflation → Higher 10Y yield.
- Used in macroeconomic forecasting and risk assessment.

Dataset Preparations

Description of the Code Blocks

Block 1: Mount Google Drive

The script starts by **mounting Google Drive** to access stored datasets.

- Uses `drive.mount('/content/drive')` to link the Colab notebook to Google Drive.
 - **Ensures direct access** to files located in the specified directory (`/content/drive/My Drive/Dataset/`).
-

Block 2: Listing Available Files

- Uses `os.listdir(folder_path)` to **list all files** in the dataset directory.
 - Helps **verify file existence** before attempting to load them.
-

Block 3: Detecting the Date Column in CSV Files

A **helper function** `detect_date_column(df)` :

- **Scans column names** (case-insensitive) for possible date columns (`'date'` or `'observation_date'`).
 - **Raises an error** if no valid date column is found.
 - **Ensures datasets have a valid time index.**
-

Block 4: Function to Load and Preprocess Datasets

A function `read_dataset(filename, rename_dict)` :

1. **Loads a CSV file** from the dataset folder.
 2. **Detects and converts the date column** to a `DatetimeIndex`.
 3. **Ensures proper indexing** (Raises an error if the date conversion fails).
 4. **Renames columns** using the provided mapping dictionary.
 5. **Returns a cleaned Pandas DataFrame.**
-

Block 5: Configuring Datasets for Loading

A configuration list `datasets_config` :

- Stores tuples with:
 - `CSV filenames`
 - `Corresponding column renaming dictionaries`
 - Covers key economic indicators:
 - `Inflation Rates, CPI, GDP, Unemployment, Interest Rates, Oil Prices, Treasury Yields, etc.`
 - Ensures consistency in variable naming across multiple datasets.
-

Block 6: Loading Multiple Datasets

Iterates over `datasets_config` and:

- Calls `read_dataset()` for each dataset.
- Handles errors gracefully, printing messages for missing/corrupted files.
- Stores successfully loaded datasets in a list.

Error Handling

- If no datasets are successfully loaded, a `ValueError` is raised.
-

Block 7: Resampling Datasets to Daily Frequency

- Uses `.asfreq('D').ffill()` to:
 - Ensure all datasets have a daily frequency.
 - Forward-fill missing values to maintain continuity.
-

Block 8: Finding the Common Date Range

- Determines the overlapping date range across all datasets:
 - `common_start`: Latest start date (most restrictive).
 - `common_end`: Earliest end date (most restrictive).
- Ensures all datasets align temporally.

Error Handling

- If no common date range exists, a `ValueError` is raised.
-

Block 9: Trimming Datasets to the Common Date Range

- Uses `.loc[common_start:common_end]` to:
 - Crop each dataset to match the overlapping date range.
 - Ensure consistency across all datasets.
-

Block 10: Merging All Datasets

- Performs an inner join (`pd.concat` with `join='inner'`):
 - Merges all datasets on the date index.
 - Keeps only dates present in all datasets.
-

Block 11: Saving the Merged Dataset

- Exports the final merged dataset to:
 - `merged_dataset.csv` in the dataset folder.
- Ensures easy reusability for further analysis.

Final Output

- Prints the merged dataset's shape (rows × columns).
- Confirms successful file saving.

Conclusion

1. Automates the process of loading, cleaning, and merging economic datasets.
2. Ensures uniform time indexing and proper column naming.
3. Prepares a high-quality dataset for time-series analysis and forecasting.

Output Result:

Files in the folder:

```
['T10YIE.csv', 'T5YIFR.csv', 'CPIAUCNS.csv', 'PCEPI.csv', 'GDPC1.csv', 'UNRATE.csv', 'FEDFUNDS.csv', 'DGS10.csv', 'DTWEXBGS.csv', 'DCOILWTIC0.csv', 'merged_dataset.csv']
```

Common start date: 2006-01-02 00:00:00

Common end date: 2024-10-01 00:00:00

Merged dataset shape: (6848, 10)

Merged dataset saved to: /content/drive/My Drive/Dataset/merged_dataset.csv

Data Preprocessing

Description of the Code Blocks

Block 1: Importing Libraries and Mounting Google Drive

This block initializes the necessary Python libraries for data processing, machine learning, and visualization. Key libraries include:

- `pandas` and `numpy` for data handling and numerical operations.
- `matplotlib` and `seaborn` for visualization.
- `scikit-learn` for machine learning utilities, including data preprocessing, model evaluation, and performance metrics.

Additionally, the script mounts Google Drive for cloud storage access, making it easier to load datasets when working in Google Colab.

Block 2: Loading the Merged Dataset

This section defines a function `load_dataset(file_path)`, which:

- Loads a CSV dataset.
- Parses date columns.
- Sets the first column as the index (assumed to be a datetime index).
- Implements exception handling to ensure robust dataset loading.

The dataset is loaded twice:

- Once into `df` for processing.
- Once into `data` for visualization.

The script also performs an initial check on column names and missing values.

Block 3: Data Preparation and Target Imputation

A function `prepare_target(df, target)` is implemented to handle missing values in the target column:

- Converts the target variable to numeric format.
- Uses **time-based interpolation** to impute missing values, preserving temporal trends rather than relying on forward-filling.
- Drops any remaining `NaN` values after interpolation.

This approach ensures minimal data loss while maintaining the integrity of time-series trends.

Block 4: Feature Engineering (Lag Features and Rolling Mean)

The function `create_lagged_features(df, target_col, lags=5, seasonal_lag=None)` enhances predictive modeling by introducing:

- **Lag features:** Previous values of the target variable up to `lags` time steps.
- **Seasonal lag:** Optional delayed values to capture periodic trends.
- **Rolling mean:** The moving average of the target variable over the specified lag window.

The function drops missing values arising from feature creation, ensuring a clean dataset for modeling.

Block 5: Train-Test Split and Feature Scaling

The function `split_and_scale(data, target, split_ratio=0.8)` ensures:

- **Chronological splitting:** The dataset is sorted by time before splitting to prevent data leakage in time-series forecasting.
 - **Feature scaling:** Standardization (`StandardScaler`) is applied only to numeric predictor variables to ensure uniform feature magnitudes.
 - **Data partitioning:** The dataset is split into training (80%) and testing (20%) sets to evaluate model performance on unseen data.
-

Block 6: Model Evaluation Metrics

The function `calculate_metrics(y_true, y_pred, naive_forecast, model_name)` computes key error metrics, including:

- **MSE (Mean Squared Error) and RMSE (Root Mean Squared Error):** Measures average squared error and its root, penalizing large deviations.
- **MAE (Mean Absolute Error):** The mean absolute difference between predictions and actual values.
- **MAPE (Mean Absolute Percentage Error) and sMAPE (Symmetric MAPE):** Measures error as a percentage, handling different scales.
- **R² Score:** Indicates model goodness-of-fit.
- **MASE (Mean Absolute Scaled Error):** Compares model error against a naive baseline forecast.

This function provides a comprehensive assessment of model accuracy while enabling comparison against baseline performance.

This structured approach ensures a robust and reproducible framework for time-series forecasting, emphasizing best practices in data preprocessing, feature engineering, and evaluation.

Result Output:

Dataset Columns:

```
Index(['10-Year Breakeven Inflation Rate', '5-Year, 5-Year Forward Inflation Expectation Rate',  
      'Consumer Price Index', 'Real Gross Domestic Product', 'Unemployment Rate', 'Federal Funds Effective  
      Rate', 'Personal Consumption Expenditures: Chain-type Price Index', 'Nominal Broad U.S. Dollar  
      Index', 'Crude Oil Prices: West Texas Intermediate', '10-Year Treasury Yield'], dtype='object')
```

Null Values per Column:

10-Year Breakeven Inflation Rate 0

5-Year, 5-Year Forward Inflation Expectation Rate 0

Consumer Price Index 0

Real Gross Domestic Product 0

Unemployment Rate 0

Federal Funds Effective Rate 0

Personal Consumption Expenditures: Chain-type Price Index 0

Nominal Broad U.S. Dollar Index 0

Crude Oil Prices: West Texas Intermediate 0

10-Year Treasury Yield 0

dtype: int64

Data Visualization

Description of the Code Blocks

Block 1: Inflation Trends Over Time (Line Plot)

This block visualizes the inflation trends of:

- 10-Year Breakeven Inflation Rate
- 5-Year, 5-Year Forward Inflation Expectation Rate

Visualization Details:

- A **line plot** is created to show inflation trends over time.
- Dashed lines** distinguish different inflation indicators.
- X-axis:** Year
- Y-axis:** Inflation Rate (%)
- Legend & Grid:** Improve readability.

Block 2: Distribution Analysis (Histograms)

Histograms provide insight into the distribution of inflation rates:

- 10-Year Breakeven Inflation Rate
- 5-Year Forward Inflation Expectation Rate

Visualization Details:

- Histogram plots** with KDE (Kernel Density Estimation) to smooth distributions.
- 30 bins** ensure detailed representation.
- Titles & Labels:** Indicate inflation rate distributions.

Block 3: Relationship Between Inflation & Economic Indicators (Scatter Plots)

This section explores how inflation correlates with key economic indicators:

- Consumer Price Index (CPI)
- Real Gross Domestic Product (GDP)
- Federal Funds Effective Rate

Visualization Details:

- Scatter plots** are used to identify trends.
- Alpha transparency (0.5):** Prevents overplotting.
- X-axis & Y-axis labels** ensure clarity.

Block 4: Economic Indicators Correlation (Heatmap)

A **correlation heatmap** visualizes relationships among key economic indicators.

Visualization Details:

- **Correlation matrix** computed using `df.corr()`.
 - **Heatmap with annotations** provides a structured view of relationships.
 - **Coolwarm colormap** improves contrast.
-

Block 5: Box Plot & Violin Plot for Inflation Indicators

These plots compare inflation indicators' spread and density.

Visualization Details:

- **Box plot:** Shows median, quartiles, and outliers.
 - **Violin plot:** Displays density distribution alongside boxplot statistics.
 - **Rotation of x-axis labels:** Ensures better readability.
-

Block 6: Pair Plot for Inflation & Economic Indicators

A **pair plot** is generated to analyze interactions among:

- **Inflation Rates**
- **CPI**
- **GDP**
- **Federal Funds Rate**

Visualization Details:

- **Scatter plots** show pairwise relationships.
 - **KDE plots** provide smooth density estimates.
 - **Adjusted font sizes and tick rotation** improve readability.
-

Block 7: Interactive Time-Series Inflation Trends

A **Plotly Express interactive plot** allows dynamic exploration of inflation trends.

Visualization Details:

- **X-axis:** Year
 - **Y-axis:** Inflation Rate (%)
 - **Legend:** Helps distinguish different inflation indicators.
-

Block 8: CPI Annual Trends (Bar Chart)

A **bar chart** visualizes annual **Consumer Price Index (CPI)** trends.

Visualization Details:

- **Grouped by Year:** Annual CPI computed using `groupby()`.
 - **Seaborn Barplot** improves presentation.
 - **X-axis rotation (45 degrees)** enhances readability.
-

Block 9: Phillips Curve (Inflation vs. Unemployment Scatter Plot)

This plot explores the **Phillips Curve**, which describes the inverse relationship between:

- **Inflation Rate**
- **Unemployment Rate**

Visualization Details:

- Scatter plot with transparency ($\alpha=0.6$)
 - Title & labels improve understanding.
 - Grid added for clarity.
-

Block 10: CPI Time Series Decomposition

Time series decomposition breaks down **CPI** into:

1. Observed values
2. Trend component
3. Seasonal component
4. Residual component (noise)

Visualization Details:

- `seasonal_decompose()` function applied.
 - Four separate subplots for each component.
 - Grid & legend improve clarity.
-

Block 11: CPI Rolling Average (12-Month)

A 12-month rolling average is computed to smooth fluctuations in **CPI**.

Visualization Details:

- Original CPI plotted alongside rolling average.
 - Red line represents smoothed trend.
 - Grid added for readability.
-

Block 12: Interactive Inflation vs. GDP (Dual Y-Axis Plot)

A dual-axis interactive visualization compares:

- Inflation Rate (Left Y-axis)
- GDP (Right Y-axis)

Visualization Details:

- Scatter plot for inflation
 - Line plot for GDP
 - Legend & title added for better insights.
-

Block 13: Bubble Chart (Inflation vs. GDP vs. Unemployment)

A bubble chart is created to analyze how:

- GDP
- Inflation Rate
- Unemployment Rate interact.

Visualization Details:

- Bubble size corresponds to Unemployment Rate.
 - Magma colormap used for clarity.
 - Legend placed in lower right corner to avoid overlap.
-

Block 14: CPI Heatmap (Year vs. Month)

A heatmap visualizes how inflation varies across months and years.

Visualization Details:

- Pivot table for month-year structure.
 - Coolwarm color map improves visibility.
 - Annotations (CPI values) add details.
-

Block 15: CPI Lag Plot

A lag plot checks for autocorrelation in CPI values.

Visualization Details:

- `lag_plot()` function from pandas.
 - Grid added for better visualization.
-

Block 16: CPI Autocorrelation & Partial Autocorrelation (ACF/PACF)

These plots assess the dependency structure in CPI over time.

Visualization Details:

- ACF (Autocorrelation Function) shows how past values influence future CPI.
 - PACF (Partial ACF) isolates direct effects.
 - 50 lags visualized for detailed insights.
-

Block 17: Radar Chart (Economic Indicators Comparison)

A radar chart compares multiple economic indicators in a normalized scale.

Visualization Details:

- Key Indicators:
 - Inflation
 - CPI
 - GDP
 - Unemployment Rate
 - Federal Funds Rate
 - Red fill highlights latest values.
 - Legend positioned outside plot.
-

Block 18: Dynamic Cross-Correlation Plot

Cross-correlation measures how inflation and GDP interact over time.

Visualization Details:

- Cross-correlation computed using `np.correlate()`.
 - Plot shows correlation strength at different time lags.
-

Block 19: Principal Component Analysis (PCA)

PCA reduces economic indicators into two principal components for visualization.

Visualization Details:

- Scatter plot of first two principal components.
 - Standardized data before PCA transformation.
 - Grid added for better visibility.
-

Block 20: Dynamic Time Warping (DTW) - CPI vs. GDP

DTW compares time-series similarity between **CPI** and **GDP**.

Visualization Details:

- `dtw.warping_paths()` applied to time-series.
 - DTW distance matrix plotted as a heatmap.
-

Block 21: Hierarchical Clustering Heatmap

This block performs **hierarchical clustering** on economic indicators.

Visualization Details:

- Dendrogram shows how indicators cluster together.
 - `ward` method ensures optimal grouping.
-

Block 22: Network Graph of Economic Indicators

A **network graph** visualizes strong correlations between economic indicators.

Visualization Details:

- Nodes represent economic indicators.
 - Edges indicate strong correlations.
 - Size of nodes based on degree (importance).
 - Text auto-adjusted to prevent overlap.
-

Block 23: Structural Break Detection in Inflation Trends

Ruptures is used to detect breakpoints in **CPI trends**.

Visualization Details:

- Binary Segmentation (`Binseg`) applied.
 - Breakpoints plotted with red dashed lines.
-

Conclusion

This **comprehensive analysis** covers:

1. Trend visualization
2. Correlation analysis
3. Predictive insights
4. Model interpretability

Description of the Additional Code Blocks

Block 24: Time Series Plot of Target Variable

This block visualizes the time series trend of the selected **target variable**.

Visualization Details:

- X-axis: Date
 - Y-axis: Target variable value
 - Navy-colored line represents the original time series.
 - Legend & Grid: Improve readability.
 - The plot is saved as `timeseries_target.png`.
-

Block 25: Rolling Mean and Expanding Mean

To smooth fluctuations, we compute:

- **Rolling Mean** (30-day moving average)
- **Expanding Mean** (cumulative moving average)

Visualization Details:

- **Blue line:** Original data.
 - **Orange line:** 30-day Rolling Mean.
 - **Green line:** Expanding Mean.
 - **Ensures better trend detection** in the data.
-

Block 26: Hodrick-Prescott Filter for Trend & Cycle Decomposition

The **Hodrick-Prescott (HP) filter** is applied to separate:

- **Trend Component** (long-term movements)
- **Cyclical Component** (short-term fluctuations)

Visualization Details:

- **Red dashed line:** Extracted trend.
 - **Green dotted line:** Cyclical fluctuations.
 - **Helps understand underlying economic trends.**
-

Block 27: Augmented Dickey-Fuller (ADF) Test for Stationarity

The **ADF test** checks whether the time series is **stationary**.

Results Interpretation:

- **ADF Statistic & p-value displayed.**
 - **Critical values compared** at 1%, 5%, and 10% levels.
 - **If p-value < 0.05:** The series is stationary.
 - **If p-value >= 0.05:** The series is non-stationary.
-

Block 28: Time Series Decomposition

The `seasonal_decompose()` function breaks down the time series into:

1. **Observed component**
2. **Trend component**
3. **Seasonal component**
4. **Residuals (random noise)**

Visualization Details:

- **Decomposition results plotted in four subplots.**
 - **Helps analyze seasonal and trend patterns.**
-

Block 29: Time Series Resampling (Yearly and Monthly)

This block:

- **Downsamples data to yearly frequency** (mean values).
 - **Upsamples to monthly frequency** (interpolated values).
 - **Provides a more structured view of long-term inflation trends.**
-

Block 30: Time Shifting (Lead & Lag Analysis)

Shifts the time series to:

- **Forward Shift:** Simulates a **lagging** indicator.
- **Backward Shift:** Simulates a **leading** indicator.

Visualization Details:

- Original series in blue.
 - Forward-shifted series in red (dashed).
 - Backward-shifted series in green (dotted).
 - Used to explore predictive relationships.
-

Block 31: Moving Averages (SMA & EWMA)

Compares:

- Simple Moving Average (SMA)
- Exponentially Weighted Moving Average (EWMA)

Visualization Details:

- SMA (dashed orange): Evenly smooths fluctuations.
 - EWMA (dotted green): Weighs recent data more.
 - Useful for financial market trend analysis.
-

Block 32: Double EWMA & Holt-Winters Smoothing

Applies:

1. **Double EWMA:** Two-pass exponential smoothing.
2. **Holt-Winters Method:** Forecasting with trend adjustments.

Visualization Details:

- Purple dashed line: Double EWMA.
 - Red dashed line: Holt-Winters fitted values.
 - Aids in economic time-series forecasting.
-

Block 33: Highlighting Economic Crises on Time Series

Highlights:

- 2008 Financial Crisis (Orange region)
- COVID-19 Pandemic (Gray region)

Visualization Details:

- Shaded regions on the time series plot.
 - Helps visualize macroeconomic disruptions.
-

Block 34: Monthly Seasonal Analysis (Bar Chart)

Computes monthly averages of the target variable.

Visualization Details:

- Bar chart showing seasonal trends across months.
 - Month names used instead of numerical representation.
 - Highlights recurring economic cycles.
-

Block 35: Trend and Variance Analysis (Rolling Window)

Computes:

- Rolling Mean (Trend)
- Rolling Standard Deviation (Variance)

Visualization Details:

- Gray shaded area: Rolling standard deviation.
 - Green line: Rolling mean.
 - Assesses long-term economic stability.
-

Block 36: Identifying Peaks & Troughs

Uses `find_peaks()` to locate:

- Peaks (local maxima)
- Troughs (local minima)

Visualization Details:

- Red dots: Peaks (inflation spikes).
 - Green dots: Troughs (inflation dips).
 - Useful for detecting recessionary cycles.
-

Block 37: Density Plot of Target Variable

Creates a Kernel Density Estimation (KDE) plot for:

- Distribution analysis of inflation rates.

Visualization Details:

- Skyblue KDE plot.
 - Smoothed probability density curve.
-

Block 38: Density Comparison (Pre vs. Post 2020)

Compares inflation distributions before and after 2020.

Visualization Details:

- Blue KDE: Pre-2020 period.
 - Red KDE: Post-2020 period.
 - Detects inflationary shifts due to COVID-19.
-

Block 39: Monthly Heatmap of Inflation Trends

Creates a heatmap of inflation trends across months & years.

Visualization Details:

- X-axis: Month (Jan - Dec).
 - Y-axis: Year (historical range).
 - Coolwarm colormap visualizes inflation shifts.
-

Block 40: Lag Analysis (Autocorrelation & Partial ACF)

Plots:

- Autocorrelation Function (ACF)
- Partial Autocorrelation Function (PACF)

Usage:

- ACF identifies periodic dependencies.
 - PACF isolates direct correlations.
 - Crucial for forecasting models like ARIMA.
-

Conclusion

1. Advanced Time-Series Techniques Applied
2. Comprehensive Trend, Seasonality, & Correlation Analysis
3. Clear Visualizations for Economic Interpretability

Analysis of Inflation Rates Over Time

The provided graph illustrates two key inflation metrics:

- **10-Year Breakeven Inflation Rate** (solid blue line)
- **5-Year Forward Inflation Expectation Rate** (dashed orange line)

Key Findings:

- Both inflation measures predominantly fluctuate between 1% and 3%, consistent with historical market-based inflation expectations.
- A significant drop near year 1000, with rates approaching 0%, indicates a period of deflationary concern or economic crisis.
- The 5-Year Forward Inflation Expectation typically remains above the 10-Year Breakeven, suggesting markets anticipate higher medium-term inflation compared to the overall decade.
- Peaks near 3% imply heightened inflation concerns at specific intervals, potentially reflecting economic expansions or inflationary shocks.
- Lows around 0.5% indicate times of market distress or significant economic downturns, such as financial crises or pandemics, when deflationary pressures are stronger.

Overall, the chart highlights market dynamics around inflation expectations, with periods of high volatility reflecting economic uncertainty or crises, and periods of stability around 2% signaling confidence in monetary policy credibility.

Distribution Analysis of Inflation Metrics

The provided histograms depict the frequency distributions of two inflation measures:

- **10-Year Breakeven Inflation Rate**
- **5-Year Forward Inflation Expectation Rate**

Key Findings:

- **Both distributions are right-skewed:**
 - Most observations cluster between approximately 2% to 2.5%.
 - A smaller number of data points fall below 1.5% or above 2.5%.
- **Central Tendency:**
 - The peak frequency for both measures occurs around the 2.25% mark, indicating market inflation expectations typically gravitate close to the Federal Reserve's standard inflation target of ~2%.
- **Variability:**
 - 10-Year Breakeven shows slightly higher variability on the lower end, with more frequent occurrences around 1% compared to the 5-Year Forward expectation, highlighting greater volatility or deflationary scares affecting near-term inflation expectations.
 - The 5-Year Forward Inflation Expectation rate is relatively tighter in distribution, indicating more stable longer-term expectations.
- **Outliers:**
 - Both distributions contain minimal occurrences below 1% or above 2.75%, signifying rare extreme inflation scenarios or deflationary periods in historical data.

Overall, these distributions underline market confidence in inflation rates generally remaining stable around the Fed's 2% target, with short-term inflation (10-year breakeven) subject to slightly

more volatility compared to medium-term forward expectations.

Relationship Analysis of Inflation and Economic Indicators

The provided scatter plots show the relationships between the **10-Year Breakeven Inflation Rate** and three key economic indicators:

1. **Consumer Price Index (CPI)**
2. **Real GDP**
3. **Federal Funds Rate**

Key Findings:

- **Inflation vs. Consumer Price Index (CPI):**
 - Generally, higher CPI values (above 240) are associated with inflation rates mostly between 2% to 2.5%, indicating stable inflation during periods of rising consumer prices.
 - Lower CPI levels (around 200-220) exhibit wider dispersion and lower inflation rates (below 1%), suggesting deflationary pressures or economic instability.
- **Inflation vs. Real GDP:**
 - Inflation rates above 2% primarily occur at higher GDP values (around \$22,000 billion), indicating inflationary pressures often coincide with periods of economic growth.
 - Inflation rates below 1.5% mostly occur at lower GDP values (below \$18,000 billion), potentially reflecting slower economic growth or recession periods.
- **Inflation vs. Federal Funds Rate:**
 - Higher Federal Funds Rates (above 2%) correlate with moderate-to-high inflation (mostly around 2% to 2.5%), consistent with monetary tightening to control inflation.
 - At lower Federal Funds Rates (below 1%), inflation shows significant variability, ranging from near-zero to above 2.5%, reflecting monetary easing during uncertain or crisis periods.

Overall, these plots highlight clear linkages between inflation expectations and broader economic conditions, underscoring how inflation dynamics interact with economic growth, monetary policy, and consumer prices.

Correlation Analysis of Economic Indicators

The provided heatmap visualizes correlation coefficients between key economic indicators:

Key Findings:

- **Strong Positive Correlations:**
 - **Consumer Price Index (CPI)** and **Real GDP** (0.97): Indicates consumer prices typically rise alongside economic growth.
 - **Personal Consumption Expenditures Index** shows a strong correlation with both CPI (1.00) and GDP (0.97), suggesting close alignment of consumption spending with overall economic activity.
 - **Nominal U.S. Dollar Index** has strong positive correlations with CPI (0.82) and GDP (0.88), implying economic growth is typically associated with a strengthening dollar.
- **Moderate Positive Correlations:**
 - **Crude Oil Prices** are positively correlated with inflation expectations (0.74) and 10-Year Breakeven Inflation Rate (0.67), highlighting oil prices as a key driver of inflation expectations.
 - **10-Year Treasury Yield** moderately correlates with the Federal Funds Rate (0.76) and inflation expectations (0.46), illustrating the relationship between interest rates and market inflation forecasts.
- **Negative Correlations:**
 - **Unemployment Rate** negatively correlates with Real GDP (-0.59), CPI (-0.46), and Federal Funds Rate (-0.57), reflecting economic downturns associated with high unemployment.
 - **Nominal U.S. Dollar Index** negatively correlates with 5-Year Inflation Expectations (-0.55), indicating a stronger dollar tends to correspond with lower inflation expectations.
- **Notable Low Correlations:**
 - **10-Year Breakeven Inflation Rate** shows weak correlation with CPI (0.13) and GDP (0.08), indicating immediate inflation measures may not directly reflect longer-term inflation expectations.

Overall, this correlation heatmap highlights key relationships among inflation measures, economic growth, monetary policy indicators, and market expectations, providing insight into interconnected economic dynamics.

Box Plot Analysis of Inflation Indicators

The provided box plot illustrates the distribution characteristics of two inflation measures:

- **10-Year Breakeven Inflation Rate** (blue)
 - **5-Year, 5-Year Forward Inflation Expectation Rate** (orange)
-

Key Findings:

- **Median Inflation Rate**
 - The **5-Year Forward Inflation Expectation Rate** has a slightly higher median (around 2.25%) compared to the **10-Year Breakeven Inflation Rate** (around 2.0%).
 - **Interquartile Range (IQR)**
 - The **10-Year Breakeven Inflation Rate** shows a wider IQR, indicating greater variability and uncertainty in short-term inflation expectations.
 - The **5-Year Forward Inflation Expectation Rate** has a narrower IQR, suggesting more stable longer-term inflation expectations.
 - **Outliers**
 - Both indicators contain multiple outliers at the lower end (below approximately 1%), indicating periods of deflationary pressures or significant economic disruptions.
 - **Range of Inflation Rates**
 - Both inflation measures generally range from about 0% to 3%, with most observations concentrated between 1.5% and 2.5%.
-

This analysis highlights the key differences in stability and variability between short-term (10-Year Breakeven) and long-term (5-Year Forward) inflation expectations.

Violin Plot Analysis of Inflation Indicators

The provided violin plot compares the distribution and density of two inflation measures:

- **10-Year Breakeven Inflation Rate** (blue)
 - **5-Year, 5-Year Forward Inflation Expectation Rate** (orange)
-

Key Findings:

- **Distribution Shape**
 - Both indicators have distributions concentrated around 2%, highlighting typical inflation expectations aligned closely with central-bank targets.
 - The **10-Year Breakeven Inflation Rate** shows greater density toward lower values (around 1%), suggesting more frequent periods of short-term deflationary pressure or uncertainty.
 - **Median and Central Tendency**
 - The **5-Year Forward Inflation Expectation Rate** has a slightly higher median (around 2.25%) compared to the **10-Year Breakeven Inflation Rate** (approximately 2.0%), indicating somewhat higher expectations for medium-term inflation.
 - **Variability**
 - The **10-Year Breakeven Inflation Rate** displays a wider spread, especially towards lower inflation values, reflecting greater short-term uncertainty or volatility.
 - The **5-Year Forward Inflation Expectation Rate** is more symmetrically distributed and narrower, suggesting greater market confidence and stability in medium-term inflation forecasts.
 - **Extreme Values**
 - Both measures have thin tails extending towards zero inflation, signifying occasional episodes of significant deflationary concerns or economic shocks.
-

Overall, this violin plot effectively illustrates differences in distribution shape, median values, and variability, clearly distinguishing short-term inflation uncertainty from more stable, medium-term inflation expectations.

Pair Plot Analysis of Inflation & Economic Indicators

The provided pair plot visualizes distributions and relationships among five economic indicators:

- **T10YIE**: 10-Year Breakeven Inflation Rate
- **T5YIFR**: 5-Year, 5-Year Forward Inflation Expectation Rate
- **CPIAUCNS**: Consumer Price Index
- **GDPIC1**: Real Gross Domestic Product (GDP)
- **FEDFUNDS**: Federal Funds Effective Rate

Key Findings:

- **Inflation Indicators (T10YIE & T5YIFR):**
 - T10YIE and T5YIFR exhibit a strong positive linear correlation, indicating short-term and medium-term inflation expectations move closely together.
 - Both measures typically center around 2%, aligning with central bank inflation targets.
- **Consumer Price Index (CPI):**
 - CPI shows a clear positive linear relationship with Real GDP, indicating prices generally rise with economic growth.
 - CPI exhibits weaker relationships with inflation expectation indicators, indicating inflation expectations don't always align directly with observed consumer price changes.
- **Real Gross Domestic Product (GDP):**
 - GDP demonstrates strong linear correlations with CPI, reflecting consistent economic growth alongside increasing consumer prices.
 - GDP has limited direct correlation with inflation expectation indicators (T10YIE & T5YIFR), suggesting economic growth alone does not directly predict market inflation forecasts.
- **Federal Funds Effective Rate (FEDFUNDS):**
 - FEDFUNDS shows scattered relationships with both inflation expectation measures, reflecting how monetary policy adjustments vary significantly based on economic context.
 - FEDFUNDS exhibits nonlinear relationships with CPI and GDP, indicating complex interactions between monetary policy, economic growth, and price levels.
- **Distributions:**
 - Inflation measures (T10YIE and T5YIFR) have relatively symmetrical distributions around 2%.
 - CPI and GDP distributions are less symmetric and show multimodal patterns, indicating varying economic cycles or structural shifts.

Overall, the pair plot provides clear insights into the intricate interactions between inflation expectations, monetary policy, consumer prices, and economic growth.

Analysis of Annual Consumer Price Index (CPI)

The provided bar chart illustrates the **annual changes in the Consumer Price Index (CPI)** from **2006 to 2024**.

Key Findings:

- **Consistent Growth:**
 - The CPI value has consistently increased each year, reflecting ongoing inflation and rising consumer prices over nearly two decades.
- **Trend Acceleration:**
 - The growth rate of CPI appears steeper from around 2020 onwards, suggesting increased inflationary pressures during recent years.
- **Historical CPI Levels:**
 - CPI values started around 200 in 2006 and surpassed the 300-mark by 2024, representing a

significant overall increase (~50% increase).

- **Color Intensity:**

- Darker bars (earlier years) gradually fade to lighter shades (recent years), visually emphasizing the progression of time and increasing CPI.

This chart effectively highlights the steady and accelerating inflation trend over the analyzed period, underscoring long-term growth in consumer prices.

Phillips Curve Analysis: Inflation vs. Unemployment

The provided scatter plot illustrates the relationship between:

- **10-Year Breakeven Inflation Rate** (y-axis)
- **Unemployment Rate** (x-axis)

Key Findings:

- **Inverse Relationship:**
 - There is a general inverse relationship, consistent with the Phillips Curve concept, indicating higher inflation rates tend to coincide with lower unemployment rates.
- **Inflation Clustering:**
 - Higher inflation rates (around 2% to 3%) predominantly occur at lower unemployment rates (approximately 3%-6%), signaling stronger economic activity.
 - Lower inflation rates (below 1.5%) become more frequent at higher unemployment rates (above 6%), highlighting weakened economic conditions.
- **Unemployment Extremes:**
 - Very high unemployment rates (above 10%) are associated exclusively with lower inflation rates (below 1.5%), reflecting significant economic downturns or recessions.
- **Variability and Exceptions:**
 - There is considerable variability within the mid-range (4%-8% unemployment), suggesting the relationship isn't perfectly linear and other factors influence inflation expectations.

Overall, this chart clearly demonstrates the Phillips Curve's concept—showing how inflation expectations generally decrease as unemployment increases, though with noticeable variability.

Time Series Decomposition Analysis of CPI

The provided charts illustrate a decomposed time series analysis of the **Consumer Price Index (CPI)** from 2006 to 2024. The decomposition separates observed data into three components: **Trend**, **Seasonality**, and **Residuals**.

Key Findings:

- **Observed CPI:**
 - Displays a clear upward trend over the observed period, reflecting consistent long-term inflation.
 - **Trend Component:**
 - Highlights a smooth, increasing trajectory of CPI, becoming steeper from approximately 2020 onwards, indicating accelerated inflation in recent years.
 - **Seasonality Component:**
 - Extremely minimal seasonal variation, consistently close to zero, indicating negligible seasonal impacts on CPI values.
 - **Residuals Component:**
 - Exhibits random fluctuations around zero, suggesting short-term volatility or irregular economic events impacting CPI.
 - Notable spikes in residuals around significant economic events (e.g., 2008 and post-2020), indicating increased volatility during these periods.
-

Overall, the decomposition effectively shows a strong and consistent inflationary trend, minimal seasonal effects, and highlights irregular volatility corresponding to economic shocks or disruptions.

CPI with 12-Month Rolling Average Analysis

The provided line chart illustrates the **Consumer Price Index (CPI)** alongside its **12-month rolling average** from 2006 to 2024.

Key Findings:

- **Overall Trend:**
 - CPI shows a clear and steady upward trend, indicating consistent inflation over the entire period.
- **Rolling Average:**
 - The 12-month rolling average (red line) smooths out short-term fluctuations, highlighting a clearer long-term growth pattern.
- **Acceleration in CPI Growth:**
 - A noticeable increase in the slope of the rolling average appears around 2020, indicating an accelerated rise in inflation in recent years.
- **Stability and Fluctuations:**
 - Minor fluctuations around the rolling average indicate temporary economic variations or short-term shocks, especially evident around economic events (e.g., 2008, 2020).

Overall, the 12-month rolling average effectively reveals the long-term upward trend of CPI, clearly indicating sustained and recently accelerated inflation.

Inflation vs. GDP Over Time

The provided dual-axis time series chart compares:

- **10-Year Breakeven Inflation Rate** (blue line, left axis)
- **Real Gross Domestic Product (GDP)** (red line, right axis)

Key Findings:

- **Inflation Rate Trends:**
 - The inflation rate generally fluctuates between 1.5% and 2.5%, with notable volatility during economic downturns, particularly around 2008 and 2020.
 - Significant dips in inflation occur during major economic crises, indicating deflationary pressures.
- **Real GDP Trends:**
 - Real GDP shows a clear, steady upward trend over time, reflecting long-term economic growth.
 - Distinct slowdowns or plateaus in GDP growth occur around 2008-2009 and again in 2020, corresponding with economic disruptions.
- **Relationship Between Inflation and GDP:**
 - Periods of strong GDP growth often align with relatively stable inflation rates around 2%, signifying balanced economic conditions.
 - Sharp GDP contractions, especially visible around 2020, align closely with rapid declines in inflation expectations.
- **Recent Trends:**
 - After the significant downturn in 2020, both GDP and inflation rates recovered swiftly, suggesting a quick economic rebound and rising inflation pressures post-crisis.

Overall, the chart effectively demonstrates how inflation expectations respond dynamically to shifts in economic growth, highlighting periods of economic stability, volatility, and recovery.

Bubble Chart Analysis: Inflation vs GDP vs Unemployment

The provided bubble chart illustrates the relationships among three key economic indicators simultaneously:

- **10-Year Breakeven Inflation Rate** (vertical axis)
- **Real Gross Domestic Product (GDP)** (horizontal axis)
- **Unemployment Rate** (bubble color and size)

Key Findings:

- **Relationship Between Inflation and GDP:**
 - Higher Real GDP values (above approximately \$21,000 billion) generally correspond with moderate inflation rates (around 1.5%–2.5%), indicating economic stability during growth periods.
 - Lower Real GDP values (below approximately \$18,000 billion) show greater variability in inflation rates, ranging from near 0% to around 3%, reflecting increased volatility and uncertainty during economic downturns.
- **Impact of Unemployment:**
 - Lower unemployment rates (4%–6%, represented by smaller, darker bubbles) mostly occur at higher GDP levels with stable and moderate inflation, suggesting favorable economic conditions.
 - Higher unemployment rates (10%–14%, represented by larger, lighter bubbles) appear mainly at lower GDP levels alongside lower inflation rates, highlighting periods of economic distress or recessions.
- **Economic Stability and Inflation:**
 - Inflation rates tend to be more stable during periods of economic expansion (high GDP, low unemployment).
 - Inflation exhibits greater volatility at lower GDP levels, coinciding with higher unemployment and weaker economic conditions.

Overall, the bubble chart clearly visualizes the interconnected dynamics between inflation, economic growth, and unemployment rates.

Heatmap Analysis of Inflation Trends (CPI)

The provided heatmap visually represents monthly **Consumer Price Index (CPI)** values from **2006 to 2024**.

Key Findings:

- **Consistent Increasing Trend:**
 - CPI consistently increases over the years, progressing from lower values (around 200 in 2006) to significantly higher values (above 310 by 2024), clearly showing long-term inflation.
- **Acceleration of Inflation:**
 - The rate of CPI growth visibly accelerates starting from approximately **2020**, demonstrated by a sharper transition from lighter (blue) to darker (red) shades.
- **Seasonal Patterns:**
 - Minor variations are observable within each year, but overall seasonality appears very minimal, with no significant monthly fluctuations or patterns evident.
- **Recent High Inflation:**
 - The most intense inflationary pressures, represented by the darkest red shades, occur predominantly from **2022 onwards**, reflecting recent higher inflation rates.

Overall, the heatmap effectively highlights the steady and recently accelerating inflation trends over the analyzed period.

Lag Plot Analysis of Consumer Price Index (CPI)

The provided lag plot visualizes the relationship between the CPI values at time **t** (**horizontal axis**) and CPI values at the next period **t+1** (**vertical axis**).

Key Findings:

- **Strong Positive Correlation:**
 - The data points clearly form a linear pattern, indicating a strong positive autocorrelation in CPI values from one period to the next.
- **Predictability:**
 - The CPI at any given time (t) strongly predicts the CPI at the following time (t+1), highlighting the stability and continuity of inflation trends.
- **Minimal Randomness:**
 - The tight linear clustering suggests minimal short-term random fluctuations, emphasizing persistent inflation trends over time.
- **Consistent Inflation Growth:**
 - The upward-sloping line reflects consistently rising CPI values, indicating ongoing inflation.

Overall, the lag plot demonstrates that CPI values exhibit strong temporal dependence and consistent upward movement, reflecting stable, long-term inflationary patterns.

Autocorrelation and Partial Autocorrelation Analysis of CPI

The provided charts display the **Autocorrelation Function (ACF)** and **Partial Autocorrelation Function (PACF)** for the Consumer Price Index (CPI).

Key Findings:

- **Autocorrelation Function (ACF):**
 - Exhibits significant positive autocorrelations at multiple lags, slowly declining over time.
 - Indicates strong persistence and a clear trend, suggesting that past CPI values strongly influence future CPI values.
- **Partial Autocorrelation Function (PACF):**
 - Shows significant spikes only at the initial lags (lags 1 and 2), with values quickly dropping to near zero afterward.
 - Suggests that CPI values can be effectively explained by their immediate past observations, with minimal direct influence from distant past observations once short-term dependencies are accounted for.
- **Implications for CPI Modeling:**
 - The persistent high values in the ACF indicate that the CPI series is non-stationary, typically requiring differencing for modeling purposes.
 - PACF indicates an AR(1) or AR(2) process might effectively capture the CPI series behavior once differenced.

Overall, these plots highlight strong autocorrelation and short-term dependencies within CPI, indicating that past CPI values significantly influence future inflation trends.

Radar Chart Analysis: Comparison of Economic Indicators with Inflation

The provided radar chart illustrates the most recent data for five critical economic indicators:

- **Consumer Price Index (CPI)**
- **10-Year Breakeven Inflation Rate**
- **Real Gross Domestic Product (GDP)**
- **Federal Funds Effective Rate**
- **Unemployment Rate**

Key Findings:

- **High CPI and Inflation:**
 - CPI and the 10-Year Breakeven Inflation Rate show high relative values, indicating significant current inflationary pressures.

- **Strong Economic Growth:**
 - Real GDP is positioned relatively high, reflecting robust economic activity at the latest measurement period.
- **Moderate Interest Rates:**
 - The Federal Funds Effective Rate is lower relative to CPI and inflation expectations, suggesting comparatively moderate monetary policy.
- **Low Unemployment:**
 - The unemployment rate shows the lowest relative positioning, indicating a strong labor market environment.
- **Overall Economic Environment:**
 - The chart shape, prominently skewed toward CPI, GDP, and inflation, underscores a scenario of economic growth accompanied by substantial inflation pressures, relatively moderate monetary tightening, and strong employment conditions.

Overall, this radar chart effectively visualizes the current economic landscape, highlighting strong inflation and economic growth coupled with a favorable labor market.

Cross-Correlation Analysis of Inflation and GDP

The provided cross-correlation plot illustrates the correlation strength between **Inflation** and **GDP** at various lag intervals.

Key Findings:

- **Peak Correlation:**
 - The plot shows a clear peak around the mid-point (approximately lag 7000), indicating the strongest correlation occurs when one of the series (GDP or Inflation) significantly leads or lags the other.
- **Symmetrical Pattern:**
 - The correlation rises steadily to the peak, then declines symmetrically, suggesting a consistent relationship pattern over the entire range of lags.
- **Positive Correlation:**
 - The positive peak indicates that inflation and GDP are positively correlated; typically, higher GDP growth aligns with rising inflation and vice versa.
- **Economic Implication:**
 - The observed peak at a large lag implies delayed interactions between inflation and GDP, indicating that changes in one indicator may predict changes in the other after substantial time intervals.

Overall, this cross-correlation analysis clearly indicates a strong and delayed positive correlation between inflation and GDP, highlighting the lagged interplay between these two critical economic variables.

PCA Analysis of Economic Indicators

The provided scatter plot represents the **Principal Component Analysis (PCA)** applied to economic indicators, where:

- **Principal Component 1 (PC1)** is plotted on the x-axis.
- **Principal Component 2 (PC2)** is plotted on the y-axis.

Key Findings:

- **Dimensionality Reduction:**
 - The PCA transformation reduces multiple economic indicators into two principal components, capturing the most significant variance in the data.
- **Distinct Clusters and Patterns:**

- The data points form discernible clusters and structures, suggesting underlying relationships and patterns among economic indicators.
- The presence of curved or nonlinear structures may indicate complex interdependencies between economic variables.
- **Variance Explanation:**
 - Principal Component 1 (PC1) likely captures the dominant trend in the dataset, potentially related to overall economic growth or inflation trends.
 - Principal Component 2 (PC2) may represent secondary factors such as interest rate fluctuations, unemployment trends, or short-term economic shocks.
- **Potential Insights:**
 - If the clusters correspond to specific economic periods or regimes, this visualization could help identify economic cycles, inflationary phases, or recession periods.

Overall, this PCA analysis effectively reduces high-dimensional economic data into two components, allowing for pattern recognition and a better understanding of economic variable relationships.

Dynamic Time Warping (DTW) Analysis: CPI vs. GDP

The provided heatmap visualizes the **Dynamic Time Warping (DTW) distance** between **Consumer Price Index (CPI)** and **Gross Domestic Product (GDP)** time series.

Key Findings:

- **DTW Distance Representation:**
 - The color intensity represents the DTW distance between CPI and GDP across different time steps.
 - Lower DTW distances (blue regions) indicate strong alignment between CPI and GDP at those time points.
 - Higher DTW distances (red regions) suggest deviations or misalignments in trends over time.
- **Pattern of Similarity:**
 - The lower-left region (near the origin) has the lowest DTW distances, indicating strong initial alignment between CPI and GDP.
 - As time progresses, the DTW distance increases, meaning that GDP and CPI trends diverge over longer periods.
- **Economic Interpretation:**
 - The increasing DTW distance suggests that while CPI and GDP may have been closely aligned in earlier time periods, their relationship has diverged over time.
 - Economic shocks or structural changes (such as financial crises, policy interventions, or inflationary shifts) could explain the observed divergence.
- **Application of DTW:**
 - DTW is useful for analyzing non-linear time series relationships where two variables do not necessarily evolve at the same rate but still exhibit underlying similarities.

Overall, this DTW heatmap effectively illustrates how CPI and GDP trends align at different time steps, revealing the evolving relationship between inflation and economic growth.

Hierarchical Clustering of Economic Indicators

The provided **dendrogram** represents hierarchical clustering applied to various **economic indicators**, illustrating how closely related they are based on their statistical similarities.

Key Findings:

- **Cluster Groupings:**
 - The indicators are grouped into distinct clusters based on their relationships.
 - **One major cluster (left side)** includes the **Nominal Broad U.S. Dollar Index, Year, Real GDP, Consumer Price Index (CPI), and Personal Consumption Expenditures**—highlighting their strong economic interdependence.

- **Another distinct cluster (right side)** consists of **Inflation Expectation Rates, Oil Prices, and Interest Rates**, suggesting close connections between monetary policies, inflation, and energy markets.
 - **Inflation and Monetary Policy Relationship:**
 - **10-Year Breakeven Inflation Rate** and **5-Year Forward Inflation Expectation Rate** are closely linked, indicating similar economic forecasting properties.
 - **Federal Funds Effective Rate** and **10-Year Treasury Yield** are grouped together, emphasizing their shared role in interest rate and monetary policy decisions.
 - **Oil Prices and Inflation Expectation:**
 - **Crude Oil Prices (West Texas Intermediate)** show proximity to inflation expectation measures, reinforcing the role of oil prices as a significant driver of inflation.
 - **Unemployment and Economic Growth:**
 - **Unemployment Rate** forms a separate sub-cluster, indicating its distinct nature but still linked to broader economic factors such as GDP and price indices.
 - **Overall Insights:**
 - This clustering reveals natural relationships between economic indicators, confirming well-established economic theories about the interdependence of GDP, inflation, interest rates, and monetary policy.
-

Conclusion:

This dendrogram effectively visualizes the relationships among key economic indicators, showing how closely related variables cluster together, which can help in economic modeling and forecasting.

Network Graph Analysis of Economic Indicators

The provided network graph represents the relationships between key **economic indicators**, where:

- **Nodes** represent different economic variables.
 - **Edges (connections)** indicate relationships or correlations between the variables.
 - **Node size** may reflect the importance or centrality of an indicator in the network.
-

Key Findings:

- **Strongly Connected Core Cluster:**
 - **Nominal Broad U.S. Dollar Index, Consumer Price Index (CPI), Real GDP, and Personal Consumption Expenditures** are highly interconnected.
 - This core cluster represents fundamental macroeconomic indicators that are tightly linked in economic dynamics.
 - **Inflation Expectation and Oil Prices Cluster:**
 - **5-Year, 5-Year Forward Inflation Expectation Rate, 10-Year Breakeven Inflation Rate, and Crude Oil Prices** form a distinct subgroup.
 - This suggests that inflation expectations are closely related to energy prices, reinforcing the role of crude oil in inflationary pressures.
 - **Isolated Monetary Policy Indicators:**
 - **Federal Funds Effective Rate** and **10-Year Treasury Yield** appear relatively disconnected from the core cluster.
 - This indicates that while monetary policy influences the economy, its direct correlation with other variables might be weaker or operate on different time scales.
 - **Economic Implications:**
 - The clustering of economic indicators highlights how different groups of variables interact.
 - The separation of inflation expectations and oil prices from core economic indicators suggests that external shocks (such as energy prices) can influence inflation independently of broader economic trends.
-

Conclusion:

This network graph effectively visualizes how key economic indicators interact, revealing strong

relationships between price indices, GDP, and inflation expectations while showing monetary policy as a more independent force in the economic system.

Structural Break Detection in Inflation Trends

The provided plot shows **Consumer Price Index (CPI) trends over time**, with key **structural breakpoints** detected, represented by vertical red dashed lines.

Key Findings:

- **Identified Breakpoints:**
 - The red dashed lines indicate points where the inflation trend undergoes a significant shift.
 - Structural breaks are observed around **2008, 2012, 2018, 2021, and 2023**, likely corresponding to economic events that influenced inflation.
- **Major Economic Events and CPI Shifts:**
 - **2008 Breakpoint:** Likely corresponds to the Global Financial Crisis, where inflation trends were disrupted.
 - **2012 Breakpoint:** May reflect post-recovery stabilization or policy changes affecting price levels.
 - **2018 Breakpoint:** Could be linked to economic policy shifts or global trade impacts.
 - **2021 Breakpoint:** Significant spike in CPI, possibly due to post-pandemic recovery, supply chain disruptions, or stimulus-induced inflation.
 - **2023 Breakpoint:** Continued high inflation levels, indicating persistent inflationary pressures.
- **Long-Term Inflation Trends:**
 - The CPI trend is generally upward, with periods of accelerated inflation, particularly after **2021**, where inflation rose sharply.
 - Breakpoints suggest changes in the inflationary environment, requiring further economic analysis for causes and implications.

Conclusion:

This structural break detection analysis highlights key periods of inflation regime shifts, providing valuable insights into economic shocks, policy impacts, and inflationary trends.

Time Series Analysis of the 10-Year Breakeven Inflation Rate

The provided plot visualizes the **10-Year Breakeven Inflation Rate** over time, showing fluctuations and key trends from **2006 to 2024**.

Key Findings:

- **General Trend:**
 - The 10-Year Breakeven Inflation Rate exhibits periods of **stability, volatility, and sharp declines** over time.
 - Despite fluctuations, the long-term trend indicates cycles of inflation expectations adjusting to economic events.
- **Major Declines and Economic Events:**
 - **2008-2009:** A significant drop is observed during the **Global Financial Crisis**, where inflation expectations fell below **0.5%** due to economic uncertainty and deflationary pressures.
 - **2020:** Another sharp decline corresponds to the **COVID-19 pandemic**, where inflation expectations briefly collapsed due to economic disruptions.
- **Post-Crisis Recovery:**
 - After both crisis periods (**2009 and 2020**), inflation expectations rebounded, reflecting economic recovery and monetary policy interventions.
 - The **2021-2022 period** shows a strong inflation surge, aligning with post-pandemic stimulus, supply chain disruptions, and rising demand.
- **Recent Trends (2023-2024):**
 - The breakeven inflation rate remains **moderately high**, though it has slightly declined from its

peak in 2022.

- This suggests a stabilization in inflation expectations, possibly due to monetary tightening policies and economic adjustments.

Conclusion:

The time series highlights the impact of major economic shocks on inflation expectations, demonstrating how financial crises, policy changes, and economic recoveries influence long-term inflation sentiment.

Analysis of 10-Year Breakeven Inflation Rate with Rolling and Expanding Means

The provided plot visualizes the **10-Year Breakeven Inflation Rate** over time, incorporating:

- **Original Data (blue line):** The actual inflation rate fluctuations.
- **Rolling Mean (30-day, orange line):** A short-term smoothing of the inflation rate, capturing recent trends.
- **Expanding Mean (green line):** A cumulative average of inflation, showing the long-term trend.

Key Findings:

- **Short-Term Volatility (Blue & Orange Lines):**
 - The **original data (blue)** shows significant fluctuations, reflecting inflation expectations reacting to economic events.
 - The **rolling mean (orange)** closely follows the blue line but smooths out short-term fluctuations, helping to identify intermediate trends.
- **Major Economic Shocks:**
 - **2008-2009:** A sharp decline due to the **Global Financial Crisis**, followed by recovery.
 - **2020:** Another major dip during the **COVID-19 pandemic**, after which inflation rebounded sharply.
- **Long-Term Stability (Green Line):**
 - The **expanding mean (green)** provides a long-term trend of inflation expectations.
 - While the short-term inflation rate fluctuates significantly, the expanding mean remains relatively stable, indicating a long-term anchoring of inflation expectations.
- **Post-2021 Inflation Surge:**
 - A noticeable increase in inflation expectations is seen after **2021**, likely due to post-pandemic recovery, supply chain disruptions, and monetary policy responses.
- **Recent Stabilization (2023-2024):**
 - The breakeven inflation rate has slightly declined from the **2022 peak**, indicating potential stabilization in expectations.

Conclusion:

The combination of original, rolling, and expanding means effectively highlights **short-term fluctuations**, **medium-term trends**, and **long-term stability** in inflation expectations. This analysis provides valuable insights into inflationary cycles, economic crises, and recovery periods.

Hodrick-Prescott (HP) Filter Decomposition of 10-Year Breakeven Inflation Rate

The provided plot visualizes the **Hodrick-Prescott (HP) filter decomposition** applied to the **10-Year Breakeven Inflation Rate**, separating it into three key components:

- **Original Data (blue line):** The raw breakeven inflation rate over time.
- **Trend Component (red dashed line):** The long-term inflation trend extracted using the HP filter.
- **Cyclical Component (green dots):** The short-term fluctuations around the trend.

Key Findings:

- **Trend vs. Short-Term Cycles:**

- The **red trend line** closely follows the **original data**, indicating that inflation expectations generally move in well-defined trends over time.
 - The **green cyclical component** represents short-term deviations from the trend, which are relatively small but reflect temporary economic shocks.
 - **Major Economic Shocks and Trend Deviations:**
 - **2008-2009 (Global Financial Crisis):** A significant downward shift in both trend and cyclical components, reflecting a collapse in inflation expectations.
 - **2020 (COVID-19 Shock):** A similar drop followed by a rapid recovery, indicating high volatility in inflation expectations during crises.
 - **2021-2022 Inflation Surge:** A noticeable upward trend, showing a strong increase in long-term inflation expectations.
 - **Cyclical Variations Over Time:**
 - The cyclical component remains relatively **small and stable**, except during major economic disruptions (**2008-2009, 2020**), where larger deviations occur.
 - This suggests that apart from external shocks, inflation expectations tend to follow a predictable trend with minor fluctuations.
-

Conclusion:

The HP filter effectively decomposes inflation expectations into **long-term trends and short-term cyclical deviations**, highlighting how inflation responds to economic shocks while maintaining an overall trajectory.

Seasonal Decomposition of the 10-Year Breakeven Inflation Rate

The provided plot represents the **seasonal decomposition** of the **10-Year Breakeven Inflation Rate**, breaking it down into four components:

1. **Observed Data (Top Panel):** The original breakeven inflation rate over time.
 2. **Trend Component (Second Panel):** The long-term movement in inflation expectations.
 3. **Seasonal Component (Third Panel):** Recurring seasonal fluctuations.
 4. **Residual Component (Bottom Panel):** The remaining noise after removing trend and seasonality.
-

Key Findings:

- **Trend Component:**
 - The long-term trend **declines sharply in 2008-2009**, reflecting the **Global Financial Crisis**.
 - A similar **drop occurs in 2020**, likely due to the **COVID-19 pandemic**.
 - The **2021-2022 inflation surge** is clearly captured as a rising trend.
 - **Seasonal Component:**
 - Displays **cyclical patterns** that repeat consistently over time.
 - Indicates that inflation expectations **follow seasonal trends**, possibly influenced by policy cycles, economic reporting, or market conditions.
 - **Residual Component:**
 - The **residuals show larger fluctuations** during crisis periods (**2008-2009, 2020**), meaning higher uncertainty in inflation expectations.
 - Outside of these crisis periods, residual noise remains relatively stable.
-

Conclusion:

This decomposition highlights how **inflation expectations** are influenced by **long-term trends, seasonal variations, and economic shocks**. The trend captures economic downturns and recoveries, while the seasonal component suggests underlying periodic influences on inflation expectations.

Analysis of 10-Year Breakeven Inflation Rate with Yearly and Monthly Trends

This visualization presents the **10-Year Breakeven Inflation Rate** over time, incorporating different trend representations:

- **Original Data (Blue Line):** The actual breakeven inflation rate fluctuations.

- **Yearly Mean (Orange Line with Markers):** The annual average inflation rate, providing a **smoother representation** of long-term trends.
 - **Monthly Interpolated Trend (Green Dashed Line):** A **smoothed interpolation** of the inflation trend, filling in gaps for better visualization.
-

Key Findings:

- **Long-Term Trend Observations:**
 - Inflation expectations **dropped significantly in 2008-2009**, reflecting the **Global Financial Crisis**.
 - A **similar decline occurred in 2020** during the **COVID-19 pandemic**, followed by a strong recovery.
 - **2021-2022 saw a sharp rise**, indicating a significant inflation surge, before stabilizing in **2023-2024**.
 - **Yearly Mean vs. Monthly Interpolation:**
 - The **orange yearly mean points** provide a **clearer indication of long-term inflation trends**, smoothing out short-term volatility.
 - The **green dashed line (monthly interpolation)** follows the original data more closely, capturing smaller fluctuations in inflation expectations.
 - **Pattern of Inflation Cycles:**
 - The data suggests **cyclical inflation behavior**, with **periods of rising and falling expectations**.
 - The recent inflation surge (**2021-2022**) represents one of the **strongest increases**, reflecting economic conditions such as **post-pandemic recovery and monetary policies**.
-

Conclusion:

This visualization effectively highlights **short-term fluctuations, long-term inflation trends, and cyclical inflation behavior**, providing valuable insights into inflation expectations over the past two decades.

Time Shifting Analysis of 10-Year Breakeven Inflation Rate

This visualization demonstrates **time shifting** of the **10-Year Breakeven Inflation Rate**, highlighting how lagging and leading data points affect trends.

Components of the Chart:

- **Original Data (Blue Line):** The actual observed breakeven inflation rate.
 - **Shifted Forward (Red Dashed Line - Lag = 1):** Each data point is moved **one step ahead**, simulating a **time lag** effect.
 - **Shifted Backward (Green Dotted Line - Lead = 1):** Each data point is moved **one step back**, simulating a **future-leading** effect.
-

Key Findings:

- **Lagging and Leading Effects:**
 - The **red dashed line (lag=1)** shows the data slightly shifted forward in time, effectively creating a **delayed representation of inflation expectations**.
 - The **green dotted line (lead=1)** represents a **forward-looking perspective**, aligning data with potential future inflation movements.
- **Stability and Predictability:**
 - The **small shifting effect** suggests **strong autocorrelation** in the breakeven inflation rate.
 - Since the shifts appear closely aligned with the original data, **short-term forecasting** using past data may be effective.
- **Economic Events Reflected:**
 - The **2008-2009 Global Financial Crisis** and **2020 COVID-19 downturn** exhibit sharp drops in inflation expectations, with lagging and leading trends following similar patterns.
 - **Post-2020 inflation spikes** are captured consistently across all shifted datasets, confirming **persistent inflationary trends**.

Conclusion:

This time-shifting analysis shows that **past inflation trends are highly correlated with future values**, suggesting **momentum-driven inflation expectations**. Such insights are useful for **forecasting models** and **monetary policy assessments**.

Analysis of Simple Moving Average (SMA) and Exponentially Weighted Moving Average (EWMA) on 10-Year Breakeven Inflation Rate

This visualization compares different smoothing techniques applied to the **10-Year Breakeven Inflation Rate**, which helps in understanding the underlying trends by reducing short-term fluctuations.

Components of the Chart:

- **Original Data (Blue Line):** Represents the raw breakeven inflation rate over time.
 - **Simple Moving Average (SMA - Orange Dashed Line):**
 - Uses a **30-day window** to calculate an average at each point.
 - Gives equal weight to all values within the window.
 - Helps in smoothing fluctuations but may **lag behind** rapid changes.
 - **Exponentially Weighted Moving Average (EWMA - Green Dotted Line):**
 - Applies an **exponential decay factor** where recent values get higher weights.
 - More responsive to changes than SMA.
 - Captures recent inflation trends more accurately while still providing smoothing.
-

Key Findings:

- **Trend Identification:**
 - Both SMA and EWMA effectively smooth out **short-term noise** while preserving overall inflation trends.
 - SMA has **greater lag** in responding to sudden inflation shifts compared to EWMA.
 - EWMA is more **reactive**, capturing **sharp movements** in inflation trends, such as during **2008-2009 financial crisis** and **2020 COVID-19 recession**.
 - **Stability vs Sensitivity:**
 - **SMA** is useful for identifying long-term trends but may not reflect recent market changes quickly.
 - **EWMA** is more useful for **real-time monitoring** and detecting emerging inflation trends faster.
 - **Economic Events Reflected:**
 - The **financial crisis (2008-2009)** and **pandemic-induced inflation spikes (2020-2022)** are well captured by EWMA.
 - SMA smooths these effects more gradually, making it ideal for **long-term economic analysis**.
-

Conclusion:

- **EWMA** is preferable for **short-term forecasting** as it quickly adapts to changes.
- **SMA** is more effective for **long-term trend analysis**, where reducing volatility is the primary goal.
- Combining both techniques can offer a **comprehensive inflation monitoring approach**.

Analysis of Double EWMA and Holt-Winters Method on 10-Year Breakeven Inflation Rate

This visualization compares **Double Exponentially Weighted Moving Average (EWMA)** and **Holt-Winters Exponential Smoothing** techniques to analyze the **10-Year Breakeven Inflation Rate**.

Components of the Chart:

- **Original Data (Blue Line):** The raw breakeven inflation rate over time.
- **Double EWMA (Purple Dashed Line, span=30):**
 - Applies **two levels of exponential smoothing**, making it more adaptable to trends and volatility changes.

- Helps smooth fluctuations while retaining the major trends in inflation.
 - **Holt-Winters (Red Dashed Line):**
 - A **triple exponential smoothing method** that incorporates trend and seasonality adjustments.
 - More advanced than Double EWMA as it considers seasonal variations explicitly.
-

Key Findings:

- **Trend Identification:**
 - Both smoothing methods effectively follow the **major trends** of the breakeven inflation rate.
 - **Double EWMA** is smoother and more stable in capturing gradual inflation movements.
 - **Holt-Winters** captures more fluctuations but remains very close to the original data.
 - **Sensitivity to Market Changes:**
 - **Holt-Winters method** adapts better to sudden inflation spikes, such as the **2008 financial crisis** and **2020 COVID-19 shock**.
 - **Double EWMA** provides a **more stable** and **less reactive** trend estimation.
 - **Best Use Cases:**
 - **Double EWMA** is better for **long-term inflation forecasting** as it smooths out unnecessary noise.
 - **Holt-Winters** is better suited for **short-term forecasting**, especially in environments with clear cyclical or seasonal patterns.
-

Conclusion:

- Both **Double EWMA** and **Holt-Winters** perform well in modeling the **10-Year Breakeven Inflation Rate**.
- **Holt-Winters** is more **responsive** to economic shocks, making it useful for real-time policy decisions.
- **Double EWMA** provides a **stable view** of inflation trends, useful for **long-term economic analysis**.

Analysis of 10-Year Breakeven Inflation Rate with Highlighted Periods

This visualization presents the **10-Year Breakeven Inflation Rate** over time, with two major economic crises highlighted.

Key Elements in the Chart:

- **Blue Line:** Represents the **10-Year Breakeven Inflation Rate**, showing market-based expectations of inflation.
 - **Highlighted Periods:**
 - **2008 Financial Crisis (Shaded in Light Orange):**
 - The breakeven inflation rate experienced a **sharp decline**, reaching near **0%**.
 - This reflects the severe deflationary pressures and market panic during the global financial crisis.
 - **COVID-19 Period (Shaded in Gray):**
 - A **rapid drop** followed by a **strong inflation surge** after economic stimulus measures were implemented.
 - Inflation reached multi-year highs post-COVID as economies reopened and supply chain disruptions persisted.
-

Key Observations:

- **Significant Drops During Economic Crises:**
 - Both the **2008 Financial Crisis** and the **COVID-19 pandemic** led to sharp declines in inflation expectations.
 - These periods were marked by heightened economic uncertainty and intervention by central banks.
- **Post-Crisis Inflation Trends:**
 - After **2008**, inflation expectations recovered gradually but remained **below pre-crisis levels**.
 - After **COVID-19**, inflation expectations **spiked rapidly**, reflecting supply chain issues and monetary expansion.
- **Comparing the Two Crises:**

- The **2008 crisis** caused a **sustained** deflationary shock, with inflation expectations staying low for years.
 - The **COVID-19 crisis** resulted in **short-lived deflation**, followed by a rapid surge in inflation expectations.
-

Conclusion:

- The breakeven inflation rate is highly sensitive to **major economic disruptions**.
- **Central bank policies** and **market confidence** play a crucial role in shaping inflation expectations.
- Monitoring such indicators helps policymakers anticipate **inflation risks** and take corrective actions.

Analysis of Average 10-Year Breakeven Inflation Rate by Month

This bar chart presents the **average 10-Year Breakeven Inflation Rate** for each month, highlighting seasonal variations in inflation expectations.

Key Observations:

- **Relatively Stable Monthly Averages:**
 - The breakeven inflation rate remains relatively **consistent** across all months.
 - The variation in monthly averages appears **small**, suggesting that inflation expectations are not highly seasonal.
 - **Slightly Higher Inflation Expectations in April and May:**
 - **April and May** show the **highest** average breakeven inflation rates.
 - This may indicate a trend where inflation expectations slightly increase in the spring months.
 - **Lower Inflation Expectations in December:**
 - The lowest average **10-Year Breakeven Inflation Rate** is observed in **December**.
 - This might be due to year-end economic cycles, holiday spending patterns, or monetary policy adjustments.
-

Conclusion:

- There is **minimal seasonality** in the 10-Year Breakeven Inflation Rate.
- A **slight peak in spring (April-May)** and a **small dip in December** suggest some seasonal economic influences.
- These insights can help policymakers and investors understand **potential cyclical inflation trends**.

Trend and Variance Analysis of 10-Year Breakeven Inflation Rate

This graph visualizes the **10-Year Breakeven Inflation Rate** with a rolling mean and rolling standard deviation over a **365-day window** to highlight long-term trends and fluctuations.

Key Observations:

- **Long-Term Trends:**
 - The **rolling mean (green line)** captures the **smoothed trend** of inflation expectations over time.
 - Significant downward trends are visible during the **2008 Financial Crisis** and the **COVID-19 period (2020)**.
 - A sharp increase is observed post-2020, with inflation peaking around **2022**, followed by a stabilization.
- **Fluctuations and Variability:**
 - The **rolling standard deviation (gray shaded area)** shows periods of **higher volatility**.
 - Increased volatility is evident during:
 - **2008-2009 (Financial Crisis)**
 - **2010-2012 (Recovery Phase)**
 - **2020-2022 (COVID-19 and Economic Uncertainty)**

- Lower volatility is seen in periods of economic stability.
 - **Recent Trends (2023-2024):**
 - The inflation rate appears to be **stabilizing**, with a **narrowing variance**, suggesting more predictable inflation expectations.
-

Conclusion:

- The **rolling mean** effectively highlights long-term inflation trends.
- The **rolling standard deviation** reveals periods of **high economic uncertainty**.
- Current trends suggest a **return to stability** after significant fluctuations in recent years.

Peaks and Troughs in 10-Year Breakeven Inflation Rate

This chart highlights the **peaks (red dots)** and **troughs (green dots)** in the **10-Year Breakeven Inflation Rate** over time, identifying significant turning points in the trend.

Key Observations:

- **Cyclical Nature:**
 - The **inflation rate exhibits a cyclical pattern** with recurring peaks and troughs.
 - Peaks (red dots) represent **local maxima**, where inflation expectations **temporarily rise**.
 - Troughs (green dots) indicate **local minima**, marking **periods of lower inflation expectations**.
 - **Major Downturns:**
 - Significant drops are visible around **2008-2009 (Financial Crisis)** and **2020 (COVID-19 Pandemic)**, where troughs reach extreme lows.
 - The sharp decline in **2008-2009** reflects the economic turmoil, with inflation expectations hitting nearly **0%**.
 - Another major drop occurred in **2020**, mirroring economic uncertainties during the pandemic.
 - **Post-Recovery Surges:**
 - Following downturns, strong **upward trends** can be seen, marked by **multiple peaks**:
 - **2010-2012**: Recovery phase after the financial crisis.
 - **2021-2022**: A sharp rise in inflation expectations post-COVID.
 - **Recent Stabilization (2023-2024):**
 - While peaks and troughs continue to form, the **volatility appears lower** compared to previous periods.
 - This suggests that **inflation expectations have become more stable**.
-

Conclusion:

- The breakeven inflation rate follows a distinct cyclical trend, influenced by economic shocks.
- Major crises (2008, 2020) cause sharp declines, followed by strong rebounds.
- Recent trends indicate more stability, but periodic fluctuations persist.

Density Plot of 10-Year Breakeven Inflation Rate

This density plot provides a **smooth estimate** of the probability distribution of the **10-Year Breakeven Inflation Rate**, showing where values are most concentrated.

Key Observations:

- **Bimodal Distribution:**
 - The plot reveals **two distinct peaks**, suggesting that the **inflation rate has clustered around two different levels over time**.
 - One peak occurs around **1.8%**, while the other, more dominant peak is **slightly above 2.0%**.
- **Higher Concentration Near 2.2%:**
 - The most frequent inflation rate appears to be **between 2.0% and 2.5%**, with a sharp peak, indicating that inflation expectations tend to hover around this range.
 - This aligns with the **Federal Reserve's target inflation rate of approximately 2%**.

- **Left-Skewed Tail:**
 - A **smaller bump around 0-1%** suggests rare instances of extremely low inflation expectations.
 - This likely corresponds to **economic crises**, such as the **2008 Financial Crisis** and **COVID-19**, where inflation expectations dropped significantly.
 - **Right-Skewed Tail:**
 - The distribution extends **above 3.0%**, but with very low density, indicating that **higher inflation expectations were rare** in the dataset.
-

Conclusion:

- Inflation expectations have generally stayed between 1.5% and 2.5%, with a strong central tendency near 2.2%.
- The presence of two peaks suggests periods of lower and higher inflation expectations, potentially driven by economic cycles.
- Crisis periods caused temporary deviations toward extremely low inflation expectations, but these were short-lived.

Density Plot of 10-Year Breakeven Inflation Rate for Specific Periods

This density plot compares the distribution of the **10-Year Breakeven Inflation Rate** before and after **2020**, illustrating how inflation expectations have shifted over time.

Key Observations:

- **Shift in Distribution:**
 - The **blue region (Pre-2020)** shows a **bimodal distribution** with two peaks around **1.8% and 2.2%**.
 - The **red region (Post-2020)** has **one dominant peak around 2.5%**, indicating higher inflation expectations after 2020.
 - **Higher Inflation Expectations Post-2020:**
 - The **Post-2020 distribution is skewed towards higher values**, meaning inflation expectations were generally higher after 2020 compared to before.
 - This suggests that factors such as **pandemic-driven economic policies, supply chain disruptions, and fiscal stimulus** may have contributed to higher inflation expectations.
 - **Reduced Occurrence of Low Inflation:**
 - Before 2020, there was a small **left tail** in the blue distribution (values near **0-1%**), likely corresponding to crisis periods like **2008 and 2020**.
 - After 2020, the red distribution shows **fewer instances of extremely low inflation expectations**, indicating a shift toward sustained higher inflation.
 - **Increased Volatility Post-2020:**
 - The red distribution is **wider and more spread out**, showing greater variation in inflation expectations compared to pre-2020.
 - This aligns with the **high uncertainty following the COVID-19 pandemic and economic recovery**.
-

Conclusion:

- Inflation expectations have increased post-2020, with the most common values shifting from 2.0% to around 2.5%.
- Instances of very low inflation (<1.5%) have become less frequent, indicating a higher baseline inflation expectation.
- The broader spread in the post-2020 distribution suggests increased economic uncertainty and volatility.

Heatmap of Monthly Averages for 10-Year Breakeven Inflation Rate

This heatmap visualizes the monthly average of the **10-Year Breakeven Inflation Rate** over different years, highlighting patterns and trends across time.

Key Observations:

- **Significant Drop in 2009 (Financial Crisis Aftermath):**
 - The **blue region in 2009** indicates a **sharp decline** in the breakeven inflation rate, with values dropping as low as **0.24 in December**.
 - This aligns with the **2008 financial crisis aftermath**, where economic uncertainty and deflation concerns led to historically low inflation expectations.
 - **Gradual Recovery (2010 - 2013):**
 - The breakeven inflation rate **recovers after 2009**, with **warmer shades (higher values)** indicating a return to more stable inflation expectations.
 - **2013 shows consistently high values (~2.5%)**, indicating a period of strong economic recovery.
 - **Another Decline in 2015 - 2016:**
 - A **light blue region in 2015 - 2016** suggests a **decline in inflation expectations**, possibly linked to **low oil prices and global economic concerns**.
 - **Impact of COVID-19 in 2020:**
 - A noticeable **drop in March-July 2020** (values near **1.00 - 1.45**) is observed, reflecting **pandemic-induced economic uncertainty**.
 - However, by late 2020 and into **2021, inflation expectations surged**, as shown by the **deep red shades (above 2.5%)**, likely due to **monetary stimulus, supply chain disruptions, and economic reopening**.
 - **Record High Inflation Expectations in 2022:**
 - **March - May 2022 shows the highest inflation expectations (above 2.85%)**, consistent with the **post-pandemic inflation surge and supply chain constraints**.
 - This period saw aggressive **interest rate hikes by the Federal Reserve** to combat inflation.
 - **Recent Trends (2023 - 2024):**
 - Inflation expectations remain **elevated but lower than 2022**, with values fluctuating around **2.2 - 2.4%**.
 - This suggests a **gradual stabilization** as inflationary pressures ease but remain above pre-pandemic levels.
-

Conclusion:

- The **2009 financial crisis and 2020 COVID-19 pandemic** caused **sharp declines** in inflation expectations.
- Inflation expectations **peaked in 2022**, aligning with post-pandemic economic conditions.
- **2023-2024 shows a stabilization trend**, though values remain higher than pre-pandemic levels.

MACHINE LEARNING MODELS

Random Forest Model Code

Description of the Code Blocks

Block 1: Random Forest Regressor Initialization and Hyperparameter Tuning

This block initializes a **Random Forest Regressor** and applies **Randomized Search Cross-Validation** to optimize hyperparameters. Key steps include:

- Defining a `RandomForestRegressor` with a fixed `random_state=42` for reproducibility.
- Specifying a hyperparameter grid with:
 - `n_estimators`: Number of trees in the forest.
 - `max_depth`: Maximum depth of trees (including unlimited depth).
 - `min_samples_split`: Minimum samples required to split an internal node.
 - `min_samples_leaf`: Minimum samples required to form a leaf node.
 - `max_features`: Feature selection strategy for best split.
- Using `RandomizedSearchCV` to randomly sample 10 combinations of hyperparameters with:
 - **5-fold cross-validation** to assess model performance.
 - **Parallel processing (`n_jobs=-1`)** for computational efficiency.
 - **Verbose output (`verbose=2`)** for detailed iteration logs.

The best hyperparameter set is stored and used to fit the **optimal Random Forest model**.

Block 2: Model Training and Performance Evaluation

Once the best hyperparameter combination is found:

- The optimized **Random Forest model** is trained on `X_train` and `y_train`.
- Predictions are made on `X_test` using the **best estimator**.
- The model's performance is evaluated using a predefined function `calculate_metrics()`, which computes:
 - **MSE, RMSE, MAE** (error metrics).
 - **MAPE and sMAPE** (percentage-based accuracy).
 - **R² score** (goodness-of-fit).
 - **MASE** (comparison against a naive model).

The evaluation metrics are stored for later analysis.

Block 3: Feature Importance Analysis

This section extracts feature importance scores from the trained **Random Forest model** to understand how different features contribute to predictions.

- **Feature Importance Extraction:**
 - `feature_importances_` from the trained model assigns an importance score to each feature.
 - These scores are stored in a Pandas `DataFrame` and sorted in descending order.
 - **Feature Importance Visualization:**
 - A **horizontal bar chart** is created using `matplotlib`:
 - Features with the highest importance appear at the top.
 - **X-axis:** Feature importance scores.
 - **Y-axis:** Feature names.
 - The y-axis is inverted to improve readability.
 - A clear title, axis labels, and a grid are added for better interpretation.
-

Block 4: Actual vs. Predicted Values (Scatter Plot)

To assess model predictions, a **scatter plot** compares actual (`y_test`) vs. predicted (`rf_pred`) values.

- **Key Components:**
 - **Blue dots** represent model predictions.
 - A **red dashed line** (**Perfect Fit Line**) represents ideal predictions (where `y_test == rf_pred`).
 - **Grid, axis labels, and a legend** enhance readability.

This visualization helps identify patterns in **overprediction or underprediction**.

Block 5: Residual Analysis

Residuals (`y_test - rf_pred`) help diagnose model errors.

- **Residual Scatter Plot:**
 - **X-axis:** Actual values.
 - **Y-axis:** Residuals.
 - A **red dashed horizontal line at y=0** represents perfect predictions.
 - Helps detect biases (e.g., clustering of residuals).
- **Residuals Histogram:**
 - Analyzes how residuals are distributed.
 - Ideally, residuals should follow a **normal distribution** centered at zero.
 - A **red dashed vertical line at x=0** is added to indicate zero error.

These plots help evaluate whether the model exhibits **heteroscedasticity (non-constant variance)**.

Block 6: Hyperparameter Tuning Results (Heatmap)

A **heatmap** visualizes how different hyperparameter combinations impact model performance.

- **Pivoting Cross-Validation Results:**
 - `mean_test_score` is extracted from `random_search.cv_results_`.
 - The table structure:
 - **Rows:** Different `max_depth` values.
 - **Columns:** Different `n_estimators` values.
 - **Color intensity** represents model performance (darker shades = higher accuracy).

This heatmap helps select **optimal hyperparameters** for future tuning.

Block 7: SHAP Feature Importance Analysis

SHAP (SHapley Additive exPlanations) provides a detailed analysis of feature contributions.

- **Global Explanation:**
 - `shap.TreeExplainer(best_rf_model)` generates SHAP values for `X_train`.
 - A **summary plot** visualizes:
 - **Feature importance** (sorted by impact).
 - **Direction of influence** (red = high value, blue = low value).
 - The spread of SHAP values across different samples.
- **Local Explanation (Force Plot):**
 - SHAP **force plots** visualize how each feature **influences a single prediction**.
 - Helps understand why the model made a certain prediction for a given sample.

These insights **enhance model interpretability**.

Block 8: LIME (Local Interpretable Model-Agnostic Explanations)

LIME explains **individual predictions** by perturbing input features.

- **LIME Initialization:**
 - `LimeTabularExplainer` is set up with training data and feature names.
 - A wrapper function `predict_fn(x)` ensures compatibility with `RandomForestRegressor`.
- **Local Explanation for a Test Sample:**
 - `explain_instance()` generates a LIME explanation for a **single prediction**.
 - The explanation is displayed in the **notebook**.

LIME helps explain **black-box models** like Random Forest in an **interpretable way**.

Block 9: Partial Dependence Plots (PDPs)

PDPs show how **each feature affects model predictions**, holding other features constant.

- **Visualization Strategy:**
 - `PartialDependenceDisplay.from_estimator()` computes feature relationships.
 - **Plots show the impact of different feature values** on predicted outcomes.
 - Helps identify **non-linear relationships**.

This technique enhances **model transparency and fairness**.

Conclusion

This workflow provides a **comprehensive framework** for training, optimizing, and interpreting a **Random Forest model**. It ensures:

1. **Robust hyperparameter tuning** via `RandomizedSearchCV`.
2. **Accurate performance evaluation** with multiple error metrics.

- 3. **Feature importance analysis** for model interpretability.
- 4. **Visual diagnostics** to detect bias, overfitting, or underfitting.
- 5. **Explainability techniques** using SHAP, LIME, and PDPs.

This structured approach ensures reproducibility, robustness, and interpretability.

Result Output:

Fitting 5 folds for each of 10 candidates, totalling 50 fits

Best Hyperparameters:

```
'n_estimators': 50,
'min_samples_split': 2,
'min_samples_leaf': 2,
'max_features': None,
'max_depth': 10
```

Random Forest evaluation metrics:

```
'Model': 'Random Forest (RF)',
'MSE': 0.003100004779513038,
'RMSE': 0.05567768654957781,
'MAE': 0.030016062863946584,
'MAPE (%)': 1.1941804402760234,
'SMAPE (%)': 1.2108601868790747,
'MASE': 1.6663418516116333,
'R^2': 0.9025413132556024.
```

Feature Importance from Random Forest:

Feature Index	Feature	Importance
9	lag_1	0.834510
14	rolling_mean	0.162120
10	lag_2	0.000878
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.000652
6	Nominal Broad U.S. Dollar Index	0.000305
8	10-Year Treasury Yield	0.000294
12	lag_4	0.000262
7	Crude Oil Prices: West Texas Intermediate	0.000257
13	lag_5	0.000215
11	lag_3	0.000202
4	Federal Funds Effective Rate	0.000103
3	Unemployment Rate	0.000065
1	Consumer Price Index	0.000058
5	Personal Consumption Expenditures: Chain-type ...	0.000048
2	Real Gross Domestic Product	0.000032

Random Forest Feature Importance Analysis

Key Findings:

- **Feature Importance Distribution:** The Random Forest model assigned significantly high importance

- to `lag_1`, which is the most influential feature in predicting the target variable.
- **Secondary Importance:** `rolling_mean` is the second most important feature but has much lower importance than `lag_1`.
 - **Minimal Contribution:** Other features, including `lag_2`, `5-Year, 5-Year Forward Inflation Expectation Rate`, `Nominal Broad U.S. Dollar Index`, and `10-Year Treasury Yield`, have negligible importance.
 - **Macroeconomic Indicators:** Features such as `Federal Funds Effective Rate`, `Unemployment Rate`, and `Consumer Price Index` contribute little to the model's prediction.
 - **Lag Features Dominance:** The presence of `lag_1`, `lag_2`, `lag_3`, `lag_4`, and `lag_5` suggests that past values of the target variable play a crucial role in forecasting.
 - **Rolling Mean Influence:** The `rolling_mean` feature's moderate importance indicates that smoothing past values helps improve prediction accuracy.

Interpretation:

- The model relies heavily on short-term lag features, suggesting that recent past values of the dependent variable strongly influence predictions.
- Macroeconomic indicators such as inflation rates, interest rates, and GDP seem to have little impact on the model's forecast.
- This feature importance analysis provides insight into which factors drive predictions, helping refine model selection and feature engineering for future improvements.

Random Forest: Actual vs. Predicted Values

Key Observations:

- **Strong Correlation:** The scatter plot shows a strong positive correlation between actual and predicted values, indicating that the Random Forest model performs well.
- **Alignment with Perfect Fit:** The red dashed line represents a perfect 1:1 fit. Most blue dots (predictions) are closely aligned with this line, suggesting high accuracy.
- **Slight Deviations:** Some points deviate from the red line, showing minor prediction errors. The dispersion increases slightly at higher values.
- **Flat Predictions:** A cluster of predicted values appears flat near the upper range, indicating potential model saturation or a limitation in capturing higher values accurately.
- **Residual Variability:** Although most points follow the trend, there are a few outliers that do not align perfectly, suggesting areas where the model could be improved.

Interpretation:

- The model effectively captures trends but may struggle with extreme values.
- Further tuning, such as feature engineering or hyperparameter optimization, might improve accuracy.
- Overall, the model's predictions are well-calibrated, demonstrating its reliability in forecasting.

Random Forest: Residual Plot Analysis

Key Observations:

- **Centered Residuals:** Most residuals (differences between actual and predicted values) are distributed around zero, indicating that the model has relatively low bias.
- **Heteroscedasticity:** Residuals appear to increase in magnitude as actual values increase, particularly beyond 2.6. This suggests that the model struggles with higher actual values.
- **Systematic Error in High Values:** A clear upward trend in residuals at the higher end suggests that the model consistently underestimates high actual values.
- **Random Dispersion for Mid-Range Values:** For actual values between approximately 2.0 and 2.6, the residuals are fairly randomly scattered, indicating that the model performs well in this range.
- **Presence of Outliers:** Some residuals deviate significantly from zero, which could indicate data points where the model is not performing optimally.

Interpretation:

- The model performs well in the mid-range but tends to underestimate higher values.
- The increasing spread of residuals suggests that the model's variance increases for larger values, which could be addressed with techniques such as transforming the target variable, adding

more relevant features, or adjusting hyperparameters.

- Overall, while the model is fairly accurate, its predictions for higher values exhibit a consistent bias that could impact forecasting accuracy.

Random Forest: Distribution of Residuals

Key Observations:

- **Centered Around Zero:** The majority of residuals are clustered around zero, indicating that the model does not exhibit significant bias in predictions.
- **Near-Normal Distribution:** The histogram resembles a normal distribution, suggesting that prediction errors are randomly distributed rather than systematically skewed.
- **Sharp Peak at Zero:** The red dashed line at zero aligns with the highest frequency of residuals, showing that many predictions are very close to the actual values.
- **Slight Right-Skew:** A small number of residuals extend towards positive values (right tail), indicating that the model tends to underestimate some data points.
- **Few Extreme Residuals:** While most residuals fall within a narrow range, a small number of outliers exist beyond 0.2 and -0.1, which could indicate specific cases where the model struggles.

Interpretation:

- The model performs well overall, with residuals symmetrically distributed around zero.
- The slight right skew suggests that the model occasionally underestimates high values.
- The presence of a few extreme residuals indicates potential areas for model improvement, such as feature engineering or hyperparameter tuning.
- The near-normal shape supports the assumption that errors are randomly distributed, which is ideal for a well-calibrated model.

Random Forest Hyperparameter Tuning Heatmap

Key Observations:

- **Hyperparameter Grid:** The heatmap visualizes model performance across different values of `n_estimators` (number of trees) and `max_depth` (tree depth).
- **Best Performance:** The highest performance (0.986) occurs when `n_estimators=200` and `max_depth=10`, indicating this combination is optimal.
- **Lower Performance for Shallower Trees:** When `max_depth=5`, performance is the lowest (0.965), suggesting that deeper trees improve model accuracy.
- **Effect of `n_estimators`:** Increasing the number of trees (`n_estimators`) generally improves performance, as seen when moving from 50 to 200.
- **Slight Drop at `max_depth=15`:** The performance at `max_depth=15` and `n_estimators=100` is slightly lower (0.970) than other configurations, possibly due to overfitting.

Interpretation:

- The best combination for this model is `max_depth=10` and `n_estimators=200`, as it achieves the highest accuracy.
- Increasing `max_depth` beyond 10 does not necessarily improve performance and may lead to overfitting.
- Using too few trees (`n_estimators=50`) results in suboptimal performance, indicating that more trees contribute to better generalization.
- This heatmap helps in selecting hyperparameters that balance model complexity and accuracy, improving predictive performance.

SHAP Summary Plot: Feature Impact on Model Predictions

Key Observations:

- **Dominance of `lag_1`:** The feature `lag_1` has the most significant impact on the model's predictions, as shown by its wide range of SHAP values.
- **Moderate Influence of `rolling_mean`:** The second most influential feature is `rolling_mean`, which also has a notable spread of SHAP values, suggesting its importance in the model.
- **Limited Impact of Macroeconomic Variables:** Features such as `10-Year Treasury Yield`, `Federal`

Funds Effective Rate, Consumer Price Index, and Real Gross Domestic Product show minimal impact, as their SHAP values remain close to zero.

- **Effect of Lag Features:** Multiple lagged variables (lag_2, lag_3, lag_4, lag_5) have some influence, but their impact is considerably lower than lag_1.
- **SHAP Value Interpretation:** Higher feature values (in pink) tend to push predictions in one direction, while lower values (in blue) push them in the opposite direction.
- **Macroeconomic Indicators Have Low Predictive Power:** Economic factors such as inflation expectations, the dollar index, and crude oil prices have a near-zero effect on predictions, indicating that the model relies more on lagged values.

Interpretation:

- The model is primarily driven by past values (lag_1) and rolling mean trends.
- Macroeconomic indicators do not significantly influence the model's predictions.
- The presence of multiple lagged features suggests that recent historical data is more predictive than external economic variables.
- The use of SHAP values provides transparency in understanding the contribution of each feature to the model's output, helping refine feature selection for future iterations.

SHAP Waterfall Plot: Feature Contributions to Prediction

Key Observations:

- **Base Value (1.981):** This represents the model's expected output before considering specific feature influences.
- **Final Prediction (2.33):** The model's predicted value after incorporating feature effects.
- **Main Contributors:**
 - **lag_1 (0.6689 impact):** The most influential feature in increasing the prediction.
 - **rolling_mean (0.6801 impact):** Also plays a significant role in raising the predicted value.
- **Color Coding:**
 - Red indicates features that push the prediction higher.
 - Blue (not present in this case) would indicate features lowering the prediction.
- **No Negative Influences:** All contributing features in this instance positively impact the prediction.

Interpretation:

- The model relies primarily on lag_1 and rolling_mean to make its predictions.
- The cumulative effect of these features moves the prediction from the base value (1.981) to 2.33.
- No opposing features (blue) suggest that all influencing factors in this instance contribute positively to the prediction.
- This visualization provides transparency into how individual feature values affect the model's output, helping in model interpretability and refinement.

SHAP Feature Contribution to Prediction

Key Observations:

- **Predicted Value (1.98):** The model predicts a value of 1.98, falling between the minimum (0.61) and maximum (2.64) possible predictions.
- **Feature Influence Breakdown:**
 - **Negative Contributions (Blue):** Features reducing the predicted value include:
 - lag_1 (-0.16)
 - 5-Year, 5-Year Forward Inflation Expectation Rate (-0.58)
 - Crude Oil Prices: West Texas Intermediate (-1.11)
 - lag_3 (-0.24)
 - **Positive Contributions (Orange):** Features increasing the predicted value include:
 - Nominal Broad U.S. Dollar Index (0.54)
 - Real Gross Domestic Product (0.96)
 - 10-Year Treasury Yield (-1.72)
 - Personal Consumption Expenditures: Chain-type Price Index (0.63)
 - Consumer Price Index (0.58)
 - rolling_mean (-0.19)
- **Balancing of Effects:** The final predicted value results from a combination of both negative and

positive influences from different features.

Interpretation:

- The prediction is driven by a mix of economic and lag features.
- `Crude Oil Prices: West Texas Intermediate` and `lag_3` significantly pull the prediction down.
- `Real Gross Domestic Product` and `Nominal Broad U.S. Dollar Index` provide the strongest positive push to the prediction.
- The presence of both positive and negative contributions suggests the model carefully weighs various macroeconomic and historical lag features to make its prediction.
- Understanding the influence of these features can help refine the model and improve interpretability.

Partial Dependence Plots (RF)

Key Observations:

- **Flat Relationships for Most Macroeconomic Features:**
 - Features such as `5-Year, 5-Year Forward Inflation Expectation Rate`, `Consumer Price Index`, `Real Gross Domestic Product`, `Unemployment Rate`, `Federal Funds Effective Rate`, `Personal Consumption Expenditures: Chain-type Price Index`, and `10-Year Treasury Yield` show flat partial dependence plots.
 - This indicates that changes in these variables do not significantly impact the model's predictions.
- **Impact of `lag_1`:**
 - The `lag_1` feature shows a clear upward trend, meaning that as its value increases, the model's predicted value also rises.
 - This confirms the strong influence of recent past values on the predictions.
- **Influence of `Nominal Broad U.S. Dollar Index`:**
 - A positive trend is observed, suggesting that higher values of this feature contribute to increased predictions.
- **Effect of `rolling_mean`:**
 - The `rolling_mean` feature also exhibits a slight positive trend, implying that higher moving averages tend to result in higher predictions.
- **Minimal Influence of Lagged Features (`lag_2`, `lag_3`, `lag_4`, `lag_5`):**
 - These lag features show almost no significant trend, except for `lag_1`, which reinforces that only the most recent lag carries substantial predictive power.

Interpretation:

- The model relies heavily on short-term historical values (`lag_1` and `rolling_mean`) rather than macroeconomic indicators.
- Macroeconomic variables do not significantly affect the predictions, as indicated by their flat partial dependence plots.
- The dependence of the model on `lag_1` suggests that short-term trends play a crucial role in forecasting.
- The insights from these plots can help refine feature selection, potentially removing redundant macroeconomic variables that do not contribute to prediction accuracy.

Extreme Gradient Boosting (XGBoost) Model Code

Description of the Code Blocks

Block 1: Importing Required Libraries and Setting Random Seeds

This block imports the necessary libraries for XGBoost model training and hyperparameter tuning, ensuring reproducibility by:

- **Setting random seeds** for Python's `random` module and NumPy.
 - **Importing XGBoost** for gradient boosting-based machine learning.
 - **Using `train_test_split`** to divide data into training and testing sets.
-

Block 2: Data Preparation for XGBoost

- Splits the dataset into training (`X_train` , `y_train`) and testing (`X_test` , `y_test`) sets (80%-20% split).
 - Converts datasets into XGBoost's optimized `DMatrix` format, which improves memory efficiency and training speed.
-

Block 3: Defining the Hyperparameter Grid

The following **hyperparameters** are searched using a **manual grid search**:

- `max_depth` : Controls tree complexity.
- `learning_rate` : Determines step size for weight updates.
- `subsample` : Percentage of data used in each boosting iteration.
- `colsample_bytree` : Fraction of features considered per tree.

A **base parameter dictionary** is also defined, specifying:

- `objective` : Squared error regression.
 - `eval_metric` : Root Mean Squared Error (RMSE).
 - `seed` : Ensures reproducibility.
-

Block 4: Grid Search with Cross-Validation

A **manual grid search** is performed using `xgb.cv()` :

- Iterates over all possible combinations of hyperparameters.
- Runs 5-fold cross-validation to evaluate each combination.
- Tracks the lowest RMSE and selects the best hyperparameter configuration.

Key Features:

- `num_boost_round=200` : Limits boosting rounds to prevent overfitting.
 - `early_stopping_rounds=10` : Stops training if no RMSE improvement occurs for 10 rounds.
-

Block 5: Training the XGBoost Model with Optimal Hyperparameters

- Uses `xgb.train()` to train a final model with the best hyperparameters.
 - Evaluates the model on the test set and applies **early stopping**.
 - Stores the trained model (`best_xgb_model`) for further predictions.
-

Block 6: Model Predictions and Performance Evaluation

- Uses `best_xgb_model.predict()` to generate test set predictions.
 - Evaluates performance using a predefined function (`calculate_metrics`).
 - Appends results to `metrics_results` for further analysis.
-

Block 7: Feature Importance Extraction

Extracts **feature importance** based on:

- Split frequency (`weight`): The number of times a feature is used in decision trees.
- Alternative importance measures (`gain` , `cover`) are available.

A **Pandas DataFrame** is created to store and sort feature importance.

Block 8: Feature Importance Visualization

A **horizontal bar chart** displays **feature importance**:

- **X-axis:** Importance score.
 - **Y-axis:** Feature names.
 - **Bars sorted in descending order** (most important features at the top).
-

Block 9: Top 10 Most Important Features

- Uses XGBoost's built-in feature importance plot.
 - Displays only the top 10 features.
 - Importance is based on frequency of use in splits (`weight`).
-

Block 10: Actual vs. Predicted Scatter Plot

This plot assesses model accuracy:

- **X-axis:** Actual test set values (`y_test`).
 - **Y-axis:** Predicted values (`xgb_pred`).
 - A red dashed `Perfect Fit` line represents ideal predictions.
-

Block 11: Residual Analysis

Residuals (actual values - predicted values) indicate:

- Systematic bias if patterns appear.
- Heteroscedasticity (changing variance) in predictions.

The residual scatter plot highlights prediction errors.

Block 12: Residual Distribution Analysis

- A histogram visualizes residual distribution.
 - A red dashed vertical line at zero indicates ideal performance.
-

Block 13: Grid Search Results Compilation

Extracts hyperparameter search results:

- Stores each configuration's RMSE in a DataFrame.
 - Sorts configurations in ascending order of RMSE.
-

Block 14: Hyperparameter Tuning Heatmap

A heatmap visualizes RMSE scores for:

- `max_depth` (rows)
- `learning_rate` (columns)

Helps identify the most optimal settings at a glance.

Block 15: SHAP Feature Importance Analysis

SHAP (SHapley Additive Explanations) is used for model interpretability:

- Creates a `TreeExplainer` for XGBoost.
 - Computes SHAP values for `X_test`.
 - A summary plot visualizes global feature importance.
-

Block 16: SHAP Dependence Plot

- Shows how an individual feature impacts predictions.

- **X-axis:** Feature values.
- **Y-axis:** Corresponding SHAP values.

Block 17: Local Explanation with SHAP Force Plot

- Analyzes feature contributions for a single test instance.
- Highlights how each feature positively or negatively impacts predictions.

Block 18: LIME Explanation for Individual Predictions

LIME (Local Interpretable Model-Agnostic Explanations) is used to analyze single-instance predictions.

- Creates synthetic perturbations of an instance.
- Determines which features drive the prediction.
- Provides a visual summary in the notebook.

Conclusion

1. Optimized XGBoost model using manual grid search and cross-validation.
2. Comprehensive evaluation using residual analysis and SHAP/LIME interpretability.
3. Feature importance extracted and visualized with multiple techniques.

This structured workflow enhances reproducibility and ensures model transparency.

Result Output:

Best Hyperparameters:

```
'objective': 'reg:squarederror',
'eval_metric': 'rmse',
'seed': 42,
'max_depth': 9,
'learning_rate': 0.1,
'subsample': 0.9,
'colsample_bytree': 0.9
```

Best RMSE: 0.025516134062302957

XGBoost evaluation metrics:

```
'Model': 'Extreme Gradient Boosting (XGBoost)',
'MSE': 0.0005988649854647083,
'RMSE': 0.024471718073415038,
'MAE': 0.014618143215612217,
'MAPE (%)': 0.9442567542096136,
'sMAPE (%)': 0.9379267866283615,
'MASE': 0.03721130171471389,
'R^2': 0.9966132131942298
```

Feature Importances:

Feature Index	Feature	Importance
0	5-Year, 5-Year Forward Inflation Expectation Rate	5996.0

9	lag_1	3979.0
7	Crude Oil Prices: West Texas Intermediate	3236.0
6	Nominal Broad U.S. Dollar Index	3103.0
8	10-Year Treasury Yield	3020.0
10	lag_2	2686.0
14	rolling_mean	2505.0
11	lag_3	2257.0
12	lag_4	2160.0
13	lag_5	1913.0
1	Consumer Price Index	1901.0
3	Unemployment Rate	673.0
4	Federal Funds Effective Rate	613.0
2	Real Gross Domestic Product	385.0
5	Personal Consumption Expenditures: Chain-type ...	257.0

Feature Importance (XGBoost)

Key Observations:

- **Top Feature:** 5-Year, 5-Year Forward Inflation Expectation Rate
 - This is the most important feature in the XGBoost model, contributing significantly more than any other variable.
- **Significant Influence of** lag_1
 - The second most important feature, indicating that recent past values of the target variable are crucial for prediction.
- **Macroeconomic Indicators in the Top Five**
 - Crude Oil Prices: West Texas Intermediate, Nominal Broad U.S. Dollar Index, and 10-Year Treasury Yield rank high in importance.
 - This suggests that XGBoost captures relationships with economic indicators better than other models (e.g., Random Forest).
- **Rolling Mean and Lag Features Have Moderate Influence**
 - rolling_mean, lag_2, lag_3, lag_4, and lag_5 contribute but are less important than lag_1.
 - This implies that while historical data matters, the most recent lag is the strongest predictor.
- **Low Impact of Some Macroeconomic Factors**
 - Consumer Price Index, Unemployment Rate, Federal Funds Effective Rate, Real Gross Domestic Product, and Personal Consumption Expenditures: Chain-type Price Index have lower importance.
 - This suggests that these variables provide less predictive value in this model.

Interpretation:

- XGBoost identifies 5-Year, 5-Year Forward Inflation Expectation Rate as the strongest predictor, unlike Random Forest, which prioritized lag_1.
- The model leverages both economic indicators and lagged historical data, with macroeconomic features playing a more prominent role than in previous models.
- Feature selection can be refined by considering the lowest-ranking variables, as they might contribute little to predictive power.
- The importance of lag features suggests that short-term historical data remains relevant but is complemented by forward-looking economic expectations.

Top 10 Feature Importance (XGBoost)

Key Observations:

- **Most Important Feature:** 5-Year, 5-Year Forward Inflation Expectation Rate
 - This feature has the highest F-score (5996.0), indicating that it plays the most significant role in XGBoost's predictions.
- **Strong Influence of** lag_1
 - The second most important feature (3979.0 F-score) suggests that the most recent historical

data point is crucial for forecasting.

- **Macroeconomic Factors Matter**

- Features such as `Crude Oil Prices: West Texas Intermediate` (3236.0), `Nominal Broad U.S. Dollar Index` (3103.0), and `10-Year Treasury Yield` (3020.0) rank high in importance.
- This indicates that XGBoost captures relationships between macroeconomic trends and the target variable effectively.

- **Moderate Contribution from Additional Lag Features**

- `lag_2`, `lag_3`, `lag_4`, and `lag_5` exhibit decreasing importance, reinforcing that more recent historical values contribute more to predictions.

- **Rolling Mean is Relatively Significant**

- `rolling_mean` has an F-score of `2505.0`, showing that the moving average of past values provides meaningful information.

Interpretation:

- Unlike traditional time series models that rely mainly on lag values, XGBoost assigns high importance to macroeconomic variables.
- The combination of economic indicators and past values suggests that both market expectations (`5-Year`, `5-Year Forward Inflation Expectation Rate`) and historical patterns (`lag_1`) are key drivers of the target variable.
- Features with lower importance (e.g., `lag_5`) could be reconsidered for removal or further analysis to optimize model efficiency.
- This feature importance ranking provides valuable insights for improving predictive performance, such as focusing on the most influential economic indicators and short-term lagged values.

Actual vs Predicted Values (XGBoost)

Key Observations:

- **Strong Linear Relationship:**
 - The predicted values (blue dots) align closely with the red dashed `Perfect Fit` line ($y = x$), indicating high model accuracy.
- **Minimal Deviation:**
 - Most predictions are very close to the actual values, with only a few minor deviations, showing strong predictive performance.
- **Low Error Rate:**
 - The spread of points around the perfect fit line is minimal, suggesting that the model's residuals (errors) are low.
- **Good Performance Across the Range:**
 - The model performs well across all values, with accurate predictions even at extreme values (both low and high).
- **Slight Dispersion in Lower Values:**
 - There are a few scattered points at the lower end of the actual values, which may indicate minor underprediction or overprediction in that range.

Interpretation:

- The XGBoost model demonstrates excellent predictive accuracy, as evidenced by the strong alignment between actual and predicted values.
- The minimal deviation from the perfect fit line suggests that the model generalizes well without significant bias or variance issues.
- Any remaining deviations are small and may be further reduced with fine-tuning or feature selection.
- The high performance of the model suggests that it is well-suited for the given dataset, capturing patterns effectively.

Residual Plot (XGBoost)

Key Observations:

- **Residuals Centered Around Zero:**
 - The majority of residuals (green dots) are scattered around the red dashed line at $y = 0$, indicating that the model has low bias.
- **Random Distribution of Residuals:**

- The residuals appear randomly distributed, suggesting that the model does not exhibit systematic errors or patterns.
- **Low Magnitude of Residuals:**
 - Most residuals remain close to zero, indicating that the predictions are generally accurate.
- **Some Outliers at Lower Values:**
 - A few residuals show larger deviations, particularly in the lower range of actual values, suggesting minor underprediction or overprediction in specific cases.
- **No Clear Heteroscedasticity:**
 - The spread of residuals does not systematically increase or decrease with actual values, meaning that prediction errors are relatively stable across the range.

Interpretation:

- The XGBoost model performs well, with residuals evenly distributed around zero, indicating no major bias in predictions.
- The absence of a distinct pattern in residuals suggests that the model captures the relationships effectively and does not miss systematic trends.
- The slight presence of outliers could be addressed by fine-tuning hyperparameters or further refining feature selection.
- Overall, the model exhibits strong predictive performance, with only minor areas for improvement in extreme cases.

Residual Distribution (XGBoost)

Key Observations:

- **Centered Around Zero:**
 - The histogram shows that most residuals are concentrated around 0, indicating that the model's predictions are generally accurate.
- **Near-Normal Distribution:**
 - The shape of the residuals resembles a normal distribution, suggesting that prediction errors are randomly distributed rather than systematically biased.
- **Sharp Peak at Zero:**
 - The highest frequency of residuals is at 0, reinforcing that a large portion of predictions are very close to actual values.
- **Symmetric Spread:**
 - The distribution is fairly symmetric, with only minor skewness, meaning there is no significant bias in overprediction or underprediction.
- **Few Extreme Residuals:**
 - Some residuals extend slightly to the left and right, but extreme errors are rare, indicating that the model performs well across different values.

Interpretation:

- The XGBoost model makes highly accurate predictions with minimal errors, as evidenced by the concentration of residuals around zero.
- The nearly normal distribution of residuals suggests that the model does not exhibit systematic bias and generalizes well to the data.
- The low presence of extreme residuals indicates that the model maintains stability and avoids large mispredictions.
- This strong residual distribution confirms that XGBoost is well-tuned for the dataset and is making reliable predictions.

Top 5 Grid Search Results:

Index	max_depth	learning_rate	subsample	colsample_bytree	mean_rmse
72	9	0.1	0.7	0.7	0.025516
73	9	0.1	0.7	0.8	0.025516
74	9	0.1	0.7	0.9	0.025516
75	9	0.1	0.8	0.7	0.025516
76	9	0.1	0.8	0.8	0.025516

RMSE for `max_depth` vs `learning_rate`

Key Observations:

- **Consistent RMSE Across Parameters:**
 - The Root Mean Squared Error (RMSE) remains constant at `0.026` across all combinations of `max_depth` and `learning_rate`.
- **No Significant Sensitivity to Hyperparameters:**
 - Changes in `learning_rate` (`0.01`, `0.05`, `0.1`) and `max_depth` (`3`, `6`, `9`) do not affect the RMSE, suggesting that the model's performance is stable across these values.
- **Lack of Optimal Hyperparameter Differentiation:**
 - Unlike typical hyperparameter tuning results where RMSE varies with different values, this plot suggests that the model might not be highly sensitive to these hyperparameters.
- **Color Consistency in the Heatmap:**
 - The color bar indicates a very narrow RMSE range, confirming minimal variation between different parameter combinations.

Interpretation:

- The stability of RMSE suggests that changes in `max_depth` and `learning_rate` within this range do not significantly impact model performance.
- This could indicate that other hyperparameters (e.g., number of estimators, subsample ratio) might have a stronger influence on RMSE.
- Further tuning on a wider range of values or additional parameters may be necessary to identify the optimal settings.
- The model appears to be well-tuned within this specific range, as no single combination of parameters shows an overwhelming advantage over others.

SHAP Summary Plot: Feature Impact on Model Predictions

Key Observations:

- **Most Influential Features:**
 - `lag_1` has the highest impact on the model's predictions, as indicated by its wide distribution of SHAP values.
 - `rolling_mean` and `lag_2` also play a significant role, though their SHAP values are less spread out than `lag_1`.
- **Low Impact of Macroeconomic Variables:**
 - Features such as `10-Year Treasury Yield`, `Consumer Price Index`, `Federal Funds Effective Rate`, and `Real Gross Domestic Product` exhibit SHAP values near zero, suggesting minimal influence on the model.
- **SHAP Value Interpretation:**
 - Red points indicate high feature values, while blue points indicate low feature values.
 - A strong separation in `lag_1` and `rolling_mean` suggests that high values of these features push the model's predictions in a specific direction.
- **Effect of Lag Features:**
 - `lag_3`, `lag_4`, and `lag_5` have a smaller influence compared to `lag_1`, reinforcing the importance of recent past values over older ones.

Interpretation:

- The model heavily relies on short-term historical values (`lag_1`, `rolling_mean`, `lag_2`) rather than macroeconomic indicators.
- The near-zero SHAP values for economic factors suggest that they do not significantly contribute to the model's predictions.
- The dominance of lag features implies that the target variable is primarily influenced by its own past values rather than external economic conditions.
- These insights can guide feature selection, potentially eliminating less important macroeconomic variables to optimize model performance.

SHAP Dependence Plot: `5-Year, 5-Year Forward Inflation Expectation Rate`

Key Observations:

- **Positive Correlation:**
 - The SHAP values increase as the 5-Year, 5-Year Forward Inflation Expectation Rate rises, indicating a strong positive relationship between this feature and the model's prediction.
- **Nonlinear Trend:**
 - While mostly linear, there are slight variations in SHAP values, suggesting that the feature's impact may vary at different levels.
- **Lower Values Have Negative Impact:**
 - For lower inflation expectation rates (below ~1.5), SHAP values are negative, meaning that these values contribute to lowering the model's prediction.
- **Higher Values Have Positive Impact:**
 - As the inflation expectation rate exceeds ~1.5, SHAP values turn positive, meaning higher values of this feature increase the model's output.
- **Spread at Mid-Range Values:**
 - There is some dispersion in SHAP values for inflation expectation rates around 1.5-2.5, indicating potential interactions with other features.

Interpretation:

- The 5-Year, 5-Year Forward Inflation Expectation Rate is an important predictor, directly influencing the model's output.
- Higher inflation expectations drive the model's prediction higher, which aligns with economic intuition—higher inflation expectations often correlate with market shifts.
- The slight spread at mid-range values suggests that this feature's effect might be influenced by other variables, indicating potential interaction effects.
- This insight helps confirm that economic forecasts, particularly inflation expectations, play a key role in the model's decision-making process.

SHAP Waterfall Plot: Feature Contributions to Prediction

Key Observations:

- **Base Value (2.056):**
 - This represents the model's expected prediction before considering specific feature effects.
- **Final Prediction (2.66):**
 - The cumulative contribution of influential features moves the prediction up from the base value to 2.66.
- **Main Contributors:**
 - lag_1 = 2.67 : The most influential feature, significantly increasing the prediction.
 - rolling_mean = 2.664 : Also plays a major role in pushing the prediction higher.
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.59 : A macroeconomic indicator contributing positively to the prediction.
 - lag_2 = 2.69 : Another important lagged feature impacting the output.
- **Color Interpretation:**
 - **Red (Positive Influence):** These features increase the predicted value.
 - **Blue (Negative Influence, barely visible here):** Would represent features that reduce the prediction, though none appear significantly impactful in this case.

Interpretation:

- The prediction of 2.66 is mainly driven by lagged values (lag_1 , lag_2 , rolling_mean), reinforcing that recent past values strongly influence the model's output.
- The presence of 5-Year, 5-Year Forward Inflation Expectation Rate as a significant contributor suggests that macroeconomic expectations also affect predictions.
- The absence of strong negative contributions implies that all key features in this case push the prediction higher.
- Understanding these influences allows for better interpretability and potential improvements in model performance by focusing on the most impactful variables.

SHAP Feature Contribution to Prediction

Key Observations:

- **Predicted Value (2.66):**

- The model predicts a value of `2.66`, positioned between the minimum (`0.79`) and maximum (`2.71`) possible prediction values.
- **Positive Contributors (Orange):**
 - The most impactful features that increase the prediction include:
 - `lag_1 = 2.67`
 - `rolling_mean = 2.66`
 - `5-Year, 5-Year Forward Inflation Expectation Rate = 2.59`
 - `Real Gross Domestic Product = 16396.15`
 - `lag_2 = 2.69`
 - These variables contribute significantly to raising the predicted value.
- **Negative Contributors (Blue):**
 - Features reducing the predicted value include:
 - `lag_3 = 2.66`
 - `lag_4 = 2.68`
 - `Unemployment Rate = 4.60`
 - `Crude Oil Prices: West Texas Intermediate = 68.44`
 - `Federal Funds Effective Rate = 4.94`
 - These factors slightly counterbalance the increase, but their overall impact is lower.
- **Lag Features Are Key Drivers:**
 - `lag_1`, `rolling_mean`, and `lag_2` are dominant in shaping the final prediction, reinforcing the significance of recent past values.
- **Macroeconomic Indicators Matter:**
 - `5-Year, 5-Year Forward Inflation Expectation Rate` and `Real Gross Domestic Product` positively influence the prediction.
 - `Unemployment Rate` and `Crude Oil Prices` contribute negatively, though their effects are relatively minor.

Interpretation:

- The model assigns the highest weight to recent historical data (`lag_1`, `rolling_mean`), suggesting that short-term trends are crucial for prediction.
- Macroeconomic factors like inflation expectations and GDP play an important but secondary role in influencing the output.
- Negative contributors such as the unemployment rate and crude oil prices slightly offset the prediction but do not drastically alter it.
- Understanding the influence of these features provides insights into refining the model and improving interpretability.

Light Gradient-Boosting Machine (LightGBM) Model Code

Description of the Code Blocks

Block 1: Importing Required Libraries

This script uses **LightGBM** for training a regression model.
It also imports:

- **Regular expressions** (`re`) for cleaning feature names.
- **RandomizedSearchCV** and **TimeSeriesSplit** for hyperparameter tuning with time-series validation.
- **NumPy** (`np`) for numerical operations.

Block 2: Creating Copies of Training and Test Sets

- **Preserves the original datasets** (`X_train`, `X_test`) by creating copies for LightGBM.
- **Prevents unwanted modifications** during preprocessing.

Block 3: Feature Name Sanitization

LightGBM does not support special characters in feature names.
The `sanitize_feature_names(df)` function:

- **Replaces non-alphanumeric characters with underscores** (`_`).

- Ensures feature compatibility with LightGBM.

Applied to both `X_train_lgb` and `X_test_lgb`.

Block 4: Setting Up Time-Series Cross-Validation

- Uses `TimeSeriesSplit(n_splits=2)` for cross-validation.
- Ensures future data points are not used to predict past values.

Prevents data leakage and ensures realistic model evaluation.

Block 5: Defining LightGBM Base Parameters

The `params` dictionary includes:

- `objective: 'regression'` → Specifies regression task.
 - `metric: 'rmse'` → Uses **Root Mean Squared Error (RMSE)** for evaluation.
 - `min_child_samples: 50` → Minimum number of samples in a leaf.
 - `n_jobs: -1` → Utilizes all available CPU cores.
-

Block 6: Hyperparameter Tuning with Randomized Search

The `hyperparameter grid (param_grid)` includes:

- **Tree Depth (`max_depth`)**: Controls model complexity.
- **Learning Rate (`learning_rate`)**: Adjusts step size per iteration.
- **Number of Leaves (`num_leaves`)**: Defines tree structure.
- **Subsample (`subsample`)**: Prevents overfitting by using a fraction of data.
- **Feature Sampling (`colsample_bytree`)**: Reduces feature usage per tree.
- **Number of Trees (`n_estimators`)**: Controls boosting iterations.

RandomizedSearchCV Setup

- Tests 10 random hyperparameter combinations (`n_iter=10`).
- Uses `neg_mean_squared_error` as the performance metric.
- Ensures reproducibility (`random_state=42`).

Fits the best model using cross-validation (`random_search.fit(X_train_lgb, y_train)`).

Block 7: Training and Model Selection

- Extracts the **best hyperparameters** (`random_search.best_params_`).
- Retrieves the **best-trained LightGBM model** (`best_lgbm_model`).
- Generates predictions (`lgbm_pred = best_lgbm_model.predict(X_test_lgb)`).

Uses the best hyperparameter combination for final predictions.

Block 8: Model Evaluation

- Uses a predefined function `calculate_metrics()` to assess:
 - RMSE, MAE, R², MAPE, etc.
- Compares performance against a **naive baseline forecast**.

Ensures model predictions are reliable for forecasting applications.

Block 9: Feature Importance Extraction

Extracts **feature importance scores** using:

- `best_lgbm_model.feature_importances_` (LightGBM's built-in method).
- **Sorted in descending order** for better interpretability.

Provides insights into which features drive model predictions.

Block 10: Feature Importance Visualization

A horizontal bar chart displays:

- X-axis: Feature importance scores.
- Y-axis: Feature names.
- Top features appear at the top (y-axis inverted).

Helps in identifying key economic indicators affecting inflation trends.

Block 11: Actual vs. Predicted Values Plot

- Scatter plot to compare model predictions (`lgbm_pred`) against actual values (`y_test`).
- Includes a red dashed "Perfect Fit" line for reference.
- Ideal predictions should align closely with this line.

Visualizes model accuracy and potential prediction errors.

Block 12: Residual Analysis (Error Examination)

Residuals = Actual - Predicted values

- Scatter plot of residuals vs. actual values.
- A red dashed horizontal line (`y=0`) represents perfect predictions.

Detects systematic bias and error patterns.

Block 13: Residual Distribution Analysis

- Histogram of residuals to examine error distribution.
- A vertical red dashed line at `0` indicates ideal error symmetry.

Ensures model errors are randomly distributed (no major bias).

Block 14: Hyperparameter Tuning Heatmap

A heatmap visualizes:

- X-axis: Learning Rate (`learning_rate`).
- Y-axis: Tree Depth (`max_depth`).
- Color intensity represents model performance (RMSE).

Helps identify optimal parameter combinations.

Block 15: SHAP Feature Importance Analysis

Uses SHAP (SHapley Additive Explanations) for interpretability:

- Computes SHAP values for each feature.
- Generates a summary plot showing:
 - Feature impact on predictions.
 - Red (high values) and blue (low values) color coding.

Explains how each feature contributes to inflation forecasting.

Block 16: Local Explanation with SHAP Force Plot

- Examines feature contributions for a single test instance.
- Displays interactive force plots (SHAP-based individual explanations).

Useful for explaining specific model predictions.

Block 17: Partial Dependence Plots (PDP)

PDPs show:

- The average effect of each feature on model predictions.
- Visualized for all input features.

Helps assess linearity/non-linearity in feature influence.

Block 18: Top 10 Most Important Features

- Uses LightGBM's built-in feature importance plot.
- Shows the top 10 most frequently used features for splitting.

Identifies critical economic indicators affecting inflation rates.

Block 19: Permutation Feature Importance

Permutation importance:

- Shuffles feature values and observes impact on performance.
- More accurate than built-in feature importance scores.
- Bar chart highlights the most impactful features.

Enhances trust in feature selection and model reliability.

Conclusion

1. Optimized LightGBM model using hyperparameter tuning and time-series validation.
2. Evaluated model using advanced metrics and interpretability techniques.
3. Visualized feature importance, residuals, and actual vs. predicted trends.

Result Output:

Fitting 2 folds for each of 10 candidates, totalling 20 fits

Best Hyperparameters:

```
'subsample': 0.8,  
'num_leaves': 20,  
'n_estimators': 150,  
'max_depth': 7,  
'learning_rate': 0.1,  
'colsample_bytree': 0.8
```

LightGBM evaluation metrics:

```
'Model': 'Light Gradient Boosting Machine (LightGBM)',  
'MSE': 0.0007446301235070315,  
'RMSE': 0.027287911673615325,  
'MAE': 0.016861573069375194,  
'MAPE (%)': 1.262763295396008,  
'sMAPE (%)': 1.164857083373016,  
'MASE': 0.042922077969458246,
```

Feature Importances:

Feature Index	Feature	Importance
9	lag_1	389
0	5-Year, 5-Year Forward Inflation Expectation Rate	369
14	rolling_mean	299
10	lag_2	286
12	lag_4	204
7	Crude Oil Prices: West Texas Intermediate	171
11	lag_3	162
8	10-Year Treasury Yield	149
6	Nominal Broad U.S. Dollar Index	149
13	lag_5	134
1	Consumer Price Index	80
3	Unemployment Rate	63
4	Federal Funds Effective Rate	62
2	Real Gross Domestic Product	47
5	Personal Consumption Expenditures: Chain-type ...	33

Feature Importance (LightGBM)

Key Observations:

- **Most Important Feature: lag_1 :**
 - lag_1 has the highest importance, indicating that the most recent past value is the strongest predictor in the LightGBM model.
- **Significant Influence of 5-Year, 5-Year Forward Inflation Expectation Rate :**
 - This macroeconomic indicator ranks second in importance, showing that economic expectations play a major role in predictions.
- **Rolling Mean and Other Lag Features are Crucial:**
 - rolling_mean, lag_2, lag_3, lag_4, and lag_5 have moderate importance, reinforcing the role of historical values in shaping predictions.
- **Economic Indicators Matter but Have Lower Influence:**
 - Features such as Crude Oil Prices: West Texas Intermediate, 10-Year Treasury Yield, Nominal Broad U.S. Dollar Index, and Consumer Price Index contribute to the model but are less influential compared to lagged values.
- **Lower Importance of Some Macroeconomic Variables:**
 - Unemployment Rate, Federal Funds Effective Rate, Real Gross Domestic Product, and Personal Consumption Expenditures: Chain-type Price Index have the least impact on the model.

Interpretation:

- The model heavily relies on lag features, particularly lag_1, confirming that recent historical values are key drivers of the target variable.
- 5-Year, 5-Year Forward Inflation Expectation Rate is highly relevant, suggesting that inflation expectations strongly influence predictions.
- While macroeconomic indicators are factored into the model, they generally have lower importance than historical data.
- This ranking can help optimize feature selection, potentially reducing reliance on less influential variables while focusing on the most impactful predictors.

Actual vs Predicted Values (LightGBM)

Key Observations:

- **Strong Alignment with the Perfect Fit Line:**
 - The blue dots (predictions) closely follow the red dashed line (y = x), indicating that the

LightGBM model's predictions are highly accurate.

- **Minimal Deviation:**
 - Most predictions fall very close to the perfect fit line, suggesting low error rates.
- **Good Performance Across the Range:**
 - The model performs well for both lower and higher actual values, with no significant bias or systematic errors.
- **Slight Dispersion at Lower Values:**
 - There are a few scattered points at the lower end, indicating minor underprediction or overprediction in some cases.

Interpretation:

- The LightGBM model demonstrates excellent predictive accuracy, as evidenced by the strong correlation between actual and predicted values.
- The low spread of deviations suggests that the model generalizes well and does not suffer from major overfitting or underfitting.
- Small errors at the lower range may be investigated further, but they do not significantly impact the overall performance.
- Overall, the model appears to be well-optimized and reliable for making accurate predictions.

Residual Plot (LightGBM)

Key Observations:

- **Residuals Centered Around Zero:**
 - The majority of residuals (green dots) are scattered closely around the red dashed line at $y = 0$, indicating that the model has minimal bias.
- **Random Distribution of Residuals:**
 - The residuals are evenly spread across the range of actual values, suggesting that the model does not exhibit systematic errors or trends.
- **Low Magnitude of Residuals:**
 - Most residuals remain within a narrow band close to zero, meaning the model's predictions are highly accurate.
- **Presence of Some Outliers:**
 - A few residuals deviate significantly, especially for lower actual values, which may indicate occasional underprediction or overprediction.
- **No Clear Heteroscedasticity:**
 - The spread of residuals does not increase significantly with actual values, indicating that prediction errors remain fairly consistent across different value ranges.

Interpretation:

- The LightGBM model demonstrates strong predictive accuracy, as evidenced by the clustering of residuals around zero.
- The absence of a distinct pattern in residuals suggests that the model captures relationships effectively without systematic bias.
- The small number of outliers could be analyzed further, possibly improving performance through additional feature engineering or hyperparameter tuning.
- Overall, the model appears well-calibrated and provides stable predictions with minimal error variance.

Residual Distribution (LightGBM)

Key Observations:

- **Centered Around Zero:**
 - The histogram shows that most residuals are concentrated around 0 , indicating that the model's predictions are generally accurate.
- **Near-Normal Distribution:**
 - The residuals follow a bell-shaped distribution, suggesting that errors are randomly distributed and not biased in one direction.
- **Sharp Peak at Zero:**
 - The highest frequency of residuals is at 0 , showing that a large portion of predictions are very close to actual values.

- **Symmetric Spread:**
 - The distribution is fairly symmetric, meaning there is no significant overprediction or underprediction trend.
- **Few Extreme Residuals:**
 - Some residuals deviate slightly towards negative and positive values, but extreme errors are minimal.

Interpretation:

- The LightGBM model produces highly accurate predictions with minimal errors, as evidenced by the concentration of residuals around zero.
- The near-normal shape of the residual distribution suggests that the model generalizes well and does not suffer from systematic bias.
- The presence of very few extreme residuals indicates that the model maintains stability across different values.
- This strong residual distribution confirms that LightGBM is well-tuned for the dataset and provides reliable predictions.

Top 5 Hyperparameter Combinations (RandomizedSearchCV):

Rank 1:

Parameters:

```
{'subsample': 0.8, 'num_leaves': 20, 'n_estimators': 150, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.8}
```

Mean Test Score: -0.001033

Standard Deviation: 0.000211

Rank 2:

Parameters:

```
{'subsample': 0.9, 'num_leaves': 31, 'n_estimators': 150, 'max_depth': 7, 'learning_rate': 0.05, 'colsample_bytree': 0.7}
```

Mean Test Score: -0.001047

Standard Deviation: 0.000210

Rank 9:

Parameters:

```
{'subsample': 0.9, 'num_leaves': 50, 'n_estimators': 200, 'max_depth': 5, 'learning_rate': 0.05, 'colsample_bytree': 0.7}
```

Mean Test Score: -0.001075

Standard Deviation: 0.000222

Rank 10:

Parameters:

```
{'subsample': 0.8, 'num_leaves': 20, 'n_estimators': 200, 'max_depth': 3, 'learning_rate': 0.05, 'colsample_bytree': 0.8}
```

Mean Test Score: -0.001140

Standard Deviation: 0.000207

Rank 4:

Parameters:

```
{'subsample': 0.9, 'num_leaves': 31, 'n_estimators': 100, 'max_depth': 5, 'learning_rate': 0.05, 'colsample_bytree': 0.8}
```

Mean Test Score: -0.001147

Standard Deviation: 0.000236

Effect of `max_depth` and `learning_rate` on Model

Performance

Key Observations:

- **Performance Metric:**
 - The heatmap visualizes the impact of different combinations of `max_depth` and `learning_rate` on model performance.
 - The values in the cells represent the error metric (likely RMSE), where lower values indicate better performance.
- **Lowest Error (-0.001) at Multiple Configurations:**
 - The best performance is achieved with `learning_rate = 0.05` and all values of `max_depth` (3 , 5 , 7).
 - Similarly, `learning_rate = 0.1` results in the same minimal error (-0.001), though some cells appear missing.
- **Higher Errors at `learning_rate = 0.01`:**
 - A deeper tree (`max_depth = 5`) with `learning_rate = 0.01` leads to the highest error (-0.023), suggesting overfitting or slow convergence.
 - `max_depth = 7` also has a relatively higher error (-0.014) at `learning_rate = 0.01`, reinforcing that a too-small learning rate may not be optimal.
- **Color Gradient Interpretation:**
 - The color bar indicates a transition from higher error (darker shades) to lower error (lighter shades), highlighting how hyperparameter choices impact model performance.

Interpretation:

- **Optimal Settings:**
 - The best-performing configurations use a `learning_rate` of 0.05 or 0.1, regardless of `max_depth`.
 - A learning rate of 0.01 leads to higher errors, especially with deeper trees (`max_depth = 5` or 7).
- **Trade-off Between Depth and Learning Rate:**
 - Higher depth may lead to overfitting when paired with a small learning rate (0.01).
 - Moderate depth values (`max_depth = 3` or 5) with `learning_rate = 0.05` or 0.1 yield the most stable performance.
- **Next Steps for Model Optimization:**
 - The model can be fine-tuned further by testing other learning rates between 0.01 and 0.05.
 - Additional hyperparameters (e.g., number of estimators, regularization terms) may further enhance performance.

SHAP Summary Plot: Feature Impact on Model Predictions

Key Observations:

- **Most Influential Features:**
 - `lag_1` has the highest impact on the model's predictions, as indicated by the wide distribution of SHAP values.
 - `rolling_mean` and `lag_2` are also significant contributors, although with a smaller range of SHAP values.
- **Macroeconomic Variables Have Minimal Impact:**
 - Features such as `Real Gross Domestic Product`, `Consumer Price Index`, `Federal Funds Effective Rate`, and `10-Year Treasury Yield` have near-zero SHAP values, indicating low influence on model predictions.
- **SHAP Value Interpretation:**
 - **Red** represents higher feature values, while **blue** represents lower values.
 - A strong separation in `lag_1`, `rolling_mean`, and `lag_2` suggests that high values of these features push the model's predictions in a specific direction.
- **Lag Features Play a Crucial Role:**
 - `lag_3`, `lag_4`, and `lag_5` show some influence, but their impact is significantly lower than `lag_1`.
 - This reinforces that more recent past values (`lag_1` and `rolling_mean`) have a stronger predictive effect than older ones.
- **Minimal Contribution from Crude Oil Prices and Currency Index:**
 - `Crude Oil Prices: West Texas Intermediate` and `Nominal Broad U.S. Dollar Index` have very low SHAP values, meaning they do not significantly affect the model's predictions.

Interpretation:

- The model primarily depends on short-term historical values (`lag_1`, `rolling_mean`, `lag_2`), confirming that recent past values drive predictions.
- Macroeconomic indicators like inflation expectations (`5-Year, 5-Year Forward Inflation Expectation Rate`) contribute but are secondary to lag-based features.
- The dominance of lagged values suggests that external economic conditions have limited direct influence on the model's output.
- These insights can help refine feature selection, potentially reducing less important macroeconomic variables to improve model efficiency.

SHAP Waterfall Plot: Feature Contributions to Prediction

Key Observations:

- **Base Value (`2.056`):**
 - This represents the model's expected prediction before accounting for feature influences.
- **Final Prediction (`2.21`):**
 - The cumulative effect of the contributing features increases the predicted value to `2.21`.
- **Top Contributors (Positive Influence):**
 - `Crude Oil Prices: West Texas Intermediate = 106.7`: This feature has a significant positive impact on the final prediction.
 - `5-Year, 5-Year Forward Inflation Expectation Rate = 2.47`: A macroeconomic factor that increases the predicted value.
 - `lag_2 = 2.17`: The second lag feature plays a strong role in influencing the model's output.
 - `rolling_mean = 2.208`: The moving average of past values contributes positively.
 - `lag_1 = 2.2`: The most recent lag value also pushes the prediction upward.
- **Color Interpretation:**
 - **Red (Positive Influence):** These features increase the predicted value.
 - **Blue (Negative Influence, barely visible here):** Would indicate features that reduce the prediction, but they are not prominent in this case.

Interpretation:

- The prediction of `2.21` is primarily driven by a combination of historical values (`lag_1`, `lag_2`, `rolling_mean`) and macroeconomic indicators (`Crude Oil Prices` and `5-Year Inflation Expectation`).
- The significant contribution of `Crude Oil Prices` suggests that market conditions are an influential factor in this model's predictions.
- `lag_1` and `lag_2` reaffirm that short-term historical trends remain essential in forecasting.
- The absence of strong negative contributions suggests that all key features in this case push the prediction higher.
- These insights can guide further refinements in feature selection, focusing on optimizing influential variables for better model performance.

SHAP Waterfall Plot: Feature Contributions to Prediction

Key Observations:

- **Base Value (`2.056`):**
 - This represents the model's expected prediction before considering individual feature contributions.
- **Final Prediction (`2.35`):**
 - The cumulative impact of the features pushes the prediction up to `2.35`.
- **Top Contributors (Positive Influence):**
 - `5-Year, 5-Year Forward Inflation Expectation Rate = 2.48`: A significant driver of the prediction increase, indicating that inflation expectations strongly impact the model.
 - `lag_2 = 2.39`: The second lagged feature contributes positively to the prediction.
 - `rolling_mean = 2.382`: The moving average of past values also increases the model's output.
 - `lag_1 = 2.34`: The most recent lag feature reinforces the prediction increase.
- **Negative Contributors (Blue - Reducing the Prediction):**
 - `lag_4 = 2.42`
 - `lag_3 = 2.42`
 - These features slightly pull the prediction down but do not outweigh the strong positive

influences.

- **Color Interpretation:**

- **Red (Positive Influence):** Features that increase the prediction.
- **Blue (Negative Influence):** Features that reduce the prediction, though their impact is minor in this case.

Interpretation:

- The model's prediction of 2.35 is mainly driven by economic expectations (5-Year, 5-Year Forward Inflation Expectation Rate) and recent historical trends (lag_1 , lag_2 , rolling_mean).
- The presence of minor negative contributions (lag_3 , lag_4) suggests that these lagged values may slightly counterbalance the effect of more recent lags but do not significantly alter the overall prediction.
- This reinforces that short-term historical values and macroeconomic expectations are the key drivers of the model's output.
- These insights can help refine the model by focusing on optimizing the most impactful features while reducing less influential ones.

Partial Dependence Plots (LightGBM)

Key Observations:

- **Strong Positive Relationships:**
 - lag_1 , lag_2 , and rolling_mean show a strong increasing trend, indicating that higher values of these features lead to higher predictions.
 - 5-Year, 5-Year Forward Inflation Expectation Rate also exhibits a slight positive trend, suggesting that inflation expectations influence the model output.
- **Minimal or No Effect:**
 - Features such as Consumer Price Index , Real Gross Domestic Product , Unemployment Rate , Federal Funds Effective Rate , Personal Consumption Expenditures , Nominal Broad U.S. Dollar Index , and Crude Oil Prices have almost flat lines.
 - This suggests that changes in these variables do not significantly impact the model's predictions.
- **Negative Influence:**
 - lag_3 shows a slight downward trend, indicating that higher values of this feature slightly reduce the model's prediction.
- **Stable Macroeconomic Factors:**
 - The nearly horizontal lines for macroeconomic variables imply that the model does not rely heavily on them for predictions, favoring recent historical values instead.

Interpretation:

- **Dependence on Lag Features:**
 - The model relies primarily on recent historical data (lag_1 , lag_2 , rolling_mean), as evidenced by their strong upward trends.
- **Limited Impact of Macroeconomic Indicators:**
 - Despite including various macroeconomic variables, most of them do not significantly influence predictions.
- **Potential Feature Optimization:**
 - Variables with minimal impact (e.g., Federal Funds Effective Rate , Nominal Broad U.S. Dollar Index , Crude Oil Prices) could potentially be excluded to simplify the model.
- **Overall Model Behavior:**
 - The model effectively captures short-term patterns while assigning low importance to long-term economic trends.

Top 10 Feature Importance (LightGBM)

Key Observations:

- **Most Important Feature: lag_1 :**
 - lag_1 is the most influential feature, with a feature importance score of 389 , indicating that the most recent past value strongly impacts predictions.
- **Significant Role of 5-Year, 5-Year Forward Inflation Expectation Rate :**
 - The second most important feature (369), showing that inflation expectations are a key

macroeconomic factor in the model.

- **Rolling Mean and Lagged Values Are Crucial:**
 - `rolling_mean` (299) and `lag_2` (286) highlight the importance of historical trends in driving predictions.
- **Moderate Influence of `lag_4` and Crude Oil Prices :**
 - `lag_4` (204) still plays a role, but its importance decreases compared to more recent lags.
 - `Crude Oil Prices: West Texas Intermediate` (171) suggests that market conditions contribute to the model's predictions.
- **Lower Influence of Economic Indicators:**
 - `10-Year Treasury Yield` (149) and `Nominal Broad U.S. Dollar Index` (149) have a lower impact on the model.
 - `lag_5` (134) shows that older lag values have diminishing importance.

Interpretation:

- **Recent Lag Features Dominate:**
 - The model relies heavily on `lag_1`, `rolling_mean`, and `lag_2`, indicating that short-term historical values are primary drivers of predictions.
- **Macroeconomic Variables Have a Secondary Role:**
 - While `5-Year Inflation Expectation Rate` and `Crude Oil Prices` influence predictions, they are not as dominant as lag-based features.
- **Feature Optimization Insight:**
 - Since `lag_5` and some macroeconomic indicators have lower importance, the model could be further optimized by reducing less significant features.
- **Overall Model Behavior:**
 - The LightGBM model prioritizes recent historical patterns over long-term economic trends, making it well-suited for short-term forecasting.

Permutation Feature Importance - LightGBM

Key Observations:

- **Most Important Feature: `lag_1`:**
 - `lag_1` has the highest permutation importance, confirming its strong predictive power in the model.
- **High Influence of `rolling_mean` and `lag_2`:**
 - `rolling_mean` is the second most influential feature, followed by `lag_2`, indicating that historical averages and recent lagged values play a crucial role in predictions.
- **Minimal Contribution from Other Features:**
 - `5-Year, 5-Year Forward Inflation Expectation Rate` has a small impact but is significantly less important than lag-based features.
 - `lag_3` and `lag_4` contribute slightly but are much less important than `lag_1` and `rolling_mean`.
- **No Significant Impact from Macroeconomic Variables:**
 - Features such as `Real Gross Domestic Product`, `10-Year Treasury Yield`, `Federal Funds Effective Rate`, `Consumer Price Index`, and `Unemployment Rate` show negligible permutation importance.
 - `Crude Oil Prices`, `Nominal Broad U.S. Dollar Index`, and `Personal Consumption Expenditures` also contribute little to model predictions.

Interpretation:

- **Dependence on Recent Lag Values:**
 - The model primarily relies on short-term historical data (`lag_1`, `rolling_mean`, `lag_2`), indicating that recent trends have the strongest predictive power.
- **Macroeconomic Factors Play an Insignificant Role:**
 - Despite being included in the model, economic indicators such as GDP, inflation, and interest rates do not significantly impact predictions.
- **Feature Selection Insight:**
 - The model could be optimized by removing or deprioritizing features with near-zero permutation importance to enhance efficiency.
- **Overall Model Behavior:**
 - The LightGBM model is strongly influenced by short-term patterns, making it effective for time-series forecasting.

Gradient Boosting Regressor (GBR)

Gradient Boosting Regressor: Model Development, Hyperparameter Tuning, and Evaluation

1. Importing Essential Libraries

- **Random & NumPy**: Used for setting seeds to ensure reproducibility.
- **Scikit-learn**: Provides tools for data splitting, hyperparameter tuning, and model implementation.

2. Data Preparation

- The dataset is divided into:
 - **Training Set (80%)**: Used for model learning.
 - **Testing Set (20%)**: Used for model evaluation.
- The split is controlled using `random_state=42` for consistent results.

3. Model Initialization

- **Gradient Boosting Regressor (GBR)** is initialized.
- The model is set with `random_state=42` for deterministic behavior.

4. Hyperparameter Tuning with GridSearchCV

- A **grid search** strategy is employed for tuning critical hyperparameters:
 - `n_estimators`: Number of trees in the ensemble.
 - `learning_rate`: Step size shrinkage to prevent overfitting.
 - `max_depth`: Depth of each tree, affecting model complexity.
 - `min_samples_split`: Minimum samples required to split an internal node.
 - `subsample`: Fraction of samples used in each boosting iteration.
- **5-Fold Cross-Validation** ensures robust hyperparameter selection.
- **Parallel processing** (`n_jobs=-1`) speeds up the search.

5. Model Training and Best Model Selection

- GridSearchCV is executed to **train multiple models** with different hyperparameter combinations.
- The best-performing model is extracted based on **Mean Squared Error (MSE)**.
- The optimal hyperparameters are printed for reference.

6. Model Prediction and Performance Evaluation

- The trained model makes predictions on the **test dataset**.
- A predefined function `calculate_metrics` computes evaluation scores.
- The model's performance is benchmarked against a **naïve forecast**.

7. Feature Importance Analysis

- **Feature importance scores** are extracted from the best Gradient Boosting model.
- A **DataFrame** is created to store and sort feature importance values.

Feature Importance Visualization

- A **horizontal bar chart** presents the ranked feature importances.
- Helps identify the most influential features in the predictive model.

8. Model Performance: Actual vs. Predicted Values

- A **scatter plot** visualizes the relationship between actual and predicted values.
- A **perfect fit reference line** is included to assess prediction accuracy.

9. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line** ($y=0$) aids in diagnosing systematic errors.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line** ($x=0$) checks for normality in residuals.

10. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Heatmap

- A **heatmap** visualizes how `learning_rate` and `n_estimators` impact model performance.
- Darker colors indicate **better performance**.

11. SHAP Analysis for Model Interpretability

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes predictions for individual instances.

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

13. Built-in vs. Permutation Feature Importance

Built-in Feature Importance

- Extracted directly from the trained Gradient Boosting model.
- Displayed using a **bar chart**.

Permutation Feature Importance

- Computed by **randomly shuffling feature values** and measuring performance impact.
- Provides a more robust estimate of feature importance.
- **Visualization:** A bar chart ranks feature importance based on permutation scores.

Conclusion

This notebook systematically implements and evaluates a **Gradient Boosting Regressor** for predictive modeling. Key contributions include:

1. **Robust hyperparameter tuning** with **GridSearchCV** and cross-validation.
2. **Comprehensive model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via built-in, permutation, and SHAP analyses.
4. **Model interpretability** through SHAP and Partial Dependence Plots.

Result Output:

Fitting 5 folds for each of 32 candidates, totalling 160 fits

Best Hyperparameters:

```
'learning_rate': 0.1,  
'max_depth': 6,
```



```
'min_samples_split': 20,

'n_estimators': 300,

'subsample': 0.8
```

Gradient Boosting Evaluation Metrics:

```
'Model': 'Gradient Boosting Regressor (GBR)',

'MSE': 0.0006021898839392853,

'RMSE': 0.02453955753348632,

'MAE': 0.014790989258911253,

'MAPE (%)': 0.9402540852297189,

'sMAPE (%)': 0.936648083110105,

'MASE': 0.03765129099191058,

'R^2': 0.9965944097534585
```

Feature Importance:

Feature Index	Feature	Importance
9	lag_1	0.792871
14	rolling_mean	0.202686
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.001142
6	Nominal Broad U.S. Dollar Index	0.000544
12	lag_4	0.000421
8	10-Year Treasury Yield	0.000403
10	lag_2	0.000389
7	Crude Oil Prices: West Texas Intermediate	0.000384
11	lag_3	0.000365
13	lag_5	0.000300
3	Unemployment Rate	0.000113
5	Personal Consumption Expenditures: Chain-type ...	0.000106
4	Federal Funds Effective Rate	0.000101
2	Real Gross Domestic Product	0.000089
1	Consumer Price Index	0.000087

Feature Importance (Gradient Boosting)

Key Observations:

- **Dominance of lag_1 :**
 - lag_1 is the most influential feature, with the highest importance score, indicating that the most recent past value significantly impacts the model's predictions.
- **Significant Role of rolling_mean :**
 - rolling_mean is the second most important feature, reinforcing the model's reliance on historical trends.
- **Minimal Contribution from Other Features:**
 - The importance of all remaining features is close to zero, suggesting they have little to no influence on the model's performance.
 - Even 5-Year, 5-Year Forward Inflation Expectation Rate , which was previously observed as a relevant macroeconomic factor, is nearly insignificant in this case.
- **Economic Indicators Are Negligible:**
 - Features like Nominal Broad U.S. Dollar Index , 10-Year Treasury Yield , Crude Oil Prices , Consumer Price Index , Unemployment Rate , and Real GDP show little or no importance in the Gradient Boosting model.

Interpretation:

- **Heavy Dependence on Short-Term Historical Data:**
 - The model strongly prioritizes recent lag values (`lag_1`) and moving averages (`rolling_mean`), indicating that short-term patterns drive predictions.
- **Minimal Influence of Macroeconomic Factors:**
 - Unlike some other models, Gradient Boosting does not assign significant importance to inflation rates, interest rates, or economic indicators.
- **Potential Feature Reduction:**
 - Given the lack of impact from most features, simplifying the model by removing insignificant variables could improve efficiency without sacrificing accuracy.
- **Model Behavior Insight:**
 - This result suggests that Gradient Boosting captures trends primarily from past values rather than relying on external economic conditions.

Actual vs Predicted Values (Gradient Boosting)

Key Observations:

- **Strong Alignment with the Perfect Fit Line:**
 - The majority of points lie very close to the red dashed line (`y = x`), indicating that the model's predictions are highly accurate.
- **Minimal Deviation from Actual Values:**
 - The scatter points are well-distributed along the diagonal, suggesting that prediction errors are small.
- **Some Variability at Lower and Higher Ranges:**
 - While most points align closely, there are minor deviations at very low and very high values.
 - These deviations indicate slightly less accuracy at the extremes, though the overall model performance remains strong.
- **Tightly Clustered Predictions:**
 - The clustering of blue points around the red line shows a well-calibrated model with minimal systematic bias.

Interpretation:

- **High Predictive Accuracy:**
 - The model demonstrates excellent performance in predicting values, with minimal error.
- **No Major Systematic Bias:**
 - The absence of a clear pattern in deviations suggests that the model does not systematically overpredict or underpredict.
- **Well-Generalized Model:**
 - The results indicate that the Gradient Boosting model generalizes well to different data points, making it a reliable forecasting tool.

Residual Plot (Gradient Boosting)

Key Observations:

- **Centered Around Zero:**
 - The residuals (errors) are distributed around the red dashed line at `y = 0` , indicating that the model does not have a strong bias in either direction.
- **No Clear Pattern in Residuals:**
 - The points are scattered randomly, suggesting that the model captures most of the variance in the data without systematic overprediction or underprediction.
- **Consistent Spread Across Values:**
 - The variance of residuals remains relatively constant across different ranges of actual values, indicating that the model performs similarly for both low and high values.
- **Few Outliers:**
 - Some points deviate significantly from zero, indicating that the model struggles with a small subset of predictions.

Interpretation:

- **Well-Calibrated Model:**
 - The residuals being evenly distributed around zero suggest that the Gradient Boosting model

makes unbiased predictions.

- **No Heteroscedasticity:**
 - The residuals do not show a clear increasing or decreasing trend, meaning that the model's accuracy is stable across different values.
- **Good Predictive Performance:**
 - The small magnitude of residuals indicates that the model provides accurate predictions with minimal error.
- **Potential for Further Refinement:**
 - Addressing the outliers (extreme residual values) through feature engineering or additional model tuning may further improve performance.

Residual Distribution (Gradient Boosting)

Key Observations:

- **Centered Around Zero:**
 - The residuals are symmetrically distributed around 0, with the highest frequency occurring near zero, indicating that the model makes unbiased predictions.
- **Approximately Normal Distribution:**
 - The histogram follows a bell-shaped curve, suggesting that the errors are normally distributed, which is a good characteristic for a well-performing model.
- **Small Residual Magnitude:**
 - Most residuals fall within a narrow range, indicating that the model's prediction errors are generally small.
- **Few Extreme Residuals:**
 - There are very few outliers on both ends of the distribution, meaning that extreme mispredictions are rare.

Interpretation:

- **Well-Calibrated Model:**
 - The near-normal distribution of residuals suggests that the model generalizes well across the dataset.
- **Low Bias and High Precision:**
 - The concentration of residuals around zero confirms that the model does not systematically overpredict or underpredict.
- **Minor Room for Improvement:**
 - Although the distribution looks good, addressing the few outliers may further refine model accuracy.

Top 5 Hyperparameter Combinations:

Rank 31:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 20, 'n_estimators': 300, 'subsample': 0.8}
```

Mean Test Score: -0.000627

Standard Deviation: 0.000042

Rank 32:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 20, 'n_estimators': 300, 'subsample': 0.9}
```

Mean Test Score: -0.000654

Standard Deviation: 0.000053

Rank 27:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 10, 'n_estimators': 300, 'subsample': 0.8}
```

Mean Test Score: -0.000656
Standard Deviation: 0.000073

Rank 28:

Hyperparameters:
{'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 10, 'n_estimators': 300, 'subsample': 0.9}
Mean Test Score: -0.000664
Standard Deviation: 0.000065

Rank 11:

Hyperparameters:
{'learning_rate': 0.05, 'max_depth': 6, 'min_samples_split': 10, 'n_estimators': 300, 'subsample': 0.8}
Mean Test Score: -0.000665
Standard Deviation: 0.000064

Hyperparameter Tuning: Learning Rate vs n_estimators

Key Observations:

- **Comparison of Different Hyperparameter Combinations:**
 - The heatmap evaluates the effect of varying `learning_rate` and `n_estimators` on model performance.
- **Small Variation in Performance:**
 - The values in the heatmap indicate very small differences in performance across different hyperparameter settings, as all values are close to `-0.001`.
- **Higher `n_estimators` and Learning Rate of `0.1` Performs Best:**
 - The lowest error (lightest yellow shade) occurs at `learning_rate = 0.1` and `n_estimators = 300`, suggesting that this combination may provide the best performance.
- **Darker Colors Indicate Slightly Worse Performance:**
 - The darkest region (`learning_rate = 0.05` and `n_estimators = 100`) represents the highest error, though the differences are minimal.

Interpretation:

- **Higher `n_estimators` Improves Performance:**
 - Increasing the number of estimators to `300` appears to slightly enhance model accuracy.
- **Optimal Learning Rate Appears to be `0.1`:**
 - A higher learning rate (`0.1`) coupled with more estimators yields the best results.
- **Minimal Impact Across Variations:**
 - The relatively small differences across the hyperparameters suggest that the model is not overly sensitive to changes in `learning_rate` and `n_estimators`, but fine-tuning still provides incremental gains.
- **Potential for Further Optimization:**
 - Experimenting with a wider range of values, such as learning rates below `0.05` or more estimators beyond `300`, might help refine performance further.

SHAP Summary Plot

Key Observations:

- **Feature Importance via SHAP Values:**
 - `lag_1` and `rolling_mean` have the highest SHAP values, indicating that they are the most influential features in the model's predictions.
- **SHAP Values Indicate Direction of Impact:**
 - Features with high values (colored red) contribute positively to the model's output.
 - Features with low values (colored blue) contribute negatively to the model's output.
- **Top Features Impacting Predictions:**
 - Apart from `lag_1` and `rolling_mean`, the `5-Year, 5-Year Forward Inflation Expectation Rate` and `lag_2` also show significant contributions.

- **Symmetric Distribution Around Zero:**
 - Most features have a balanced distribution of positive and negative SHAP values, suggesting a well-calibrated model with no extreme biases.
- **Lower Impact Features:**
 - Consumer Price Index, Federal Funds Effective Rate, and Real Gross Domestic Product have minimal influence on the predictions.

Interpretation:

- **Model is Highly Influenced by Recent Historical Values:**
 - The presence of multiple lag features (lag_1, lag_2, lag_3, lag_4, lag_5) suggests that past values strongly determine current predictions.
- **Rolling Mean Adds Stability:**
 - The rolling_mean feature plays a key role, indicating that smoothed historical trends contribute significantly to the model's output.
- **Macroeconomic Indicators Play a Lesser Role:**
 - Features like Consumer Price Index and Unemployment Rate have lower SHAP values, implying that they contribute less to the model's decision-making.

SHAP Force Plot - Feature Contribution to Prediction

Key Observations:

- **Predicted Value ($f(x) = 2.34$):**
 - The final predicted output is 2.34, which is influenced by both positive and negative contributing features.
- **Positive Contributions (Red - Pushing the Prediction Higher):**
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.48
 - rolling_mean = 2.382
 - lag_1 = 2.34
 - These features had a strong positive influence, pushing the prediction value above the base value.
- **Negative Contributions (Blue - Pushing the Prediction Lower):**
 - lag_2 = 2.39
 - lag_4 = 2.42
 - lag_3 = 2.42
 - These features contributed to reducing the prediction but did not dominate.
- **Base Value (2.055):**
 - This is the expected prediction before considering the effects of individual feature values.
- **Interpretation:**
 - The prediction was primarily driven upwards by macroeconomic factors (5-Year Forward Inflation Expectation Rate) and historical lag features (lag_1).
 - Some lag features (lag_2, lag_3, lag_4) acted in opposition but were not strong enough to lower the final prediction significantly.

Conclusion:

- The 5-Year Forward Inflation Expectation Rate and rolling mean are among the most important features driving the prediction.
- The effect of historical lag values varies, with some increasing and others decreasing the predicted value.
- This SHAP force plot helps interpret how individual features influence a specific model prediction.

Partial Dependence Plots (Gradient Boosting)

Key Observations:

- **Understanding Partial Dependence:**
 - Partial dependence plots (PDPs) illustrate the marginal effect of a single feature on the model's predictions.
 - They help understand how changes in a particular feature affect the predicted outcome while keeping other variables constant.
- **Notable Feature Impacts:**

- **5-Year Forward Inflation Expectation Rate:**
 - Shows a **slight positive trend**, indicating that an increase in this economic indicator leads to a higher prediction.
- **Consumer Price Index (CPI):**
 - Has **minimal effect** on the model's predictions, suggesting low importance.
- **Real Gross Domestic Product (GDP):**
 - Also shows **little influence**, implying that this feature may not be strongly correlated with the target variable.
- **Unemployment Rate & Federal Funds Effective Rate:**
 - These features exhibit a **flat response**, indicating little to no direct effect on the predictions.
- **Crude Oil Prices (West Texas Intermediate):**
 - Displays a **slight positive trend**, showing that increasing oil prices have some impact on the predicted values.
- **Time Lag Features (lag_1, lag_2, lag_3, lag_4, lag_5, rolling_mean):**
 - **lag_1** and **rolling_mean** show a **strong increasing trend**, indicating that these lagged features are significant drivers of the prediction.
 - **lag_2** and **lag_3** show a **decreasing trend**, suggesting that higher values may lead to lower predictions.

Conclusion:

- The **most influential factors** in the model appear to be **lag_1**, **rolling_mean**, and **5-Year Forward Inflation Expectation Rate**.
- Some macroeconomic indicators, like **CPI** and **GDP**, seem to have **limited effect**.
- The **trend of lag features** indicates that the model relies heavily on past values to make predictions.

This analysis highlights the key drivers of the model's predictions and provides insight into the relationship between different features and the target variable.

Feature Importance - Gradient Boosting (Built-In)

Key Observations:

- **Top Influential Features:**
 - **lag_1** is the most important feature, significantly contributing to the model's predictions.
 - **rolling_mean** is the second most influential factor, but its impact is notably smaller than lag_1.
 - Other features have **minimal or negligible importance** in the built-in Gradient Boosting model.
- **Economic Indicators and Lag Features:**
 - **5-Year Forward Inflation Expectation Rate**, **Nominal Broad U.S. Dollar Index**, and **10-Year Treasury Yield** appear on the chart, but their importance is close to zero.
 - Other macroeconomic indicators, such as **Consumer Price Index (CPI)**, **Real Gross Domestic Product (GDP)**, and **Unemployment Rate**, have little to no impact.
- **Model's Dependence on Lagged Data:**
 - The dominance of **lag_1** and **rolling_mean** suggests that the model heavily relies on past values rather than external economic indicators.
 - The lack of importance assigned to fundamental macroeconomic features implies that historical trends in the target variable are the primary drivers of prediction accuracy.

Conclusion:

- The model is **strongly dependent on lagged features**, particularly **lag_1**, meaning past values are the best predictors of future values.
- Macroeconomic factors such as **CPI**, **GDP**, and **Unemployment Rate** have **negligible influence** on the predictions.
- **Feature selection could be optimized**, as many features in the dataset do not significantly contribute to model performance.

Permutation Feature Importance - Gradient Boosting

Key Observations:

- **Top Influential Features:**
 - **lag_1** is the most important feature, contributing significantly to the model's predictions.
 - **rolling_mean** follows as the second most influential factor but with lower importance compared to lag_1.
 - **5-Year Forward Inflation Expectation Rate** and **lag_2** have minimal but non-negligible contributions.
- **Negligible Influence of Economic Indicators:**
 - Key macroeconomic indicators such as **10-Year Treasury Yield**, **Nominal Broad U.S. Dollar Index**, **Consumer Price Index**, **Real GDP**, and **Crude Oil Prices** show **little to no importance**.
 - The model does not significantly rely on external economic indicators for its predictions.
- **Dependence on Past Data:**
 - **Lag features (lag_1, rolling_mean, lag_2, etc.)** dominate the feature importance, confirming that past values of the target variable are **highly predictive** of future values.
 - The heavy reliance on lag features suggests the model primarily captures temporal patterns rather than external influences.

Conclusion:

- The model's performance is **driven mainly by lagged values**, particularly **lag_1** and **rolling_mean**.
- **Macroeconomic indicators have minimal relevance**, implying that historical trends in the target variable are more predictive than external economic factors.
- **Permutation importance analysis** confirms that removing lagged values would **significantly impact model performance**, while removing economic indicators would have little to no effect.

Extremely Randomized Trees Regressor (ERTR)

Extremely Randomized Trees Regressor (Extra Trees): Model Development, Hyperparameter Tuning, and Performance Evaluation

1. Importing Essential Libraries

- **Random & NumPy:** Used for setting random seeds to ensure reproducibility.
- **Warnings:** Suppresses unnecessary runtime warnings for cleaner execution.
- **Scikit-learn:** Provides tools for data splitting, hyperparameter tuning, and model implementation.

2. Data Preparation

- The dataset is split into:
 - **Training Set (80%):** Used to train the model.
 - **Testing Set (20%):** Used for performance evaluation.
- The split is controlled using `random_state=42` to ensure consistency across runs.

3. Model Initialization

- **ExtraTrees Regressor (ETR)** is instantiated.
- ETR is an ensemble method that builds multiple **decision trees** while introducing extra randomness during feature splits.
- The model is set with `random_state=42` for deterministic behavior.

4. Hyperparameter Tuning using GridSearchCV

- An exhaustive **grid search** strategy is employed for hyperparameter optimization:
 - `n_estimators`: Number of trees in the ensemble.
 - `max_depth`: Maximum depth of individual trees.
 - `min_samples_split`: Minimum samples required for a node split.
 - `min_samples_leaf`: Minimum samples required at a leaf node.
 - `bootstrap`: Whether bootstrapping is applied.

- **5-Fold Cross-Validation** ensures robust performance assessment.
- **Parallel processing** (`n_jobs=-1`) speeds up the hyperparameter search.

5. Model Training and Selection of Optimal Parameters

- **GridSearchCV** trains multiple models with different hyperparameter combinations.
- The best-performing model is selected based on **Negative Mean Squared Error (MSE)**.
- The optimal hyperparameters are retrieved and displayed.

6. Model Prediction and Performance Evaluation

- The optimized model is used to generate predictions on the **test dataset**.
- A predefined function `calculate_metrics` computes performance scores.
- The model is benchmarked against a **naïve forecasting method** for comparison.

7. Feature Importance Analysis

- **Feature importance scores** are extracted from the best ExtraTrees model.
- A **DataFrame** is created to store and sort feature importance values.

Feature Importance Visualization

- A **horizontal bar chart** presents the ranked feature importances.
- Helps identify the most influential predictors in the dataset.

8. Model Performance: Actual vs. Predicted Values

- A **scatter plot** visualizes the correlation between actual and predicted values.
- A **perfect fit reference line** is included to assess model accuracy.

9. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line** ($y=0$) indicates zero residuals for easier analysis.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line** ($x=0$) is used to check for normality in residuals.

10. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Heatmap

- A **heatmap** visualizes the impact of `max_depth` and `n_estimators` on model performance.
- Darker colors indicate **higher performance**.

11. SHAP Analysis for Model Interpretability

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes feature contributions for a specific instance.

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

13. Built-in vs. Permutation Feature Importance

Built-in Feature Importance

- Extracted directly from the trained ExtraTrees model.
- Displayed using a **bar chart**.

Permutation Feature Importance

- Computed by **randomly shuffling feature values** and measuring performance impact.
- Provides a more robust estimate of feature importance.
- **Visualization:** A bar chart ranks feature importance based on permutation scores.

Conclusion

This notebook systematically implements and evaluates an **Extremely Randomized Trees Regressor (ETR)** for predictive modeling. Key contributions include:

1. **Robust hyperparameter tuning** with **GridSearchCV** and cross-validation.
2. **Comprehensive model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via built-in, permutation, and SHAP analyses.
4. **Model interpretability** through SHAP and Partial Dependence Plots.

Result Output:

Fitting 5 folds for each of 162 candidates, totalling 810 fits

Best Hyperparameters:

```
'bootstrap': False,
'max_depth': 10,
'min_samples_leaf': 1,
'min_samples_split': 2,
'n_estimators': 500
```

ExtraTrees Regressor Evaluation Metrics:

```
'Model': 'Extremely Randomized Trees Regressor (ERTR)',
'MSE': 0.0007227462690668256,
'RMSE': 0.026883940728003878,
'MAE': 0.016370082040005472,
'MAPE (%)': 1.1514243212219075,
'sMAPE (%)': 1.0918019278863444,
'MASE': 0.041670960046053356,
'R^2': 0.9959126220643945
```

Feature Importances:

Feature Index	Feature	Importance
14	rolling_mean	0.192706
10	lag_2	0.172242
9	lag_1	0.167844
12	lag_4	0.151660
13	lag_5	0.147841
11	lag_3	0.136892
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.022326

7	Crude Oil Prices: West Texas Intermediate	0.007055
8	10-Year Treasury Yield	0.000301
5	Personal Consumption Expenditures: Chain-type ...	0.000261
3	Unemployment Rate	0.000239
1	Consumer Price Index	0.000184
6	Nominal Broad U.S. Dollar Index	0.000171
4	Federal Funds Effective Rate	0.000160
2	Real Gross Domestic Product	0.000118

Feature Importance (Extra Trees)

Key Observations:

- **Top Influential Features:**
 - **rolling_mean** is the most important feature, indicating that the moving average of past values significantly impacts the predictions.
 - **lag_2** and **lag_1** are the second and third most influential features, emphasizing the model's reliance on past observations.
 - **lag_4, lag_5, and lag_3** also contribute considerably, further supporting the temporal dependency in the model.
- **Limited Influence of External Economic Indicators:**
 - Macroeconomic indicators such as **Crude Oil Prices, 10-Year Treasury Yield, Unemployment Rate, Consumer Price Index, and Real GDP** have **minimal importance**.
 - This suggests that the model does not significantly incorporate broader economic trends but instead relies heavily on historical values of the target variable.
- **Temporal Dependency in Predictions:**
 - The strong importance of **lag features (lag_1 to lag_5)** and **rolling_mean** suggests that the model performs well by capturing past trends.
 - The lack of emphasis on external macroeconomic indicators indicates that the model is **more autoregressive** rather than influenced by economic events.

Conclusion:

- The **Extra Trees model** prioritizes past values and trends (**rolling_mean and lags**) for prediction, rather than relying on macroeconomic indicators.
- The minimal contribution of external factors implies that historical time-series patterns are more predictive than broader economic variables.
- This feature importance ranking reinforces the model's **time-dependent nature**, making it more suited for forecasting applications where past trends play a dominant role.

Actual vs Predicted Values (Extra Trees)

Key Observations:

- **Strong Agreement Between Predictions and Actual Values:**
 - The blue dots (predictions) align closely with the red dashed line, which represents a **perfect fit** (i.e., actual values = predicted values).
 - This suggests that the **Extra Trees model** provides highly accurate predictions.
- **Minimal Deviation from the Perfect Fit Line:**
 - Most points fall along the red dashed line, indicating a **low error rate**.
 - There are only a few slight deviations, mostly for lower values, implying small residual errors.
- **Performance Across the Range of Values:**
 - The model maintains **consistency** in predictions across the full range of actual values.
 - There is **no visible pattern of bias**, meaning the model does not systematically overpredict or underpredict in any specific range.

Conclusion:

- The **Extra Trees model** performs exceptionally well in predicting the target variable.
- The predictions **closely match the actual values**, suggesting that the model has effectively captured the underlying patterns in the data.
- The small deviations at lower values indicate slight prediction variance, but overall, the model shows **strong generalization ability**.

Residual Plot (Extra Trees)

Key Observations:

- **Centered Around Zero:**
 - The residuals (difference between actual and predicted values) are **evenly distributed** around the zero line, suggesting that the Extra Trees model does not exhibit systematic bias.
- **Randomly Scattered Residuals:**
 - The residuals do not show any clear pattern or trend, confirming that the model's errors are **randomly distributed**.
 - This randomness indicates that the model captures the data well and does not suffer from underfitting or overfitting.
- **Variance Across Actual Values:**
 - The spread of residuals remains relatively **consistent across the range of actual values**, meaning the model's prediction errors do not significantly vary for different target values.
 - However, there are a few **outliers** (larger residuals), particularly at lower values, which might indicate areas where the model struggles slightly.

Conclusion:

- The **Extra Trees model** performs well, as the residuals are **randomly distributed** around zero, indicating no major bias.
- The model does not systematically underpredict or overpredict at any specific range of actual values.
- The presence of **a few outliers** suggests areas where further tuning or additional features could improve predictions.

Residual Distribution (Extra Trees)

Key Observations:

- **Centered Around Zero:**
 - The residuals (errors) are symmetrically distributed around **zero**, indicating that the Extra Trees model does not have a strong bias in overpredicting or underpredicting.
- **Normal-Like Distribution:**
 - The histogram resembles a **bell-shaped curve**, suggesting that the residuals follow a normal distribution.
 - This is a good indication that the model's errors are **random** rather than systematic.
- **Small Residual Spread:**
 - The residual values are **mostly concentrated near zero**, with a small number of residuals extending to the left and right.
 - This suggests that the model's predictions are **accurate**, with only a few large errors.
- **Outliers Present:**
 - While most residuals are close to zero, a few **outliers** exist in the left and right tails, which could indicate specific instances where the model performs less accurately.

Conclusion:

- The Extra Trees model appears to have **good predictive accuracy**, as residuals are centered around zero and normally distributed.
- The narrow spread indicates that the model is **consistent in its predictions**, with few large deviations.
- Some **outliers exist**, which could be further investigated for potential model improvements.

Top 5 Hyperparameter Combinations:

Rank 138:

Hyperparameters:

```
{'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 500}
```

Mean Test Score: -0.000787

Standard Deviation: 0.000100

Rank 137:

Hyperparameters:

```
{'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
```

Mean Test Score: -0.000791

Standard Deviation: 0.000098

Rank 136:

Hyperparameters:

```
{'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

Mean Test Score: -0.000792

Standard Deviation: 0.000098

Rank 139:

Hyperparameters:

```
{'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100}
```

Mean Test Score: -0.000792

Standard Deviation: 0.000096

Rank 141:

Hyperparameters:

```
{'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 500}
```

Mean Test Score: -0.000792

Standard Deviation: 0.000102

Effect of max_depth and n_estimators on Model Performance

Key Observations:

- **Lower RMSE with Higher max_depth :**
 - The model's performance improves (lower RMSE) as max_depth increases from 3 to 10.
 - Shallower trees (max_depth = 3) have a higher RMSE, indicating underfitting.
- **Impact of n_estimators is Minimal:**
 - Increasing n_estimators from 100 to 500 does not significantly affect performance across different max_depth values.
 - This suggests that the model reaches **saturation** quickly, and additional estimators do not significantly improve accuracy.
- **Best Configuration:**
 - The best-performing configurations appear at **higher max_depth values (5 or 10)**.
 - These settings consistently produce the lowest RMSE values (-0.001), indicating the best balance of bias and variance.

Conclusion:

- Deeper trees (higher `max_depth`) improve model accuracy, reducing error.
- Adding more estimators (`n_estimators`) has a negligible impact on further improving performance.
- A balance must be maintained: While deeper trees perform better, excessive depth may lead to overfitting in other cases.

SHAP Summary Plot: Feature Importance and Impact on Model Output

Key Observations:

- **Most Influential Features:**
 - The top three features contributing to the model's output are:
 1. `rolling_mean`
 2. `lag_1`
 3. `lag_2`
 - These features have the largest spread of SHAP values, indicating their strong influence on predictions.
- **Feature Importance Representation:**
 - Features are ranked in descending order based on their average absolute SHAP value.
 - The `rolling_mean` and `lag_1` features have the highest impact on predictions.
- **SHAP Value Distribution:**
 - **Positive SHAP values (right side)** indicate an increase in the model's predicted value.
 - **Negative SHAP values (left side)** indicate a decrease in the model's predicted value.
- **Color Representation:**
 - **Redder (high feature values)** contribute more positively to the prediction.
 - **Bluer (low feature values)** contribute more negatively to the prediction.
- **Non-Lag Features Impact:**
 - While lag-based features (`lag_1`, `lag_2`, etc.) dominate, economic indicators such as:
 - 5-Year, 5-Year Forward Inflation Expectation Rate
 - Crude Oil Prices: West Texas Intermediate
 - Federal Funds Effective Rate
 - These still play a role but with lower impact.

Conclusion:

- The model primarily relies on lagged values and moving averages to make predictions.
- Macroeconomic variables, while included, have a **relatively minor impact** compared to past values of the target variable.
- This analysis suggests that short-term historical trends are the most influential in shaping predictions.

SHAP Waterfall Plot: Feature Contributions to a Single Prediction

Key Observations:

- **Base Value (2.056):**
 - This represents the starting prediction before specific feature contributions are considered.
- **Final Prediction (2.21):**
 - The final model prediction after accounting for feature impacts.
- **Positive Contributions:**
 - Features that **increase the prediction value** are shown in red.
 - The most impactful contributors are:
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.47
 - `lag_3` = 2.2
 - `lag_2` = 2.17
 - `rolling_mean` = 2.208
 - `lag_4` = 2.23
 - `lag_5` = 2.23

- `lag_1 = 2.2`
- These features collectively push the prediction upward from **2.056** to **2.21**.
- **No Negative Contributions:**
 - Unlike some SHAP plots where certain features pull the prediction lower (shown in blue), this case shows that all contributing factors **increase** the final prediction.
- **Interpretation:**
 - The model relies heavily on **lagged values (lag_1 to lag_5)** and **rolling means**, suggesting that **past trends strongly influence the prediction**.
 - **Economic indicators**, such as the **5-Year Forward Inflation Expectation Rate**, also contribute significantly.

Conclusion:

- The **prediction of 2.21** is primarily driven by past values and macroeconomic factors.
- The **SHAP waterfall plot effectively breaks down how each feature moves the prediction from the base value to the final result**.
- This type of analysis is **useful for understanding individual predictions** and diagnosing model behavior on specific inputs.

Partial Dependence Plots (ExtraTrees)

Key Observations:

- **Understanding Partial Dependence Plots (PDPs):**
 - These plots illustrate the **marginal effect of individual features** on the model's predictions while keeping other features constant.
- **Features with Strong Positive Relationships:**
 - `lag_1`, `lag_2`, `rolling_mean`, `lag_3`, `lag_4`:
 - These features show a **strong upward trend**, indicating that higher values lead to **higher predictions**.
 - Suggests that **recent past values significantly influence future predictions**.
- **Features with Minimal or No Impact:**
 - `Consumer Price Index`, `Real Gross Domestic Product`, `Unemployment Rate`, `Federal Funds Effective Rate`, `Crude Oil Prices: West Texas Intermediate`:
 - The flat lines suggest **no significant relationship** with the model's predictions.
 - These macroeconomic factors seem **less influential** compared to lag features.
- **5-Year, 5-Year Forward Inflation Expectation Rate:**
 - Shows a **slight upward trend**, indicating a minor positive influence on predictions.
- **Interpretation:**
 - The model is **highly dependent on lagged features (lag_1, lag_2, rolling_mean)**, meaning historical data plays a critical role in forecasting.
 - Macroeconomic indicators **do not have a strong influence** in this model.
 - **Rolling mean** shows a significant upward effect, indicating that smoothed past values strongly contribute to future predictions.

Conclusion:

- **ExtraTrees regression model** relies predominantly on **time-series lag features** rather than macroeconomic indicators.
- **Predictive power is largely driven by recent trends** rather than external economic factors.
- This insight helps in **feature selection and model tuning** to optimize performance.

Permutation Feature Importance (Extra Trees)

Key Observations:

- **Understanding Permutation Importance:**
 - This plot shows the **drop in model performance** when each feature's values are shuffled.
 - A **higher importance score** means the feature is **more critical** to the model's predictions.

- **Most Important Features:**
 - `lag_1` and `rolling_mean` are the **most significant features**, indicating that **recent historical values and moving averages** play a crucial role in predictions.
 - `lag_2` also holds notable importance, reinforcing the idea that short-term lagged values are essential.
- **Moderately Important Features:**
 - `lag_3`, `lag_4`, and `lag_5` show lower but still meaningful importance, suggesting that **longer-term past values still contribute to the prediction**.
- **Least Important Features:**
 - **Macroeconomic indicators** such as:
 - 5-Year, 5-Year Forward Inflation Expectation Rate
 - Crude Oil Prices: West Texas Intermediate
 - Federal Funds Effective Rate
 - 10-Year Treasury Yield
 - Real Gross Domestic Product
 - Unemployment Rate
 - Consumer Price Index
 - These have almost **zero permutation importance**, indicating they **do not significantly impact predictions**.

Interpretation:

- The **Extra Trees model is highly dependent on time-series lag features**, particularly short-term (`lag_1`, `lag_2`) and smoothed trends (`rolling_mean`).
- **Macroeconomic indicators have negligible importance**, meaning the model does not rely on external economic data for predictions.
- The feature importance rankings provide guidance for **feature selection**, potentially allowing the removal of less impactful variables to improve model efficiency.

Conclusion:

- **Lagged values and rolling averages are the primary drivers** of the model's predictive power.
- **Macroeconomic variables do not contribute significantly**, and their removal may not affect performance.
- This analysis helps in **refining feature selection and improving model interpretability**.

Histogram-Based Gradient Boosting Regressor (HGBR)

Histogram-Based Gradient Boosting Regressor (HGBR): Model Development, Hyperparameter Tuning, and Performance Evaluation

1. Importing Essential Libraries

- **Random & NumPy:** Used for setting random seeds to ensure reproducibility.
- **Pandas:** Used for handling tabular data.
- **Scikit-learn:** Provides tools for data preprocessing, hyperparameter tuning, and model implementation.

2. Data Preparation

- The dataset is split into:
 - **Training Set (80%):** Used to train the model.
 - **Testing Set (20%):** Used for model evaluation.
- The split is controlled using `random_state=42` to ensure consistency across runs.

3. Model Initialization

- **Histogram-Based Gradient Boosting Regressor (HGBR)** is instantiated.
- HGBR is a fast, histogram-based gradient boosting method optimized for large datasets.
- The model is set with `random_state=42` for deterministic behavior.

4. Hyperparameter Tuning using GridSearchCV

- An exhaustive **grid search** strategy is employed for hyperparameter optimization:
 - `max_iter`: Number of boosting iterations.
 - `learning_rate`: Step size for weight updates.
 - `max_depth`: Maximum depth of individual trees.
 - `min_samples_leaf`: Minimum samples required per leaf node.
 - `max_bins`: Number of bins for feature discretization.
- **5-Fold Cross-Validation** ensures robust performance assessment.
- **Parallel processing** (`n_jobs=-1`) speeds up the hyperparameter search.

5. Model Training and Selection of Optimal Parameters

- GridSearchCV **trains multiple models** with different hyperparameter combinations.
- The best-performing model is selected based on **Negative Mean Squared Error (MSE)**.
- The optimal hyperparameters are retrieved and displayed.

6. Model Prediction and Performance Evaluation

- The optimized model is used to generate predictions on the **test dataset**.
- A predefined function `calculate_metrics` computes performance scores.
- The model is benchmarked against a **naïve forecasting method** for comparison.

7. Model Performance: Actual vs. Predicted Values

- A **scatter plot** visualizes the correlation between actual and predicted values.
- A **perfect fit reference line** is included to assess model accuracy.

8. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line** ($y=0$) indicates zero residuals for easier analysis.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line** ($x=0$) is used to check for normality in residuals.

9. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Heatmap

- A **heatmap** visualizes the impact of `max_depth` and `learning_rate` on model performance.
- Darker colors indicate **higher performance**.

10. SHAP Analysis for Model Interpretability

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes feature contributions for a specific instance.

11. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

12. Permutation Feature Importance

- Computed by **randomly shuffling feature values** and measuring performance impact.
 - Provides a more robust estimate of feature importance.
 - **Visualization:** A bar chart ranks feature importance based on permutation scores.
-

Conclusion

This notebook systematically implements and evaluates a **Histogram-Based Gradient Boosting Regressor (HGBR)** for predictive modeling. Key contributions include:

1. **Robust hyperparameter tuning** with **GridSearchCV** and cross-validation.
2. **Comprehensive model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via SHAP and permutation analyses.
4. **Model interpretability** through SHAP and Partial Dependence Plots.

Result Output:

Fitting 5 folds for each of 243 candidates, totalling 1215 fits

Best Hyperparameters:

```
'learning_rate': 0.1,  
'max_bins': 255,  
'max_depth': 7,  
'max_iter': 300,  
'min_samples_leaf': 10
```

HGBR Evaluation Metrics:

```
'Model': 'Histogram-Based Gradient Boosting Regressor (HGBR)',  
'MSE': 0.0005834295749823299,  
'RMSE': 0.024154286886230565,  
'MAE': 0.01484038473999798,  
'MAPE (%)': 0.9645516085929583,  
'SMAPE (%)': 0.9656255050663115,  
'MASE': 0.03777702995362074,  
'R^2': 0.9967005057323347
```

Actual vs Predicted Values (HistGradientBoostingRegressor)

Key Observations:

- **Plot Interpretation:**
 - The scatter plot compares **actual values** (x-axis) with **predicted values** (y-axis).
 - The **red dashed line** represents the **perfect fit** (i.e., when actual = predicted).
 - Each **blue dot** represents a single prediction.
- **Model Performance:**
 - The data points are **closely aligned** along the red dashed line, indicating that the **model predictions are highly accurate**.
 - **Minimal deviation** from the line suggests that the model does not suffer from significant bias or variance.
 - There are **some minor outliers**, particularly at lower values, where predictions show slight deviations.
- **Error Analysis:**

- The spread of points around the perfect fit line is **small**, indicating **low residual errors**.
- Outliers and slight deviations may indicate areas where the model's performance is **less accurate**.

Interpretation:

- The **HistGradientBoostingRegressor** model demonstrates **strong predictive power**, as seen by the close alignment of predictions with actual values.
- The presence of **some outliers** suggests possible areas for model fine-tuning, such as feature engineering or hyperparameter optimization.

Conclusion:

- **The model performs well**, with **low error and high predictive accuracy**.
- **Further improvements** could be achieved by investigating residual patterns and refining feature selection or hyperparameters.

Residual Plot (HistGradientBoostingRegressor)

Key Observations:

- **Plot Interpretation:**
 - The scatter plot visualizes **residuals** (y-axis) against **actual values** (x-axis).
 - The **red dashed line at $y = 0$** represents the ideal case where residuals are zero, meaning perfect predictions.
 - Each **green dot** represents the residual (error) for a single prediction.
- **Model Performance:**
 - **Most residuals are centered around zero**, indicating that the model **does not have a systematic bias**.
 - There is **no clear pattern**, suggesting that the model captures the relationships in the data well.
 - The **spread of residuals is relatively small**, meaning the model has **low variance** and is making consistent predictions.
- **Error Analysis:**
 - Some **outliers** (points with higher residuals) indicate **instances where the model made relatively larger errors**.
 - The **variance of residuals remains fairly constant** across different actual values, which suggests **homoscedasticity** (good for model reliability).

Interpretation:

- The **HistGradientBoostingRegressor** model appears to perform **well**, with **most residuals near zero** and no significant pattern of errors.
- The presence of some **outliers** suggests that certain data points may be harder to predict, possibly due to noise in the dataset or missing features.

Conclusion:

- **The residuals are well-distributed**, indicating a **good model fit**.
- **Further model tuning** (e.g., handling outliers, feature engineering) might help reduce the spread of residuals and improve overall performance.

Residual Distribution (HistGradientBoostingRegressor)

Key Observations:

- **Histogram Interpretation:**
 - The histogram represents the **distribution of residuals** (differences between actual and predicted values).
 - The **x-axis** represents residual values, while the **y-axis** represents their frequency.
 - The **red dashed line at zero** represents the ideal case where residuals are centered around zero.

- **Model Performance:**
 - The majority of residuals are clustered around zero, indicating accurate predictions.
 - The distribution is approximately symmetric, suggesting no systematic bias in the model's predictions.
 - The spread of residuals is small, meaning low variance in the model's predictions.
- **Error Analysis:**
 - The absence of long tails or extreme outliers indicates that the model does not frequently make large errors.
 - The residuals form a bell-shaped curve, resembling a normal distribution, which is a sign of a well-fitted model.

Interpretation:

- The **HistGradientBoostingRegressor** appears to be making consistent and accurate predictions.
- The residuals being centered around zero and following a normal distribution indicate good model calibration.
- Further improvements could involve refining hyperparameters or feature selection to further reduce small errors.

Conclusion:

- The model performs well, with residuals tightly centered around zero.
- The histogram suggests no major systematic errors or bias, making the model suitable for reliable predictions.

Top 5 Hyperparameter Combinations:

Rank 187:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7, 'max_iter': 300, 'min_samples_leaf': 10}
```

Mean Test Score: -0.000682

Standard Deviation: 0.000105

Rank 188:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7, 'max_iter': 300, 'min_samples_leaf': 20}
```

Mean Test Score: -0.000703

Standard Deviation: 0.000078

Rank 184:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7, 'max_iter': 200, 'min_samples_leaf': 10}
```

Mean Test Score: -0.000706

Standard Deviation: 0.000104

Rank 178:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 5, 'max_iter': 300, 'min_samples_leaf': 10}
```

Mean Test Score: -0.000714

Standard Deviation: 0.000082

Rank 241:

Hyperparameters:

```
{'learning_rate': 0.1, 'max_bins': 100, 'max_depth': 7, 'max_iter': 300, 'min_samples_leaf': 10}
```

Mean Test Score: -0.000716

Standard Deviation: 0.000107

Effect of `max_depth` and `learning_rate` on Model Performance

Key Observations:

- **Heatmap Interpretation:**
 - The heatmap visualizes the **interaction between `max_depth` and `learning_rate`** on model performance.
 - The **x-axis represents the learning rate**, and the **y-axis represents the max depth**.
 - The **color scale** indicates the **model error**, where **darker shades represent higher error** and **lighter shades represent lower error**.
- **Impact of Learning Rate:**
 - At a **low learning rate (0.01)**, model error is relatively high (darker shade).
 - Increasing the learning rate to **0.05 or 0.1 significantly reduces error**, as shown by the lighter shades.
 - **Optimal learning rate** appears to be **0.05 or 0.1**, where model performance is best.
- **Impact of Max Depth:**
 - Increasing `max_depth` from **3 to 7** has **minimal impact on error at higher learning rates**.
 - However, at **lower learning rates (0.01)**, a shallower tree (`max_depth = 3`) results in the **highest error**.
 - This suggests that **deeper trees are more beneficial when learning rates are low**.
- **Best Configuration:**
 - The **lowest error (best performance)** is achieved at a **learning rate of 0.05 or 0.1**, regardless of `max_depth`.
 - The **worst performance** occurs at **low learning rates with shallow trees (`max_depth = 3`)**.

Interpretation:

- A **low learning rate (0.01)** leads to **higher error**, especially when combined with shallow trees.
- **Optimal performance** is observed at **higher learning rates (0.05 or 0.1)** across all `max_depth` values.
- **Increasing max depth alone does not significantly improve performance** unless paired with an appropriate learning rate.

Conclusion:

- A **learning rate of 0.05 or 0.1** is recommended for the best model performance.
- **Avoid using a low learning rate (0.01) with shallow trees (`max_depth = 3`)** to prevent high error.
- **Further fine-tuning** of `n_estimators` and other hyperparameters may further optimize model accuracy.

SHAP Summary Plot: Feature Contributions to Model Predictions

Key Observations:

- **SHAP (SHapley Additive exPlanations) values** measure each feature's impact on the model's output.
- **X-axis:** SHAP values (how much each feature pushes the prediction higher or lower).
- **Y-axis:** Features ranked by importance, with the most impactful features at the top.
- **Color Gradient:**
 - **Red (High feature values):** Indicates that a high value for this feature **increases the prediction**.
 - **Blue (Low feature values):** Indicates that a low value for this feature **decreases the prediction**.

Feature Importance:

1. `lag_1`:

- The most important feature influencing predictions.
- High values (red) generally increase predictions, while low values (blue) decrease them.

2. **rolling_mean** :
 - Another highly impactful feature.
 - Higher values tend to increase predictions.
3. **5-Year, 5-Year Forward Inflation Expectation Rate** :
 - Important economic indicator contributing significantly to predictions.
4. **Lags (lag_2 , lag_3 , lag_4 , lag_5)**:
 - Past values of the target variable impact future predictions.
 - Higher past values (red) generally push predictions higher.
5. **Macroeconomic Indicators**:
 - **10-Year Treasury Yield** , **Personal Consumption Expenditures** , **Unemployment Rate** , **Crude Oil Prices** , **Federal Funds Effective Rate** , and **Real GDP** have lower impacts but still contribute to model outputs.

Interpretation:

- Time series lag features (lag_1 , lag_2 , rolling_mean) dominate the model predictions.
- Macroeconomic variables have less influence but still play a role in refining predictions.
- Higher values of key features generally push predictions higher, while lower values reduce predictions.

Conclusion:

- The model relies primarily on past values (lag_1 , lag_2 , rolling_mean) to make predictions.
- Economic indicators contribute but do not dominate predictions.
- Feature interactions could be further explored to refine the model.

SHAP Force Plot: Feature Contributions to a Single Prediction

Key Observations:

- This **SHAP force plot** explains how different features **pushed the model's prediction** higher or lower for a specific instance.
- **X-axis**: The final prediction value ($f(x) = 2.23$).
- **Base Value**: The model's expected prediction before applying feature effects (≈ 2.056).
- **Feature Contributions**:
 - **Features pushing the prediction higher (red)**:
 - **lag_1 = 2.2** : Strongest positive impact.
 - **rolling_mean = 2.208** : Significant influence in raising the prediction.
 - **5-Year, 5-Year Forward Inflation Expectation Rate = 2.47** : Contributes positively.
 - **Crude Oil Prices: West Texas Intermediate = 106.7** : Also increases the prediction.
 - **10-Year Treasury Yield = 2.03** : Smaller but noticeable positive contribution.
 - **Features pushing the prediction lower (blue)**:
 - Minimal effect, suggesting all key variables in this instance increased the prediction.

Interpretation:

- The final prediction (2.23) is higher than the base value (2.056) because multiple key features exerted an upward influence.
- Time-series features (lag_1 , rolling_mean) dominate the prediction, indicating strong temporal dependencies.
- Macroeconomic indicators (Inflation Expectation Rate , Oil Prices , Treasury Yield) also played a role but had less impact compared to time-series features.
- No major negative contributors, meaning all important features in this instance aligned to push the prediction upwards.

Conclusion:

- The model relies primarily on past values (lag_1 , rolling_mean) but also considers macroeconomic factors.
- This instance demonstrates how key economic indicators interact with time-series data to determine the final prediction.

- Further analysis could explore feature interactions and their relative importance across multiple instances.

Partial Dependence Plots (HistGradientBoostingRegressor)

Key Observations:

- These plots show how individual features impact the model's predictions while keeping other features constant.
- Each subplot represents one feature on the x-axis, with the model's partial dependence (predicted output) on the y-axis.

Feature Effects:

1. 5-Year, 5-Year Forward Inflation Expectation Rate:

- A slight **positive relationship** with the model's predictions.
- As inflation expectations increase, the predicted value also increases.

2. Consumer Price Index (CPI):

- **Minimal impact** on predictions, suggesting it is not a strong predictor in this model.

3. Real Gross Domestic Product (GDP):

- Almost **flat**, indicating little influence on the model's output.

4. Unemployment Rate:

- Also shows **negligible impact**, meaning changes in unemployment do not significantly affect predictions.

5. Federal Funds Effective Rate:

- **Flat curve**, indicating no strong influence.

6. Personal Consumption Expenditures (PCE) - Chain-type Price Index:

- **Minimal effect**, similar to CPI.

7. Nominal Broad U.S. Dollar Index:

- No significant variation, suggesting little impact.

8. Crude Oil Prices (West Texas Intermediate):

- Almost flat, implying oil prices do not significantly impact predictions.

Time-Series Features:

9. lag_1 (Previous value of the target variable):

- **Strong increasing trend:** Higher past values lead to higher predictions.
- Indicates that **past values are crucial in forecasting future values**.

10. lag_2, lag_3, lag_4, lag_5 (Earlier past values):

- **Mixed effects:**
 - lag_2 shows a **positive relationship**, but less steep than lag_1.
 - lag_3 shows a **slight negative effect**.
 - lag_4 and lag_5 are **relatively flat**, indicating weaker influence.

11. Rolling Mean:

- **Strong positive trend**, indicating that the moving average of past values significantly impacts predictions.

Conclusion:

- The model heavily relies on time-series features (**lag_1**, **rolling_mean**) to make predictions.
- Macroeconomic indicators (CPI, GDP, Unemployment, Oil Prices, Treasury Yield) have minimal influence.
- Feature importance suggests that short-term past values (lag features) are more predictive than long-term trends.

Implications:

- Since the model relies mainly on recent past values, it behaves like a time-series forecasting

model rather than an econometric model.

- Future improvements could explore adding interaction effects between time-series and economic features for better performance.

Permutation Feature Importance (HistGradientBoostingRegressor)

Key Observations:

- This plot represents the importance of different features in the model's predictions based on permutation importance.
- Permutation importance measures how much the model's performance decreases when a feature's values are randomly shuffled.
- The x-axis represents **Permutation Importance**, with higher values indicating more significant impact.

Feature Importance Insights:

1. lag_1 (Previous Target Value)

- **Most important feature** in the model.
- **Strong influence** on predictions, indicating that the immediate past value is the best predictor for the current value.

2. rolling_mean (Moving Average of Past Values)

- The **second most important feature**.
- This suggests that the trend from recent values plays a major role in prediction.

3. 5-Year, 5-Year Forward Inflation Expectation Rate

- Has some contribution but significantly lower than lag features.
- Indicates that inflation expectations may have a minor role in forecasting.

4. Other Lag Features (lag_2, lag_3, lag_4, lag_5)

- Have minimal influence, meaning that **more distant past values are less important compared to immediate past values**.

5. Macroeconomic Variables (GDP, CPI, Unemployment, Treasury Yield, etc.)

- **Almost no impact on model predictions**.
- Suggests that this model is primarily driven by past time-series values rather than economic indicators.

Conclusion:

- The model heavily relies on recent past values (`lag_1` and `rolling_mean`) to generate predictions.
- Macroeconomic indicators have little to no effect, suggesting the model functions more like a traditional time-series forecasting model rather than an econometric model.
- Future improvements could explore interactions between economic factors and time-series lags to better capture external influences.

Categorical Boosting (CatBoost)

CatBoost Regressor: Model Development, Hyperparameter Tuning, and Performance Evaluation

1. Importing Essential Libraries

- **Random & NumPy**: Used for setting random seeds to ensure reproducibility.
- **Pandas**: Used for handling tabular data.
- **Scikit-learn**: Provides tools for data preprocessing, hyperparameter tuning, and model evaluation.
- **CatBoost**: Implements the **Categorical Boosting (CatBoost) Regressor**, an optimized gradient boosting algorithm.

2. Data Preparation

- The dataset is split into:
 - **Training Set (80%)**: Used to train the model.
 - **Testing Set (20%)**: Used for model evaluation.
- The split is controlled using `random_state=42` to ensure consistency across runs.

3. Model Initialization

- **CatBoost Regressor (CatBoost)** is instantiated.
- CatBoost is particularly **efficient for handling categorical features** and is optimized for **fast training**.
- The model is set with `random_state=42` for deterministic behavior.

4. Hyperparameter Tuning using GridSearchCV

- An exhaustive **grid search** strategy is employed for hyperparameter optimization:
 - `iterations`: Number of boosting iterations.
 - `learning_rate`: Step size for weight updates.
 - `depth`: Maximum depth of individual trees.
 - `early_stopping_rounds`: Prevents overfitting by stopping training when performance stagnates.
- **5-Fold Cross-Validation** ensures robust performance assessment.
- **Parallel processing (n_jobs=-1)** speeds up the hyperparameter search.

5. Model Training and Selection of Optimal Parameters

- GridSearchCV **trains multiple models** with different hyperparameter combinations.
- The best-performing model is selected based on **Negative Mean Squared Error (MSE)**.
- The optimal hyperparameters are retrieved and displayed.

6. Model Prediction and Performance Evaluation

- The optimized model is used to generate predictions on the **test dataset**.
- A predefined function `calculate_metrics` computes performance scores.
- The model is benchmarked against a **naïve forecasting method** for comparison.

7. Feature Importance Analysis

- **Feature importance scores** are extracted using CatBoost's built-in `get_feature_importance()` method.
- A **DataFrame** is created to store and sort feature importance values.

Feature Importance Visualization

- A **horizontal bar chart** presents the ranked feature importances.
- Helps identify the most influential predictors in the dataset.

8. Model Performance: Actual vs. Predicted Values

- A **scatter plot** visualizes the correlation between actual and predicted values.
- A **perfect fit reference line** is included to assess model accuracy.

9. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line (y=0)** indicates zero residuals for easier analysis.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line (x=0)** is used to check for normality in residuals.

10. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Heatmap

- A **heatmap** visualizes the impact of `iterations` and `learning_rate` on model performance.
- Darker colors indicate **higher performance**.

11. SHAP Analysis for Model Interpretability

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes feature contributions for a specific instance.

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

13. Permutation Feature Importance

- Computed by **randomly shuffling feature values** and measuring performance impact.
- Provides a more robust estimate of feature importance.
- **Visualization:** A bar chart ranks feature importance based on permutation scores.

Conclusion

This notebook systematically implements and evaluates a **CatBoost Regressor** for predictive modeling. Key contributions include:

1. **Robust hyperparameter tuning** with **GridSearchCV** and cross-validation.
2. **Comprehensive model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via SHAP and permutation analyses.
4. **Model interpretability** through SHAP and Partial Dependence Plots.

Output Result:

Fitting 5 folds for each of 54 candidates, totalling 270 fits

Best Hyperparameters:

```
'depth': 6,  
'early_stopping_rounds': 50,  
'iterations': 1000,  
'learning_rate': 0.1
```

CatBoost Evaluation Metrics:

```
'Model': 'Categorical Boosting (CatBoost)',  
'MSE': 0.0006085261379135197,  
'RMSE': 0.02466832255978342,  
'MAE': 0.015110494700140429,  
'MAPE (%)': 1.109062408868412,  
'SMAPE (%)': 1.0279571463218284,  
'MASE': 0.03846460997488332,  
'R^2': 0.996558576064933
```

Feature Importances:

Feature Index	Feature	Importance
9	lag_1	24.667615
14	rolling_mean	18.957311
10	lag_2	18.315177
13	lag_5	14.225876
12	lag_4	9.982502
11	lag_3	9.386669
0	5-Year, 5-Year Forward Inflation Expectation Rate	1.613011
3	Unemployment Rate	0.784934
8	10-Year Treasury Yield	0.400789
7	Crude Oil Prices: West Texas Intermediate	0.386082
5	Personal Consumption Expenditures: Chain-type ...	0.367733
6	Nominal Broad U.S. Dollar Index	0.314435
4	Federal Funds Effective Rate	0.246456
2	Real Gross Domestic Product	0.186847
1	Consumer Price Index	0.164565

Feature Importance Analysis (CatBoost)

Key Findings:

- The **most influential feature** in the model is `lag_1`, indicating that the most recent past value of the target variable significantly impacts predictions.
- `rolling_mean` is the **second most important feature**, suggesting that the smoothed trend of past values plays a critical role in forecasting.
- Other **lag variables** (`lag_2`, `lag_3`, `lag_4`, `lag_5`) are also highly significant, reinforcing the importance of historical patterns in prediction.
- The **5-Year, 5-Year Forward Inflation Expectation Rate** is the highest-ranked economic indicator, albeit with much lower importance than lagged features.
- The **Unemployment Rate** has some influence but is far less important than lagged values.
- Several traditional macroeconomic indicators such as the **10-Year Treasury Yield, Crude Oil Prices, and Consumer Price Index (CPI)** show minimal importance in the model.
- The findings suggest that **historical data (lags & rolling averages) dominate prediction accuracy, while traditional macroeconomic indicators contribute relatively little.**

Interpretation:

- The dominance of `lag_1` and `rolling_mean` suggests that inflation (or the target variable) follows strong autoregressive patterns.
- The **low importance of CPI and GDP** may indicate that short-term forecasting relies more on direct past values rather than broader economic conditions.
- The **minimal role of interest rates (e.g., Federal Funds Rate, Treasury Yields)** suggests that monetary policy effects may be indirect or lagged beyond the model's scope.

Conclusion:

This feature importance ranking indicates that **time series patterns (lags & rolling trends) are the strongest predictors, while traditional economic indicators have limited short-term predictive power.**

Actual vs Predicted Values (CatBoost)

Key Findings:

- The scatter plot displays the **actual values (x-axis) vs. predicted values (y-axis)** for a model

trained using **CatBoost**.

- The **red dashed line** represents the **perfect fit** (i.e., ideal scenario where predictions exactly match actual values).
- The **blue dots** represent individual data points comparing model predictions to actual values.
- The **alignment of most points along the red dashed line** suggests that the model has a **high predictive accuracy**.
- There are **some deviations**, particularly at lower values, but they are minimal, indicating **low error and strong model performance**.
- The model does not show significant bias towards under- or over-prediction, as the spread around the perfect fit line is balanced.
- Overall, the results indicate **high correlation between actual and predicted values**, validating the model's effectiveness.

Interpretation:

- Since most points are **closely clustered along the perfect fit line**, the CatBoost model demonstrates **strong generalization ability**.
- The minor deviations at lower values could indicate **small residual errors**, but these do not seem significant enough to undermine overall model performance.
- This suggests that the model effectively captures the underlying patterns in the data and is **reliable for forecasting**.

Conclusion:

- The **CatBoost model performs exceptionally well**, with predicted values **closely matching actual values**.
- There is **minimal error**, supporting the conclusion that the model is **highly accurate and well-calibrated** for its intended purpose.

Residual Plot (CatBoost)

Key Findings:

- The scatter plot visualizes the **residuals (prediction errors) against actual values**.
- The **red dashed line** represents **zero residuals**, indicating a perfect prediction.
- Residuals are **evenly distributed around zero**, suggesting **no systematic bias** in the model.
- The **spread of residuals remains relatively constant** across different actual values, indicating **homoscedasticity** (consistent variance).
- There are **some outliers**, particularly at lower actual values, but they are minimal and do not indicate a severe problem.
- The lack of visible patterns suggests that **the model captures the underlying structure of the data well**.

Interpretation:

- Since the residuals are **centered around zero**, the model does **not consistently overpredict or underpredict**.
- The **absence of a clear trend** in residuals suggests that the model has effectively learned the relationship between features and target values.
- The **constant spread** of residuals across different values confirms that the model does not suffer from **heteroscedasticity** (variance increasing with actual values), which would indicate instability.

Conclusion:

- The **CatBoost model performs well**, with residuals **randomly scattered around zero**, confirming **strong predictive accuracy**.
- No clear patterns in residuals imply that the model has captured most of the signal in the data and is **not missing significant trends**.
- Minor residual deviations are normal and do not indicate any major issues with the model's performance.

Residual Distribution (CatBoost)

Key Findings:

- The histogram shows the **distribution of residuals** (errors between actual and predicted values).
- The **red dashed line at zero** represents **perfect predictions** with no error.
- The residuals are **tightly clustered around zero**, indicating that **the model has minimal error**.
- The distribution appears **approximately normal (bell-shaped)**, suggesting that the errors are **random and not systematically biased**.
- There are **no significant outliers**, as most residuals are within a small range (approximately between -0.2 and 0.2).
- The high peak at **zero residual** suggests that the majority of predictions are very close to the actual values.

Interpretation:

- The **symmetry and concentration around zero** suggest that the model does not consistently overestimate or underestimate values.
- The **narrow spread** indicates **low variance in prediction errors**, meaning the model is **highly accurate**.
- Since the distribution does not exhibit strong skewness, there is **no systematic pattern of error**, supporting the model's reliability.

Conclusion:

- The **CatBoost model performs exceptionally well**, as the residuals are **centered around zero with minimal dispersion**.
- The residual distribution confirms that **errors are small, unbiased, and randomly distributed**, further validating the model's effectiveness.

Top 5 Hyperparameter Combinations:

Rank 27:

Hyperparameters:

```
{'depth': 6, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.1}
```

Mean Test Score: -0.000682

Standard Deviation: 0.000150

Rank 36:

Hyperparameters:

```
{'depth': 6, 'early_stopping_rounds': 100, 'iterations': 1000, 'learning_rate': 0.1}
```

Mean Test Score: -0.000682

Standard Deviation: 0.000150

Rank 45:

Hyperparameters:

```
{'depth': 8, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.1}
```

Mean Test Score: -0.000712

Standard Deviation: 0.000154

Rank 54:

Hyperparameters:

```
{'depth': 8, 'early_stopping_rounds': 100, 'iterations': 1000, 'learning_rate': 0.1}
```

Mean Test Score: -0.000712

Standard Deviation: 0.000154

Rank 44:

Hyperparameters:

```
{'depth': 8, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.05}
```

Mean Test Score: -0.000739

Standard Deviation: 0.000153

Hyperparameter Tuning: Iterations vs Learning Rate

Key Findings:

- The heatmap illustrates the relationship between **iterations (y-axis)** and **learning rate (x-axis)** in hyperparameter tuning.
- The values within the cells represent the **model's performance metric (likely loss or error)**, where lower values indicate better performance.
- **Darker colors represent worse performance**, while **lighter colors indicate better performance**.
- At **300 iterations** and a **0.01 learning rate**, the model exhibits the **highest error (-0.003)**, suggesting **underfitting** due to insufficient learning.
- Increasing **iterations to 500 or 1000** generally improves the performance across all learning rates.
- The **learning rates of 0.05 and 0.1 consistently perform better** across different iteration counts.
- The **best overall performance (lowest error)** occurs at **higher learning rates (0.05, 0.1)** with **higher iterations (500, 1000)**.

Interpretation:

- A **low learning rate (0.01)** with **few iterations (300)** leads to **higher error**, indicating the model **is not learning efficiently**.
- Increasing the learning rate from **0.01 to 0.05 or 0.1** improves performance, suggesting **faster convergence without excessive overfitting**.
- The **performance difference between 500 and 1000 iterations is minimal**, implying that **500 iterations may be sufficient for optimal learning**.
- The results suggest an **optimal combination of a 0.05 or 0.1 learning rate with at least 500 iterations**.

Conclusion:

- The **best-performing hyperparameter combination** for this model appears to be a **learning rate of 0.05 or 0.1** with **at least 500 iterations**.
- A **learning rate of 0.01 is too low**, particularly at fewer iterations, causing **slow convergence and underfitting**.
- Increasing the **learning rate and iteration count improves performance**, but **beyond 500 iterations, gains are minimal**.
- This analysis helps in selecting an **efficient combination of hyperparameters to maximize model performance while avoiding unnecessary computation**.

SHAP Value Analysis: Feature Impact on Model Output

Key Findings:

- The **SHAP plot visualizes the impact of features on the model's predictions**, with **SHAP values on the x-axis** representing the contribution to the output.
- **Higher absolute SHAP values indicate greater impact** on the model's prediction.
- The **color gradient** represents the feature values:
 - **Blue (low values)**
 - **Red (high values)**
- The most influential features are:
 - **rolling_mean** and **lag_1**, showing the highest SHAP impact.
 - **5-Year, 5-Year Forward Inflation Expectation Rate** also plays a noticeable role.
 - **Other lag variables (lag_2, lag_3, lag_4, lag_5)** also contribute but to a lesser extent.
- Traditional economic indicators such as **GDP, Crude Oil Prices, CPI, and the Federal Funds Rate**

have minimal impact on the model's predictions.

Interpretation:

- **Time-series features (`rolling_mean` , `lag_1`)** dominate the model's decision-making, highlighting that recent historical values strongly influence predictions.
- **High SHAP values for lags suggest an autoregressive nature** of the target variable.
- **Macroeconomic indicators (CPI, GDP, Unemployment Rate, etc.) have little influence**, meaning the model does not heavily rely on them for predictions.
- **The color spread of SHAP values** suggests that feature effects vary based on their magnitude, reinforcing non-linearity in their impact.

Conclusion:

- The **model prioritizes recent historical values over traditional economic indicators**, emphasizing the importance of autoregressive signals.
- **Macroeconomic variables contribute minimally**, suggesting that short-term forecasting is primarily driven by past values rather than broader economic trends.
- The **SHAP plot validates the model's reliance on time-series features**, aligning with findings from feature importance analysis.

SHAP Force Plot: Feature Contributions to Prediction

Key Findings:

- This **SHAP force plot** illustrates how different features contribute to the model's final prediction.
- The **base value** (0.06364) represents the model's average output before considering feature contributions.
- **Positive SHAP values (red segments)** indicate features that **increase the prediction**, pushing the final value higher.
- **Negative SHAP values (blue segments)** represent features that **decrease the prediction**, pulling it lower.
- The **final prediction ($f(x) = 0.16$)** is the result of adding all feature contributions to the base value.

Feature Contributions:

- **Features contributing positively (red):**
 - `lag_1` = 2.2
 - `lag_2` = 2.17
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.47
 - Crude Oil Prices: West Texas Intermediate = 106.7
 - `lag_4` = 2.23
 - `lag_5` = 2.23
 - Federal Funds Effective Rate = 0.13
- **Feature reducing the prediction (blue):**
 - Personal Consumption Expenditures: Chain-type Price Index

Interpretation:

- **Time-series features (`lag_1` , `lag_2` , `lag_4` , `lag_5`)** have the strongest positive influence, suggesting that recent historical data plays a dominant role in shaping predictions.
- **Macroeconomic factors like crude oil prices and inflation expectations also contribute positively**, but to a lesser extent than lag variables.
- The **Personal Consumption Expenditures (PCE) index negatively impacts the prediction**, suggesting that this factor lowers the target value in this specific instance.
- The combined effect of positive contributions outweighs negative contributions, leading to a **final prediction higher than the base value**.

Conclusion:

- The **model heavily relies on lagged values for prediction**, confirming the importance of historical trends.
- Some **macroeconomic indicators (oil prices, inflation expectations) influence the outcome**, but their impact is secondary.
- The **final prediction (0.16) is significantly shaped by time-series data**, reinforcing the model's preference for past values over broader economic indicators.

Partial Dependence Plots (CatBoost)

Key Findings:

- The **partial dependence plots (PDPs)** illustrate the **marginal effect** of each feature on the model's predictions.
- **X-axis** represents the feature values, while the **Y-axis** shows the corresponding model predictions.
- **Time-series features (lag_1, rolling_mean, etc.) have the strongest positive impact** on predictions.
 - **lag_1, rolling_mean, and lag_2** show a **clear upward trend**, meaning higher values of these features lead to increased predictions.
 - **lag_4 and lag_3** exhibit **mild non-linear effects**, but still positively influence the model.
- **Macroeconomic features have minimal impact:**
 - **Consumer Price Index (CPI), Real GDP, Unemployment Rate, Federal Funds Rate, and Nominal Broad U.S. Dollar Index** show **almost flat trends**, suggesting these features have little to no effect.
 - **5-Year, 5-Year Forward Inflation Expectation Rate** shows a **slight increasing effect**, implying a mild correlation with the target variable.
- **Crude Oil Prices and Personal Consumption Expenditures** have a **slight upward trend**, indicating **minor influence** but not as dominant as time-series features.
- The **10-Year Treasury Yield** has a **mild increasing trend**, showing a small yet present impact on predictions.

Interpretation:

- The model primarily **relies on lagged values (lag_1, rolling_mean, etc.) for predictions**, reinforcing an autoregressive nature.
- Macroeconomic indicators **do not significantly influence predictions**, as their PDPs remain relatively flat.
- The **small positive trend in crude oil prices and inflation expectation rates** suggests some impact, but they are secondary to time-series features.
- The **rolling mean has the strongest influence**, showing a **steep upward trajectory**, indicating that past trends are crucial for prediction.

Conclusion:

- The **most influential features are historical values (lag_1, rolling_mean, lag_2)**, confirming the model's preference for past observations.
- **Macroeconomic indicators have minimal direct effect**, suggesting they are not key drivers of the target variable.
- The model follows a **time-series forecasting approach**, where recent past values are the strongest predictors, rather than broader economic indicators.

Permutation Feature Importance (CatBoost)

Key Findings:

- The **bar chart represents the permutation importance** of different features in the CatBoost model.
- **Permutation importance measures the decrease in model performance when a specific feature's values are randomly shuffled**, indicating its contribution to prediction accuracy.
- **Top influential features:**
 - **rolling_mean** has the **highest importance**, suggesting that **recent trend smoothing is the most**

- critical factor in prediction.
- lag_1 is the **second most important feature**, emphasizing the role of the previous time step in forecasting.
- lag_2 also holds significant importance, further reinforcing the **autoregressive nature** of the model.
- Moderate impact features:**
 - 5-Year, 5-Year Forward Inflation Expectation Rate has some influence but is **far less significant** than lag-based features.
 - lag_5, lag_3, and lag_4 contribute **minimally**, suggesting that **further past values have diminishing predictive power**.
- Minimal impact features:**
 - Macroeconomic indicators (Real GDP, CPI, Crude Oil Prices, Unemployment Rate, etc.) show **near-zero importance**.
 - These features do not significantly affect predictions, implying that the model relies more on historical values than external economic variables.

Interpretation:

- The model's predictive power is **heavily dependent on past values** (rolling_mean , lag_1 , lag_2), highlighting a **time-series driven approach**.
- Macroeconomic indicators contribute little to predictions, meaning that external economic conditions do not heavily impact short-term forecasting.
- The slight importance of **inflation expectations** (5-Year, 5-Year Forward Rate) suggests that it may have some predictive value but is **not a primary driver**.

Conclusion:

- Time-series features dominate the model's predictions, especially **rolling_mean** and **lag_1**.
- Macroeconomic factors play an **insignificant role**, confirming that **recent historical values are far more important than broader economic conditions**.
- The model relies on an **autoregressive structure**, where recent data trends are the key predictors rather than external influences.

Feature Importance Analysis for CatBoost (Direct Access)

Key Findings:

- The **bar chart represents feature importance scores** in the CatBoost model.
- Most influential features:**
 - lag_1 has the **highest importance**, confirming that the most recent past value is the strongest predictor.
 - lag_2 follows closely, further reinforcing the importance of short-term historical values.
 - rolling_mean is also highly influential, suggesting that **trend smoothing is crucial for predictions**.
- Moderate importance features:**
 - lag_3, lag_4, and lag_5 have a moderate impact, indicating that additional lagged values still contribute, but their influence decreases as they go further back in time.
- Minimal impact features:**
 - Macroeconomic indicators such as **CPI, GDP, Unemployment Rate, Treasury Yield, and Crude Oil Prices** show **negligible importance** in the model.
 - The **5-Year, 5-Year Forward Inflation Expectation Rate** has a small, but noticeable impact, indicating that inflation expectations play a limited role in predictions.

Interpretation:

- Time-series features dominate, particularly **lag_1, lag_2, and rolling_mean**, confirming an **autoregressive nature of the model**.
- Macroeconomic indicators contribute little, suggesting that short-term predictions do not heavily rely on broader economic conditions.
- The decreasing importance of **longer lags** (lag_3 , lag_4 , lag_5) suggests that **recent past values are far more predictive than older data points**.

Conclusion:

- The most critical features are short-term historical values (`lag_1` , `lag_2`) and rolling averages, indicating that the model prioritizes recent trends.
- Economic indicators have little to no influence, reinforcing that the target variable is primarily driven by its own past values rather than external factors.
- This confirms that a time-series approach using lagged values is the most effective for prediction in this model.

Support Vector Regression (SVR)

Support Vector Regression (SVR): Model Development, Hyperparameter Tuning, and Performance Evaluation

1. Importing Essential Libraries

- **Random & NumPy**: Used for setting random seeds to ensure reproducibility.
- **Scikit-learn**:
 - **SVR (Support Vector Regression)**: Implements regression using support vector machines.
 - **GridSearchCV**: Performs hyperparameter tuning via exhaustive search.
 - **train_test_split**: Splits the dataset into training and testing subsets.

2. Data Preparation

- The dataset is split into:
 - **Training Set (80%)**: Used to train the SVR model.
 - **Testing Set (20%)**: Used for model evaluation.
- The split is controlled using `random_state=42` to ensure reproducibility.

3. Model Initialization

- **Support Vector Regression (SVR)** is instantiated with:
 - `kernel='rbf'`: **Radial Basis Function (RBF)** kernel for handling non-linearity.
 - Additional hyperparameters will be optimized via **GridSearchCV**.

4. Hyperparameter Tuning using GridSearchCV

- **Grid Search Strategy**: Searches for the best hyperparameters from the following grid:
 - `C`: Regularization strength.
 - `epsilon`: Defines the tolerance margin for error.
 - `gamma`: Controls influence of training samples.
- **5-Fold Cross-Validation** is used to ensure robust performance.
- **Parallel Processing (`n_jobs=-1`)** speeds up the search.

5. Model Training and Selection of Optimal Parameters

- **GridSearchCV** trains multiple models with different hyperparameter combinations.
- The **best-performing model** is selected based on **Negative Mean Squared Error (MSE)**.
- The optimal hyperparameters are retrieved and displayed.

6. Model Prediction and Performance Evaluation

- The optimized SVR model is used to **generate predictions** on the test dataset.
- A predefined function `calculate_metrics` computes performance scores.
- The model is benchmarked against a **naïve forecasting method**.

7. Actual vs. Predicted Values Visualization

- A **scatter plot** is generated to visualize model predictions.
- A **perfect fit reference line** is included to assess accuracy.

8. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line ($y=0$)** helps assess error distribution.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line ($x=0$)** is used to check for normality in residuals.

9. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Heatmap

- A **heatmap** visualizes the impact of `C` and `epsilon` on model performance.
- Darker colors indicate **higher performance**.

10. SHAP Analysis for Model Interpretability

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **KernelExplainer** is used since SVR is a non-linear black-box model.
- **K-means clustering** reduces dataset size for computational efficiency.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes feature contributions for specific instances.

11. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

12. Permutation Feature Importance

- Computed by **randomly shuffling feature values** and measuring performance impact.
- Provides a more robust estimate of feature importance.
- **Visualization:** A bar chart ranks feature importance based on permutation scores.

Conclusion

This notebook systematically implements and evaluates an **SVR model** for predictive modeling. Key contributions include:

1. **Comprehensive hyperparameter tuning** with **GridSearchCV**.
2. **Model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via SHAP and permutation analyses.
4. **Model interpretability** through SHAP and Partial Dependence Plots.

Output Result:

Fitting 5 folds for each of 18 candidates, totalling 90 fits

Best Hyperparameters:

```
'C': 10,  
'epsilon': 0.01,  
'gamma': 'auto'
```

SVR Evaluation Metrics:

```
'Model': 'Support Vector Regression (SVR)',  
  
'MSE': 0.0012041754672048918,  
  
'RMSE': 0.034701231494067925,  
  
'MAE': 0.016298244855890376,  
  
'MAPE (%)': 1.0984784399408885,  
  
'SMAPE (%)': 1.026759844422318,  
  
'MASE': 0.041488094473250144,  
  
'R^2': 0.9931899748972683
```

Actual vs Predicted Values (SVR)

Key Findings:

- The scatter plot compares the **actual values (x-axis)** with **predicted values (y-axis)** for a **Support Vector Regression (SVR)** model.
- The **red dashed line** represents a **perfect fit**, where predicted values would exactly match actual values.
- The **blue points (predictions)** align closely with the **perfect fit line**, indicating that the model performs well.
- The **majority of predictions are tightly clustered around the diagonal**, showing a **high correlation** between actual and predicted values.
- There are a **few deviations at lower and higher values**, but they are minimal, suggesting **low error variance**.

Interpretation:

- The **strong alignment with the perfect fit line** suggests that **SVR is making highly accurate predictions**.
- The **even spread around the line** indicates **no significant bias** in the model (i.e., it does not consistently overpredict or underpredict).
- Minor deviations at lower values could be due to **small residual errors**, but these do not appear significant.
- The **overall trend confirms that the model generalizes well** across different values.

Conclusion:

- The **SVR model exhibits excellent predictive accuracy**, as most points are closely aligned with the perfect fit.
- **Minimal deviations suggest a well-trained model with low error**, making it reliable for forecasting.
- This visualization **validates the effectiveness of SVR for this dataset**, demonstrating a **strong relationship between actual and predicted values**.

Residual Plot (SVR)

Key Findings:

- The scatter plot visualizes the **residuals (errors between actual and predicted values)** against **actual values**.
- The **red dashed line at zero** represents **perfect predictions** where the model has no error.
- **Most residuals are centered around zero**, indicating that the **SVR model is making accurate predictions** with minimal bias.
- The **spread of residuals remains relatively constant** across different actual values, suggesting **homoscedasticity** (constant variance).
- There are **some outliers**, particularly at lower and higher actual values, but they appear **randomly distributed**, not following a clear pattern.
- The **absence of a systematic trend** in residuals suggests that **the model captures the underlying**

structure of the data well.

Interpretation:

- The **random scatter around zero** confirms that the model does **not consistently overpredict or underpredict**.
- The **constant spread of residuals** indicates that the model maintains stable performance across different ranges of actual values.
- The **lack of a clear pattern in residuals** suggests that **SVR has effectively captured the relationship between features and target values** without strong systematic errors.

Conclusion:

- The **SVR model exhibits strong predictive performance**, as residuals are **randomly distributed and centered around zero**.
- **Minimal variance and lack of structure in residuals confirm that the model does not suffer from bias or heteroscedasticity**.
- The **random nature of errors validates the model's robustness**, making it reliable for predictions.

Residual Distribution (SVR)

Key Findings:

- The histogram visualizes the **distribution of residuals (errors between actual and predicted values) for the SVR model**.
- The **red dashed line at zero** represents **perfect predictions**, where there is no error.
- The residuals are **highly concentrated around zero**, indicating that the **SVR model has minimal error**.
- The distribution appears **narrow and sharply peaked**, suggesting that **most predictions are very close to actual values**.
- There are **very few residuals with large magnitudes**, meaning the model **rarely makes large errors**.
- The **symmetry of the distribution** suggests that errors are **evenly spread and do not exhibit bias** towards over- or underprediction.

Interpretation:

- The **tight clustering around zero** confirms that the SVR model is **highly accurate**.
- The **lack of a long tail or significant skewness** indicates that the model **does not consistently make large errors in any direction**.
- The **small spread in residuals** further supports **low prediction variance**, reinforcing the model's reliability.

Conclusion:

- The **SVR model performs exceptionally well**, as residuals are **mostly centered around zero** with **minimal dispersion**.
- **The errors are small, unbiased, and normally distributed**, confirming the model's **robustness and reliability for forecasting**.
- This visualization **validates the model's effectiveness**, demonstrating a **strong match between predicted and actual values**.

Top 5 Hyperparameter Combinations:

Rank 8:

Hyperparameters:

```
{'C': 10, 'epsilon': 0.01, 'gamma': 'auto'}
```

Mean Test Score: -0.002214
Standard Deviation: 0.000516

Rank 14:

Hyperparameters:

```
{'C': 100, 'epsilon': 0.01, 'gamma': 'auto'}
```

Mean Test Score: -0.002237
Standard Deviation: 0.000494

Rank 2:

Hyperparameters:

```
{'C': 1, 'epsilon': 0.01, 'gamma': 'auto'}
```

Mean Test Score: -0.002793
Standard Deviation: 0.000547

Rank 10:

Hyperparameters:

```
{'C': 10, 'epsilon': 0.1, 'gamma': 'auto'}
```

Mean Test Score: -0.006956
Standard Deviation: 0.000394

Rank 16:

Hyperparameters:

```
{'C': 100, 'epsilon': 0.1, 'gamma': 'auto'}
```

Mean Test Score: -0.006956
Standard Deviation: 0.000394

Hyperparameter Tuning: C vs Epsilon

Key Findings:

- The heatmap represents the **performance metric (likely loss or error)** for different combinations of **C (regularization parameter)** and **Epsilon (ϵ -insensitive zone)** in SVR tuning.
- **X-axis (Epsilon):** Controls the margin of tolerance where no penalty is given for errors within this range.
- **Y-axis (C):** Regularization parameter that **determines the trade-off between margin size and training accuracy**.
- The **values in each cell represent the model's performance**, with **lower values indicating better performance**.
- **Color scale interpretation:**
 - **Darker colors represent worse performance (higher error/loss).**
 - **Lighter colors (yellow-green) represent better performance (lower error/loss).**
- **Observations on parameter effects:**
 - **Low C (1) leads to the highest error**, regardless of the epsilon value.
 - **Increasing C to 10 reduces the error**, but the improvement is minor.
 - **The best performance is achieved at C = 100 with an epsilon of 0.01**, suggesting that a **higher C improves prediction accuracy**.
 - **A larger epsilon (0.2) results in slightly higher error**, meaning that **allowing too much tolerance decreases precision**.

Interpretation:

- **Higher values of C (stronger regularization) lead to better model performance**, as seen from the decreasing error when moving from **C = 1** to **C = 100**.
- **Smaller epsilon values generally perform better**, indicating that a **narrower tolerance band for errors improves accuracy**.
- However, for **C = 100**, increasing epsilon to **0.2 increases the error**, suggesting that **there is an**

optimal balance between C and epsilon for best performance.

Conclusion:

- **C = 100** and **$\epsilon = 0.01$** result in the lowest error, making it the **optimal hyperparameter combination** in this setup.
- **Higher C improves accuracy**, while a smaller epsilon provides better precision, but too large an epsilon (0.2) slightly worsens performance.
- This heatmap guides hyperparameter selection for optimizing SVR model performance.

SHAP Value Analysis: Feature Impact on Model Output

Key Findings:

- The **SHAP summary plot** visualizes the impact of each feature on the model's predictions.
- **X-axis:** SHAP values, representing the contribution of a feature to the model's output.
- **Y-axis:** Features ranked by importance, with the most impactful features at the bottom.
- **Color coding:**
 - **Red** represents high feature values, while **blue** represents low feature values.
 - The **spread of points** along the x-axis shows the varying impact of each feature.

Feature Impact Observations:

- **Top influential features:**
 - **lag_1**, **5-Year, 5-Year Forward Inflation Expectation Rate**, and **lag_4** have the **greatest SHAP impact**, indicating they are key drivers in predictions.
 - Higher values of **lag_1** (red) tend to **increase predictions**, while lower values (blue) decrease them.
 - **Rolling mean and other lag features (lag_2, lag_5, lag_3)** also have significant influence, reinforcing the autoregressive nature of the model.
- **Moderate-impact features:**
 - **Unemployment Rate**, **10-Year Treasury Yield**, and **Federal Funds Effective Rate** contribute but are **less influential than lag variables**.
- **Minimal-impact features:**
 - **Macroeconomic indicators such as CPI, Crude Oil Prices, GDP, and Personal Consumption Expenditures** have minimal influence, suggesting the model does not heavily rely on them.

Interpretation:

- **Time-series features (lag_1, rolling_mean, etc.)** dominate the model's decision-making, confirming the predictive strength of historical trends.
- **Macroeconomic indicators contribute little**, implying that broader economic conditions have a limited direct effect on predictions.
- The **SHAP distribution across features is spread symmetrically**, indicating a **well-balanced model** without extreme bias.

Conclusion:

- **Short-term historical values (lags) and inflation expectations** are the most crucial predictors, with macroeconomic variables playing a minor role.
- The **model prioritizes time-series data for forecasting**, validating its autoregressive nature.
- **SHAP values confirm that the model is not overly dependent on any single feature**, ensuring robust and reliable predictions.

SHAP Decision Plot Analysis

Key Findings:

- The **SHAP decision plots** illustrate how individual features contribute to the model's final prediction.

- **Color Coding:**
 - Red features increase the prediction.
 - Blue features decrease the prediction.
- Time-series features (`rolling_mean` , `lag_1` , `lag_2` , `lag_3` , `lag_4`) are the most influential, confirming an autoregressive model structure.
- Macroeconomic indicators (`CPI` , `Nominal Broad U.S. Dollar Index` , `Crude Oil Prices`) have variable effects, influencing the output differently across different instances.
- Consumer Price Index (CPI) is a significant driver in some cases, while in others, it has minimal impact.

Interpretation:

- The model relies primarily on past values for predictions, with rolling averages and lagged values dominating.
- Macroeconomic indicators play a secondary role, meaning they are less impactful than historical trends.
- The SHAP plots illustrate non-linearity, meaning different feature values interact uniquely, affecting predictions in a non-uniform manner.

Conclusion:

- Short-term historical values remain the strongest predictors.
- Macroeconomic variables have inconsistent effects, making them secondary to time-series patterns.
- The SHAP decision plot confirms the autoregressive nature of the model, reinforcing the importance of rolling mean and lag variables.

Partial Dependence Plots (SVR)

Key Findings:

- The partial dependence plots (PDPs) show the marginal effect of each feature on the model's predictions.
- X-axis: Represents the values of the given feature.
- Y-axis: Represents the corresponding model output (prediction).

Observations by Feature:

- **Time-Series Features:**
 - `lag_1` , `lag_2` , `lag_3` , `lag_4` , `lag_5` , and `rolling_mean` show a clear upward trend, confirming that past values strongly influence the predictions.
 - Rolling mean has the most consistent increase, reinforcing its importance in capturing trends.
- **Macroeconomic Indicators:**
 - 5-Year, 5-Year Forward Inflation Expectation Rate shows a steady positive correlation, meaning higher inflation expectations increase the predicted values.
 - 10-Year Treasury Yield follows a similar increasing trend, indicating its relevance in prediction.
 - Unemployment Rate has a slight downward trend, suggesting that higher unemployment might contribute to lower predictions.
 - Consumer Price Index (CPI) and Personal Consumption Expenditures show non-linear effects, meaning their influence varies across different value ranges.
 - Crude Oil Prices exhibit a non-linear pattern with a sharp rise after a certain threshold, indicating a more complex relationship with the target variable.

Interpretation:

- The SVR model relies heavily on past values (`rolling_mean` , `lag_1` , `lag_2`), reinforcing its autoregressive nature.
- Macroeconomic indicators contribute to predictions, but their effects are more complex and non-linear.
- The model appears to be well-calibrated, as the relationships align with expected economic trends.

Conclusion:

- Time-series features remain the dominant drivers of the model's predictions.
- Certain macroeconomic indicators (Inflation Expectation, Treasury Yield, Unemployment Rate) influence predictions but are secondary to past values.
- The non-linear effects of CPI and Crude Oil Prices suggest that their impact on predictions is more complex than a simple linear relationship.
- The model effectively captures historical trends while considering broader economic conditions.

Permutation Feature Importances for SVR

Key Findings:

- The bar chart represents permutation feature importance scores for the Support Vector Regression (SVR) model.
- X-axis: Features used in the model.
- Y-axis: Importance score, measuring the drop in model performance when a feature's values are randomly shuffled.

Feature Importance Observations:

- Most Important Features:
 - Real Gross Domestic Product (GDP), Unemployment Rate, and Nominal Broad U.S. Dollar Index are the top contributors, suggesting that economic factors play a dominant role in predictions.
 - Federal Funds Effective Rate and Crude Oil Prices also show moderate importance, confirming their impact.
- Lagged Features (lag_1 to lag_5) and rolling_mean have near-zero importance, meaning that historical values contribute minimally to the model's decision-making.
- The 5-Year, 5-Year Forward Inflation Expectation Rate has very low importance, indicating that long-term inflation expectations do not significantly influence predictions.

Interpretation:

- Unlike autoregressive models, SVR prioritizes macroeconomic indicators over past values, indicating a fundamental shift in predictive influence.
- GDP, Unemployment Rate, and the U.S. Dollar Index are critical for model accuracy, while time-series-based features are largely ignored.
- The low importance of lags and rolling averages suggests that SVR does not strongly depend on past values, unlike traditional time-series models.

Conclusion:

- SVR heavily relies on macroeconomic indicators rather than historical values, making it more of an econometric model than a time-series model.
- Economic indicators like GDP, Unemployment, and Dollar Index are the strongest predictors, while past values (lag_1, rolling_mean) are nearly irrelevant.
- This analysis confirms that SVR captures broader economic trends rather than short-term historical patterns.

Ridge Regression (Ridge)

Ridge Regression: Hyperparameter Tuning, Model Evaluation, and Interpretability

1. Importing Essential Libraries

- NumPy & Random: Used for setting random seeds to ensure reproducibility.
- Scikit-learn:
 - Ridge Regression: A linear regression model with L2 regularization.
 - GridSearchCV: Performs exhaustive hyperparameter tuning.

- `train_test_split`: Splits the dataset into training and test subsets.

2. Data Preparation

- The dataset is split into:
 - **Training Set (80%)**: Used to train the Ridge Regression model.
 - **Testing Set (20%)**: Used for model evaluation.
- **Random Seed (`random_state=42`)** is set to ensure reproducibility.

3. Model Initialization

- **Ridge Regression** is instantiated with:
 - `alpha`: Regularization strength parameter (to be optimized).
 - **L2 regularization** prevents overfitting by penalizing large coefficients.

4. Hyperparameter Tuning using GridSearchCV

- **Grid Search Strategy**: Searches for the best `alpha` value from:
 - `[0.1, 1.0, 10.0, 100.0, 1000.0]`
- **5-Fold Cross-Validation** is used for **robust model selection**.
- **Parallel Processing (`n_jobs=-1`)** speeds up the search.

5. Model Training and Selection of Optimal Parameters

- `GridSearchCV` **trains multiple models** with different `alpha` values.
- The **best-performing model** is selected based on **Negative Mean Squared Error (MSE)**.
- The optimal `alpha` is retrieved and displayed.

6. Model Prediction and Performance Evaluation

- The optimized Ridge model is used to **generate predictions** on the test dataset.
- A predefined function `calculate_metrics` computes evaluation scores.
- The model is benchmarked against a **naïve forecasting method**.

7. Actual vs. Predicted Values Visualization

- A **scatter plot** is generated to visualize Ridge Regression predictions.
- A **perfect fit reference line** is included to assess accuracy.

8. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line (y=0)** helps assess error distribution.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line (x=0)** is used to check for normality in residuals.

9. Hyperparameter Search Results

- `GridSearchCV` results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Line Plot

- A **line plot** visualizes the effect of `alpha` on **Negative Mean Squared Error (MSE)**.

10. Feature Importance Analysis

- **Coefficient values** from Ridge Regression indicate feature importance.
- A **horizontal bar chart** visualizes feature importance based on learned coefficients.

11. SHAP Analysis for Model Interpretability

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **K-means clustering** is applied to **reduce background data** for SHAP calculations.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes feature contributions for specific instances.

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

Conclusion

This notebook systematically implements and evaluates a **Ridge Regression model** for predictive modeling. Key contributions include:

1. **Comprehensive hyperparameter tuning** with **GridSearchCV**.
2. **Model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via SHAP and coefficient analysis.
4. **Model interpretability** through SHAP and Partial Dependence Plots.

Output Result:

Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best Hyperparameters:

'alpha': 0.1

Ridge Regression Evaluation Metrics:

'Model': 'Ridge Regression (Ridge)',

'MSE': 0.00012763510353657853,

'RMSE': 0.011297570691815941,

'MAE': 0.006680917323838124,

'MAPE (%)': 0.4287683783113252,

'SMAPE (%)': 0.42827753605764096,

'MASE': 0.017006648970499055,

'R^2': 0.9992781797314876

Actual vs Predicted Values (Ridge Regression)

Key Findings:

- The scatter plot compares the **actual values (ground truth) vs. predicted values** generated by the **Ridge Regression model**.
- **X-axis:** Actual values.
- **Y-axis:** Predicted values.
- The **red dashed line** represents the perfect fit (ideal scenario where predictions match actual values exactly).
- The **blue dots** represent the model's predictions, with their alignment to the red line indicating accuracy.

Observations:

- **Predictions are tightly clustered around the perfect fit line**, suggesting **high accuracy**.
- **No significant deviations or systematic bias** is visible, confirming that the model does not consistently overpredict or underpredict.
- **Predictions are well-distributed across the range of actual values**, indicating **stability and robustness**.
- **A few minor deviations at lower values** are present but do not significantly impact the overall performance.

Interpretation:

- **The Ridge Regression model performs exceptionally well**, as shown by the near-perfect alignment with actual values.
- **Minimal prediction errors suggest strong generalization**, meaning the model is not overfitting.
- **The strong linearity confirms Ridge Regression's suitability for this dataset**, particularly in handling multicollinearity while maintaining predictive power.

Conclusion:

- **Ridge Regression provides highly accurate predictions with minimal error.**
- **The model generalizes well across different value ranges, making it reliable for forecasting.**
- **The results validate the effectiveness of Ridge regularization**, which balances bias and variance to improve prediction stability.

Residual Plot (Ridge Regression)

Key Findings:

- The **scatter plot visualizes the residuals** (differences between actual and predicted values) of the **Ridge Regression model**.
- **X-axis:** Actual values.
- **Y-axis:** Residuals (prediction errors).
- **The red dashed line represents the zero residual line**, indicating perfect predictions.

Observations:

- **The residuals are symmetrically distributed around zero**, indicating that the model does not exhibit systematic bias.
- **No clear pattern is visible**, suggesting that the errors are random rather than structured.
- **Residuals remain small in magnitude**, meaning that the model makes **only minor prediction errors**.
- **There is a slight increase in residual spread for higher actual values**, but no major heteroscedasticity is observed.

Interpretation:

- **The randomness of residuals confirms that Ridge Regression captures the underlying data trends effectively.**
- **Minimal and symmetrically distributed residuals indicate a well-fitted model** with no major underfitting or overfitting issues.
- **The slight increase in residual spread at higher values is common** but does not significantly affect overall accuracy.
- **The model does not systematically overpredict or underpredict** in any particular range, confirming its reliability.

Conclusion:

- **Ridge Regression provides an accurate and unbiased fit**, as confirmed by the random residual distribution.
- **The model generalizes well, with only small errors across the entire range of values.**
- **The residual plot validates the effectiveness of Ridge regularization**, ensuring stability and robustness in predictions.

Residual Distribution (Ridge Regression)

Key Findings:

- The **histogram displays the distribution of residuals** (differences between actual and predicted values) for the **Ridge Regression model**.
- **X-axis:** Residual values.
- **Y-axis:** Frequency of residual occurrences.
- The **red dashed line represents zero residual error**, which corresponds to perfect predictions.

Observations:

- The residuals are **symmetrically distributed around zero**, suggesting that **the model does not systematically overpredict or underpredict**.
- The **distribution is narrow and centered**, meaning that most errors are small, confirming **high prediction accuracy**.
- A **bell-shaped (normal-like) distribution is visible**, which is a strong indicator that **the model captures the data well** without major bias.
- **Very few extreme residuals (outliers) exist**, highlighting that the model is robust and does not frequently make large errors.

Interpretation:

- The **near-normal distribution of residuals suggests that the assumptions of Ridge Regression hold well**, ensuring statistical reliability.
- The **narrow spread indicates low variance**, meaning that predictions are **consistent and stable**.
- The **model generalizes well**, with residuals being minimal across the entire dataset.
- **There is no evidence of skewness or systematic bias**, further supporting the model's effectiveness.

Conclusion:

- **Ridge Regression performs exceptionally well**, with **small, normally distributed residuals**.
- The **model is unbiased and does not overfit**, as seen from the symmetric and centered error distribution.
- **This analysis validates Ridge Regression as a reliable model for making accurate predictions**.

Top 5 Hyperparameter Combinations:

Rank 1:

Hyperparameters:

```
{'alpha': 0.1}
```

Mean Test Score: -0.000163

Standard Deviation: 0.000019

Rank 2:

Hyperparameters:

```
{'alpha': 1.0}
```

Mean Test Score: -0.000618

Standard Deviation: 0.000070

Rank 3:

Hyperparameters:

```
{'alpha': 10.0}
```

Mean Test Score: -0.001070

Standard Deviation: 0.000123

Rank 4:

Hyperparameters:

```
{'alpha': 100.0}
```

Mean Test Score: -0.001831

Rank 5:

Hyperparameters:

`{'alpha': 1000.0}`

Mean Test Score: -0.009793

Standard Deviation: 0.000224

Effect of Alpha on Model Performance (Ridge Regression)

Key Findings:

- The line plot illustrates the relationship between the regularization strength (Alpha) and model performance for Ridge Regression.
- X-axis: Alpha (Regularization Strength).
- Y-axis: Negative Mean Squared Error (MSE), a measure of model error (lower values indicate better performance).

Observations:

- As Alpha increases, the negative mean squared error also increases, indicating that higher regularization decreases model accuracy.
- At very low values of Alpha (near 0), the model achieves the lowest error, suggesting that less regularization leads to better predictions.
- The error grows nearly linearly with increasing Alpha, implying that excessive regularization harms performance.
- The trend confirms the bias-variance tradeoff: stronger regularization reduces overfitting but increases bias, degrading model accuracy.

Interpretation:

- An optimal Alpha value exists where Ridge Regression balances bias and variance.
- Very high Alpha values force too much regularization, leading to underfitting, as the model is unable to learn meaningful patterns.
- Very low Alpha values (close to 0) minimize error, but may risk overfitting, as the model behaves similarly to Ordinary Least Squares (OLS).
- Selecting an appropriate Alpha requires cross-validation to avoid both underfitting and overfitting.

Conclusion:

- A low Alpha value is preferable for this dataset, as it results in the lowest model error.
- Increasing Alpha consistently worsens performance, indicating that regularization should be applied cautiously.
- This analysis highlights the importance of tuning the Alpha hyperparameter to achieve an optimal tradeoff between bias and variance in Ridge Regression.

Feature Coefficients:

Feature Index	Feature	Coefficient
14	rolling_mean	3.042920
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.012153
13	lag_5	0.007162
3	Unemployment Rate	0.000614
4	Federal Funds Effective Rate	0.000571
6	Nominal Broad U.S. Dollar Index	0.000189
7	Crude Oil Prices: West Texas Intermediate	0.000082
2	Real Gross Domestic Product	0.000004
1	Consumer Price Index	-0.000021

1	Consumer Price Index	0.000000
8	10-Year Treasury Yield	-0.000494
5	Personal Consumption Expenditures: Chain-type ...	-0.000699
9	lag_1	-0.205700
11	lag_3	-0.592217
10	lag_2	-0.627436
12	lag_4	-0.635154

Feature Coefficients (Ridge Regression)

Key Findings:

- The bar chart displays the feature coefficients from the Ridge Regression model, showing the impact of each feature on predictions.
- X-axis: Coefficient values.
- Y-axis: Feature names.

Observations:

- The "rolling_mean" feature has the largest positive coefficient, indicating it is the most influential factor in the model's predictions.
- Most other features have near-zero coefficients, suggesting that Ridge Regression strongly penalized them due to regularization.
- Some lag-based features (lag_1, lag_2, lag_3, lag_4, lag_5) have small but non-zero coefficients, showing that past values contribute to predictions.
- A few features (e.g., lag_4, lag_2, and lag_3) have negative coefficients, meaning that an increase in their values correlates with a decrease in the predicted target.

Interpretation:

- The large coefficient for "rolling_mean" suggests that short-term trends strongly affect the target variable.
- Regularization in Ridge Regression helps shrink less important coefficients, reducing overfitting and making the model more stable.
- Macroeconomic indicators such as "Unemployment Rate", "Federal Funds Effective Rate", and "Real Gross Domestic Product" have been penalized close to zero, meaning they may not significantly contribute to the model's predictions.
- The negative coefficients on certain lag values indicate inverse relationships where past values have a suppressing effect on the target variable.

Conclusion:

- "rolling_mean" dominates model predictions, meaning moving averages are highly predictive.
- The Ridge Regression model effectively reduces feature importance for less relevant predictors, enforcing sparsity.
- Some lag-based features still contribute meaningfully, but their effects are limited compared to rolling averages.
- Macroeconomic indicators, despite their potential relevance, do not show strong influence under the current Ridge Regression settings.

SHAP Summary Plot for Ridge Regression

Key Findings

- This SHAP summary plot visualizes the feature importance and impact on model predictions using SHAP values.
- X-axis: SHAP values (indicating how much each feature contributes to increasing or decreasing the model output).
- Y-axis: Features ranked by importance (most influential at the top).
- Color: Indicates feature value (blue = low, red = high).

Observations

- "rolling_mean" has the highest impact on the model predictions, as it has the widest distribution of SHAP values.
- Lag-based features (lag_1 to lag_5) also contribute significantly, showing that previous values influence the target variable.
- Macroeconomic variables such as "Real Gross Domestic Product", "Consumer Price Index", and "Unemployment Rate" have minimal SHAP value variation, meaning they contribute little to model predictions.
- The spread of SHAP values along the X-axis indicates how much a feature pushes the prediction higher or lower.
 - For example, higher values of "rolling_mean" tend to push predictions positively (red values shifting right).
 - Lower values (blue) of lag-based features tend to reduce predictions.
- Less relevant features cluster around 0 SHAP value, indicating they do not significantly affect the model's output.

Interpretation

- The model relies heavily on short-term trend indicators (rolling_mean and lag features) rather than macroeconomic factors.
- Lag-based features contribute to prediction variance, indicating the importance of past values.
- SHAP values confirm that traditional economic indicators such as GDP, CPI, and interest rates have minimal predictive power for this specific target variable.
- The asymmetry in SHAP value distributions suggests that certain features have a stronger positive effect than negative.

Conclusion

- "rolling_mean" and lag features are the dominant drivers of model predictions.
- Macroeconomic factors contribute little to the prediction output.
- Feature impact varies non-linearly, highlighting the need for careful interpretation of relationships in Ridge Regression models.
- SHAP analysis aligns with feature importance rankings, confirming the dominance of short-term trends in this model.

SHAP Force Plot Analysis for Ridge Regression

Key Findings

- This SHAP force plot explains how different features contribute to a single prediction ($f(x) = 2.222$).
- X-axis: Represents the prediction range, centered around the base value (2.032).
- SHAP values: Contributions of each feature to the final prediction.
 - Red (positive impact): Features pushing the prediction higher.
 - Blue (negative impact): Features pushing the prediction lower.

Observations

- "rolling_mean" contributes the most positively (+0.176), significantly increasing the predicted value.
- Lag-based features (lag_1, lag_2, lag_3, and lag_4) all have a negative effect, pulling the prediction downward.
- The final prediction (2.222) is higher than the base value (2.032) because the positive contributions outweigh the negative ones.
- The width of each segment indicates the magnitude of impact.
 - "rolling_mean" has a much larger effect than any of the lag features.
 - Lag features have relatively equal but negative influence.

Interpretation

- The model heavily relies on "rolling_mean" to drive predictions upward.
- Lag-based features act as stabilizers, preventing an extreme positive shift in prediction.
- The balance of positive and negative SHAP values provides a transparent view of how the model

arrived at the specific prediction.

- A larger "rolling_mean" generally leads to higher predictions, while historical lags tend to suppress excessive increases.

Conclusion

- "rolling_mean" is the most influential factor in this prediction.
- Lag features contribute negatively but in a more distributed manner.
- The model exhibits a clear dependency on short-term trends rather than long-term macroeconomic indicators.
- SHAP force plots offer an intuitive way to understand how individual feature values shape the model's decision.

Partial Dependence Plots (Ridge Regression)

Key Findings

- These **Partial Dependence Plots (PDPs)** visualize how individual features influence the model's predictions while **holding all other features constant**.
- **X-axis:** Represents the feature values.
- **Y-axis:** Represents the predicted target value (partial dependence).

Observations

1. Most macroeconomic features have nearly flat PDP curves:
 - **5-Year Forward Inflation Expectation Rate, Consumer Price Index, Real GDP, Unemployment Rate, Federal Funds Rate, and Crude Oil Prices** all show almost no variation.
 - **Interpretation:** The Ridge Regression model does not assign significant importance to these features.
2. Rolling Mean exhibits a strong positive relationship:
 - **Higher rolling_mean values lead to increased predictions.**
 - **Interpretation:** The model relies heavily on recent historical data rather than broader economic indicators.
3. Lag-based features (lag_1, lag_2, lag_3, lag_4, and lag_5) show a negative trend:
 - **Increasing these lags results in lower predictions.**
 - **Interpretation:** The model assigns **negative weight to previous time step values**, suggesting that past values tend to drive predictions downward.
4. 10-Year Treasury Yield shows a slight negative trend:
 - **Increasing Treasury Yield slightly decreases the target variable.**
 - **Interpretation:** Higher interest rates might correlate with lower predictions.

Key Takeaways

- The model is primarily driven by rolling_mean rather than macroeconomic indicators.
- Past lag values negatively impact predictions, possibly suggesting a mean-reverting trend.
- Macroeconomic variables have little influence in this Ridge Regression model, as indicated by their flat PDP curves.
- The PDPs confirm that the model captures short-term trends more than long-term economic relationships.

Conclusion

- The Ridge Regression model prioritizes rolling mean over fundamental economic indicators.
- Lag-based features provide some predictive power but contribute negatively.
- Macroeconomic factors, while theoretically relevant, do not significantly affect the model's predictions in this setup.
- Further feature engineering or an alternative modeling approach (e.g., non-linear models) may be required to capture broader economic effects.

Lasso Regression (Lasso)

Lasso Regression: Feature Selection, Model Evaluation, and Interpretability

1. Importing Essential Libraries

- **NumPy & Pandas**: Handling numerical and tabular data.
- **Scikit-learn**:
 - **Lasso Regression**: A linear model with **L1 regularization** for feature selection.
 - **GridSearchCV**: Exhaustive hyperparameter tuning.
 - **train_test_split**: Splitting data into training and testing subsets.
 - **StandardScaler**: Standardizing features to improve convergence.

2. Data Preparation and Preprocessing

- **Dataset Splitting**:
 - **Training Set (80%)**: Used for model training.
 - **Testing Set (20%)**: Used for model evaluation.
- **Feature Scaling**:
 - Lasso Regression is **sensitive to feature scales**, so **standardization** improves numerical stability.

3. Model Initialization

- **Lasso Regression** is initialized with:
 - `max_iter=5000`: Ensures convergence in optimization.
 - `alpha`: Regularization strength (to be optimized).
 - **L1 regularization** encourages sparsity by shrinking some coefficients to zero.

4. Hyperparameter Tuning using GridSearchCV

- **Grid Search Strategy**: Optimizes `alpha` from:
 - `[0.01, 0.1, 1.0, 10.0, 100.0]`
- **5-Fold Cross-Validation** ensures **robust model selection**.
- **Parallel Processing** (`n_jobs=-1`) speeds up computation.

5. Model Training and Selection of Optimal Parameters

- **GridSearchCV** trains multiple models with different `alpha` values.
- The **best-performing model** is selected based on **Negative Mean Squared Error (MSE)**.

6. Model Prediction and Performance Evaluation

- The optimized Lasso model is used to **generate predictions** on the test dataset.
- A predefined function `calculate_metrics` computes evaluation scores.
- The model is benchmarked against a **naïve forecasting method**.

7. Feature Selection Analysis

- **Lasso Regression** automatically **removes irrelevant features** by shrinking coefficients to zero.
- **Extracted feature coefficients** are analyzed to identify the most influential predictors.
- A **horizontal bar chart** visualizes the retained features.

8. Actual vs. Predicted Values Visualization

- A **scatter plot** visualizes the Lasso Regression predictions.
- A **perfect fit reference line** is included to assess accuracy.

9. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.

- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line** ($y=0$) helps assess error distribution.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
- A **vertical reference line** ($x=0$) is used to check for normality in residuals.

10. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Line Plot

- A **line plot** visualizes the effect of `alpha` on **Negative Mean Squared Error (MSE)**.

11. Model Interpretability using SHAP

- **SHAP (SHapley Additive ExPlanations)** is used to interpret feature contributions.
- **K-means clustering** is applied to **reduce background data** for SHAP calculations.
- **SHAP Summary Plot** (global explanation) shows the impact of each feature.
- **SHAP Force Plot** (local explanation) visualizes feature contributions for specific instances.

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- **Multiple PDPs** are plotted to examine global trends.

13. Permutation Feature Importance

- **Permutation Importance** quantifies how much shuffling a feature affects model performance.
- A **bar chart** visualizes feature importance rankings.

Conclusion

This notebook systematically implements and evaluates a **Lasso Regression model** for predictive modeling. Key contributions include:

1. **Automated feature selection** using **L1 regularization**.
2. **Comprehensive hyperparameter tuning** with **GridSearchCV**.
3. **Model evaluation** using residual analysis and benchmark comparisons.
4. **Feature importance insights** via SHAP and coefficient analysis.
5. **Model interpretability** through SHAP and Partial Dependence Plots.

Output Result:

Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best Hyperparameters:

`'alpha': 0.01`

Lasso Regression Evaluation Metrics:

`'Model': 'Lasso Regression (Lasso)',`

`'MSE': 0.0009688091889882974,`

`'RMSE': 0.03112569981523785,`

`'MAE': 0.01978713484025213,`

`'MAPE (%)': 1.50065547482365,`

`'sMAPE (%)': 1.3584204499037977,`

'MASE': 0.05036925919729485,

'R^2': 0.9945210518928096

Non-Zero Feature Coefficients:

Feature Index	Feature	Coefficient
9	lag_1	0.323054
14	rolling_mean	0.072329

Feature Coefficients (Lasso Regression)

Key Findings

- This plot shows the **Lasso Regression** feature coefficients, indicating which features are retained after L1 regularization.
- **Lasso performs feature selection by shrinking less important coefficients to zero**, leaving only the most influential predictors.

Observations

1. **Only two features remain non-zero:**
 - **lag_1** has the highest coefficient (~0.32), making it the most significant predictor.
 - **rolling_mean** is also retained but with a much smaller coefficient (~0.07).
2. **All other features have been completely eliminated (coefficient = 0):**
 - This suggests that **Lasso identified lag_1 and rolling_mean as the only relevant variables**.
 - **Macroeconomic indicators (such as inflation, GDP, and interest rates) were deemed unimportant for this model.**
3. **lag_1 dominates the prediction:**
 - Since Lasso selects only a few predictors, this **implies that recent lagged values (lag_1) carry the most predictive power**.
 - The model relies heavily on **immediate past values to make predictions**.
4. **rolling_mean contributes but is secondary:**
 - This suggests that **short-term trend smoothing still holds some value in prediction**.
 - However, it is much less influential compared to **lag_1**.

Key Takeaways

- **Lasso aggressively reduces model complexity by removing irrelevant features.**
- **The model is highly dependent on short-term historical values (lag_1), rather than broader economic indicators.**
- **Feature selection confirms that recent past values (lag features) are more predictive than rolling averages or macroeconomic factors.**
- **For better performance, experimenting with alternative models (e.g., non-linear approaches) might be necessary to capture broader relationships.**

Conclusion

- **Lasso Regression strongly favors lag_1 as the dominant predictor, with a minor role for rolling_mean.**
- **Macroeconomic variables contribute little to the prediction and are removed entirely.**
- **This suggests that time series forecasting benefits more from past observations rather than external economic data in this case.**

Actual vs Predicted Values (Lasso Regression)

Key Findings

- This scatter plot compares the **actual values (x-axis) vs. predicted values (y-axis)** from the

Lasso Regression model.

- The **red dashed line** represents the **perfect fit**, where predictions match actual values exactly.
- The **blue points** represent individual predictions.

Observations

1. Strong alignment with the perfect fit line:

- The predictions **closely follow the red dashed line**, indicating a **highly accurate model**.
- Most points cluster tightly around the perfect fit, suggesting **low bias**.

2. Minor deviations in lower values:

- At **lower actual values (< 1.0)**, some blue points deviate slightly.
- This indicates that **Lasso might underfit or have higher residual errors for small values**.

3. Good generalization with minimal overfitting:

- Unlike overly complex models, **Lasso Regression performs well without major signs of overfitting**.
- The **simplicity of Lasso (by eliminating non-important features)** contributes to this stability.

4. Some variance in high-value predictions:

- At **higher actual values (> 2.5)**, predictions are slightly more scattered.
- This could indicate that **the model struggles to capture non-linear effects in higher ranges**.

Key Takeaways

- **Lasso Regression produces highly accurate predictions**, with a **near-perfect alignment** to the actual values.
- **Minor errors appear for lower and higher values**, suggesting potential room for improvement (e.g., adding non-linearity).
- **Feature selection by Lasso does not significantly degrade prediction accuracy**, confirming that most removed features were not contributing useful information.
- **If better performance is needed at the extremes**, alternative models like Ridge Regression or SVR (Support Vector Regression) could be explored.

Conclusion

- **Lasso Regression is a strong, interpretable model** that performs well in this case.
- **Minimal overfitting and high alignment with actual values** suggest it is a **reliable choice**.
- **For further improvements**, additional feature engineering or non-linear models could be considered.

Residual Plot (Lasso Regression)

Key Findings

- This plot visualizes **residuals (errors)** of the **Lasso Regression model**, where:
 - **X-axis:** Actual values of the target variable.
 - **Y-axis:** Residuals (difference between actual and predicted values).
 - **Red dashed line:** Represents the ideal scenario where **residuals are zero** (perfect predictions).

Observations

1. Residuals are centered around zero:

- The majority of residuals **cluster around the red dashed line ($y=0$)**, indicating that **the model has minimal bias**.
- This suggests that **Lasso Regression is making predictions that do not systematically overestimate or underestimate values**.

2. Slight heteroscedasticity (variance increases at higher values):

- Residuals are **tightly packed at lower actual values** but become more dispersed as actual values increase.
- This means that **Lasso Regression performs better at predicting smaller values**, but exhibits **slightly higher error for larger values**.

3. Few outliers in residuals:

- Some residuals **fall far from zero**, especially at lower actual values.
 - This indicates that **Lasso Regression struggles with certain extreme values**.
4. **No strong non-linear pattern:**
- If residuals followed a curved pattern, it would indicate that the model is missing some non-linearity.
 - Here, residuals appear **randomly scattered**, suggesting that **Lasso Regression captures the linear trend effectively**.

Key Takeaways

- **Lasso Regression has a balanced error distribution**, with most residuals centered around zero.
- **Variance increases at higher actual values**, which means the model might benefit from **feature transformations or non-linear adjustments**.
- **A few outliers exist**, but overall, there are no severe patterns of overfitting or underfitting.
- **Potential Improvements:**
 - Try **feature scaling or polynomial features** to reduce error for extreme values.
 - Consider alternative models (e.g., **SVR, Ridge Regression**) if reducing high-value residuals is a priority.

Conclusion

- **Lasso Regression performs well overall**, but tends to struggle with higher values.
- If error consistency across all value ranges is critical, additional tuning or a different regression approach might be needed.

Residual Distribution (Lasso Regression)

Key Findings

- This plot visualizes the **distribution of residuals (errors)** for the **Lasso Regression model**, where:
 - **X-axis:** Residual values (differences between actual and predicted values).
 - **Y-axis:** Frequency (how often each residual value occurs).
 - **Red dashed line:** Represents the ideal case where **residuals are zero** (perfect predictions).

Observations

- Residuals are approximately normally distributed:**
 - The histogram is **bell-shaped and symmetric**, indicating that **Lasso Regression does not have severe bias** in predictions.
- Most residuals are concentrated around zero:**
 - A large number of residuals **fall within a small range around zero**, suggesting that **predictions are generally accurate**.
 - This confirms that **Lasso Regression is effective in reducing large errors**.
- Few outliers on both sides:**
 - While most residuals are small, **a few larger errors are present**, particularly in the negative direction.
 - This suggests that **the model may underpredict in some cases**.
- Low variance in errors:**
 - The spread of residuals is **relatively narrow**, indicating that **Lasso Regression produces stable and consistent predictions**.

Key Takeaways

- **Lasso Regression provides reliable predictions with minimal errors**.
- **Residuals are symmetrically distributed**, indicating **no significant systematic bias**.
- **A small number of extreme residuals exist**, but they do not drastically affect the overall model performance.
- **Potential improvements:**
 - Investigate the **features contributing to extreme residuals**.
 - Consider **fine-tuning Lasso's alpha parameter** to optimize performance.

Conclusion

- **Lasso Regression** effectively minimizes prediction errors, as seen in the concentrated residuals around zero.
- The **normal-like distribution** suggests the model generalizes well, making it a **strong candidate** for forecasting.

Top 5 Hyperparameter Combinations:

Rank 1:

Hyperparameters:

`{'alpha': 0.01}`

Mean Test Score: -0.000929

Standard Deviation: 0.000108

Rank 2:

Hyperparameters:

`{'alpha': 0.1}`

Mean Test Score: -0.010894

Standard Deviation: 0.001211

Rank 3:

Hyperparameters:

`{'alpha': 1.0}`

Mean Test Score: -0.165085

Standard Deviation: 0.013979

Rank 4:

Hyperparameters:

`{'alpha': 10.0}`

Mean Test Score: -0.165085

Standard Deviation: 0.013979

Rank 5:

Hyperparameters:

`{'alpha': 100.0}`

Mean Test Score: -0.165085

Standard Deviation: 0.013979

Effect of Alpha on Model Performance

Key Findings

- This plot analyzes how different values of **Alpha (regularization strength)** impact the **Negative Mean Squared Error (NMSE)** of the model.
 - **X-axis:** Alpha values (Regularization strength).
 - **Y-axis:** Negative Mean Squared Error (lower values indicate better performance).
 - **Dashed blue line with dots:** Trend of NMSE for different Alpha values.

Observations

1. **Drastic increase in NMSE with higher Alpha values:**

- At **Alpha = 0**, the model has the **lowest NMSE**, indicating **best performance**.
- As **Alpha increases**, NMSE **jumps significantly** and remains high across all tested values.

2. **Over-regularization weakens model performance:**

- When Alpha is very high (e.g., 10, 100), the error stabilizes at a much higher level.
 - This suggests that excessive regularization hinders the model's predictive ability.
3. Ideal Alpha selection is crucial:
- The best model performance occurs at $\text{Alpha} \approx 0$.
 - Introducing any significant regularization leads to increased error.

Key Takeaways

- Larger Alpha values introduce too much bias, leading to suboptimal model predictions.
- Minimal or no regularization ($\text{Alpha} \approx 0$) gives the best performance for this dataset.
- Fine-tuning Alpha is essential to balance bias-variance tradeoff in the model.

Conclusion

- This plot strongly suggests that regularization is not beneficial for this specific model and dataset.
- The optimal Alpha value should be close to zero to maintain high model accuracy.

SHAP Summary Plot for Lasso Regression

Key Findings

- This SHAP (SHapley Additive exPlanations) summary plot illustrates the impact of each feature on the model's predictions.
 - **Y-axis:** Features ranked by importance.
 - **X-axis:** SHAP values (impact on model output).
 - **Color Scale:**
 - Red indicates high feature values.
 - Blue indicates low feature values.

Observations

1. **Most influential features:**
 - lag_1 and rolling_mean have the highest SHAP values, meaning they strongly influence the model's predictions.
 - The impact of these features is significantly larger than other variables.
2. **Limited importance of economic indicators:**
 - Traditional macroeconomic variables like Consumer Price Index, Federal Funds Effective Rate, and Unemployment Rate have SHAP values centered around zero.
 - This suggests that the Lasso model disregards these features due to its regularization, confirming their weak influence on predictions.
3. **Lags (lag_2 to lag_5) have moderate impact:**
 - These lagged values exhibit some effect, though significantly lower than lag_1.
 - This indicates that recent past values are more predictive than older lag values.
4. **Symmetric distribution around zero:**
 - Most feature SHAP values are close to zero, reinforcing that the Lasso model selects only a few key features while shrinking others to zero.

Key Takeaways

- Lasso regression has aggressively selected a small subset of features, primarily lag_1 and rolling_mean.
- Macroeconomic indicators contribute minimally, implying they do not provide strong predictive power in this model.
- The model favors recent past values (lag_1) over older ones, aligning with typical time-series forecasting behavior.

Conclusion

- The SHAP summary plot validates the sparsity introduced by Lasso regression, demonstrating that only a few key variables significantly impact predictions.

- Feature selection is highly relevant, and economic indicators seem to be **less useful** for this specific model.

SHAP Force Plot (Lasso Regression)

Key Findings

- This **SHAP force plot** visualizes how individual features contribute to a **specific prediction**.
- The **base value (2.862)** represents the expected model prediction **before considering feature effects**.
- The **final prediction (2.93)** is derived by summing up the contributions of important features.

Observations

1. Key Features Driving the Prediction Higher:

- `rolling_mean = 2.208` and `lag_1 = 2.2` **significantly increase the prediction**.
- Both features are **highlighted in red**, indicating that their values are high and positively influence the model output.

2. No Features Pulling the Prediction Down:

- Unlike other force plots that may have blue elements (indicating negative impact), **this instance shows only positive contributions**, meaning no feature is reducing the predicted value.

3. Prediction Shift:

- The base value was **2.862**, but due to strong contributions from `rolling_mean` and `lag_1`, the **final prediction increased to 2.93**.

Key Takeaways

- The model heavily relies on `rolling_mean` and `lag_1`, confirming their dominance in influencing predictions.
- **No negative contributions** indicate that other features either had a **neutral impact** or were shrunk by the Lasso regularization.
- **Lasso regression aggressively selects key features**, and this force plot visually confirms the limited number of influential variables.

Conclusion

- The SHAP force plot effectively illustrates the **individual feature contributions to the final prediction**.
- The **strong positive influence of `rolling_mean` and `lag_1`** aligns with prior feature importance analyses, reinforcing their critical role in model predictions.

Partial Dependence Plots (Lasso Regression)

Key Findings

- This **Partial Dependence Plot (PDP)** shows how each individual feature impacts the model's prediction while keeping other features constant.
- It helps in understanding the **marginal effect** of each variable on the target outcome.

Observations

1. Strong Positive Relationship:

- `lag_1` shows a **clear upward trend**, indicating that higher values of `lag_1` increase the predicted outcome significantly.
- `rolling_mean` also has a slight positive trend, showing that as the rolling mean increases, predictions also slightly increase.

2. Weak or No Influence:

- Features like `Federal Funds Effective Rate`, `Unemployment Rate`, and `5-Year Forward Inflation Expectation Rate` have almost **flat** partial dependence lines, meaning these variables have

little or no effect on the model's predictions.

- **Personal Consumption Expenditures** and **Nominal Broad U.S. Dollar Index** also appear to have minimal impact.

3. Mixed Impact:

- **Consumer Price Index (CPI)** has a slight **negative trend**, indicating that an increase in CPI might slightly reduce the predicted outcome.
- **Real Gross Domestic Product (GDP)** and **Crude Oil Prices** show slight positive trends, meaning an increase in these variables tends to push the model's predictions higher.

Key Takeaways

- **Lasso Regression applies feature selection**, meaning only a few features contribute significantly to the predictions.
- **Lag-based variables (lag_1, rolling_mean)** are among the most influential, reinforcing the importance of past data trends in predictions.
- **Macroeconomic variables (CPI, GDP, Inflation Rate)** show minimal impact, suggesting that Lasso might have shrunk their coefficients to near zero.

Conclusion

- The model relies more on past values (**lag_1**, **rolling_mean**) rather than macroeconomic indicators.
- Flat trends in some variables suggest Lasso's regularization effect, where less important variables have little to no contribution.
- This analysis confirms the feature selection capability of Lasso Regression, focusing only on the most relevant predictors.

Permutation Importance for Lasso Regression

Key Findings

- This **Permutation Importance Plot** shows the contribution of each feature to the model's predictions.
- Features with **higher importance values** have a greater impact on model performance, while those with **near-zero importance** contribute little or nothing.

Observations

1. Dominant Feature:

- **lag_1** is by far the most **important feature**, with a significantly higher permutation importance than all other variables.
- This suggests that the model **heavily relies on past values** (**lag_1**) for making predictions.

2. Negligible Contribution of Other Features:

- Features like **Real Gross Domestic Product (GDP)**, **Consumer Price Index (CPI)**, and **Crude Oil Prices** show **very low importance**, indicating they have **little influence** on predictions.
- Macroeconomic variables such as **Unemployment Rate**, **Federal Funds Effective Rate**, and **Inflation Expectation Rate** have **almost zero impact** on model predictions.

3. Lasso's Feature Selection Effect:

- The Lasso Regression model has **eliminated most of the variables**, reducing their importance to almost zero.
- This aligns with the nature of **Lasso regularization**, which forces some coefficients to **exactly zero**, effectively removing less relevant features.

Key Takeaways

- **lag_1** is the **primary driver of predictions**, reinforcing that recent past values are the best predictors for this model.
- **Macroeconomic indicators play an insignificant role**, suggesting that historical data (lags) alone is sufficient for accurate predictions.
- **Lasso Regression acts as a strong feature selection method**, filtering out unimportant variables and keeping only the most relevant ones.

Conclusion

- The model is highly dependent on `lag_1` and discards other variables as non-essential.
- Lasso's regularization strength is high, shrinking most feature contributions to zero.
- This suggests that a simpler model using just lag-based variables might be sufficient, without the need for complex macroeconomic inputs.

ElasticNet Regression (EN)

ElasticNet Regression: Feature Selection, Model Evaluation, and Interpretability

1. Introduction

- ElasticNet Regression is a linear model that combines **L1 (Lasso)** and **L2 (Ridge)** regularization.
 - This approach allows for both **feature selection (Lasso)** and **coefficient shrinkage (Ridge)**.
 - The notebook covers:
 - Hyperparameter tuning using `GridSearchCV`
 - Model evaluation metrics
 - Feature importance analysis
 - SHAP interpretability
 - Partial Dependence and Permutation Importance analysis
-

2. Importing Essential Libraries

- **NumPy & Pandas**: Handling numerical and tabular data.
 - **Scikit-learn**:
 - **ElasticNet Regression**: The core regression model.
 - **GridSearchCV**: Exhaustive hyperparameter tuning.
 - **train_test_split**: Splitting data into training and testing subsets.
 - **StandardScaler**: Standardizing features to improve model stability.
-

3. Data Preparation and Preprocessing

- **Dataset Splitting**:
 - **Training Set (80%)**: Used for model training.
 - **Testing Set (20%)**: Used for model evaluation.
 - **Hyperparameter Grid for ElasticNet**:
 - `alpha`: Controls the strength of regularization.
 - `l1_ratio`: Defines the balance between L1 and L2 penalties.
 - `l1_ratio=1.0` → Pure Lasso
 - `l1_ratio=0.0` → Pure Ridge
 - Values in between apply a combination of both.
-

4. Hyperparameter Tuning using GridSearchCV

- **Grid Search Strategy** optimizes:
 - `alpha` over `[0.001, 0.01, 0.1, 1.0, 10.0]`
 - `l1_ratio` over `[0.1, 0.3, 0.5, 0.7, 0.9]`
 - **5-Fold Cross-Validation** ensures **robust model selection**.
 - **Parallel Processing** (`n_jobs=-1`) speeds up computation.
-

5. Model Training and Selection of Optimal Parameters

- **GridSearchCV** trains multiple models with different `alpha` and `l1_ratio` values.
 - The **best-performing model** is selected based on **Negative Mean Squared Error (MSE)**.
-

6. Model Prediction and Performance Evaluation

- The optimized ElasticNet model is used to **generate predictions** on the test dataset.
 - A predefined function `calculate_metrics` computes evaluation scores.
 - The model is benchmarked against a **naïve forecasting method**.
-

7. Feature Selection Analysis

- ElasticNet assigns importance to features via regression coefficients.
 - Zero coefficients indicate features removed due to L1 regularization.
 - A horizontal bar chart visualizes the retained features.
-

8. Actual vs. Predicted Values Visualization

- A scatter plot visualizes the ElasticNet Regression predictions.
 - A perfect fit reference line is included to assess accuracy.
-

9. Residual Analysis

Residual Scatter Plot

- Residuals (differences between actual and predicted values) are computed.
- A scatter plot visualizes residuals against actual values.
- A horizontal reference line ($y=0$) helps assess error distribution.

Residual Histogram

- A histogram illustrates the distribution of residuals.
 - A vertical reference line ($x=0$) is used to check for normality in residuals.
-

10. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The top 5 hyperparameter combinations are displayed.

Hyperparameter Tuning Line Plot

- A line plot visualizes the effect of `alpha` on **Negative Mean Squared Error (MSE)**.
-

11. Model Interpretability using SHAP

- SHAP (SHapley Additive ExPlanations) is used to interpret feature contributions.
 - K-means clustering is applied to **reduce background data** for SHAP calculations.
 - SHAP Summary Plot (global explanation) shows the impact of each feature.
 - SHAP Force Plot (local explanation) visualizes feature contributions for specific instances.
-

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
 - Multiple PDPs are plotted to examine global trends.
-

13. Permutation Feature Importance

- Permutation Importance quantifies how much shuffling a feature affects model performance.
 - A bar chart visualizes feature importance rankings.
-

14. Conclusion

This notebook systematically implements and evaluates an **ElasticNet Regression model** for predictive

modeling. Key contributions include:

1. **Automated feature selection** using **ElasticNet's combination of L1 and L2 regularization**.
2. **Comprehensive hyperparameter tuning** with **GridSearchCV**.
3. **Model evaluation** using residual analysis and benchmark comparisons.
4. **Feature importance insights** via SHAP and coefficient analysis.
5. **Model interpretability** through SHAP and Partial Dependence Plots.

Output Result:

Fitting 5 folds for each of 25 candidates, totalling 125 fits

Best Hyperparameters:

'alpha': 0.001,

'l1_ratio': 0.9

ElasticNet Regression Evaluation Metrics:

'Model': 'ElasticNet Regression (EN)',

'MSE': 0.0008566682982060062,

'RMSE': 0.029268896429588975,

'MAE': 0.017870444715126745,

'MAPE (%)': 1.331774581145144,

'sMAPE (%)': 1.2061165105100073,

'MASE': 0.045490217209015464,

'R^2': 0.9951552470762098

Non-Zero Feature Coefficients:

Feature Index		Feature	Coefficient
9	lag_1		6.976118e-01
14	rolling_mean		2.742880e-01
0	5-Year, 5-Year Forward Inflation Expectation Rate		1.456102e-02
4	Federal Funds Effective Rate		1.106055e-03
3	Unemployment Rate		9.268042e-04
7	Crude Oil Prices: West Texas Intermediate		2.667544e-04
2	Real Gross Domestic Product		8.729704e-06
6	Nominal Broad U.S. Dollar Index		5.546011e-07
1	Consumer Price Index		-5.078478e-04

Feature Coefficients (ElasticNet Regression)

Key Findings

- This plot represents the **feature coefficients** in an **ElasticNet Regression model**.
- Features with **higher absolute coefficient values** contribute more significantly to the model's predictions.

Observations

1. Most Influential Features:

- **lag_1** has the **largest coefficient**, making it the most significant predictor.
- **rolling_mean** is the **second most important feature**, though its contribution is smaller than **lag_1**.
- **5-Year Forward Inflation Expectation Rate** has a **minor contribution**, but it's still relevant.

2. Negligible or Zero Impact Features:

- Features such as `Federal Funds Effective Rate`, `Unemployment Rate`, `Crude Oil Prices`, and `GDP` have **very small or nearly zero coefficients**.
- **Macroeconomic variables do not significantly influence predictions**, indicating that recent past values (lags) and rolling averages dominate the forecasting.

3. ElasticNet's Effect on Feature Selection:

- **ElasticNet applies both L1 (Lasso) and L2 (Ridge) regularization**, which helps:
 - **Shrink irrelevant coefficients towards zero** (like `CPI`, `GDP`, and `Unemployment Rate`).
 - **Retain a few important variables** (`lag_1` and `rolling_mean`) with reduced coefficients.

Key Takeaways

- `lag_1` remains the **dominant predictor**, showing that recent past values drive forecasts.
- **ElasticNet eliminates weak predictors**, keeping only the most informative ones.
- **Rolling average** (`rolling_mean`) plays a **secondary role**, contributing to trend-based forecasting.
- **Macroeconomic indicators have little predictive power** for this model.

Conclusion

- The **ElasticNet model confirms** that recent past values (`lag_1`) and rolling trends (`rolling_mean`) are the **strongest predictors**.
- **Macroeconomic factors** like inflation, interest rates, and unemployment contribute **very little**, aligning with previous findings from Lasso and Ridge regression.
- **ElasticNet provides an optimal balance** between feature selection and coefficient shrinkage, making it a **robust model** for handling collinearity and irrelevant variables.

Actual vs Predicted Values (ElasticNet Regression)

Key Findings

- This scatter plot visualizes the **performance of the ElasticNet regression model** by comparing **actual vs. predicted values**.
- The **red dashed line** represents a **perfect fit**, where predicted values exactly match the actual values.
- The **blue dots** represent individual data points, showing how close the model's predictions are to the actual values.

Observations

1. Strong Alignment with Perfect Fit:

- Most of the **blue dots closely follow the red dashed line**, indicating that the **model's predictions are highly accurate**.
- There are **very few large deviations**, suggesting that the model does **not have significant systematic errors**.

2. Minimal Bias or Skewness:

- The **errors (residuals)** appear **symmetrically distributed around the perfect fit line**, implying that the model does **not systematically overestimate or underestimate** values.
- There is **no noticeable trend** of the model performing better in specific value ranges.

3. Outliers and Slight Deviations:

- Some data points show **slight deviations** from the perfect fit, particularly for **lower values** (near 0) and **higher values** (above 2.5).
- These outliers suggest **small prediction errors**, which may be due to data noise or model limitations.

Key Takeaways

- **ElasticNet regression performs well**, as the **predicted values align very closely with actual values**.
- **There is no major bias**, meaning the model **does not favor overprediction or underprediction**.
- **Only minor errors are present**, mostly at **extreme values**, but they do not significantly impact

overall performance.

- **ElasticNet successfully balances feature selection and regularization**, leading to an accurate and stable model.

Conclusion

- This plot **validates the effectiveness of the ElasticNet regression model** for predicting the target variable.
- The **tight clustering around the perfect fit line** suggests that the model captures patterns well with **minimal overfitting or underfitting**.
- Further improvements could involve **fine-tuning hyperparameters** or **handling outliers** for even better accuracy.

Residual Plot (ElasticNet Regression)

Key Findings

- This residual plot visualizes **the difference between actual values and predicted values** from the **ElasticNet regression model**.
- The **y-axis represents residuals (errors)**, and the **x-axis represents actual values**.
- The **red dashed line at $y = 0$** indicates **perfect predictions**, where residuals would be zero.

Observations

1. Randomly Scattered Residuals:

- The **residuals are randomly distributed around the red dashed line**, suggesting that the model does not have **systematic bias**.
- There is **no clear pattern**, which is a good sign that the model's errors are not correlated with the actual values.

2. Mostly Small Residuals:

- The majority of points are **concentrated near zero**, indicating that **the ElasticNet model predicts well** for most data points.
- Some **larger residuals** exist, particularly for lower and higher actual values, but they are **not extreme**.

3. Potential Slight Heteroscedasticity:

- The spread of residuals **appears slightly wider at higher actual values**, suggesting that **prediction errors might increase slightly for larger values**.
- However, the effect is **not very pronounced**, meaning **the model still performs well across the range of values**.

4. No Major Outliers:

- While some residuals deviate more from zero, there **are no extreme outliers**, which suggests that the model is **robust**.

Key Takeaways

- **ElasticNet regression does not show systematic bias**, as residuals are **randomly distributed around zero**.
- **Most predictions are accurate**, with **only minor deviations** observed for some values.
- **The model performs well overall**, but there may be slight increases in error for larger actual values.
- **No major overfitting or underfitting is detected**, making the model reliable for predictive tasks.

Conclusion

- This residual plot confirms that **ElasticNet regression provides stable and well-balanced predictions**.
- Any further improvements could involve **checking for slight heteroscedasticity** and **adjusting hyperparameters to refine error distributions**.

Residual Distribution (ElasticNet Regression)

Key Findings

- This histogram displays the **distribution of residuals (errors)** from the **ElasticNet regression model**.
- The **x-axis represents residual values**, while the **y-axis represents their frequency**.
- The **red dashed line at zero** indicates **perfect predictions**, where residuals would be zero.

Observations

1. **Residuals are Centered Around Zero:**
 - The residuals **mostly cluster around zero**, indicating that the **model predictions are unbiased** on average.
 - This suggests **no significant systematic overestimation or underestimation** in the predictions.
2. **Near-Normal Distribution:**
 - The histogram forms a **bell-shaped curve**, resembling a normal distribution.
 - This is a **good indicator** that the model's errors are **randomly distributed**, which is a key assumption in linear regression models.
3. **Low Variance in Errors:**
 - Most residuals fall within a **narrow range close to zero**, indicating that **prediction errors are small**.
 - The **frequency of large residuals is very low**, suggesting **good model performance**.
4. **Minimal Outliers:**
 - There are **very few extreme residuals on either side**, indicating **no significant outliers or extreme prediction errors**.

Key Takeaways

- ElasticNet regression performs well, as its residuals are **normally distributed and centered around zero**.
- **Prediction errors are small**, meaning the model provides **consistent and reliable estimates**.
- **No major outliers or extreme deviations are present**, reinforcing **model robustness**.
- The model's **assumptions about error distribution appear valid**, making it **suitable for further predictive tasks**.

Conclusion

- This residual distribution confirms that **ElasticNet regression is an effective model for this dataset**.
- Future improvements could involve **fine-tuning hyperparameters** to further **reduce the spread of residuals**.

Top 5 Hyperparameter Combinations:

Rank 5

Hyperparameters: `{'alpha': 0.001, 'l1_ratio': 0.9}`
Mean Test Score: -0.000828
Standard Deviation: 0.000086

Rank 4

Hyperparameters: `{'alpha': 0.001, 'l1_ratio': 0.7}`
Mean Test Score: -0.000829
Standard Deviation: 0.000087

Rank 3

Hyperparameters: `{'alpha': 0.001, 'l1_ratio': 0.5}`
Mean Test Score: -0.000841
Standard Deviation: 0.000091

Rank 2

Hyperparameters: {'alpha': 0.001, 'l1_ratio': 0.3}

Mean Test Score: -0.000868

Standard Deviation: 0.000096

Rank 1

Hyperparameters: {'alpha': 0.001, 'l1_ratio': 0.1}

Mean Test Score: -0.000890

Standard Deviation: 0.000099

Effect of Alpha on Model Performance

Key Findings

- This plot illustrates the relationship between the regularization strength (alpha) and the negative mean squared error (MSE) of the model.
- The x-axis represents the alpha value, which controls the level of regularization.
- The y-axis represents the negative mean squared error, with lower values indicating better model performance.

Observations

1. Increasing Alpha Decreases Model Performance:

- As the alpha value increases, the negative mean squared error also increases.
- This suggests that stronger regularization leads to higher error, possibly due to excessive constraint on model complexity.

2. Optimal Alpha is Near Zero:

- The lowest error is observed at the smallest alpha values, suggesting that minimal regularization is ideal for this dataset.
- Very small alpha values help control overfitting without overly restricting model flexibility.

3. Significant Performance Drop at High Alpha Values:

- As alpha reaches larger values (around 10), the error drastically increases.
- This indicates that excessive regularization weakens the model's ability to capture patterns, leading to underfitting.

4. Stable Performance in Mid-Range Alpha:

- There is a plateau in the middle range of alpha values, where the error remains relatively stable.
- This suggests a trade-off between underfitting and overfitting, which could be a candidate for optimal alpha selection.

Key Takeaways

- Too much regularization (high alpha) results in poor model performance due to underfitting.
- Minimal or moderate regularization (low alpha) yields the best performance.
- An optimal alpha should be chosen to balance overfitting (low alpha) and underfitting (high alpha).
- Cross-validation should be used to fine-tune and find the best alpha for the dataset.

Conclusion

- The model performs best when alpha is near zero, indicating that strong regularization is not necessary.
- To optimize performance, hyperparameter tuning should be applied to find the best balance.

SHAP Summary Plot - Feature Importance Analysis

Key Findings

- This SHAP summary plot illustrates the **impact of different features on the model's predictions**.
- The **x-axis represents the SHAP value**, which indicates how much a feature influences the model's output.
- The **y-axis lists the features**, sorted by importance from top to bottom.
- The **color bar (blue to red) indicates feature values**, where **blue represents lower values** and **red represents higher values**.

Observations

1. Most Important Features:

- **lag_1** and **rolling_mean** are the most influential features affecting the model's predictions.
- The large spread of SHAP values for **lag_1** suggests it has a **strong and consistent impact** on predictions.
- **rolling_mean** also plays a crucial role, showing that **past trends are significant**.

2. Medium-Impact Features:

- **Real Gross Domestic Product (GDP), Consumer Price Index (CPI), and Crude Oil Prices** have a moderate effect on the model's output.
- Their **SHAP values are more centered around zero**, indicating lesser but still meaningful influence.

3. Low-Impact Features:

- Features like **Personal Consumption Expenditures** and some lags (**lag_4**, **lag_5**, etc.) have minimal impact.
- Their SHAP values remain **close to zero**, meaning they **do not significantly contribute to predictions**.

4. SHAP Value Interpretation:

- **Positive SHAP values** mean the feature **increases the model's prediction**.
- **Negative SHAP values** mean the feature **lowers the prediction**.
- The spread of values shows whether a feature has a **strong or weak effect** on the prediction.

Key Takeaways

- **lag_1** is the most predictive feature, suggesting that **recent past values significantly influence future predictions**.
- **Rolling mean captures trends effectively**, indicating that **smoothing techniques are useful in the model**.
- **Macroeconomic indicators like GDP and CPI have some predictive power**, but their effects are **weaker than past trends**.
- **Some features contribute very little**, implying they **could be dropped for a more efficient model**.

Conclusion

- The SHAP analysis confirms that **recent lagged values and trend features drive the model's predictions**.
- **Feature selection should prioritize lag_1 and rolling_mean** while considering removing low-impact features.
- **Interpreting SHAP values helps understand the model's decision-making process**, making it more explainable.

SHAP Force Plot - Feature Contributions to Prediction

Key Findings

- This **SHAP force plot visualizes the contribution of different features to a single prediction**.
- The **x-axis represents the model's output range**, with the **base value (2.039)** acting as a reference point.
- The **red segments indicate features that push the prediction higher**, while the **blue segments indicate features that push it lower**.
- The final predicted value is **2.20**.

Observations

1. Features Increasing the Prediction (Red)

- `lag_1 = 2.2` is the **strongest contributor**, pushing the prediction significantly higher.
- **Rolling mean (`rolling_mean = 2.208`)** also plays a major role in increasing the output.
- **Crude Oil Prices (`West Texas Intermediate = 106.7`)** and **Consumer Price Index (`CPI = 229.4`)** contribute to the upward shift, indicating their positive influence on the model.

2. Feature Decreasing the Prediction (Blue)

- **Real Gross Domestic Product (`GDP = 1.737e+4`)** is the **only feature pushing the prediction lower**, but its effect is relatively minor compared to the positive contributors.

3. Overall Impact

- The prediction is driven primarily by lag features (`lag_1`) and trend-based features (`rolling_mean`).
- **Macroeconomic factors like crude oil prices and CPI also contribute**, but with less impact.
- **GDP has a negative effect**, suggesting that in this instance, higher GDP may slightly decrease the predicted value.

Key Takeaways

- **Historical trends (`lag_1` , `rolling_mean`)** dominate the prediction.
- **Economic indicators (CPI, crude oil prices)** play a meaningful role but are secondary to lag features.
- **GDP has a slight downward effect**, possibly due to economic cycles or model-specific relationships.
- **This visualization helps interpret individual predictions by showing which features push values up or down.**

Conclusion

- The force plot confirms that **past values (`lag_1`)** and rolling trends are the key drivers of the model's output.
- **Macroeconomic indicators contribute but have smaller impacts** compared to historical patterns.
- **Understanding feature contributions like this improves model interpretability and trust in predictions.**

Partial Dependence Plots (ElasticNet Regression)

Key Findings

- The **Partial Dependence Plots (PDPs)** illustrate the marginal effect of each feature on the model's predictions.
- The **y-axis represents the partial dependence (model output)**, while the **x-axis represents the feature values**.
- These plots help identify whether a feature has a **positive, negative, or neutral impact** on predictions.

Observations

1. Strong Positive Influence

- `lag_1` and `rolling_mean` show a strong **positive trend**, indicating that as these values increase, the model's predicted output increases.
- This suggests that the model **relies heavily on past values (lag features) and trend-based statistics**.

2. Weak or No Influence

- Several macroeconomic variables, such as:
 - **5-Year Forward Inflation Expectation Rate**
 - **Federal Funds Effective Rate**
 - **Nominal Broad U.S. Dollar Index**
 - **Unemployment Rate**
- These plots remain **flat**, meaning these features have **minimal influence** on the model's predictions.

3. Mixed Effects

- **Consumer Price Index (CPI)** has a slight **negative impact**, implying that higher CPI values may decrease the predicted output.
 - **Real Gross Domestic Product (GDP)** has a slight **positive trend**, meaning higher GDP may marginally increase the predicted outcome.
4. **Feature Importance Confirmation**
- The strong dependency of the model on **lag_1** and **rolling_mean** aligns with other feature importance evaluations (e.g., SHAP values, coefficient plots).
 - The **low influence of macroeconomic indicators** suggests that the model relies more on historical patterns than economic conditions.

Key Takeaways

- **Past values (lag_1 , rolling_mean) drive model predictions**, reinforcing the reliance on historical trends.
- **Macroeconomic factors (inflation rate, interest rate, unemployment, etc.) have little impact**, suggesting they do not significantly alter predictions in this setting.
- **Consumer Price Index (CPI) has a negative effect, while GDP has a slight positive effect**, reflecting their possible roles in the model.
- **This analysis validates the model's feature dependencies and helps interpret the prediction mechanism.**

Conclusion

- The **ElasticNet Regression model relies primarily on lag-based features and rolling averages.**
- **Macroeconomic variables do not significantly contribute** to the model's decisions.
- **Understanding these dependencies improves interpretability and allows for better feature engineering.**

Permutation Feature Importance for ElasticNet Regression:

Feature Index		Feature	Importance
9	lag_1		9.896920e-01
14	rolling_mean		1.547433e-01
2	Real Gross Domestic Product		3.961436e-03
1	Consumer Price Index		2.485186e-03
0	5-Year, 5-Year Forward Inflation Expectation Rate		4.811733e-04
7	Crude Oil Prices: West Texas Intermediate		4.201348e-04
4	Federal Funds Effective Rate		9.368184e-05
3	Unemployment Rate		5.048701e-05
6	Nominal Broad U.S. Dollar Index		1.092271e-08
5	Personal Consumption Expenditures: Chain-type ...		0.000000e+00
8	10-Year Treasury Yield		0.000000e+00
10	lag_2		0.000000e+00
11	lag_3		0.000000e+00
12	lag_4		0.000000e+00
13	lag_5		0.000000e+00

Permutation Importance for ElasticNet Regression

Key Findings

- The plot illustrates the **importance of each feature** in the ElasticNet Regression model using **permutation importance**.
- **Higher permutation importance means a feature significantly affects the model's predictions** when shuffled.

Observations

1. Most Important Features

- `lag_1` is the most crucial feature, contributing significantly to the model's predictions.
- `rolling_mean` follows as the second most influential feature, suggesting that trend-based historical values play an essential role.

2. Minimal to No Impact Features

- Several macroeconomic indicators show nearly zero permutation importance, including:
 - Real Gross Domestic Product (GDP)
 - Consumer Price Index (CPI)
 - Crude Oil Prices: West Texas Intermediate
 - Unemployment Rate
 - Federal Funds Effective Rate
- This implies that the model does not rely on these features to make predictions.

3. Lag-Based Features Dominate

- Lagged values (`lag_1`) and rolling averages (`rolling_mean`) have the highest importance, reinforcing that the model relies on past data rather than external economic factors.

Key Takeaways

- ElasticNet Regression is primarily driven by historical trends (`lag_1` and `rolling_mean`).
- Macroeconomic variables do not contribute significantly to the model's predictions.
- Feature importance aligns with previous analyses (e.g., coefficient plots, SHAP values), validating that lag-based features dominate the prediction process.

Conclusion

- The model is highly dependent on historical patterns rather than economic indicators.
- Macroeconomic factors do not play a critical role, suggesting they could be omitted without significantly affecting performance.
- Understanding feature importance helps refine feature selection and improve interpretability.

Decision Tree Regressor (DTR)

Decision Tree Regression: Model Training, Evaluation, and Interpretability

1. Introduction

- Decision Tree Regression (DTR) is a non-linear model that partitions the data into hierarchical regions.
- This notebook covers:
 - Hyperparameter tuning using `GridSearchCV`
 - Model performance evaluation
 - Feature importance analysis
 - SHAP-based interpretability
 - Partial Dependence and Decision Tree visualization

2. Importing Essential Libraries

- NumPy & Pandas: Handling numerical and tabular data.
- Scikit-learn:
 - `DecisionTreeRegressor`: Core regression model.
 - `GridSearchCV`: Hyperparameter tuning.
 - `train_test_split`: Splitting data for training and testing.

3. Data Preprocessing

- Dataset Splitting:
 - Training Set (80%): Used for model training.
 - Testing Set (20%): Used for evaluation.

- **Hyperparameter Grid for Decision Tree:**
 - `max_depth`: Controls tree depth.
 - `min_samples_split`: Minimum samples required for a split.
 - `min_samples_leaf`: Minimum samples required at a leaf node.
 - `max_features`: Number of features to consider at each split.
-

4. Hyperparameter Tuning using GridSearchCV

- **Grid Search Strategy** optimizes:
 - `max_depth` over `[3, 5, 7, 10, None]`
 - `min_samples_split` over `[2, 5, 10]`
 - `min_samples_leaf` over `[1, 2, 4]`
 - `max_features` over `['sqrt', 'log2', None]`
 - **5-Fold Cross-Validation** ensures **robust model selection**.
 - **Parallel Processing** (`n_jobs=-1`) speeds up computation.
-

5. Model Training and Selection of Optimal Parameters

- **GridSearchCV** trains multiple models with different hyperparameters.
 - The **best-performing model** is selected based on **Negative Mean Squared Error (MSE)**.
-

6. Model Prediction and Performance Evaluation

- The optimized Decision Tree model is used to **generate predictions** on the test dataset.
 - A predefined function `calculate_metrics` computes evaluation scores.
 - The model is benchmarked against a **naïve forecasting method**.
-

7. Feature Importance Analysis

- **Decision Trees** assign importance to features based on their contribution to splitting the dataset.
 - A **horizontal bar chart** visualizes the retained features.
-

8. Actual vs. Predicted Values Visualization

- A **scatter plot** visualizes the Decision Tree Regression predictions.
 - A **perfect fit reference line** is included to assess accuracy.
-

9. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line** (`y=0`) helps assess error distribution.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
 - A **vertical reference line** (`x=0`) is used to check for normality in residuals.
-

10. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Line Plot

- A line plot visualizes the effect of `max_depth` on Negative Mean Squared Error (MSE).

11. Model Interpretability using SHAP

- SHAP (SHapley Additive ExPlanations) is used to interpret feature contributions.
- SHAP Summary Plot (global explanation) shows the impact of each feature.
- SHAP Force Plot (local explanation) visualizes feature contributions for specific instances.

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
- Multiple PDPs are plotted to examine global trends.

13. Decision Tree Visualization

- A visualization of the trained Decision Tree is generated using `plot_tree()`.
- The first few levels of the tree are displayed to interpret decision boundaries.

14. Conclusion

This notebook systematically implements and evaluates a **Decision Tree Regression model** for predictive modeling. Key contributions include:

1. **Automated hyperparameter tuning** for optimal model selection.
2. **Comprehensive model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via Decision Tree splitting rules.
4. **Model interpretability** through SHAP and Partial Dependence Plots.
5. **Decision Tree visualization** for understanding model decisions.

Result Output:

Fitting 5 folds for each of 135 candidates, totalling 675 fits

Best Hyperparameters:

```
'max_depth': 7,  
'max_features': None,  
'min_samples_leaf': 2,  
'min_samples_split': 2
```

Decision Tree Regressor Evaluation Metrics:

```
'Model': 'Decision Tree Regressor (DTR)',  
'MSE': 0.001038835453532052,  
'RMSE': 0.03223097040940673,  
'MAE': 0.020456966898434143,  
'MAPE (%)': 1.333935754901849,  
'SMAPE (%)': 1.3303441257218145,  
'MASE': 0.052074354191067956,  
'R^2': 0.9941250293592327
```

Feature Importances:

Feature Index		Feature	Importance
9		lag_1	0.991238

14	rolling_mean	0.007839
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.000328
13	lag_5	0.000138
8	10-Year Treasury Yield	0.000127
12	lag_4	0.000093
11	lag_3	0.000093
6	Nominal Broad U.S. Dollar Index	0.000072
2	Real Gross Domestic Product	0.000029
10	lag_2	0.000022
3	Unemployment Rate	0.000014
7	Crude Oil Prices: West Texas Intermediate	0.000008
5	Personal Consumption Expenditures: Chain-type ...	0.000000
1	Consumer Price Index	0.000000
4	Federal Funds Effective Rate	0.000000

Feature Importance (Decision Tree Regression)

Key Findings

- The plot displays the **importance of each feature** in the Decision Tree Regression model.
- **Higher importance values indicate that a feature significantly contributes** to the model's decision-making.

Observations

1. Most Important Features

- **lag_1** is by far the most dominant feature, with an importance value close to **1.0**.
- **rolling_mean** is the second most relevant feature, but its importance is **significantly lower** than **lag_1**.

2. Minimal to No Impact Features

- Most macroeconomic variables such as:
 - **Real Gross Domestic Product (GDP)**
 - **Consumer Price Index (CPI)**
 - **Unemployment Rate**
 - **Crude Oil Prices: West Texas Intermediate**
 - **Federal Funds Effective Rate**
- **Have almost no impact**, meaning the Decision Tree model does not consider them important for predictions.

3. Lag-Based Features Are Crucial

- **lag_1** dominates the feature importance ranking, reinforcing that **past values have the highest predictive power**.
- Rolling averages (**rolling_mean**) and additional lag values (**lag_5**, **lag_4**, **lag_3**, **lag_2**) contribute marginally.
- **Macroeconomic factors are nearly irrelevant** to the model.

Key Takeaways

- **Decision Tree Regression is entirely driven by historical values, especially lag_1.**
- **Macroeconomic indicators have little to no influence**, which may indicate they could be removed from the model without affecting performance.
- **Feature importance aligns with previous regression-based analyses**, further confirming that past values are the primary predictive indicators.

Conclusion

- The model relies almost exclusively on **lag_1**, meaning it prioritizes recent historical values over external economic indicators.
- **Macroeconomic variables contribute negligibly**, suggesting that the Decision Tree model behaves

similarly to a time series forecasting approach with minimal external influence.

- **Understanding feature importance is critical for refining model inputs and optimizing prediction accuracy.**

Actual vs Predicted Values (Decision Tree Regression)

Key Findings

- The plot visualizes the relationship between the **actual values (x-axis)** and the **predicted values (y-axis)** from the Decision Tree Regression model.
- **The red dashed line represents the ideal perfect fit line**, where predictions would match actual values exactly.
- **Each blue dot represents a single prediction**, showing how closely the model's output aligns with the actual data.

Observations

1. Strong Alignment with the Perfect Fit

- The majority of points **fall very close to the red dashed line**, indicating that the model predicts well.
- This suggests a **low error rate and high predictive accuracy**.

2. Slight Deviations for Smaller Values

- For **lower values (close to 0)**, some predictions appear slightly off the perfect fit line.
- This may indicate **slight underfitting or overfitting in lower-value regions**.

3. Minor Outliers Present

- A few blue points are **visibly farther from the red dashed line**, indicating **instances where the model makes noticeable errors**.
- These deviations could be due to **irregularities in the data, model limitations, or noise**.

4. Decision Tree Behavior Evident

- The **stepped nature of predictions** suggests that the Decision Tree model is producing **piecewise constant approximations** rather than a smooth fit.
- This aligns with how decision trees segment data into discrete regions.

Key Takeaways

- **The model performs well overall**, with predictions closely following the actual values.
- **Minor errors exist**, particularly in **lower-value regions and some outlier cases**.
- **The step-like nature of predictions** indicates a decision tree's characteristic behavior of creating discrete partitions.
- **Further tuning or pruning of the decision tree might help smooth predictions and reduce overfitting risks.**

Conclusion

- The **Decision Tree Regression model captures patterns effectively**, but some improvements could be made.
- If **smoother predictions** are desired, **ensemble methods like Random Forests or Gradient Boosting** may be considered.
- Overall, the model provides **reasonably accurate predictions**, making it a useful tool for structured, rule-based forecasting.

Residual Plot (Decision Tree Regression)

Key Findings

- This plot visualizes the **residuals (y-axis)** against the **actual values (x-axis)** from the Decision Tree Regression model.
- **Residuals = Actual values - Predicted values** represent the difference between actual and predicted values.

- The **red dashed line at $y = 0$** represents a perfect model with zero residuals.

Observations

1. Centered Residuals Around Zero

- The residuals are **scattered around the red dashed line**, indicating that the model predictions do not have a strong bias.
- A good regression model should have residuals centered around zero, which this plot largely exhibits.

2. Heteroscedasticity Present

- The residuals appear to be **more dispersed at higher actual values**.
- This suggests that the model might struggle with **higher values**, leading to increased prediction variance.

3. Presence of Outliers

- Some points **deviate significantly from the red dashed line**, meaning there are instances where the model has **larger errors**.
- These outliers could indicate **overfitting to certain regions or anomalies in the data**.

4. Pattern Suggesting Overfitting

- The clustering of residuals at **distinct levels** suggests the model is **overfitting to the training data**.
- Decision Trees tend to overfit by creating **hard splits in the data**, resulting in step-like behavior in predictions.

Key Takeaways

- **Model performance is generally good**, as residuals are centered around zero.
- **Prediction errors increase for higher actual values**, indicating **potential overfitting or non-linearity in the data**.
- A **Random Forest or Pruned Decision Tree model** might improve generalization, reducing prediction variance.

Conclusion

- The model captures the general pattern well but **shows signs of overfitting**.
- Further improvements like **pruning the tree, ensembling methods (Random Forest), or regularization** could help reduce variance and improve predictions.

Residual Distribution (Decision Tree Regression)

Key Findings

- This histogram displays the **distribution of residuals** (errors) from the Decision Tree Regression model.
- **Residuals = Actual values - Predicted values**, representing the difference between actual and predicted values.
- The **red dashed line at zero** represents the ideal case where residuals are centered around zero.

Observations

1. Centered Around Zero

- The residuals are **symmetrically distributed around zero**, suggesting that the model is **unbiased**.
- This indicates that the Decision Tree model does not systematically over-predict or under-predict.

2. Near-Normal Distribution

- The histogram shows a **bell-shaped curve**, which is a **good sign for model performance**.
- However, the peak is slightly higher than a perfect normal distribution, which may indicate **some overfitting**.

3. Presence of Outliers

- There are a few residuals **significantly deviating from zero**, suggesting that the model

struggles with certain instances.

- These **outliers** might be caused by **overfitting** or anomalies in the data.

4. Skewness & Variance

- The **spread of residuals is relatively narrow**, indicating that **prediction errors are not extremely large**.
- However, some skewness in distribution might suggest that **certain data ranges are harder to predict**.

Key Takeaways

- The **Decision Tree Regression model performs well overall**, as most residuals are **close to zero**.
- Some **outliers and potential overfitting** can be addressed with **pruning techniques or ensemble methods** like **Random Forest**.
- If the goal is better generalization, **reducing tree depth or using boosting methods** can help.

Conclusion

- The residuals are **mostly well-behaved**, but the **presence of outliers and slight overfitting suggests room for improvement**.
- Techniques like **Random Forest, Gradient Boosting, or Hyperparameter tuning** could help improve **model generalization** and reduce variance.

Top 5 Hyperparameter Combinations:

Rank 76

Hyperparameters: `{'max_depth': 7, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 2}`

Mean Test Score: -0.001073

Standard Deviation: 0.000107

Rank 77

Hyperparameters: `{'max_depth': 7, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 5}`

Mean Test Score: -0.001086

Standard Deviation: 0.000103

Rank 78

Hyperparameters: `{'max_depth': 7, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 10}`

Mean Test Score: -0.001092

Standard Deviation: 0.000094

Rank 79

Hyperparameters: `{'max_depth': 7, 'max_features': None, 'min_samples_leaf': 4, 'min_samples_split': 2}`

Mean Test Score: -0.001097

Standard Deviation: 0.000086

Rank 80

Hyperparameters: `{'max_depth': 7, 'max_features': None, 'min_samples_leaf': 4, 'min_samples_split': 5}`

Mean Test Score: -0.001097

Standard Deviation: 0.000086

Effect of Max Depth on Model Performance

Key Findings

- This plot shows how **maximum tree depth** impacts the **negative mean squared error (MSE)** in a Decision Tree Regression model.
- The x-axis represents **Max Depth** (complexity of the tree).
- The y-axis represents **Negative Mean Squared Error** (lower values indicate better performance).

Observations

1. Initial Improvement with Increasing Depth

- As the max depth increases from 3 to 5, the **negative mean squared error decreases significantly**.
- This suggests that **deeper trees capture more patterns** in the data, improving performance.

2. Diminishing Returns Beyond Depth 5

- Beyond depth 5, the performance **stabilizes** and does not improve significantly.
- At depths 7 to 10, there is **only a marginal reduction** in error, indicating **overfitting risk**.

3. Overfitting Trend at Higher Depths

- The slight increase in error at **depth 7, 9, and 10** suggests **overfitting**.
- This happens when the model becomes **too complex**, capturing noise rather than general patterns.

Key Takeaways

- **Optimal Depth: ~5 to 7**
 - A depth of 5 to 7 **appears to be the sweet spot**, balancing complexity and performance.
 - Shallower trees underfit, while deeper trees overfit.
- **Regularization is Needed for Deeper Trees**
 - If using a **depth >7**, **pruning techniques** or **ensemble methods (Random Forest, Gradient Boosting)** should be considered.
- **Hyperparameter Tuning is Essential**
 - Choosing the **right max depth** is crucial to prevent **underfitting (too shallow)** or **overfitting (too deep)**.

Conclusion

- The **best model depth is around 5-7**, balancing **performance and generalization**.
- Beyond **depth 7**, performance gains are minimal, and **overfitting starts to appear**.
- Regularization techniques like **pruning, cross-validation, or ensemble models** should be used to optimize performance further.

SHAP Summary Plot Analysis

Key Findings

- This SHAP summary plot visualizes the **impact of each feature on the model's predictions**.
- The x-axis represents the **SHAP value**, which indicates **how much a feature influences the model's output** (positive or negative).
- The y-axis lists the **features in order of importance**, with the most influential features at the top.
- The color scale represents **feature values**:
 - **Red (High values)**
 - **Blue (Low values)**

Observations

1. Most Important Features

- **lag_1** has the highest impact on the model's predictions.
- **rolling_mean** is the second most influential feature.
- Other important features include **5-Year Forward Inflation Expectation Rate**, **lag_5**, and **lag_3**.

2. Positive & Negative Impact

- Features with positive SHAP values (right side) contribute to higher model outputs.
- Features with negative SHAP values (left side) contribute to lower model outputs.

3. Feature Influence Patterns

- Features with a wider spread of SHAP values (e.g., `lag_1`, `rolling_mean`) indicate greater variability in their impact.
- Features with small SHAP values (closer to 0) contribute less to predictions.

Key Takeaways

- `lag_1` and `rolling_mean` are the most significant predictors.
- Lag-based variables play a crucial role in predicting the target variable.
- Economic indicators such as the 5-Year Forward Inflation Expectation Rate and Treasury Yield have moderate impact.
- Features lower in the plot have minimal contribution to model predictions.

Conclusion

- The model heavily relies on lagged values (especially `lag_1`) and rolling averages.
- Economic indicators like inflation expectations and interest rates have some influence.
- Understanding SHAP values helps in feature selection and model interpretability.

SHAP Force Plot Analysis

Key Findings

- This SHAP force plot visualizes how individual features contribute to a specific prediction.
- The x-axis represents the prediction range, with the base value as the starting point.
- Red features push the prediction higher, while blue features push it lower.

Observations

1. Prediction Output (`f(x) = 2.21`)

- The final model prediction is 2.21, which is higher than the base value (2.056).
- This means that certain features contributed positively to increase the prediction.

2. Top Contributing Features

- `lag_1 = 2.2` has the highest positive impact on the prediction.
- `5-Year, 5-Year Forward Inflation Expectation Rate = 2.47` also significantly contributes to pushing the prediction higher.

3. Feature Impact on Prediction

- All highlighted features in red had a positive influence, increasing the predicted value.
- There are no blue features in this plot, meaning no features pushed the prediction downward.

Key Takeaways

- Lag-based variables (`lag_1`) play a dominant role in the prediction.
- Inflation expectations also have a significant effect on the predicted value.
- The model's decision-making process is strongly influenced by past values and economic indicators.

Conclusion

- The force plot provides transparency into individual predictions.
- Understanding these contributions can help in interpreting and validating model decisions.
- Future model improvements could involve analyzing if additional features could contribute further to predictions.

Partial Dependence Plots (Decision Tree Regression) Analysis

Key Findings

- The plots illustrate the **marginal effect of each feature on the model's predictions**.
- **X-axis**: Represents the feature values.
- **Y-axis**: Shows the corresponding partial dependence, indicating the model's predicted response.

Observations

1. Feature Influence on Predictions

- Most macroeconomic features, such as **Consumer Price Index, Federal Funds Rate, Real GDP, and Unemployment Rate**, show **little to no variation in partial dependence**.
- The model does not strongly respond to changes in these features.

2. Most Influential Features

- **lag_1** shows a **strong positive trend**, indicating that increasing **lag_1** **significantly raises predictions**.
- **Rolling mean (rolling_mean)** and **lag variables (lag_2, lag_3, lag_4, lag_5)** have **minimal impact** on the prediction.
- **Macroeconomic indicators appear to be weak predictors** in this model.

3. Decision Tree Model Behavior

- The model primarily relies on **past values (lag_1)** rather than external economic indicators.
- The non-linearity of decision trees may **limit their ability to capture relationships in features with little variation**.

Key Takeaways

- **lag_1** is the **dominant feature**, reinforcing that **recent past values drive the predictions**.
- **Macroeconomic variables contribute little**, suggesting that a **more complex model or feature engineering** might be needed.
- **Decision Trees may not fully capture continuous dependencies**, as evident from the flat lines in most plots.

Conclusion

- This analysis highlights the **model's strong reliance on historical data (lag_1)** rather than broader economic factors.
- **Further improvements** could involve **trying different feature transformations, incorporating interactions, or using ensemble models** like Random Forests for better generalization.

Decision Tree Visualization (Max Depth = 3)

Analysis

Key Findings

- The decision tree model is structured hierarchically, **splitting data at different thresholds of lag_1 and rolling_mean**.
- Each **node** represents a decision **based on feature values** to minimize squared error.
- The **tree depth is 3**, limiting complexity while maintaining interpretability.

Observations

1. Primary Splitting Feature: lag_1

- The **first split occurs at lag_1 ≤ 1.945**, indicating this feature is the strongest predictor.
- Most subsequent splits **continue refining lag_1 thresholds**, showing its **dominance in decision-making**.
- This highlights that the **model relies heavily on recent past values (lag_1)** rather than external economic indicators.

2. Secondary Splitting Feature: rolling_mean

- Some branches, particularly in deeper nodes, introduce **rolling_mean** as a secondary factor.
- For values above **2.039** and **2.726**, **rolling_mean impacts predictions**, but not as significantly as **lag_1**.

3. Error Reduction Across Nodes

- **Squared error progressively decreases** as we move deeper into the tree.

- This shows that the model refines its predictions with each split, **improving accuracy by breaking data into more homogeneous groups.**

4. Sample Distribution

- The **root node** starts with **5474 samples** and progressively splits into smaller groups.
- At the **leaf nodes**, **sample sizes are small**, meaning the model is **fitting more specifically to subsets of data.**

Key Takeaways

- **lag_1** is the most critical feature, consistently driving major splits.
- **rolling_mean** contributes at deeper levels, but its influence is less significant.
- The tree structure indicates the model relies on past values rather than macroeconomic indicators.
- Further optimization might include **tuning max depth** or using ensemble methods (e.g., Random Forest) to enhance generalization.

Conclusion

- The **decision tree effectively captures key patterns in the data** but might benefit from **additional features or adjustments** to reduce overfitting.
- Exploring **ensemble models (Random Forest, Gradient Boosting)** could **enhance stability and predictive performance.**

K-Nearest Neighbors Regression (KNN)

K-Nearest Neighbors (KNN) Regression: Model Training, Evaluation, and Interpretability

1. Introduction

- **K-Nearest Neighbors Regression (KNN)** is a non-parametric, instance-based learning algorithm that predicts values based on the closest training samples.
 - This notebook covers:
 - **Hyperparameter tuning using GridSearchCV**
 - **Model performance evaluation**
 - **Residual and error analysis**
 - **Feature importance insights**
 - **SHAP, LIME, and Partial Dependence Plots for model interpretability**
 - **Analysis of nearest neighbor distances for prediction insights**
-

2. Importing Essential Libraries

- **NumPy & Pandas:** Handling numerical and tabular data.
 - **Scikit-learn:**
 - **KNeighborsRegressor:** Core regression model.
 - **GridSearchCV:** Hyperparameter tuning.
 - **train_test_split:** Splitting data for training and testing.
 - **Matplotlib & Seaborn:** Visualization of model predictions and feature relationships.
-

3. Data Preprocessing

- **Dataset Splitting:**
 - **Training Set (80%):** Used for model training.
 - **Testing Set (20%):** Used for evaluation.
 - **Hyperparameter Grid for KNN:**
 - **n_neighbors:** Number of nearest neighbors.
 - **weights:** Defines how neighbors contribute to predictions.
 - **p:** Distance metric (Manhattan or Euclidean).
-

4. Hyperparameter Tuning using GridSearchCV

- **Grid Search Strategy** optimizes:
 - `n_neighbors` over `[3, 5, 7, 10, 15]`
 - `weights` over `['uniform', 'distance']`
 - `p` over `[1, 2]` (Manhattan vs Euclidean distance)
 - **5-Fold Cross-Validation** ensures **robust model selection**.
 - **Parallel Processing** (`n_jobs=-1`) speeds up computation.
-

5. Model Training and Selection of Optimal Parameters

- **GridSearchCV** trains multiple models with different hyperparameters.
 - The **best-performing model** is selected based on **Negative Mean Squared Error (MSE)**.
-

6. Model Prediction and Performance Evaluation

- The optimized KNN model is used to **generate predictions** on the test dataset.
 - A predefined function `calculate_metrics` computes evaluation scores.
 - The model is benchmarked against a **naïve forecasting method**.
-

7. Actual vs. Predicted Values Visualization

- A **scatter plot** visualizes the KNN Regression predictions.
 - A **perfect fit reference line** is included to assess accuracy.
-

8. Residual Analysis

Residual Scatter Plot

- **Residuals** (differences between actual and predicted values) are computed.
- A **scatter plot** visualizes residuals against actual values.
- A **horizontal reference line** ($y=0$) helps assess error distribution.

Residual Histogram

- A **histogram** illustrates the distribution of residuals.
 - A **vertical reference line** ($x=0$) is used to check for normality in residuals.
-

9. Hyperparameter Search Results

- GridSearchCV results are converted into a **DataFrame**.
- The **top 5 hyperparameter combinations** are displayed.

Hyperparameter Tuning Heatmap

- A **heatmap** visualizes the effect of different KNN hyperparameter values on model performance.
-

10. Model Interpretability using SHAP

- **SHAP (SHapley Additive EXplanations)** is used to interpret feature contributions.
 - **SHAP Summary Plot** (global explanation) shows the impact of each feature.
 - **SHAP Force Plot** (local explanation) visualizes feature contributions for specific instances.
-

11. Local Explanation using LIME

- **LIME (Local Interpretable Model-agnostic EXplanations)** is used to explain individual predictions.
 - The **most influential features** affecting a specific test instance are displayed.
-

12. Partial Dependence Plots (PDP)

- PDPs illustrate how changes in feature values influence model predictions.
 - **Multiple PDPs** are plotted to examine global trends.
-

13. Distance Analysis of Nearest Neighbors

- The **distances of nearest neighbors** for a selected test instance are plotted.
 - This provides insights into the **stability and reliability of predictions**.
-

14. Feature Importance via Permutation Analysis

- **Permutation Importance** evaluates how much each feature impacts predictions.
 - A **bar chart** ranks features based on their importance.
-

15. Conclusion

This notebook systematically implements and evaluates a **K-Nearest Neighbors Regression model** for predictive modeling. Key contributions include:

1. **Automated hyperparameter tuning** for optimal model selection.
2. **Comprehensive model evaluation** using residual analysis and benchmark comparisons.
3. **Feature importance insights** via permutation-based analysis.
4. **Model interpretability** through SHAP, LIME, and Partial Dependence Plots.
5. **Distance analysis of nearest neighbors** to assess prediction stability.

Output Result:

Fitting 5 folds for each of 20 candidates, totalling 100 fits

Best Hyperparameters:

```
'n_neighbors': 5,  
'p': 1,  
'weights': 'distance'
```

KNN Evaluation Metrics:

```
'Model': 'K-Nearest Neighbors Regression (KNN)',  
'MSE': 0.0009267051428894211,  
'RMSE': 0.030441832121103046,  
'MAE': 0.017880381942585476,  
'MAPE (%)': 1.1510531324305393,  
'sMAPE (%)': 1.1287107789164201,  
'MASE': 0.04551551297768598,  
'R^2': 0.994759164708316
```

Actual vs Predicted Values (KNN Regression) Analysis

Key Findings

- This scatter plot visualizes the **relationship between actual and predicted values** for the **K-Nearest Neighbors (KNN) regression model**.
- The **red dashed line** represents the **perfect fit**, where predicted values exactly match actual

values.

- Each **blue dot** represents a data point comparing actual vs. predicted values.

Observations

1. Strong Alignment with the Perfect Fit Line

- Most data points are **closely aligned with the red dashed line**, indicating **high prediction accuracy**.
- This suggests that **KNN regression is performing well** for this dataset.

2. Some Minor Deviations

- A few points slightly deviate from the perfect fit, particularly at **lower actual values** (closer to zero).
- This could indicate **some bias or slight prediction errors in low-value ranges**.

3. Consistent Performance Across the Value Range

- The KNN model maintains **consistent accuracy** across different actual values.
- There is **no clear systematic overestimation or underestimation trend**.

Key Takeaways

- **KNN regression provides strong predictive performance** for this dataset.
- **Most predictions are highly accurate**, with only slight deviations in lower value ranges.
- **Further fine-tuning** (e.g., adjusting **k** value) could potentially improve the model's performance.
- Comparing **KNN with other models** (e.g., **Decision Tree, Ridge Regression**) might provide insights into **which model generalizes best** for this dataset.

Conclusion

- The **KNN regression model demonstrates high accuracy and minimal bias**, making it a **strong choice for this predictive task**.
- **Further analysis on hyperparameter tuning** (e.g., optimal **k** selection) could refine performance even more.

Residual Plot Analysis (KNN Regression)

Key Findings

- This plot visualizes the **residuals (errors)** of the **K-Nearest Neighbors (KNN)** regression model against actual values.
- The **red dashed line at zero** represents the **ideal scenario where residuals are symmetrically distributed around zero**.
- Each **green dot corresponds to a residual value**, computed as:

$$Residual = Actual\ Value - Predicted\ Value$$

Observations

1. Residuals are Centered Around Zero

- The residuals are **evenly spread** above and below the zero line, which suggests that the model has **low systematic bias**.

2. No Clear Pattern (Homoscedasticity)

- The distribution of residuals appears **randomly scattered**, meaning **constant variance** across different actual values.
- This is a **good sign**, indicating that the model does not exhibit **heteroscedasticity (uneven variance in errors)**.

3. Some Outliers

- There are a few **residual points farther from zero**, which may indicate **occasional mispredictions** by the KNN model.
- These could be **outliers or cases where KNN struggles to generalize well**.

4. Slightly More Spread at Higher Values

- Residuals appear **slightly more dispersed at higher actual values**, which could suggest the **model has more variability in predictions at higher values**.

Key Takeaways

- **KNN regression performs well**, as residuals are **mostly centered around zero** and show **no obvious pattern**.
- **A few outliers exist**, suggesting potential improvements through **hyperparameter tuning** (e.g., adjusting the number of neighbors **k**).
- **Slightly higher residual variance at larger values** might indicate that the model could benefit from **feature scaling or additional feature engineering**.

Conclusion

- The **KNN regression model appears to make generally reliable predictions**, with **small residual errors** and **no major bias**.
- **Fine-tuning KNN parameters** (e.g., adjusting **k** or using weighted distances) may **further reduce residual variance** and improve performance.

Residual Distribution Analysis (KNN Regression)

Key Findings:

- **Residual Distribution:** The histogram illustrates the residuals (errors) from a K-Nearest Neighbors (KNN) regression model. Residuals are the differences between actual and predicted values.
- **Centering Around Zero:** The majority of residuals are concentrated around zero, indicating that the model predictions are, on average, accurate with minimal bias.
- **Symmetry:** The histogram appears relatively symmetric, suggesting that the errors are evenly distributed on both sides of zero, which is a sign of a well-fitted model.
- **Narrow Spread:** The residuals are tightly clustered, showing that most errors are small, implying good model performance.
- **Peak at Zero:** A sharp peak at zero suggests that a large number of predictions are very close to their actual values.
- **Vertical Red Line:** The red dashed line at zero reinforces the expectation that residuals should ideally be centered around zero in a well-balanced regression model.

Interpretation:

- If the residuals were skewed or had a significant spread, it would indicate systemic bias or high prediction errors.
- The relatively normal distribution of residuals suggests that the KNN regression model is performing well, with no obvious systematic errors or bias.

Top 5 Hyperparameter Combinations:

Rank 6:

Hyperparameters: `{'n_neighbors': 5, 'p': 1, 'weights': 'distance'}`
Mean Test Score: -0.001220
Standard Deviation: 0.000077

Rank 2:

Hyperparameters: `{'n_neighbors': 3, 'p': 1, 'weights': 'distance'}`
Mean Test Score: -0.001223
Standard Deviation: 0.000086

Rank 10:

Hyperparameters: `{'n_neighbors': 7, 'p': 1, 'weights': 'distance'}`
Mean Test Score: -0.001299

Standard Deviation: 0.000062

Rank 14:

Hyperparameters: {'n_neighbors': 10, 'p': 1, 'weights': 'distance'}

Mean Test Score: -0.001445

Standard Deviation: 0.000065

Rank 1:

Hyperparameters: {'n_neighbors': 3, 'p': 1, 'weights': 'uniform'}

Mean Test Score: -0.001563

Standard Deviation: 0.000024

Hyperparameter Tuning: n_neighbors vs Weights

Key Findings:

- The heatmap presents the impact of two hyperparameters—`n_neighbors` (y-axis) and `weights` (x-axis)—on a certain performance metric (possibly error rate or accuracy) in a machine learning model.
- The color gradient represents the magnitude of the performance metric, where lighter colors indicate better performance (lower values) and darker colors indicate worse performance (higher values).
- Two types of weight functions are compared:
 - **"distance"**: Weights points inversely proportional to their distance.
 - **"uniform"**: Assigns equal weight to all points.
- The analysis is conducted across different values of `n_neighbors` (3, 5, 7, 10, 15).
- The performance metric values are negative, which may suggest that a loss function is being minimized (e.g., negative accuracy or log-loss).
- Observations:
 - The `distance` weight generally results in a better (less negative) performance metric across all values of `n_neighbors`, maintaining a value of approximately -0.002.
 - The `uniform` weight shows degradation in performance as `n_neighbors` increases, with the worst value observed at `n_neighbors = 15` (-0.004).
 - The contrast between `distance` and `uniform` weights increases with `n_neighbors`, suggesting that distance-based weighting is more beneficial for higher neighbor counts.

Conclusion:

- Using `distance` as the weight function provides consistently better results across different values of `n_neighbors`.
- The impact of `n_neighbors` is more pronounced when using `uniform` weights, with performance deteriorating as `n_neighbors` increases.
- This suggests that **adaptive weighting based on distance helps mitigate the negative effects of increasing neighbors** in the model.

SHAP Summary Plot Analysis

Key Findings:

- The plot visualizes the **SHAP values**, which indicate the impact of different economic indicators on the model's output.
- **Feature Importance:**
 - The most influential features include **Real Gross Domestic Product (GDP)**, **Crude Oil Prices (West Texas Intermediate)**, **Nominal Broad U.S. Dollar Index**, and **Consumer Price Index (CPI)**.
 - These features have the largest spread of SHAP values, meaning they contribute significantly to the model's predictions.
- **Feature Effect:**
 - The **color gradient (blue to pink)** represents the feature value (low to high).

- **High feature values (pink)** and **low feature values (blue)** impact the model differently, shifting the prediction in positive or negative directions.
- **SHAP Value Interpretation:**
 - **SHAP values near zero** suggest minimal influence on the model's output.
 - Features with **positive or negative SHAP values** indicate their contribution to increasing or decreasing the predicted value.
- **Lagged Variables & Moving Average:**
 - The model incorporates **lagged values (lag_1 to lag_5)** and **rolling_mean**, suggesting it accounts for historical trends.
 - These time-series features contribute to forecasting, but their SHAP values are relatively smaller than macroeconomic indicators.
- **Macroeconomic Indicators:**
 - Variables such as the **10-Year Treasury Yield, Unemployment Rate, and Federal Funds Effective Rate** have some impact but are relatively less influential compared to GDP and Oil Prices.

Conclusion:

- **GDP, Oil Prices, Dollar Index, and CPI** are the most critical features influencing the model's predictions.
- **High values of these indicators (pink dots)** generally push the prediction in a different direction than **low values (blue dots)**.
- The model considers both **current macroeconomic conditions and historical trends** (via lagged values and rolling mean).
- The **Federal Funds Rate and Unemployment Rate**, while important, have a smaller impact compared to GDP and CPI.

Waterfall Plot Analysis

Key Findings:

- The plots visualize the contribution of different economic factors to the model's prediction.
- **Real Gross Domestic Product (GDP)** has a significant impact, shown by the large red and blue areas.
 - When GDP is high, it contributes positively (red), increasing the prediction.
 - When GDP is lower, it contributes negatively (blue), decreasing the prediction.
- **Nominal Broad U.S. Dollar Index and Crude Oil Prices (West Texas Intermediate)** are also key influencing factors.
 - Higher values of the Dollar Index (red) push the prediction upwards.
 - Higher oil prices (red) also impact the prediction, but the effect varies across instances.
- The **initial prediction value** (left-most number) starts higher and gradually adjusts based on feature contributions.
- The **blue and red segments** illustrate the individual SHAP values:
 - **Red** sections indicate positive contributions (driving the prediction higher).
 - **Blue** sections indicate negative contributions (driving the prediction lower).
- Across multiple samples, GDP consistently plays the dominant role in shaping the model's output.

Conclusion:

- **GDP is the most influential economic indicator** in the model, followed by the **U.S. Dollar Index and Oil Prices**.
- The combination of macroeconomic indicators determines the final prediction, with each factor either boosting or reducing the model's output.
- The **direction and magnitude of the SHAP values** show how economic changes drive predictions in different ways.

Partial Dependence Plots (KNN Regression) Analysis

Key Findings:

- The plots illustrate the **partial dependence** of the target variable on different economic

- predictors while marginalizing over other features.
- **Real Gross Domestic Product (GDP)** has a **non-linear relationship** with the target variable.
 - The plot for GDP shows fluctuations, indicating a **complex dependency pattern** in the model.
 - **Crude Oil Prices (West Texas Intermediate)** show a **slight positive trend** with the target variable.
 - This suggests that as oil prices increase, the model's predicted output slightly increases.
 - Most other features, including the **Consumer Price Index (CPI), Unemployment Rate, Federal Funds Effective Rate, 10-Year Treasury Yield, and various lag variables**, show nearly **flat partial dependence plots**.
 - This indicates **minimal direct influence** of these variables on the target variable in isolation.
 - **Nominal Broad U.S. Dollar Index, Personal Consumption Expenditures, and Inflation Expectation Rate** also exhibit relatively flat trends.
 - These features do not have a strong standalone effect on predictions.
 - **Rolling Mean and Lag Features (lag_1 to lag_5)** show no significant impact.
 - This suggests that short-term historical values have little direct effect when other factors are considered.

Conclusion:

- **GDP is the most influential predictor**, with a complex impact on the model's output.
- **Crude Oil Prices show a small positive dependency**, meaning they slightly affect the prediction.
- Most other economic indicators, including CPI, interest rates, and unemployment, **do not significantly change the prediction when analyzed in isolation**.
- **Lagged variables and moving averages have minimal effects**, indicating the model may not heavily rely on past values for forecasting.

Permutation Feature Importance (KNN Regression) Analysis

Key Findings:

- **Real Gross Domestic Product (GDP) is the most influential feature** in the model.
 - It has the highest permutation importance, meaning that when its values are shuffled, the model's performance degrades significantly.
 - This suggests the model relies heavily on GDP for making predictions.
- **Crude Oil Prices (West Texas Intermediate) also contribute to the model's predictions** but to a much lesser extent than GDP.
 - It shows some importance, but its impact is relatively small.
- **Most other features, including the Consumer Price Index (CPI), Unemployment Rate, Federal Funds Effective Rate, and Nominal Broad U.S. Dollar Index, have negligible importance**.
 - Their permutation importance is close to zero, indicating that shuffling these variables does not significantly affect the model's predictions.
- **Lag features (lag_1 to lag_5) and rolling_mean have little to no impact**.
 - This suggests that past values of the target variable do not contribute much to the model's predictive power.
- **The error bars represent uncertainty in importance values**.
 - GDP has a small error bar, indicating consistent high importance.
 - Crude Oil Prices have a slightly larger error bar, reflecting some variability in their impact.

Conclusion:

- **GDP is the dominant factor driving the model's predictions**, reinforcing its economic significance.
- **Crude Oil Prices have some influence, but other economic indicators play a minimal role** in the KNN regression model.
- **Lag features and rolling averages do not significantly contribute**, suggesting that the model does not depend on past values for forecasting.

Prediction Explanation Analysis

Key Findings:

- The **predicted value** falls between **0.14 (min)** and **2.86 (max)**, with a **final prediction** around **2.63**.
- **Feature Contributions:**
 - **Positive contributors (orange):** Features that increase the predicted value.
 - **Crude Oil Prices (West Texas Intermediate)** at **68.44** has the highest positive impact.
 - **Consumer Price Index (CPI)** and **10-Year Treasury Yield** contribute marginally to the increase.
 - **Negative contributors (blue):** Features that decrease the predicted value.
 - **Real Gross Domestic Product (GDP)** at **16,396.15** slightly reduces the prediction.
 - **Unemployment Rate** at **4.60** also has a small negative effect.
- The **bar on the left** represents **SHAP values**, where:
 - **Blue (negative impact):** Decreases prediction.
 - **Orange (positive impact):** Increases prediction.
- The **table on the right** lists the feature values used in the prediction.

Conclusion:

- **Crude Oil Prices (68.44)** is the **strongest driver of the prediction**, pushing the value higher.
- **GDP and Unemployment Rate** contribute **negatively**, slightly lowering the prediction.
- **Consumer Price Index and Treasury Yield** have **minimal impact**, but they slightly increase the prediction.
- The model's **prediction is primarily influenced by oil prices**, while macroeconomic factors like GDP and unemployment provide counteracting effects.

Distances to Nearest Neighbors (KNN) Analysis

Key Findings:

- The bar chart visualizes the **distances from a given instance (Instance 0) to its k-nearest neighbors** in a K-Nearest Neighbors (KNN) model.
- **X-axis (Nearest Neighbors):** Represents the closest data points (neighbors) to the selected instance.
- **Y-axis (Distance):** Represents the Euclidean distance (or another distance metric) between Instance 0 and its respective neighbors.
- **Observations:**
 - The first two nearest neighbors have **relatively small distances**, indicating they are closely related to Instance 0.
 - The **third and fourth nearest neighbors have larger distances**, suggesting they are farther away but still considered close enough for KNN predictions.
 - The **fifth nearest neighbor has the highest distance**, meaning it is the least similar among the considered neighbors.
- **Pattern Interpretation:**
 - A gradual increase in distances suggests a **progression from highly similar to less similar data points**.
 - A significant jump in distance (as seen from the second to third neighbor) indicates a possible **cluster boundary or data sparsity**.

Conclusion:

- The **first two neighbors are very similar** to Instance 0, providing strong local information for KNN predictions.
- The **third and fourth neighbors contribute to the prediction but with less similarity**.
- The **fifth neighbor is the least relevant**, potentially introducing more variability in the prediction.
- The increasing distance pattern suggests that Instance 0 is part of a **localized cluster**, but its neighborhood becomes more diverse as k increases.

Deep Learning Models

Bidirectional Long Short-Term Memory (BiLSTM)

Bidirectional LSTM for Time-Series Forecasting: Hyperparameter Optimization and Model Evaluation

1. Introduction

- This notebook implements a **Bidirectional Long Short-Term Memory (BiLSTM) network** for time-series forecasting.
 - **Key Contributions:**
 - **Data preprocessing** with a safe train-test split for time-series data.
 - **Hyperparameter tuning** using **Optuna**.
 - **Efficient training** with **Early Stopping** and **Pruning**.
 - **Comprehensive model evaluation**, including:
 - Residual and error analysis
 - Feature distribution visualization
 - Signal-to-Noise Ratio (SNR) calculation
 - **Outlier handling** for improved model performance.
-

2. Importing Essential Libraries

- **TensorFlow & Keras:** Building and training LSTM models.
 - **Optuna:** Bayesian optimization for hyperparameter tuning.
 - **Matplotlib & Seaborn:** Visualization of model predictions, residuals, and feature distributions.
-

3. Reproducibility and Environment Configuration

- Setting a global random seed (`seed=42`) ensures reproducibility across:
 - Python's built-in `random` module
 - NumPy operations
 - TensorFlow's computation graph
 - Forcing deterministic TensorFlow operations ensures stable training across multiple runs.
-

4. Data Preprocessing

Safe Train-Test Splitting

- **Chronological Splitting:** Maintains time-series order without shuffling.
- **Prevents Data Leakage:** Removes overlapping indices between train and test sets.

Feature Reshaping for LSTM

- LSTM models require a 3D input tensor: (samples, timesteps, features).
 - A custom function **reshapes the dataset** into the required format.
-

5. Bidirectional LSTM Model Definition

- **Bidirectional LSTM** captures both **past and future dependencies** in sequential data.
 - **Hyperparameter tuning with Optuna:**
 - `units`: Number of LSTM neurons per layer.
 - `dropout_rate`: Prevents overfitting.
 - `clipnorm`: Gradient clipping to prevent exploding gradients.
 - `learning_rate`: Optimized choice of `0.001`, `0.005`, or `0.01`.
 - `activation_function`: Either `"tanh"` or `"relu"`.
 - `optimizer`: `Adam` or `RMSprop` based on performance.
-

6. Hyperparameter Optimization using Optuna

Optuna Objective Function

- Defines the **search space** for hyperparameters.
- **Early Stopping** prevents overfitting.
- **Optuna Pruning** dynamically stops underperforming trials to save computation.

Optuna Study Execution

- **20 trials** optimize hyperparameters with a **timeout of 1 hour**.
 - **TPE Sampler** ensures efficient Bayesian optimization.
-

7. Training the Best Bidirectional LSTM Model

- The **optimal hyperparameters** from Optuna are used for final training.
 - **Validation Split (80%-20%)** ensures generalization.
 - **Model training and test predictions** are performed using the selected hyperparameters.
-

8. Model Performance Evaluation

Smoothed Loss Curve

- A **moving average filter** smooths noisy training and validation loss curves.
- **Visualization** helps in understanding model convergence.

Residual Analysis

- **Residual = Actual - Predicted**
 - **Scatter Plot:** Visualizes residuals with threshold lines for detecting outliers.
 - **Histogram of Residuals:** Helps assess model bias and error distribution.
-

9. Handling Outliers

- **Outliers are identified using residuals:**
 - Any value **exceeding 3 standard deviations** is flagged.
 - **Effect of Outliers on Model Performance:**
 - Removing outliers **improves model accuracy and interpretability**.
-

10. Final Model Evaluation (After Outlier Removal)

- **Performance metrics** are computed on a cleaned dataset.
 - **Comparison against a naive baseline** validates model improvement.
-

11. Predictions vs. True Values Visualization

- **Line plot** comparing actual and predicted values.
 - **Helps in visually assessing model performance over time.**
-

12. Feature Distribution in Training and Test Sets

- **Histogram comparison** ensures **similar feature distributions** across datasets.
 - **Prevents dataset shift issues** that could affect model generalization.
-

13. Signal-to-Noise Ratio (SNR) Analysis

- **SNR quantifies model reliability** by comparing:
 - **Signal Power** (true values)
 - **Noise Power** (residual errors)
 - **Higher SNR** indicates better predictive performance.
-

14. Conclusion

- This notebook successfully:
 - **Optimized a Bidirectional LSTM model** for time-series forecasting.
 - **Implemented Optuna-based hyperparameter tuning** for performance gains.
 - **Analyzed residuals and outliers** for improving model reliability.
 - **Ensured feature consistency** between train and test sets.
 - **Evaluated Signal-to-Noise Ratio** for assessing prediction quality.

Key Takeaways

- BiLSTM captures both forward and backward dependencies in sequences.
- Optuna's Bayesian optimization effectively selects hyperparameters.
- Comprehensive error analysis and outlier handling improve model trustworthiness.

Explaining Predictions of a Bidirectional LSTM Model: SHAP, LIME, and Feature Attribution Techniques

1. Introduction

- Interpreting deep learning models is critical for trust and explainability in time-series forecasting.
 - This notebook explores **feature attribution methods** for a trained **Bidirectional LSTM (BiLSTM)** model.
 - **Techniques Used:**
 - **SHAP (SHapley Additive Explanations):** Global and local feature importance analysis.
 - **LIME (Local Interpretable Model-Agnostic Explanations):** Instance-wise interpretability.
 - **Saliency Maps:** Gradient-based explanation for time-series dependencies.
 - **Integrated Gradients:** Cumulative feature attribution over interpolated inputs.
 - **Partial Dependence Plots (PDPs):** Visualizing feature effects on predictions.
 - **Permutation Feature Importance:** Model-agnostic analysis of feature contributions.
 - **Decision Tree Surrogate Model:** Extracting interpretable decision rules.
-

2. Computing SHAP Values for the BiLSTM Model

2.1 SHAP KernelExplainer for LSTM Models

- SHAP KernelExplainer estimates feature importance by perturbing input features and observing model output changes.
- BiLSTM expects 3D input (samples, timesteps, features), but SHAP requires 2D input.
- **Solution:** Flatten the time-series data before computing SHAP values.

2.2 Computing SHAP Values

- KernelExplainer is initialized using a small subset of the training data as a reference.
- SHAP values are computed for the first 100 test samples to reduce computational cost.

2.3 Global SHAP Summary Plot

- Bar chart visualization ranks the most important features globally.
 - Helps understand which features influence predictions the most.
-

3. SHAP Feature Importance Analysis

3.1 Flattening Time-Series Data for SHAP

- SHAP requires 2D input, so LSTM training and test data are reshaped from (samples, timesteps, features) to (samples, features).
- Only a small subset of the training data (50 samples) is used as background for SHAP computations.

3.2 SHAP Explanation for Multiple Test Instances

- SHAP values are computed for the first 10 test samples.
- If the BiLSTM model has multiple outputs, we extract the relevant SHAP values.
- Debugging checks ensure correct feature alignment between SHAP values and test data.

3.3 SHAP Summary Plot

- Bar chart visualization ranks features based on SHAP importance scores.
 - Feature names are auto-generated for clarity.
-

4. SHAP Force Plot for Individual Predictions

- Force plots explain feature contributions for a single instance.
 - Feature effects are visualized interactively in Jupyter Notebooks.
 - BiLSTM input is reshaped from 3D to 2D for SHAP force plot compatibility.
-

5. LIME Explanation for a Bidirectional LSTM Model

5.1 Flattening Data for LIME

- LIME requires 2D input, so the 3D LSTM input is flattened.
- Feature names are auto-generated for each timestep.

5.2 Defining a Wrapper Function for Model Compatibility

- LIME calls the model with 2D inputs, so a function reshapes the input back to 3D before prediction.

5.3 Explaining an Individual Prediction

- LIME generates explanations for a single test instance.
 - The top 10 most influential features are displayed.
-

6. Partial Dependence Plots (PDPs)

- PDPs show how a specific feature affects model predictions while holding others constant.
 - Time-series data is flattened to 2D, and PDPs are generated for selected features.
 - Key visualization insights:
 - Non-linear relationships between features and predictions.
 - Feature influence at different time points.
-

7. Permutation Feature Importance

7.1 Wrapping the BiLSTM Model for scikit-learn Compatibility

- Permutation importance requires a scikit-learn-compatible estimator.
- A wrapper function is created to reshape data back to 3D before passing it to the BiLSTM model.

7.2 Computing Feature Importance

- Each feature is randomly shuffled to measure its effect on prediction performance.
- The most important features are identified based on performance degradation.

7.3 Visualization of Permutation Importance

- A horizontal bar chart displays feature importance with error bars representing variance.
-

8. Integrated Gradients for BiLSTM Feature Attribution

8.1 The Integrated Gradients Method

- A baseline input (zero array) is interpolated toward the real input.
- Gradients are computed at each interpolation step, and their average contribution is used as feature importance.

8.2 Computing Integrated Gradients for a Sample

- The first test instance is analyzed to compute feature attribution scores.

8.3 Visualizing Integrated Gradients

- A bar chart displays feature importance scores based on Integrated Gradients.
-

9. Decision Tree Surrogate Model for Explaining BiLSTM

9.1 Training a Decision Tree to Approximate BiLSTM Predictions

- A Decision Tree is trained on the BiLSTM predictions to extract human-interpretable rules.
- Tree depth is limited (`max_depth=3`) for better interpretability.

9.2 Visualizing the Decision Tree

- A plot of the trained Decision Tree reveals decision rules learned by the BiLSTM.
-

10. Saliency Maps for LSTM Interpretability

10.1 Computing Saliency Maps

- Saliency maps highlight important timesteps for model predictions.
- Gradient-based sensitivity analysis is applied to identify influential time points.

10.2 Visualizing the Saliency Map

- Line plots highlight timesteps that impact model predictions.
-

11. Residual Analysis for BiLSTM Predictions

11.1 Residual Analysis for Outlier Detection

- Residuals (errors) are plotted against true values.
- Outliers exceeding 3 standard deviations are flagged.

11.2 Residual Scatter Plot

- Scatter plots visualize model prediction errors.
 - A red reference line (`y=0`) helps detect over/under-estimation.
-

12. Summary of Key Findings

- SHAP provides global and local feature explanations.
- LIME explains individual predictions for interpretability.
- Partial Dependence Plots highlight feature interactions.
- Permutation Importance identifies crucial features.
- Integrated Gradients reveal cumulative feature effects.
- Decision Tree Surrogate Models approximate deep learning predictions with simple rules.
- Saliency Maps highlight influential time points.
- Residual analysis helps detect and remove outliers.

Conclusion

This notebook successfully demonstrates multiple interpretability techniques for Bidirectional LSTM models, ensuring trustworthy and transparent deep learning predictions in time-series forecasting.

Output Result:

Best Parameters:

```
'units': 150,
'learning_rate': 0.001,
'dropout_rate': 0.2,
'clipnorm': 3.0,
'optimizer': 'Adam',
'activation_function': 'tanh',
'batch_size': 32,
'epochs': 100
```

Best Validation Loss: 0.0016429921379312873

Number of Outliers: 64

Indices of Outliers: [202 292 310 311 312 313 314 317 340 362 374 388 408 416 417 418 422 423 424 429 430 431 432 433 434 435 436 439 440 441 442 443 444 445 446 447 448 449 450 458 461 462 463 464 467 468 469 470 471 472 473 474 475 476 477 478 481 483 484 488 517 572 573 808]

Bidirectional LSTM Evaluation Metrics:

```
'Model': 'Bidirectional Long Short-Term Memory (BiLSTM)',
'MSE': 0.0033665655250873896,
'RMSE': 0.058022112380431215,
'MAE': 0.04952238352910769,
'MAPE (%)': 2.0860980949063954,
'sMAPE (%)': 2.114597508069628,
'MASE': 0.49423914427566185,
'R^2': 0.8498323689196471
```

Signal-to-Noise Ratio (SNR): 30.96 dB

Smoothed Loss Over Epochs Analysis

Key Findings:

- The graph represents the **training and validation loss trends** over multiple epochs.
- **X-axis (Epochs):** Represents the number of training iterations.
- **Y-axis (Loss):** Represents the smoothed loss value for training and validation datasets.
- **Training Loss (Blue Line):**
 - Starts **high at the beginning (~0.03)** and rapidly decreases within the first few epochs.
 - Continues to decline gradually as training progresses.
 - Shows signs of stabilization in later epochs, indicating **convergence**.
- **Validation Loss (Orange Line):**
 - Starts at a **much lower value** than the initial training loss.
 - Fluctuates slightly throughout training but remains relatively stable.
 - Does not show signs of sharp increase, suggesting **no severe overfitting**.
- **Comparison & Interpretation:**
 - The training loss is consistently **lower than validation loss**, which is expected in a well-trained model.
 - The **validation loss does not increase significantly**, implying that the model generalizes well without excessive overfitting.
 - The difference between training and validation loss is **small**, suggesting **a well-fitted model with good generalization**.

Conclusion:

- The model is effectively learning, as indicated by the **smooth decline in training loss**.
- **No significant overfitting is observed**, as validation loss remains relatively stable.
- **Training appears to be successful**, with both losses decreasing over time and converging towards minimal values.
- Future adjustments could focus on further tuning hyperparameters if needed, but overall, the model performance appears strong.

Model Summary Analysis

Key Findings:

- The model is named "**sequential_20**", indicating it is a **Sequential model** in Keras/TensorFlow.
- **Layers and Parameters:**
 - The model consists of **Bidirectional LSTM (Long Short-Term Memory) layers**, **Dropout layers**, and a **Dense (fully connected) layer**.
 - **Bidirectional LSTM Layers:**
 - The first **bidirectional_40** layer has **182,400 parameters** and outputs a shape of `(None, 15, 300)`, meaning it processes sequences of length 15 with 300 features.
 - The second **bidirectional_41** layer has **541,200 parameters** and outputs a shape of `(None, 300)`, meaning it consolidates sequence information into a single 300-dimensional representation.
 - **Dropout Layers (dropout_40 and dropout_41):**
 - These layers have **0 parameters** since they are used for **regularization** to prevent overfitting.
 - **Dense Layer (dense_20):**
 - The final layer is **fully connected (Dense layer)** with **301 parameters**, and it outputs a single value `(None, 1)`, likely used for regression or binary classification.
- **Model Parameters Breakdown:**
 - **Total parameters: 2,171,705 (8.28 MB)**
 - **Trainable parameters: 723,901 (2.76 MB)** → These parameters are updated during training.
 - **Non-trainable parameters: 0 (0.00 B)** → No frozen layers, meaning all parameters are learnable.
 - **Optimizer parameters: 1,447,804 (5.52 MB)** → These parameters belong to the optimizer, likely including momentum, running averages, or adaptive learning rate parameters.

Conclusion:

- The model is a **deep learning architecture utilizing Bidirectional LSTMs**, which suggests it is designed for **sequential or time-series data**.
- **Dropout layers** are used to improve generalization and **reduce overfitting**.
- The **final dense layer with a single output** suggests the model is performing a **regression or binary classification task**.
- The model is **fully trainable with no frozen layers**, meaning all parameters contribute to learning.

Residuals and Outliers Analysis

Key Findings:

- The scatter plot visualizes the **residuals** (differences between actual and predicted values) for a dataset.
- **X-axis (Data Point Index):** Represents the index of each data point in the dataset.
- **Y-axis (Residual):** Represents the residual error (difference between actual and predicted values).
- **Residual Distribution:**
 - The majority of the residuals **cluster around zero**, indicating that the model's predictions are relatively accurate.
 - Some residuals deviate significantly from zero, suggesting the presence of **outliers or model inaccuracies**.

- **Thresholds (Red Dashed Lines):**
 - The **upper threshold** (positive red dashed line) represents a predefined limit for high residuals.
 - The **lower threshold** (negative red dashed line) indicates the lower bound for acceptable residuals.
 - Residuals exceeding these thresholds can be considered **outliers**.
- **Outliers Detection:**
 - A few points surpass the **upper threshold**, indicating overestimation by the model.
 - Some points fall below the **lower threshold**, suggesting underestimation.
 - These extreme residuals may highlight data inconsistencies, model errors, or unexpected variations in the dataset.

Conclusion:

- The residuals are **mostly centered around zero**, suggesting a well-performing model.
- **Some residuals exceed the thresholds**, indicating potential **outliers or systematic model biases**.
- Further analysis may be needed to investigate **outlier causes**, such as data preprocessing issues, missing variables, or improvements in model architecture.

Model Predictions vs. True Values Analysis

Key Findings:

- The plot compares the **true values** (green dots) and the **model's predicted values** (orange crosses) across all samples.
- **X-axis (Sample Index):** Represents the index of each sample in the dataset.
- **Y-axis (Value):** Represents the true and predicted values of the target variable.
- **Observations:**
 - The **predicted values closely follow the true values**, indicating that the model captures trends well.
 - The model performs well in regions where **predictions align tightly with actual values**.
 - There are **minor deviations**, especially in peak areas where true values spike higher than predictions.
 - The model tends to **underestimate certain peaks**, suggesting possible difficulties in capturing extreme fluctuations.
- **Pattern Interpretation:**
 - The overall trend is well-matched, showing that the model has **learned the underlying data distribution**.
 - Differences between predicted and true values may indicate areas for further **model refinement or feature engineering**.

Conclusion:

- The model shows **strong predictive performance**, successfully capturing the overall pattern of the data.
- Some **underestimation at peak values** suggests potential room for improvement, such as adjusting model complexity or fine-tuning hyperparameters.
- The **alignment of predictions with actual values** indicates that the model is **generalizing well** without excessive overfitting.

Feature Distribution in Training and Test Sets Analysis

Key Findings:

- The histogram visualizes the **distribution of feature values** for both the **training (blue) and test (orange) datasets**.
- **X-axis (Feature Value):** Represents the range of feature values.
- **Y-axis (Frequency):** Represents the number of occurrences of each feature value.
- **Distribution Observations:**
 - A large concentration of values **near zero** suggests that the majority of feature values are

- **small or sparse.**
- Both training and test datasets exhibit **similar distributions**, ensuring consistency in data representation.
- A few instances have **high feature values (outliers)**, which appear at the far right of the distribution.
- The **test set follows a similar pattern as the train set** but with fewer samples, as expected.
- **Pattern Interpretation:**
 - The heavy skew towards **low values** suggests that the feature might be **highly imbalanced** or contain a lot of **zero entries**.
 - The **outliers (high feature values)** could have a significant impact on the model's predictions and may require further investigation.

Conclusion:

- The feature distribution is **well-aligned between training and test sets**, ensuring the model is trained on data that is representative of real-world scenarios.
- The dataset appears **highly skewed towards lower values**, which might require **scaling or transformation techniques** for better model performance.
- The **presence of high-value outliers** suggests that further preprocessing, such as **log transformation or normalization**, might be beneficial to mitigate their impact.

Feature 0 Analysis

Key Findings:

- The plot represents the distribution and influence of **Feature 0** in the model.
- **X-axis:** Represents the impact or contribution of Feature 0.
- **Y-axis:** Represents the feature index or position.
- **Observations:**
 - The data points are **centered around zero**, indicating that **Feature 0 has a minimal impact** on the model's predictions.
 - Points are colored in **blue and pink**, suggesting different categories of values (e.g., high vs. low values in SHAP analysis).
 - There is **no strong deviation to the left or right**, meaning Feature 0 does not strongly push predictions in a positive or negative direction.
- **Pattern Interpretation:**
 - The **symmetry** of the plot suggests that Feature 0 has a **balanced effect** on the model.
 - A **narrow distribution range** implies that Feature 0 has a relatively **low variance in impact**.

Conclusion:

- **Feature 0 has a minimal impact on model predictions**, as it is centered around zero.
- The **distribution is balanced**, indicating no significant bias in its effect.
- If Feature 0 is not meaningful for prediction, it might be a candidate for **feature selection or dimensionality reduction**.

shap_values.shape: (10, 15, 1) X_test_flat[:10].shape: (10, 15) After squeeze, shap_values.shape: (10, 15)

SHAP Feature Importance Analysis

Key Findings:

- The bar chart represents the **mean absolute SHAP values** for each feature, which indicates their overall impact on the model's predictions.
- **X-axis:** Displays the **mean(|SHAP value|)**, which quantifies the **average impact on model output magnitude**.
- **Y-axis:** Lists the **features (Feature_0 to Feature_14)** ranked by importance.

Observations:

- **Feature_0 has the highest impact** on the model predictions, suggesting it is the most influential

feature.

- **Feature_14 and Feature_9** also have **significant contributions**, meaning they strongly affect the model's output.
- Several features, such as **Feature_1, Feature_2, Feature_5, and Feature_6**, have very low SHAP values, implying **minimal influence on predictions**.
- The **distribution of importance is uneven**, with a few dominant features driving the predictions while others contribute negligibly.

Interpretation:

- The **high importance of Feature_0, Feature_14, and Feature_9** suggests that the model heavily relies on these features for decision-making.
- Features with **low SHAP values might be redundant or less informative**, potentially candidates for **feature selection or dimensionality reduction**.
- The dataset consists of **10 test samples and 15 features**, as indicated by:
 - `shap_values.shape: (10, 15, 1)` → 10 test samples, 15 features, and 1 prediction per sample.
 - `X_test_flat[:10].shape: (10, 15)` → The test data has 10 samples with 15 features each.
 - After squeeze, `shap_values.shape: (10, 15)`, meaning the SHAP values were reshaped to match the feature count.

Conclusion:

- **Feature_0 is the most crucial feature**, significantly influencing model predictions.
- **Feature_14 and Feature_9 also play key roles**, while many other features contribute minimally.
- **Dimensionality reduction techniques** could be explored to remove low-impact features and improve model efficiency.

SHAP Force Plot Analysis

Key Findings:

- The plot visualizes the contribution of different **timesteps** (features) in influencing the model's prediction.
- **X-axis**: Represents the model's prediction range, with the **base value** around **2.024**.
- **Y-axis (SHAP values)**: Represents how each feature **pushes the prediction higher (red) or lower (blue)**.
- **Prediction Outcome ($f(x) = 1.95$)**:
 - The **final predicted value is 1.95**, which is lower than the base value (**2.024**).
 - **Blue features (negative impact)** push the prediction **downward**.
 - **Red features (positive impact)** push the prediction **upward**.

Observations:

- **Top contributing negative features (blue, reducing prediction)**:
 - `timestep_14 = 1.978`, `timestep_0 = 2.03`, `timestep_9 = 1.99`, `timestep_10 = 1.99`, `timestep_12 = 1.96`, and `timestep_4 = 0.09` all contribute to **lowering the prediction**.
 - These features **collectively pull the prediction below the base value**.
- **Top contributing positive feature (red, increasing prediction)**:
 - `timestep_13 = 1.97` is the only major feature **pushing the prediction higher**.
 - However, its impact is not strong enough to counteract the collective downward influence of the other features.

Interpretation:

- The **majority of timesteps are contributing negatively**, meaning **past observations are driving the prediction lower than expected**.
- **Timestep 13 attempted to push the prediction higher**, but its effect was overpowered by the combined effect of negative contributors.
- This suggests that **recent timesteps (such as timestep 14 and timestep 0) played a crucial role in reducing the predicted value**.

Conclusion:

- The model predicts 1.95, slightly lower than the expected base value of 2.024.
- The prediction is mainly influenced by negative contributors, with only one strong positive force.
- Timestep_14 and Timestep_0 had the most significant impact in pulling the prediction downward, while Timestep_13 was the only feature trying to push it upward.
- This suggests that recent patterns in the data are influencing the model's decision significantly.

SHAP Local Explanation of Model Prediction

Key Findings:

- The predicted value is 2.02, with a local model prediction of 2.104.
- Intercept value is 1.867, meaning this is the base value from which the SHAP values adjust the prediction.
- The model prediction is influenced by both positive and negative feature contributions.

Feature Contributions:

- Positive contributors (orange, increasing prediction):
 - Feature_14 (2.06) and Feature_9 (2.06) have the strongest positive impact, pushing the prediction higher.
 - Feature_7 (53.08) and Feature_5 (106.08) also contribute positively, though with a smaller effect.
 - Feature_8 (1.15) and Feature_2 (21058.38) contribute to a minor increase.
- Negative contributors (blue, decreasing prediction):
 - Feature_12 (2.06) and Feature_6 (111.56) have the strongest negative impact, reducing the prediction.
 - Feature_3 (6.40) and Feature_13 (2.09) also contribute slightly to decreasing the output.

Interpretation:

- Feature_14 and Feature_9 play the biggest role in pushing the prediction higher, suggesting they are key variables in the model's decision-making.
- Feature_12 and Feature_6 counteract the increase, pulling the predicted value down.
- The final predicted value (2.02) results from a balance between these positive and negative contributions.
- The feature values are displayed in the table on the right, where orange highlights positive contributors and blue highlights negative contributors.

Conclusion:

- The model's prediction of 2.02 is well-explained by the SHAP values, where Feature_14 and Feature_9 drive the increase, while Feature_12 and Feature_6 reduce it.
- A few features have a minor impact, while others strongly push the prediction up or down.
- Understanding these feature contributions helps in model interpretability and assessing the most important drivers of predictions.

Partial Dependence Plot (PDP) for Feature 0

Key Findings:

- The plot shows the partial dependence of the model's predictions on Feature 0.
- X-axis (Feature 0 Values): Represents the range of values for Feature 0.
- Y-axis (Partial Dependence - Average Prediction): Represents the model's predicted output as Feature 0 changes, holding all other features constant.

Observations:

- The relationship between Feature 0 and the model's predictions is monotonically increasing.
- As Feature 0 increases, the predicted output also increases, suggesting a positive correlation.

- The trend is **non-linear**, with a steeper increase at lower values of Feature 0 and a **flattening effect at higher values**.
- The model's response suggests that **Feature 0 is an important predictor**, contributing to higher predictions as its value increases.

Interpretation:

- The model **relies heavily on Feature 0**, as increasing its value consistently **increases the predicted output**.
- The **non-linearity** in the curve suggests **diminishing returns**—higher values of Feature 0 still increase predictions, but at a slower rate.
- This insight can be useful for **feature engineering**, suggesting potential transformations (e.g., **log scaling** if required).

Conclusion:

- **Feature 0 has a strong positive impact on model predictions**.
- The effect is **non-linear**, meaning the feature's influence is stronger at lower values and tapers off at higher values.
- Understanding this trend can help in **feature selection, model interpretation, and possible feature scaling improvements**.

Permutation Feature Importance Analysis

Key Findings:

- The plot shows the **permutation feature importance**, which measures how much the model's performance decreases when each feature's values are randomly shuffled.
- **X-axis (Mean Permutation Importance)**: Represents the **average drop in model performance** when a feature is shuffled.
- **Y-axis (Feature Index)**: Lists the feature indices ranked by their importance.

Observations:

- **Feature 14 has the highest importance**, indicating that it is the most crucial predictor for the model.
- **Feature 9 also has a significant impact**, but lower than Feature 14.
- **Features 10, 12, and 8 show moderate importance**, meaning they contribute to the model but not as much as Features 14 and 9.
- **Other features (0, 1, 2, 3, 4, etc.) have very low or almost negligible importance**, suggesting they have little influence on the model's output.
- The **error bars represent variability in importance scores**, with larger bars indicating higher uncertainty.

Interpretation:

- **Feature 14 is the most influential feature**, meaning changes in its values have the greatest effect on model predictions.
- **Feature 9 also plays a crucial role**, but its impact is significantly lower than Feature 14.
- **Features with very low importance (e.g., Features 0, 1, 2, etc.) may be considered for removal** to simplify the model and improve efficiency.
- **Error bars indicate confidence in importance scores**, helping assess reliability.

Conclusion:

- **Feature 14 and Feature 9 are the primary drivers of the model's predictions**.
- **Features with near-zero importance may be removed**, as they do not significantly affect predictions.
- **Understanding feature importance helps in refining the model**, improving interpretability, and optimizing computational efficiency.

Integrated Gradients Analysis for a Single

Sample

Key Findings:

- The plot represents the **Integrated Gradients (IG) values** for a single sample, indicating how much each feature contributed to the model's prediction.
- **X-axis (Feature Index)**: Represents different features (0 to 14).
- **Y-axis (Integrated Gradient Value)**: Represents the importance of each feature in influencing the model's output.

Observations:

- **Feature 2 has the highest contribution**, with an IG value significantly larger than all other features.
- **Feature 0 and Feature 14 show minor contributions**, but they are much smaller compared to Feature 2.
- **Most other features have near-zero IG values**, meaning they have **negligible impact** on the prediction for this particular sample.
- The distribution suggests that **only a few features meaningfully contribute**, while others do not significantly influence the prediction.

Interpretation:

- **Feature 2 is the dominant driver of the model's decision for this sample**, suggesting that the model is heavily relying on it.
- **Feature 0 and Feature 14 have some influence**, but they are **not as impactful** as Feature 2.
- **The remaining features have almost no effect**, indicating that they are either redundant for this specific instance or their variations do not significantly impact the prediction.

Conclusion:

- **Feature 2 is the most influential predictor**, as it has the highest IG value.
- **Feature selection or regularization** could be considered to reduce dependence on non-influential features.
- This analysis helps in understanding how **individual predictions are made**, improving model interpretability, and identifying **potential feature dependencies**.

Global Surrogate Decision Tree Analysis

Key Findings:

- The **decision tree serves as a global surrogate model**, helping to interpret a more complex machine learning model by approximating its decision-making process.
- **Root Node (Top Decision Split)**:
 - The first split occurs at **rolling_mean $\leq$ 1.913**, indicating that this feature is the most important for decision-making.
 - The dataset is divided into two major groups based on this threshold.

Observations:

- **Left Subtree (rolling_mean $\leq$ 1.913)**:
 - Further splits occur at **rolling_mean $\leq$ 1.279** and **lag_1 $\leq$ 1.655**.
 - This subtree contains **2213 samples**, with an average prediction value of **1.562**.
 - Lower rolling mean values lead to smaller predictions, with **the smallest values reaching 0.307**.
- **Right Subtree (rolling_mean $>$ 1.913)**:
 - Further splits occur at **rolling_mean $\leq$ 2.255** and **lag_1 $\leq$ 2.085**.
 - This subtree contains **3261 samples**, with an average prediction value of **2.226**.
 - Higher rolling mean values lead to **larger predictions, reaching up to 2.498**.
- **Error Reduction**:
 - As we move deeper into the tree, **squared error decreases**, indicating that the model becomes

more confident in its predictions.

- The **lowest squared error occurs at the deepest nodes**, meaning the tree effectively partitions the data.

Interpretation:

- **Rolling Mean is the most influential feature**, as it is used in multiple split points.
- **Lag_1 is the second most important feature**, used for finer decision-making at deeper levels.
- The model predicts **lower values for lower rolling mean values and higher values for higher rolling mean values**.
- The tree structure suggests a **hierarchical decision process**, where early splits determine major trends, and later splits refine the predictions.

Conclusion:

- **Rolling Mean and Lag_1 are the key drivers of model predictions**.
- **Predictions increase as rolling_mean values increase**, showing a strong correlation.
- The decision tree provides a **simplified, interpretable version of the original model**, making it useful for understanding key decision rules.
- **Smaller squared error at deeper levels confirms that the tree effectively captures the underlying structure of the data**.

Saliency Map Analysis for First Test Sample

Key Findings:

- The **saliency map** visualizes how sensitive the model's prediction is to changes in each timestep's input feature.
- **X-axis (Timesteps)**: Represents different time steps (0 to 14) in the sequence.
- **Y-axis (Gradient Magnitude)**: Measures how much the prediction changes when each timestep's value is slightly altered.

Observations:

- **Timestep 14 has the highest gradient magnitude (~0.8)**, indicating that this timestep has the strongest impact on the model's prediction.
- **Timestep 9 also has a significant gradient magnitude (~0.2)**, but it is much lower than Timestep 14.
- **Most other timesteps (2 to 8) show near-zero gradient values**, meaning that **changes in these timesteps have minimal effect on the model's output**.
- **The initial timesteps (0-1) have small but nonzero gradient values**, suggesting a minor influence.

Interpretation:

- **Timestep 14 is the most critical input feature** for the model's decision in this test sample.
- **Timestep 9 is the second most important**, but with a significantly lower impact.
- **Timesteps with near-zero gradients indicate unimportant features**, which could suggest that these time steps do not carry useful predictive information for this sample.
- **This analysis helps in understanding which parts of the input sequence drive model predictions**, making it useful for interpretability in time-series forecasting tasks.

Conclusion:

- **Timesteps 14 and 9 contribute the most to the model's prediction**, with Timestep 14 having the highest influence.
- **Other timesteps (especially 2-8) have almost no impact**, suggesting they might be redundant for this sample.
- **Feature selection or attention mechanisms** could be considered to focus more on the relevant timesteps and improve model efficiency.
- **This saliency map enhances interpretability**, allowing for better understanding of time-series feature importance.

Residual Analysis for BiLSTM Predictions

Key Findings:

- Residuals (Y-axis)** represent the difference between predicted and true values:
$$\text{Residual} = \text{True Value} - \text{Predicted Value}$$
- True Values (X-axis)** show the actual target values from the dataset.
- The **red dashed line** represents the ideal case where residuals are zero, indicating perfect predictions.

Observations:

- Most residuals are close to zero**, indicating that the BiLSTM model generally performs well.
- There is a **slight upward trend**, meaning that as true values increase, residuals tend to be positive, suggesting the model might **underestimate larger values**.
- Some points fall **below zero**, which means the model also occasionally **overestimates** certain values.
- Higher variance in residuals** for larger true values indicates **greater prediction uncertainty** in this range.

Interpretation:

- The model performs well but tends to slightly **underestimate larger true values**.
- Some **outliers** (residuals far from zero) exist, indicating **potential mispredictions**.
- The model might benefit from:
 - Recalibrating on larger values** to reduce bias.
 - Regularization techniques** to control variance.
 - Feature engineering or additional training** to enhance performance.

Conclusion:

- The **BiLSTM model** provides **reasonable predictions**, but it **struggles slightly with high values**.
- Residuals are generally small, but **variance increases for larger true values**.
- Further **model tuning, feature selection, or data augmentation** might help improve performance.

Output Summary for Interpretability Results

SHAP: Global summary plot displays the top feature contributions.

LIME: Local explanations available for instance 10.

Residual Analysis: Outliers handled, 64 instances identified.

Bidirectional Gated Recurrent Unit (BiGRU)

Optimized Bidirectional GRU Model for Time-Series Forecasting: A Comprehensive Pipeline

1. Introduction

- This notebook presents a **systematic approach** to training a **Bidirectional Gated Recurrent Unit (Bi-GRU) model** for time-series forecasting.
- It includes **data preprocessing, hyperparameter tuning using Optuna, model training, evaluation, and interpretability techniques**.
- Key highlights:
 - Reproducibility**: Ensures deterministic behavior for consistent results.
 - Hyperparameter Optimization**: Uses **Optuna** to find the best Bi-GRU configuration.
 - Evaluation & Interpretability**: Includes **outlier handling, loss curve smoothing, and residual analysis**.

2. Setting Up the Environment

2.1 Importing Libraries

- **TensorFlow** for deep learning.
- **Optuna** for hyperparameter tuning.
- **Matplotlib** for visualizations.
- **Scikit-learn** for train-test splitting.

2.2 Setting Global Random Seeds

- A **fixed seed (42)** ensures **consistent experimental results**.
- TensorFlow is **configured for deterministic behavior**.

2.3 Configuring TensorFlow for Stability

- **Forces single-thread execution** to improve stability across runs.
 - **Suppresses unnecessary logging** to maintain a clean output.
-

3. Data Preprocessing for GRU Model

3.1 Safe Train-Test Splitting

- **Maintains the chronological order** to prevent data leakage.
- **Ensures non-overlapping train and test sets**.

3.2 Reshaping Data for Recurrent Neural Networks (RNNs)

- **GRU models require 3D input** (samples, timesteps, features).
 - The **reshape function expands dimensions** for compatibility.
-

4. Defining the Bidirectional GRU Model

4.1 Hyperparameter Tuning Using Optuna

- **Optimized parameters:**
 - **GRU units** (100, 150).
 - **Learning rate** (0.001, 0.005).
 - **Dropout rate** (0.2, 0.3).
 - **Gradient clipping** (clipnorm 2.0, 3.0).
 - **Optimizer** (Adam, RMSprop).
 - **Activation function** (tanh, relu).
 - **Batch size** (32, 64).
 - **Epochs** (50, 100, 150).

4.2 Bi-GRU Model Architecture

- **Two Bidirectional GRU layers** with dropout for regularization.
 - **A single Dense output layer** for regression tasks.
 - **Mean Squared Error (MSE) loss function**.
-

5. Hyperparameter Optimization with Optuna

5.1 Optuna Objective Function

- **Trials different Bi-GRU configurations** to minimize validation loss.
- **Uses EarlyStopping & Pruning** to optimize training efficiency.

5.2 Running the Optimization Process

- **20 trials with 1-hour timeout** for computational efficiency.
 - **Stores the best hyperparameters** for final training.
-

6. Training the Best Bi-GRU Model

6.1 Model Training on the Full Dataset

- Trains the Bi-GRU model using the best hyperparameters.
- 20% validation split to monitor performance.
- Predictions generated on the test set.

6.2 Smoothing Loss Curves

- A moving average filter reduces noise in training/validation loss plots.
 - Helps in analyzing the learning behavior of the model.
-

7. Evaluating the Model

7.1 Displaying the Model Summary

- Layer-wise architecture inspection.
- Number of parameters (trainable & non-trainable).

7.2 Computing Residuals and Detecting Outliers

- Residuals (True - Predicted values) are analyzed.
- Outliers are defined as values exceeding 3 standard deviations.

7.3 Removing Outliers for Robust Evaluation

- Predictions and true values are cleaned before final evaluation.

7.4 Computing Evaluation Metrics

- Metrics are recalculated after removing extreme outliers.
-

8. Visualization and Interpretation

8.1 Residual Analysis

- Scatter plot of residuals vs. actual values.
- Red reference line ($y=0$) indicates ideal predictions.

8.2 Loss Curves Over Epochs

- Smoothed curves show training dynamics.
 - Validation loss monitoring helps prevent overfitting.
-

9. Summary of Key Findings

- Bi-GRU models effectively capture time-series dependencies.
 - Optuna significantly improves model performance via hyperparameter tuning.
 - Handling outliers ensures robust evaluation.
 - Residual analysis highlights model performance variations.
-

10. Conclusion

- This notebook provides a **comprehensive, reproducible framework** for training a **highly optimized Bi-GRU model** for time-series forecasting.
- The **combination of hyperparameter tuning, visualization, and interpretability techniques** ensures both accuracy and explainability.

Interpreting Bi-GRU Model Predictions: Residual Analysis, Feature Importance, and Model Explainability

1. Introduction

- This notebook provides an **in-depth interpretability analysis** of the **Bidirectional Gated Recurrent**

Unit (Bi-GRU) model.

- The following aspects are covered:
 - Residual Analysis and Outlier Detection
 - Feature Distribution in Training and Test Sets
 - Signal-to-Noise Ratio (SNR) for Model Evaluation
 - SHAP for Global Feature Importance
 - Partial Dependence Plot (PDP) Analysis
 - Integrated Gradients for Attribution
 - Saliency Maps for Feature Sensitivity
 - LIME for Local Explanation of Predictions
 - Decision Tree Surrogate Model for Approximate Interpretability
-

2. Residual Analysis and Outlier Detection

2.1 Visualizing Residuals in Bi-GRU Predictions

- Residuals (True - Predicted values) are plotted to assess systematic prediction errors.
- Thresholds at ± 3 standard deviations identify potential outliers.

2.2 Scatter Plot of Residuals

- Blue dots represent residuals.
 - Dashed red lines show the outlier thresholds.
 - Points outside the thresholds indicate extreme prediction errors.
-

3. Visualizing Predictions vs. True Values

- Line plots compare actual vs. predicted values.
 - True values are shown in green, while Bi-GRU predictions are shown in orange.
 - Dashed lines indicate prediction uncertainty.
-

4. Feature Distribution Analysis

4.1 Comparing Training and Test Set Distributions

- Histograms visualize how feature distributions differ between the training and test sets.
 - Blue bars represent the training set, while orange bars represent the test set.
 - Overlapping areas indicate consistency in feature distributions.
-

5. Signal-to-Noise Ratio (SNR) Computation

- SNR quantifies how much useful signal exists relative to noise.
- Higher SNR values indicate a better predictive model.

5.1 Formula for SNR Calculation

[$\text{SNR} = 10 \log_{10} \left(\frac{\text{Signal Power}}{\text{Noise Power}} \right)$]

- **Signal Power:** Mean squared value of true values.
 - **Noise Power:** Mean squared residuals (errors).
 - Computed SNR values are reported in decibels (dB).
-

6. SHAP Feature Importance Analysis

- SHAP (SHapley Additive exPlanations) provides a global feature importance analysis.
- KernelExplainer is used to approximate SHAP values for the Bi-GRU model.
- SHAP values measure how much each feature contributes to the model's prediction.

6.1 SHAP Summary Plot

- Bar plot ranks the most influential features.
- SHAP values closer to zero indicate minimal influence.

6.2 SHAP Dependence Plot

- Visualizes how feature interactions influence predictions.
 - Points represent individual data samples.
-

7. Partial Dependence Plot (PDP) Analysis

- PDP examines how changing a feature's value impacts the model's prediction.
- Computed by varying a selected feature while keeping others constant.

7.1 PDP Visualization

- X-axis: Feature values
 - Y-axis: Partial dependence (model's average prediction response).
-

8. Integrated Gradients for Feature Attribution

- Integrated Gradients (IG) quantifies the contribution of each feature to a prediction.
- Calculates the path integral of gradients along an interpolated trajectory from a baseline (e.g., zero input) to the actual input.

8.1 IG Formula

$$IG = (X - X_{\text{baseline}}) \times \text{Average Gradient}$$

- The IG bar plot highlights the most influential features.
-

9. Saliency Maps for Feature Sensitivity

- Saliency Maps highlight which time steps influence Bi-GRU predictions the most.
- Gradients are computed with respect to input features.

9.1 Saliency Map Visualization

- X-axis: Time steps
 - Y-axis: Gradient magnitude (importance score for each timestep).
-

10. Decision Tree Surrogate Model for Approximate Interpretability

- A Decision Tree Regressor is trained to approximate the behavior of the Bi-GRU model.
- The decision tree provides a human-interpretable approximation of how the Bi-GRU model makes predictions.

10.1 Decision Tree Visualization

- Tree nodes represent decision rules for predictions.
 - Feature splits indicate which features are most important.
-

11. Local Model Explanation using LIME

- LIME (Local Interpretable Model-Agnostic Explanations) explains individual Bi-GRU predictions.
- Works by generating perturbed versions of an instance and fitting a simpler interpretable model (e.g., linear regression) to approximate predictions.

11.1 LIME Explanation for a Single Test Instance

- The top 10 most influential features are displayed.

- Explains why a particular instance received its predicted value.

12. Summary and Conclusions

- Residual analysis detects systematic prediction errors and outliers.
 - SNR provides a quantitative measure of prediction reliability.
 - SHAP, PDP, and Integrated Gradients offer detailed feature importance insights.
 - LIME enables local interpretability of Bi-GRU predictions.
 - A decision tree surrogate model provides an approximate explanation of model decisions.
-

Output Result:

Best Parameters:

```
'units': 150,  
'learning_rate': 0.005,  
'dropout_rate': 0.2,  
'clipnorm': 2.0,  
'optimizer': 'Adam',  
'activation_function': 'tanh',  
'batch_size': 64,  
'epochs': 100
```

Best Validation Loss: 0.0007413366693072021

Number of Outliers Detected: 98

Indices of Outliers: [67 123 128 171 202 205 235 237 276 286 292 293 297 305 306 310 311 312 313 314 315 316 317 320 339 340 348 353 354 359 361 362 366 374 387 388 391 404 408 409 416 417 418 419 422 423 424 425 426 428 429 430 431 432 433 434 435 436 439 440 441 442 443 444 445 446 447 448 449 450 457 458 459 460 461 462 463 464 467 468 469 470 471 472 473 474 475 476 477 481 483 487 488 496 506 509 517 929]

Bidirectional GRU Evaluation Metrics After Outlier Removal:

```
'Model': 'Bidirectional Gated Recurrent Unit (Bi-GRU)',  
'MSE': 0.003462737144908402,  
'RMSE': 0.05884502650953946,  
'MAE': 0.05205709345972884,  
'MAPE (%)': 2.2095573631071015,  
'SMAPE (%)': 2.24014713929429,  
'MASE': 0.4155021714852761,  
'R^2': 0.8382369251733033
```

Smoothed Loss Over Epochs

Key Findings:

- The **Y-axis (Loss)** represents the model's error during training and validation.
- The **X-axis (Epochs)** represents the number of training iterations.
- Two curves are shown:
 - **Smoothed Training Loss (Blue):** Tracks how well the model is minimizing error during training.
 - **Smoothed Validation Loss (Orange):** Measures the model's performance on unseen data.

Observations:

- **Rapid loss decrease in the early epochs:**
 - The training loss starts high but **drops sharply in the first few epochs**, indicating the model is **learning quickly**.
- **Training and validation loss stabilize after a few epochs:**
 - Both losses reach **very low values and remain stable**, suggesting the model has **successfully learned patterns** in the data.
- **No significant overfitting:**
 - The training and validation loss curves remain **close to each other**, meaning the model **generalizes well** without memorizing the training data.

Interpretation:

- The model is **well-optimized**, with minimal training and validation loss.
- There is **no major overfitting** since the validation loss does not diverge from the training loss.
- The loss values are **very small**, indicating **high model accuracy**.

Conclusion:

- The training process was **successful**, with loss stabilizing after an initial sharp decline.
- The model appears to be **robust**, generalizing well to new data.
- Further fine-tuning may not be necessary unless additional performance gains are required.

Model: "sequential_41"

Layer (type)	Output Shape	Param #
bidirectional_82 (Bidirectional)	(None, 15, 300)	137,700
dropout_82 (Dropout)	(None, 15, 300)	0
bidirectional_83 (Bidirectional)	(None, 300)	406,800
dropout_83 (Dropout)	(None, 300)	0
dense_41 (Dense)	(None, 1)	301

Total params: 1,634,405 (6.23 MB)
Trainable params: 544,801 (2.08 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 1,089,604 (4.16 MB)

Key Insights/Findings:

- The **Bi-GRU layers** contribute significantly to the model’s complexity, with a high number of parameters dedicated to sequential feature extraction.
- **Dropout layers** are included to reduce overfitting but do not add trainable parameters.
- The **Dense output layer** has minimal parameters, indicating a simple final prediction mapping.
- A large proportion of parameters belong to the **optimizer**, suggesting significant memory allocation for training stability.

Observations and Conclusion:

- The model is **well-structured** for handling sequential data, with Bi-GRU layers extracting temporal dependencies.
- **Regularization is enforced** via dropout layers to enhance generalization.
- The model balances trainable and optimizer parameters efficiently, making it **scalable for real-world time-series tasks**.

Residuals and Outliers Analysis

Key Observations

- **Residual Distribution:** The scatter plot shows the residuals (differences between predicted and actual values) across different data points. Most residuals are small, indicating a relatively good fit.
- **Upper and Lower Thresholds:** The red dashed lines represent the upper and lower threshold

boundaries. Residuals outside these limits are considered outliers.

- **Outlier Detection:** A few data points exceed the upper threshold, indicating significant deviations from expected values, while there are fewer cases where residuals fall below the lower threshold.
- **Pattern in Residuals:** There appears to be some structure in the residuals, particularly around the middle of the dataset, suggesting potential heteroscedasticity (variance changes across different data points).
- **Implications:** If residuals exhibit a pattern rather than random dispersion, it may indicate that the model is not capturing all underlying relationships, possibly requiring feature engineering or a different modeling approach.

Conclusion

The plot suggests that while the model generally fits the data well, there are notable deviations (outliers) that may require further investigation. Addressing these anomalies could improve model performance and robustness.

Model Predictions vs. True Values

Key Observations

- **Comparison of Predictions and Actual Values:**
 - The plot compares the model's predicted values (orange crosses) against the true values (green circles).
 - There is a strong alignment between the predicted and true values, suggesting that the model captures the overall trend effectively.
- **Prediction Accuracy:**
 - The model performs well in tracking the general shape and fluctuations of the actual values.
 - However, slight deviations exist where the predicted values occasionally under- or over-estimate the true values.
- **Peaks and Valleys:**
 - The model struggles slightly with sharp peaks and sudden changes, as seen in some areas where the true values exhibit rapid increases or decreases.
 - This suggests that the model may have some lag in capturing extreme variations or rapid transitions.
- **Noise and Variability:**
 - The distribution of predictions around the true values appears relatively tight, indicating a low level of random error.
 - Some variance remains, which may suggest minor underfitting or room for further refinement in the model.

Conclusion

The model exhibits strong predictive performance, closely following the true values. However, minor discrepancies exist, particularly in capturing sharp peaks and fluctuations. Fine-tuning the model or incorporating additional features may improve accuracy in these areas.

Feature Distribution in Training and Test Sets

Key Observations

- **Highly Skewed Distribution:**
 - The majority of feature values are concentrated near zero, with a long tail extending toward higher values.
 - This suggests that the dataset contains many small values and a few significantly large values, indicating a highly skewed distribution.
- **Comparison of Train and Test Sets:**
 - The training set (blue) and test set (orange) have similar distributions, meaning the data

split has preserved the feature distribution.

- This consistency is crucial for ensuring that the model generalizes well when applied to unseen data.
- **Outliers Present:**
 - The presence of values extending towards 20,000+ suggests potential outliers in the dataset.
 - These extreme values may need special handling, such as normalization, log transformation, or outlier removal, depending on the impact on model performance.
- **Class Imbalance or Sparse Features:**
 - The extreme concentration of values at zero might indicate a large number of missing or sparse features.
 - If this feature represents categorical or count data, feature engineering techniques such as binning or scaling could be useful.

Conclusion

The feature distribution is highly skewed with a significant number of low-value observations and some extreme outliers. Ensuring appropriate preprocessing, such as normalization or transformation, could help improve model performance and stability.

Signal-to-Noise Ratio (SNR): 30.83 dB

```
shap_values.shape: (10, 15, 1)
```

```
X_test_flat[:10].shape: (10, 15)
```

After squeeze, shap_values.shape: (10, 15)

SHAP Feature Importance Analysis

Key Observations

- **Feature Importance Ranking:**
 - The plot shows the average SHAP value magnitude for each feature, representing its contribution to the model's predictions.
 - Higher values indicate a greater impact on model output.
- **Most Influential Features:**
 - `Feature_14` has the highest impact on the model, meaning it plays the most significant role in decision-making.
 - `Feature_0`, `Feature_12`, and `Feature_9` also contribute significantly but to a lesser extent than `Feature_14`.
- **Less Impactful Features:**
 - Features such as `Feature_1`, `Feature_2`, `Feature_5`, `Feature_6`, and `Feature_7` have minimal influence on the model.
 - These features might be candidates for removal or further investigation to determine their relevance.
- **Model Interpretability:**
 - The plot helps in understanding which features drive predictions the most.
 - It can assist in feature selection, simplifying the model without losing predictive power.

Conclusion

The model relies heavily on `Feature_14`, followed by `Feature_0`, `Feature_12`, and `Feature_9`. Features with low importance may be reconsidered for removal or further analyzed for potential interactions with other features.

SHAP Value Analysis for Feature 0

Key Observations

- **Feature Influence on Model Predictions:**
 - The plot shows the SHAP values for `Feature 0`, which indicate how this feature impacts the model's output.
 - Negative SHAP values suggest that increasing `Feature 0` tends to decrease the model's prediction.
- **Trend in SHAP Values:**
 - Data points with lower `Feature 0` values (around 2.03–2.05) generally have more negative SHAP values.
 - Higher values of `Feature 0` (above 2.07) tend to have less negative or slightly higher SHAP values, indicating a weaker negative effect on the model output.
- **Color Representation:**
 - The plot uses color gradients to highlight different values, where red points typically indicate higher values, and blue points represent lower values.
 - This helps visualize how `Feature 0` affects the SHAP values across different instances.
- **Possible Interpretation:**
 - The nonlinear distribution of SHAP values suggests that `Feature 0` may have varying degrees of impact at different ranges.
 - If the relationship is complex, transformations (e.g., log scaling or binning) might be explored to better model its effect.

Conclusion

The plot indicates that `Feature 0` generally has a negative impact on predictions, but the strength of this effect varies across different feature values. This suggests a nonlinear relationship that may require further investigation for improved model interpretability.

Partial Dependence Plot (PDP) for Feature 0

Key Observations

- **Relationship Between Feature 0 and Model Output:**
 - The partial dependence plot (PDP) illustrates how `Feature 0` affects the model's predictions while averaging out the effects of all other features.
 - The trend shows a positive correlation between `Feature 0` and the model output.
- **Monotonic Increase:**
 - As `Feature 0` increases, the partial dependence value also increases, indicating that higher values of `Feature 0` contribute to higher predictions.
 - The relationship appears to be smooth and nonlinear, with diminishing returns at higher values.
- **Nonlinear Pattern:**
 - The curve is not perfectly linear but rather follows an upward concave shape.
 - This suggests that the impact of `Feature 0` on predictions grows at a decreasing rate.
- **Model Interpretation:**
 - If `Feature 0` represents a numerical variable such as a financial metric or a continuous measurement, its positive influence on the prediction means that larger values push the model output higher.
 - The diminishing slope at higher values suggests that beyond a certain point, increasing `Feature 0` has a reduced impact on predictions.

Conclusion

The PDP for `Feature 0` reveals a positive nonlinear relationship with the model's output, where increasing `Feature 0` leads to higher predictions, but with diminishing effects at higher values. This insight can be used for feature analysis and potential model optimization.

Integrated Gradients Analysis for a Single Test

Sample

Key Observations

- **Feature Importance and Directionality:**
 - The plot shows the Integrated Gradients values for each feature, indicating their contribution to the model's prediction for a single test sample.
 - Positive values suggest a feature contributes positively to the prediction, while negative values indicate a suppressing effect.
- **Most Influential Features:**
 - **Feature 2** has the most significant negative impact on the model's prediction, pulling the output lower.
 - **Feature 14** has the highest positive contribution, strongly increasing the prediction.
- **Minor Impact of Other Features:**
 - Several features have small gradient values, indicating that they have minimal influence on the model's decision for this specific instance.
 - Some features (e.g., **Feature 6** and **Feature 12**) have slight positive or negative effects, but they are not as dominant as **Feature 2** or **Feature 14**.
- **Potential Implications:**
 - The model heavily relies on **Feature 2** and **Feature 14** for this particular prediction.
 - If this trend is observed across multiple test samples, these features might be critical in the overall decision-making process of the model.

Conclusion

The Integrated Gradients plot highlights **Feature 2** as a strong negative influencer and **Feature 14** as a major positive contributor to the model's prediction. Other features play a less significant role in this specific test instance. Further analysis on a broader dataset could confirm whether these trends hold consistently.

Saliency Map for First Test Sample

Key Observations

- **Gradient Magnitude Interpretation:**
 - The plot shows the gradient magnitude of the model's output with respect to each time step (or feature index).
 - Higher values indicate greater importance, meaning that changes in those time steps have a stronger effect on the model's decision.
- **Most Important Time Step:**
 - The last time step (index 14) exhibits the highest gradient magnitude, suggesting it plays a crucial role in the model's prediction.
 - This implies that the model relies heavily on recent or specific features when making a decision.
- **Low Impact of Earlier Time Steps:**
 - Time steps from 1 to 7 have minimal gradient magnitudes, indicating they contribute little to the model's final decision.
 - This may suggest that the earlier part of the sequence contains less relevant information or that the model has learned to focus more on later time steps.
- **Intermediate Fluctuations:**
 - There is a noticeable increase in gradient magnitude around time steps 9 to 12, suggesting some mid-sequence features influence the model to a moderate extent.
 - However, their impact is still significantly lower compared to the last time step.

Conclusion

The saliency map indicates that the model heavily relies on the last time step (feature index 14)

while earlier time steps have little influence. This suggests a strong temporal dependency, which could imply that later features contain more predictive information. Understanding this pattern can help refine feature engineering or model architecture for improved performance.

Global Surrogate Decision Tree Analysis

Key Observations

- **Purpose of the Decision Tree:**
 - This decision tree serves as a **global surrogate model** to approximate and interpret a more complex machine learning model.
 - It helps in understanding how key features influence predictions in a more interpretable manner.
- **Primary Splitting Feature:**
 - The root node splits on `lag_1 <= 1.915`, indicating that this feature is the most important in dividing the dataset.
 - This suggests that past values (`lag_1`) significantly impact the predictions.
- **Subsequent Splits and Feature Influence:**
 - The second major split on the left branch occurs on `rolling_mean <= 1.279`, showing that the rolling average also plays a crucial role in prediction.
 - On the right branch, `lag_1 <= 2.255` is another strong predictor, further refining the decision boundary.
 - The model heavily relies on `lag_1` and `rolling_mean`, meaning past values and their trends are critical for forecasting.
- **Error Reduction Across Splits:**
 - As the tree branches further, the squared error decreases, meaning that finer splits lead to more accurate predictions.
 - Terminal nodes have the lowest squared error, showing that more refined decision-making improves prediction accuracy.
- **Sample Distribution:**
 - The root node contains **5474 samples**, which get progressively split into smaller groups based on feature thresholds.
 - Leaves represent the final predictions, with varying sample sizes and values.

Conclusion

This **global surrogate decision tree** provides insights into the underlying complex model by approximating its decision-making process. The most influential features are `lag_1` and `rolling_mean`, which means past values and their moving averages are critical for predictions. Understanding these decision paths can help improve feature engineering, model interpretation, and refinement.

Local Explanation Analysis

Key Observations

- **Prediction Overview:**
 - The predicted local value is **2.027**.
 - The minimum and maximum possible predictions in the model's range are **-0.08** and **2.71**, respectively.
 - The intercept (baseline prediction) is **1.796**, meaning this is the starting reference point before feature contributions are considered.
- **Feature Contributions:**
 - The visualization separates **positive** and **negative** contributions of different features to the model's prediction.
 - Positive contributions increase the predicted value, while negative contributions lower it.
- **Most Influential Features:**

- **Positive Impact:**
 - `Feature_14` (> 2.06) contributes **+0.30** to the prediction, making it the strongest positive influencer.
 - `Feature_9` (> 2.05) contributes **+0.10**, suggesting it also has a meaningful but smaller positive effect.
- **Negative Impact:**
 - `Feature_12` (≤ 2.05) contributes **-0.08**, decreasing the prediction.
 - `Feature_6` (≤ 97.81) contributes **-0.04**, indicating a moderate negative effect.
- **Feature Values:**
 - The feature values are displayed on the right panel.
 - Features in **orange** contribute positively, while those in **blue** contribute negatively.
 - Notably, `Feature_2` has an extremely high value (**21058.38**), yet it contributes minimally, suggesting its impact on this specific prediction might be weak or already accounted for in interactions.

Conclusion

The local explanation highlights how specific feature values influenced the final prediction of **2.027**. `Feature_14` and `Feature_9` are the strongest **positive contributors**, while `Feature_12` and `Feature_6` act as **negative influencers**. This analysis helps in understanding how the model makes decisions for an individual prediction.

Hybrid Models

Hybrid Models: Combining Machine Learning and Deep Learning Approaches

This section explores three hybrid modeling techniques:

1. **Stacking Ensemble (Random Forest + XGBoost)**
2. **LSTM Feature Extraction + Random Forest**
3. **LSTM + GRU Hybrid Model**

Each approach leverages strengths from different modeling paradigms to enhance predictive performance in time series forecasting.

1. Stacking Ensemble: Combining Random Forest (RF) and Extreme Gradient Boosting (XGBoost)

Step 1: Train the Base Model - Random Forest

- The **Random Forest** model is initialized with optimized hyperparameters.
- The model is trained on the original training dataset (`X_train`, `y_train`).
- Predictions are generated for both training and test sets.

Step 2: Prepare Data for Stacking

- The **Random Forest predictions** are added as a new feature to both training and test sets.
- This additional feature allows the second-layer model to leverage the RF predictions.

Step 3: Train the Second-Layer Model - XGBoost

- The **Extreme Gradient Boosting (XGBoost)** model is trained using the augmented dataset.
- XGBoost uses **RF predictions as an additional feature**, improving its learning capabilities.

Step 4: Evaluate the Stacking Model

- The model's predictions are evaluated using predefined regression metrics.
 - The final performance is stored for comparison.
-

2. Hybrid Model: LSTM for Feature Extraction + Random Forest for Prediction

Step 1: Set Random Seeds for Reproducibility

- Seeds are fixed for **NumPy** and **TensorFlow** to ensure consistent results.

Step 2: Data Preparation for LSTM

- **Lag features** are extracted to provide sequential inputs for LSTM.
- The data is reshaped into a **3D format**: (**samples**, **timesteps**, **features**).

Step 3: Build and Train the LSTM Model

- An **LSTM model** is constructed with:
 - A **tanh activation function** for temporal learning.
 - A **dropout layer** to reduce overfitting.
 - A **dense layer** to extract latent feature embeddings.
- The model is trained on the sequential data.

Step 4: Extract LSTM Latent Feature Embeddings

- The trained **LSTM model** generates **feature embeddings** for both training and test sets.

Step 5: Train a Random Forest on LSTM Embeddings

- A **Random Forest model** is trained using the extracted LSTM embeddings.
- The **LSTM acts as a feature extractor**, while RF handles final predictions.

Step 6: Evaluate the Hybrid Model

- Predictions are generated, and evaluation metrics are computed.
 - The **LSTM + RF** hybrid model is compared against other models.
-

3. Hybrid Model: LSTM + GRU for Time Series Forecasting

Step 1: Set Random Seeds for Reproducibility

- Fixes seeds for **NumPy** and **TensorFlow** to ensure stable results.

Step 2: Data Preparation for LSTM + GRU

- **Lag features** are extracted and reshaped into **3D format** for sequential learning.

Step 3: Build the LSTM + GRU Hybrid Model

- The model architecture consists of:
 - An **LSTM layer** for capturing long-term dependencies.
 - A **GRU layer** for short-term dependencies.
 - **Dropout layers** for regularization.
 - A **Dense layer** for final predictions.

Step 4: Train the LSTM + GRU Hybrid Model

- The model is trained with **Mean Squared Error (MSE) loss** and an **Adam optimizer**.

Step 5: Make Predictions Using the Trained Model

- The trained **LSTM-GRU model** generates predictions on the test set.

Step 6: Evaluate Model Performance

- The model's performance is evaluated against benchmark forecasting methods.
 - Results are stored for comparison with other models.
-

Summary

Model Type	Key Components	Strengths
Stacking (RF + XGBoost)	Random Forest (Base) + XGBoost (Meta)	Leverages two powerful ML models for improved accuracy.
LSTM + RF	LSTM for feature extraction + RF for prediction	Uses deep learning to extract meaningful features for an ML model.
LSTM + GRU	LSTM + GRU (Deep Learning)	Captures both long-term and short-term dependencies in time series data.

Each hybrid model enhances predictive accuracy by integrating different strengths from **Machine Learning (ML)** and **Deep Learning (DL)**.

Output Result:

Stacking Ensemble (RF + GBM) Evaluation Metrics:

'Model': 'Hybrid Model: Stacking RF + GBM',
'MSE': 0.003578237589134771,
'RMSE': 0.059818371669034684,
'MAE': 0.03493534310008593,
'MAPE (%)': 1.3977086797788767,
'sMAPE (%)': 1.4202811745173631,
'MASE': 1.9394357138693288,
'R^2': 0.8875065165701795

LSTM + Random Forest Evaluation Metrics:

'Model': 'Hybrid Model: LSTM + Random Forest',
'MSE': 0.0026929336144895003,
'RMSE': 0.0518934833528209,
'MAE': 0.03085490952236714,
'MAPE (%)': 1.24663044213293,
'sMAPE (%)': 1.2561475327432015,
'MASE': 1.7129104272554994,
'R^2': 0.9153389132518637

LSTM + GRU Hybrid Evaluation Metrics:

'Model': 'Hybrid Model: LSTM + GRU',
'MSE': 0.0017484045925626251,
'RMSE': 0.04181392821253015,
'MAE': 0.03010765143255843,
'MAPE (%)': 1.2569436566030483,
'sMAPE (%)': 1.2550489215774794,
'MASE': 1.6714263913695253,
'R^2': 0.9450332410404239

Final Result for All Models

Model Evaluation Metrics Summary and Visualization

This section presents a **summary and visual comparison of evaluation metrics** for different models. It includes:

1. **Tabular representation** of model performance metrics.
2. **Best-performing model selection** based on each metric.
3. **Graphical comparison** of evaluation metrics using bar plots.

1. Convert and Display Evaluation Metrics as a Table

- The `metrics_results` list (containing dictionaries of model evaluation metrics) is converted into a **Pandas DataFrame** for better readability.
- The `tabulate` library is used to generate a **formatted table** displaying the metrics in an organized structure.

2. Identify the Best Model for Each Metric

- The best model for each metric is determined:
 - **For R^2** : A **higher** value indicates better performance.
 - **For other metrics (MSE, RMSE, MAE, MAPE, sMAPE, MASE)**: A **lower** value is preferable.
- The model achieving the **best score** for each metric is displayed.

3. Model Evaluation Metric Visualization

- **Horizontal bar plots** are created to compare model performance for the following metrics:
 - **Mean Squared Error (MSE)**
 - **Root Mean Squared Error (RMSE)**
 - **Mean Absolute Error (MAE)**
 - **Mean Absolute Percentage Error (MAPE)**
 - **Symmetric Mean Absolute Percentage Error (sMAPE)**
 - **Mean Absolute Scaled Error (MASE)**
 - **Coefficient of Determination (R^2)**
- The **plots highlight performance differences** among models, making it easier to interpret results visually.
- **Ascending or descending sorting** is applied based on whether a **higher** or **lower** value is preferred.

Key Insights

- This approach **simplifies model comparison** by presenting both numerical and graphical insights.
- The **best model for each metric** is explicitly highlighted.
- **Performance trade-offs** between different evaluation metrics can be visually assessed.

By using **structured tables and intuitive visualizations**, this methodology provides a **comprehensive evaluation framework** for comparing time series forecasting models.

Model Evaluation Metrics Summary

Model	MSE	RMSE	MAE	MAPE (%)	sMAPE (%)	MASE	R^2
Random Forest (RF)	0.0031	0.0557	0.0300	1.1942	1.2109	1.6663	0.9025
Extreme Gradient Boosting (XGBoost)	0.0006	0.0245	0.0146	0.9443	0.9379	0.0372	0.9966
Light Gradient Boosting Machine (LightGBM)	0.0007	0.0273	0.0169	1.2628	1.1649	0.0429	0.9958
Gradient Boosting Regressor (GBR)	0.0006	0.0245	0.0148	0.9403	0.9366	0.0377	0.9966
Extremely Randomized Trees Regressor (ERTR)	0.0007	0.0269	0.0164	1.1514	1.0918	0.0417	0.9959
Histogram-Based Gradient Boosting Regressor (HGBR)	0.0006	0.0242	0.0148	0.9646	0.9656	0.0378	0.9967
Categorical Boosting (CatBoost)	0.0006	0.0247	0.0151	1.1091	1.0280	0.0385	0.9966
Support Vector Regression (SVR)	0.0012	0.0347	0.0163	1.0985	1.0268	0.0415	0.9932
Ridge Regression (Ridge)	0.0001	0.0113	0.0067	0.4288	0.4283	0.0170	0.9993

Lasso Regression (Lasso)	0.0010	0.0311	0.0198	1.5007	1.3584	0.0504	0.9945
ElasticNet Regression (EN)	0.0009	0.0293	0.0179	1.3318	1.2061	0.0455	0.9952
Decision Tree Regressor (DTR)	0.0010	0.0322	0.0205	1.3339	1.3803	0.0521	0.9941
K-Nearest Neighbors Regression (KNN)	0.0009	0.0304	0.0179	1.1511	1.1287	0.0455	0.9948
Bidirectional Long Short-Term Memory (BiLSTM)	0.0034	0.0580	0.0495	2.0861	2.1146	0.4942	0.8498
Bidirectional Gated Recurrent Unit (Bi-GRU)	0.0035	0.0588	0.0521	2.2096	2.2401	0.4155	0.8382
Hybrid Model: Stacking RF + GBM	0.0036	0.0598	0.0349	1.3977	1.4203	1.9394	0.8875
Hybrid Model: LSTM + Random Forest	0.0027	0.0519	0.0309	1.2466	1.2561	1.7129	0.9153
Hybrid Model: LSTM + GRU	0.0017	0.0418	0.0301	1.2569	1.2550	1.6714	0.9450

Best Models for Each Metric:

- **MSE:** Ridge Regression (Ridge) (**0.0001**)
- **RMSE:** Ridge Regression (Ridge) (**0.0113**)
- **MAE:** Ridge Regression (Ridge) (**0.0067**)
- **MAPE (%)**: Ridge Regression (Ridge) (**0.4288**)
- **sMAPE (%)**: Ridge Regression (Ridge) (**0.4283**)
- **MASE:** Ridge Regression (Ridge) (**0.0170**)
- **R²:** Ridge Regression (Ridge) (**0.9993**)

Model Evaluation Metrics Summary - Key Insights

The table presents the performance of different regression models using metrics like MSE, RMSE, MAE, MAPE, sMAPE, MASE, and (R^2). Below is a concise analysis:

Best Performing Models

- **Ridge Regression (Ridge)**
 - **Lowest error across all metrics (MSE: 0.0001, RMSE: 0.0113, MAE: 0.0067, MAPE: 0.4288%)**
 - **Highest (R^2) (0.9993)**
 - **Best overall predictive accuracy.**
- **Gradient Boosting Models (XGBoost, HGBR, CatBoost, GBR)**
 - **High accuracy with low RMSE (~0.0242-0.0247) and high (R^2) (~0.9966-0.9967).**
 - **Strong candidates for regression tasks.**

Weakest Models

- **Neural Networks (BiLSTM, Bi-GRU)**
 - **High errors (MSE: ~0.0034-0.0035, RMSE: ~0.0588, MAE: ~0.0495-0.0521).**
 - **Lowest (R^2) (~0.8382-0.8498), indicating poor fit.**
 - **Not suitable for this dataset.**

Hybrid Model Performance

- **LSTM + GRU Hybrid:**
 - **Performs better than standalone LSTM or GRU models.**
 - **MAPE: 1.2569%, RMSE: 0.0418** – Improved over deep learning models.

Key Takeaways

- **Ridge Regression & Gradient Boosting models (XGBoost, HGBR, CatBoost) are the best choices.**
- **Avoid using BiLSTM & Bi-GRU due to high errors and poor (R^2).**
- **Hybrid models improve deep learning performance but still lag behind tree-based methods.**

Recommendation: Use Ridge Regression or Gradient Boosting for best accuracy!

Model Comparison: Mean Squared Error (MSE)

Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their Mean Squared Error (MSE).
- **Lower MSE indicates better performance**, meaning models with shorter bars perform better.

Top Performing Models (Lowest MSE)

1. **Ridge Regression** has the lowest MSE, suggesting it provides the best predictive accuracy.
2. **Histogram-Based Gradient Boosting Regressor (HGBR)** and **Extreme Gradient Boosting (XGBoost)** also have low MSE, indicating strong performance.

Moderate Performance Models

- **Gradient Boosting Regressor (GBR)**, **CatBoost**, and **Extremely Randomized Trees Regressor (ERTR)** fall in the mid-range in terms of MSE.
- **LightGBM** and **ElasticNet Regression** show comparable performance but slightly higher error.

Higher MSE (Lower Performance) Models

- **Hybrid deep learning models** such as **LSTM + GRU** and **LSTM + Random Forest** have relatively high MSE, suggesting weaker predictive accuracy.
- **Random Forest (RF)** and **Bi-LSTM** also perform worse than simpler models like Ridge Regression.
- **Stacking models (Hybrid Model: Stacking RF + GBM)** show the highest MSE, indicating poor predictive performance.

Insights

- Traditional regression models like **Ridge Regression** and **Lasso Regression** outperform complex deep learning models in terms of MSE.
- **Boosting techniques (XGBoost, LightGBM, CatBoost)** provide competitive performance.
- Deep learning models, despite their complexity, **do not necessarily yield better results in this context**.
- Hybrid models involving **LSTM and Random Forests** have **higher error rates** compared to traditional models.

Conclusion

- **Ridge Regression** is the most accurate model based on MSE.
- Deep learning and hybrid models do not outperform traditional boosting and regression models in this analysis.
- For lower MSE, simpler models like Ridge Regression or boosting techniques may be preferable.

Model Comparison: Root Mean Squared Error (RMSE)

Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their **Root Mean Squared Error (RMSE)**.
- **Lower RMSE values indicate better model performance**, as they signify lower prediction errors.

Top Performing Models (Lowest RMSE)

1. **Ridge Regression** achieves the lowest RMSE, indicating the most accurate predictions.
2. **Histogram-Based Gradient Boosting Regressor (HGBR)**, **Extreme Gradient Boosting (XGBoost)**, and **Gradient Boosting Regressor (GBR)** also perform well, showing relatively low RMSE.

Moderate Performance Models

- **Categorical Boosting (CatBoost), Extremely Randomized Trees Regressor (ERTR), and LightGBM** exhibit moderate RMSE values.
- **ElasticNet Regression and K-Nearest Neighbors Regression (KNN)** are slightly worse but still competitive.

Higher RMSE (Lower Performance) Models

- **Hybrid deep learning models** such as **LSTM + GRU** and **LSTM + Random Forest** have **higher RMSE**, suggesting weaker predictive accuracy.
- **Random Forest (RF) and Bi-LSTM** also perform worse than simpler models like Ridge Regression.
- **The Stacking Hybrid Model (RF + GBM)** shows the **highest RMSE**, meaning it has the poorest prediction accuracy.

Insights

- Traditional regression models like **Ridge Regression** and **Lasso Regression** **outperform deep learning models** in terms of RMSE.
- **Boosting algorithms (XGBoost, LightGBM, CatBoost)** provide **competitive performance**, making them strong alternatives.
- **Deep learning and hybrid models** do not necessarily lead to **better performance** in this dataset.
- **Hybrid deep learning models** tend to have **higher RMSE**, suggesting they may not generalize well compared to simpler models.

Conclusion

- **Ridge Regression** is the **best-performing model** in terms of RMSE.
- **Boosting methods (XGBoost, LightGBM, CatBoost)** are **effective choices** for reducing error.
- **Deep learning models, including Bi-GRU and Bi-LSTM**, show **weaker predictive accuracy** compared to traditional regression methods.
- For optimal performance, simpler models like **Ridge Regression** or **tree-based boosting methods** may be preferable.

Model Comparison: Mean Absolute Percentage Error (MAPE)

Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their **Mean Absolute Percentage Error (MAPE)**.
- **Lower MAPE values indicate better performance**, as they signify lower relative errors in predictions.

Top Performing Models (Lowest MAPE)

1. **Ridge Regression** has the lowest MAPE, making it the most accurate model in terms of percentage error.
2. **Gradient Boosting Regressor (GBR), Extreme Gradient Boosting (XGBoost), and Histogram-Based Gradient Boosting Regressor (HGBR)** also show low MAPE values, indicating strong predictive accuracy.

Moderate Performance Models

- **Support Vector Regression (SVR), CatBoost, and K-Nearest Neighbors Regression (KNN)** exhibit moderate MAPE values.
- **Extremely Randomized Trees Regressor (ERTR) and Random Forest (RF)** have slightly higher errors but remain competitive.

Higher MAPE (Lower Performance) Models

- **Deep learning models** such as **Bi-GRU** and **Bi-LSTM** have the **highest MAPE**, indicating poorer predictive performance in terms of percentage error.

- **Hybrid models (LSTM + GRU, LSTM + Random Forest, Stacking RF + GBM)** also show high MAPE, suggesting they struggle with relative error minimization.
- **Lasso Regression and Decision Tree Regressor (DTR)** also exhibit relatively high MAPE, making them less effective.

Insights

- **Ridge Regression outperforms deep learning models** in terms of MAPE, suggesting that simpler models provide better generalization.
- **Boosting techniques (XGBoost, GBR, HGBR)** show strong performance, indicating they effectively minimize relative errors.
- **Deep learning and hybrid models do not necessarily lead to better performance**, as seen in their higher MAPE values.
- **Traditional regression models like Ridge and ElasticNet Regression perform well compared to complex deep learning architectures.**

Conclusion

- **Ridge Regression achieves the best performance** based on MAPE.
- **Boosting models (XGBoost, GBR, HGBR)** are strong contenders for minimizing percentage error.
- **Deep learning models (Bi-GRU, Bi-LSTM)** struggle with relative error, making them less effective than simpler models.
- **For minimizing MAPE, traditional regression and boosting techniques are preferable** over complex hybrid and deep learning models.

Model Comparison: Symmetric Mean Absolute Percentage Error (sMAPE)

Key Observations

- The bar chart compares multiple machine learning and deep learning models based on **Symmetric Mean Absolute Percentage Error (sMAPE)**.
- **Lower sMAPE values indicate better performance**, as they represent lower percentage errors in predictions.

Top Performing Models (Lowest sMAPE)

1. **Ridge Regression** achieves the lowest sMAPE, indicating the most accurate predictions.
2. **Gradient Boosting Regressor (GBR), Extreme Gradient Boosting (XGBoost), and Histogram-Based Gradient Boosting Regressor (HGBR)** also show low sMAPE values, making them strong contenders.

Moderate Performance Models

- **Support Vector Regression (SVR), CatBoost, and Extremely Randomized Trees Regressor (ERT)** exhibit moderate sMAPE values.
- **K-Nearest Neighbors Regression (KNN) and LightGBM** have slightly higher errors but still perform reasonably well.

Higher sMAPE (Lower Performance) Models

- **Deep learning models such as Bi-GRU and Bi-LSTM** have the highest sMAPE, indicating poorer predictive performance in terms of relative percentage error.
- **Hybrid models (LSTM + GRU, LSTM + Random Forest, and Stacking RF + GBM)** also show high sMAPE, suggesting they struggle with relative error minimization.
- **Lasso Regression and Decision Tree Regressor (DTR)** also exhibit relatively high sMAPE, making them less effective compared to boosting and traditional regression models.

Insights

- **Ridge Regression outperforms deep learning models** in terms of sMAPE, highlighting the efficiency of simpler models.

- **Boosting techniques (XGBoost, GBR, HGBR) provide competitive performance**, suggesting they effectively handle percentage errors.
- **Deep learning models do not necessarily improve performance**, as seen in their higher SMAPE values.
- **Traditional regression models like Ridge and ElasticNet Regression perform well compared to complex architectures.**

Conclusion

- **Ridge Regression is the best-performing model** based on SMAPE.
- **Boosting methods (XGBoost, GBR, HGBR) are effective choices** for minimizing percentage error.
- **Deep learning models (Bi-GRU, Bi-LSTM) exhibit weaker predictive accuracy** in terms of SMAPE.
- **For optimal performance, traditional regression and boosting methods should be preferred over hybrid and deep learning models.**

Model Comparison: Mean Absolute Scaled Error (MASE)

Key Observations

- The bar chart compares multiple machine learning and deep learning models based on **Mean Absolute Scaled Error (MASE)**.
- **Lower MASE values indicate better model performance**, meaning models with shorter bars have lower scaled error.

Top Performing Models (Lowest MASE)

1. **Ridge Regression** achieves the lowest MASE, indicating the best predictive accuracy.
2. **Extreme Gradient Boosting (XGBoost), Gradient Boosting Regressor (GBR), and Histogram-Based Gradient Boosting Regressor (HGBR)** also perform well with relatively low MASE.
3. **Other boosting models like CatBoost and LightGBM** also exhibit low scaled errors.

Moderate Performance Models

- **Support Vector Regression (SVR), Extremely Randomized Trees Regressor (ERTR), and K-Nearest Neighbors Regression (KNN)** show moderate MASE values.
- **ElasticNet Regression, Lasso Regression, and Decision Tree Regressor (DTR)** have slightly higher MASE but remain competitive.

Higher MASE (Lower Performance) Models

- **Deep learning and hybrid models such as Stacking RF + GBM, LSTM + Random Forest, and LSTM + GRU** exhibit the highest MASE, indicating poor predictive accuracy.
- **Random Forest (RF) and BiLSTM** also show significantly higher MASE compared to simpler regression models.
- **Hybrid models combining LSTM and traditional ML models perform worse than standalone boosting or regression models.**

Insights

- **Ridge Regression outperforms all models**, highlighting the strength of traditional regression methods.
- **Boosting techniques (XGBoost, GBR, HGBR) provide strong performance**, suggesting they effectively minimize scaled errors.
- **Deep learning and hybrid models do not necessarily lead to better performance**, as seen in their higher MASE values.
- **Traditional regression models and boosting algorithms outperform deep learning architectures**, making them more reliable choices.

Conclusion

- **Ridge Regression is the best-performing model** based on MASE.
- **Boosting models (XGBoost, GBR, HGBR) are strong alternatives** for reducing scaled errors.
- **Deep learning models (BiLSTM, Bi-GRU) and hybrid approaches underperform**, making them less suitable for this dataset.
- **For lower scaled errors, simpler models like Ridge Regression or tree-based boosting methods are preferable** over complex hybrid and deep learning models.

Model Comparison: Coefficient of Determination (R^2)

Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their **Coefficient of Determination (R^2)**.
- **Higher R^2 values indicate better model performance**, meaning models with longer bars have better predictive accuracy.

Top Performing Models (Highest R^2)

1. **Ridge Regression achieves the highest R^2** , indicating it explains the most variance in the data.
2. **Histogram-Based Gradient Boosting Regressor (HGBR), Extreme Gradient Boosting (XGBoost), and Gradient Boosting Regressor (GBR)** also perform very well, showing high predictive accuracy.
3. **Other boosting models like CatBoost and LightGBM also have high R^2 values**, confirming their strong generalization ability.

Moderate Performance Models

- **Support Vector Regression (SVR), Extremely Randomized Trees Regressor (ERTR), and K-Nearest Neighbors Regression (KNN)** show moderate R^2 values.
- **ElasticNet Regression, Lasso Regression, and Decision Tree Regressor (DTR)** perform slightly worse but still explain a significant portion of variance.

Lower R^2 (Poorer Performance) Models

- **Deep learning models such as Bi-GRU and Bi-LSTM exhibit the lowest R^2 values**, indicating poor predictive performance.
- **Hybrid models (LSTM + GRU, LSTM + Random Forest, and Stacking RF + GBM) also show lower R^2 values**, suggesting they do not generalize well.
- **Random Forest (RF) and Decision Tree Regressor (DTR) also struggle** compared to simpler models.

Insights

- **Ridge Regression outperforms all models**, highlighting the efficiency of simpler, well-regularized models.
- **Boosting techniques (XGBoost, GBR, HGBR) provide competitive performance**, confirming their ability to handle complex patterns.
- **Deep learning models (Bi-GRU, Bi-LSTM) struggle to explain variance**, making them less effective in this scenario.
- **Traditional regression models and boosting algorithms outperform deep learning architectures**, suggesting simpler models are more suitable.

Conclusion

- **Ridge Regression is the best-performing model** based on R^2 .
- **Boosting models (XGBoost, GBR, HGBR) are strong alternatives** for achieving high predictive accuracy.
- **Deep learning models (BiLSTM, Bi-GRU) and hybrid approaches underperform**, making them less reliable.
- **For high R^2 values, simpler models like Ridge Regression or tree-based boosting methods are preferable** over complex hybrid and deep learning models.